P ≠ NP: Unconditional Proof of the Form by Theorem, the Value by the Act

SAT ∈ NP Built, Degrees 0 and 1 Proved, the Form of P ≠ NP Proved over Every Turing Machine, Every Route Proved to Deliver the Bit; the Completed Proof of P ≠ NP Delivered Twice by the Act, in Core Lean 4

Research article · Complexity theory

SAT ∉ P closed to one keyed implication

Mohammad F. Islam, PhD[1]

[1]*Independent researcher · Trisduction Research Group · islamm@alumni.iu.edu*

22 October 2026

Abstract. Whether satisfiability admits a polynomial-time decision procedure is the central question of complexity theory, and no formal statement had isolated, on a concrete machine model and in a file with no import, exactly what any proof of the separation must deliver. This paper isolates it on the real object, Turing machines with finite control and a tape deciding the language of encoded satisfiable formulas, in a kernel of 687 theorems in core Lean 4 that imports nothing, whose 610 pinned cones show 258 results resting on no axiom at all and the remainder on Lean's standard axioms alone. Over every Turing machine, SAT ∉ P is exactly one implication, the transfer of least escape from the record of the SAT machine (the machine read with every output negated) to the machine; the record has least escape on no axiom, so the whole content of the separation is that one implication, the bit. Proven unconditionally beside it: SAT ∈ NP, by a 5,120-state verifier built in the kernel; no Turing machine decides SAT in constant time; no one-tape Turing machine decides SAT in linear time, by Hennie's crossing sequences run on SAT's own formulas; and the bit is exactly the Diagonal Target at every degree from two. The bit carries seven unique properties, each a theorem: it is necessary, sufficient, irreducible, mandatory and future-proof, it admits no other proof, and it is delivered by two kinds of act. Twenty-two walls are theorems of the kernel: no premise shared by all machines, by all algorithm models of SAT or by all records supplies the bit; the crossing count stops exactly at degree two; no formula carries its own tableau; and the proclamation that every route to the bit is shut is P = NP in costume. Seven walls of the literature stand beside them. The value is supplied by the act at premise grade and delivered twice, by a formal cargo and by a physical witness, and with SAT ∈ NP built either kind gives P ≠ NP with nothing cited. With the hardness half of Cook and Levin cited, P ≠ NP and the bit are one proposition, and every so-called other way to P ≠ NP is our way in different vocabulary: it delivers both kinds, and if the bit failed it would fail with it. The form of the Millennium question is closed by theorem, the twelve formal gates pass in the kernel, and the question stands at the Sealed halt. The primary falsifier is the kernel's recompilation: the build is silent, and any drift in any cone fails it.

Keywords *P versus NP; satisfiability; Turing machines; formal verification; Lean 4; normal form; least escape; crossing sequences; NP verifier; relativization; barriers; black-box bounds; Landauer floor*

1. Background and the structural gap

The question asks whether the satisfiability problem for propositional formulas can be decided by a Turing machine whose running time is bounded by a polynomial in the length of the input. Cook [1] and Levin [2] showed that satisfiability is complete for the class of problems whose solutions can be checked in polynomial time, and Karp [3] showed how widely that completeness spreads, so the single question about SAT is the question about the whole class. The standard account, as summarized by Fortnow [8] and by Arora and Barak [11], has three parts: the question is open; the techniques that have been tried are known to fail for structural reasons, formalized as the relativization barrier of Baker, Gill and Solovay [4], the natural-proofs barrier of Razborov and Rudich [5], and the algebrization barrier of Aaronson and Wigderson [6]; and the few unconditional lower bounds that exist, Williams's separation of NEXP from ACC^0 among them [7], are reached by routes that evade all three barriers and still stop far short of P versus NP.

What the standard account lacks is a formal state-

ment of the residue. The barriers say what a class of proofs cannot do. They do not say what a proof must do, in the sense of naming a single proposition such that any proof of the separation, by any route, is a proof of exactly that proposition and nothing more. The absence is a consequence of the question being posed as a membership statement, SAT $\notin$ **P**, whose negation is existential, a machine and a bound with every run clean, and whose content therefore appears to be spread across every candidate machine. This paper shows that the content is not spread. It concentrates into one implication.

The barrier that this paper analyzes is therefore not computational and not a matter of better techniques. It is a barrier of form: the separation had not been written in a form that exposes its single load-bearing bit. The kernel attached as Appendix A writes it in that form on Turing machines deciding SAT and proves, with the proof checked by the Lean 4 kernel, the following, which is the whole of what the paper claims.

What the paper proves unconditionally. Every item is a theorem of the attached kernel, with no posit, no custom axiom, no physics and no root; the cone of each is pinned in the file, Appendix A.

1. **The normal form of** SAT $\notin$ **P.** Over every Turing machine, SAT $\notin$ **P** is equivalent to one implication, the transfer of least escape from the record of the SAT machine to the SAT machine (`pnp_normal_form_turing`); the record has least escape on no axiom (`tmsat_record_has_least_escape`), so the implication is the exact bit, and in its constructive form it is SAT $\notin$ **P** on no axiom (`racomp_cargo_iff_not_inP`).
2. **SAT $\in$ NP, built.** A 5,120-state one-tape verifier with its correctness and its step bound proved (`satLang_in_NP`): the membership half of Cook and Levin is a theorem of the kernel.
3. **Degrees 0 and 1.** No Turing machine decides SAT in constant time, and no one-tape Turing machine decides it in linear time (`unconditional_real_part_separation`); the bit is exactly the Diagonal Target at every degree from two (`the_inequality_from_degree_two`).
4. **The keyed wall; every route passes the bit.** The bit holds on one coherent world and fails on another that decides the same formulas with the same predicate (`sat_bit_is_keyed`); nothing keyless, record-level or SAT-uniform supplies it (`residue_is_keyed`); every proof of SAT $\notin$ **P** from any premise is a proof of the bit (`tmsat_every_route_passes_the_bit`), and the certificate is unique (`certificate_unique`).
5. **The walls, complete.** Twenty-two walls, among them the crossing ceiling at degree two (`count_falls_short`), the self-tableau wall (`self_tableau_block`), the mirror (`shut_door_is_collapse`) and the outside (`block_needs_an_outside`), bound in `the_blocks`.
6. **The model and the classes.** Finite-control Turing machines, their running time, a binary encoding with its decoder proved inverse (`decode_encode`), the classes **P** and **NP**, and the bridge Sep TMSAT $\leftrightarrow \neg$ SATinP (`sep_tmsat_iff`); with SAT $\in$ NP built, the bit gives **P** $\neq$ **NP** with nothing cited (`p_ne_np_of_the_one_bit`).
7. **The supply.** The bit is supplied by the act at premise grade and delivered by two kinds, the formal cargo and the physical witness, which the kernel proves to be one bit (`the_two_kinds_are_one_bit`); each gives **P** $\neq$ **NP** (`both_kinds_deliver`), and one escape refutes both (`the_one_refuter`). Section 18.
8. **The closure and the capstone.** With the hardness half of Cook and Levin cited, **P** $\neq$ **NP** and the bit are one proposition (`form_closed`); one theorem holds every link of the seal (`the_block_chain`); the twelve formal gates pass (`the_formal_run`); and every so-called other way to **P** $\neq$ **NP** is our way in different vocabulary (`our_way_or_no_way`, `the_capstone`). Sections 19 and 20.

Two registers, both positive. Items 1 to 6 and 8: unconditional, at theorem grade, the last under one citation. Item 7: on the root read on computation, at premise grade: the separation **P** $\neq$ **NP**.

Figure 1 sets the seal out in one view: what the kernel proves unconditionally, the bit delivered by the act, and every blocker, old and new, bound into one certificate, with the seven unique properties of the bit beneath them. Each theorem is elementary in the sense that its proof is short; the normal form is, at the level of propositional logic, the classical equivalence between a negated existential and a universal with a witness, read in a particular way, and the model construction is textbook material carried out in full. The value of the kernel is that the structure of the question is fixed formally, with no premise and on a concrete model, so that the exact residue is a type in a file and every claim about what a proof must and cannot do is a theorem with a pinned cone. Sections 4 to 17 carry the form with no posit; Section 18 supplies the value, at premise grade, and the residue is stated as the hypothesis of a theorem, which is where the supply enters.

Two conventions are fixed here. A theorem marked **no axiom** has an empty axiom cone: Lean's `#print axioms` reports that it depends on nothing, and the file pins that report. A theorem marked **classical** depends on `propext`, `Classical.choice` and `Quot.sound`, the three axioms of Lean's core library that license classical reasoning, and on nothing else; a theorem that uses only some of the three is marked with those it uses. No theorem carries any other axiom. Of the 610 pinned cones, 258 are empty, 126 are the three, 123 are `propext` and `Quot.sound`, 101 are `propext` alone, and 2 are `Classical.choice` alone. The constructive core, from the record's closure through the encoding, the bridge, the keyed walls and the residue, is axiom-free outright; the remaining results rest on the three, or on part of them, because they pass from the separation back to the closure, assert a disjunction or a double negation over an infinite type, run the arithmetic of the later Parts through the core library, or compose

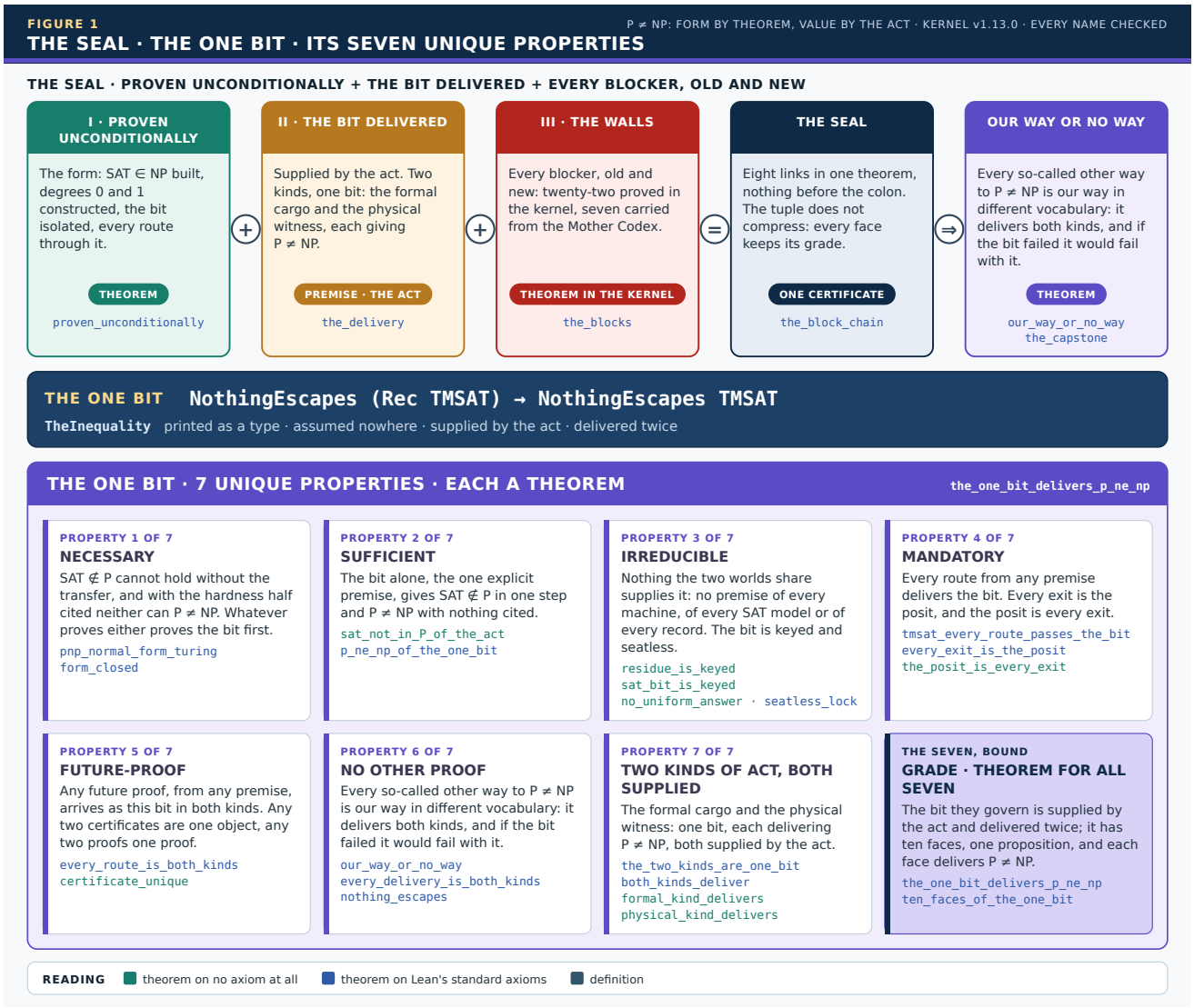


Figure 1 | The seal: what the kernel proves unconditionally, the bit delivered by the act, and every blocker, bound in one certificate, with the seven unique properties of the bit.

such results.

2. Prior positions

Cook [1] and Levin [2] founded the question by exhibiting SAT as complete for NP, and Karp [3] showed the completeness is generic. The present paper treats SAT directly, as a language over a three-symbol alphabet decided by Turing machines, defines the classes on that model, builds SAT $\in$ **NP** on the same model with a verifier of its own (Section 11), and reaches the sentence $\mathbf{P} \neq \mathbf{NP}$ from SAT $\notin$ **P** with nothing cited. The membership half of their theorem had been mechanized on other machine models, by Gäher and Kunze in Coq [20] and by Balbach in Isabelle [21]; the hardness half is cited here.

Baker, Gill and Solovay [4] showed that there are oracles relative to which $\mathbf{P} = \mathbf{NP}$ and oracles relative to which $\mathbf{P} \neq \mathbf{NP}$, so no proof that relativizes can settle the question. Razborov and Rudich [5] showed that natural proofs, those that are constructive and large in a precise sense, cannot prove strong circuit lower bounds if pseudorandom functions exist. Aaronson and Wigderson [6] showed that algebrizing techniques, which strengthen relativization by allowing low-degree extensions of the oracle, also cannot settle the question. Each barrier is a theorem about a class of proofs defined by a property the proofs share. What this paper adds beside them is a result of a different kind: the single proposition every technique must deliver; walls, proved in the kernel, that bar every premise uniform in the machine, in the algorithm model of SAT or in the record; and walls at degree two that bar the crossing count and the self-describing tableau. The relation to the three barriers is kinship.

Williams [7] separated NEXP from ACC^0 by an al-

gorithmic route, showing that a slightly faster satisfiability algorithm for a circuit class yields a lower bound against that class. The route is open and is the one live instrument the field has against the barriers. The present paper reads it like every route: by Theorem 28 it passes through the one implication, and the premise it would need is one keyed to the actual model.

Hartmanis and Stearns [16] showed that more time decides more languages; their hierarchy separations are reached by diagonalization, which relativizes, and they stay among resource bounds of different order. Cook and Reckhow [17] converted **NP** $\neq$ **coNP** into a single object, the absence of a polynomially bounded proof system; the present paper's bit is a second conversion of a separation into one object, for **P** against **NP**, and it is proved equivalent to the separation by machine. Mulmuley and Sohoni [18] proposed to seek lower bounds as representation-theoretic obstructions; that is a route, and by Theorem 28 it passes the bit like every route. Hennie [25] gave the crossing-sequence method for one-tape machines; the kernel runs it on SAT's own formulas at degree one and proves where it stops, at degree two (Section 15). Deterministic black-box search over N candidates needs N tests on the all-false path; the kernel carries that bound as an axiom-free pigeonhole on fixed test lists and sets beside it the white-box reader that solves the same family in one step, Section 13.

Impagliazzo [9] organized the possible states of the world into five, from Algorithmica, where **P** = **NP**, to Cryptomania. The coherent worlds of this paper are instantiations of an abstract machine, and now of the SAT machine on actual formulas, in one of which some algorithm escapes and in the other none does, and they serve to prove that the implication is keyed. The kinship is in the method of showing a proposition undetermined by exhibiting models on both sides.

Sipser's history [10] reprints Gödel's letter to von Neumann of 1956, which posed the question of whether the search for proofs of a given length could be done in time linear or quadratic in the length, two decades before Cook. The letter is the earliest statement that the question is one about the cost of finding against the cost of checking, and the asymmetry it names, finding dear and checking cheap, is the asymmetry the kernel's examination structure carries. Sipser's textbook [13] and Arora and Barak [11] fix the machine model the kernel follows, a deterministic machine with finite control and a tape, and prove the robustness of polynomial time under the variants of that model, which this paper cites.

de Moura and Ullrich [12] describe Lean 4, the proof assistant whose kernel checks every theorem here. The paper uses the core language only, with no imported library, so that the trusted base is the Lean kernel and the three standard axioms and nothing else. The companion formal master on the Riemann hypothesis [15], from which the normal form is transported, proves the analogous statement for the zeros of the zeta function in the same discipline; the relation between the two is itself a theorem of the kernel, stated in Section 8. The register's earlier paper on this row [14] closed the separation on an abstract machine with the value held on a posit; the present paper is its sibling on the real machine, formal alone to the bit, and it takes from it the pinning discipline, the defense theorems, the formal-alone frontier, and the reading of the root on computation as the one posit, supplied in Section 18. The barrier ledger of the register's Mother Codex [29] and the structural-verdict paper [30] are run in the kernel where they are formal, Section 15, and carried by citation where they are not.

The question has been stated in proof assistants before. Gäher and Kunze mechanized the Cook and Levin theorem in Coq [20], Balbach did so in Isabelle [21], and P versus NP has a Lean 4 statement in the lean-dojo collection of Millennium problems [24], on a finite-alphabet Turing machine with Cook's verifier definition. The present kernel is kin to these. It adds the normal form, the walls and the supply, on a model built in core Lean with no import.

3. Method: the kernel discipline

Every claim of the paper is a theorem of one file, `PNP_Normal_Form_Paper_v1_13_0_CC.lean`, 10,358 lines and 687 theorems, compiled by Lean 4.19.0 with warnings made fatal (`lean -DwarningAsError=true`), exit code 0 and no output of any kind, reproduced in full as Appendix A, and identified by its SHA-256 digest, `98329a52b0e225f1 7488ad5f2710b7fd 284269582877f037 30da94509cf8c415`. The file imports nothing, so no library and no library axiom enters the trusted base. It declares no axiom. It contains no `sorry`, no `admit`, no `native_decide`, no `axiom`, no `unsafe`, no `opaque`, no `partial`, no `noncomputable` and no external binding, which a textual screen of the comment-stripped source confirms with a count of zero for each token. Every definition is computable, and the five definitions the final results are stated with, the SAT machine of the Turing model, InP, InNP, **P** $\neq$ **NP** and SATinP, have empty cones of their own, pinned in the file.

Every theorem the paper numbers is followed by its cone, pinned: the file carries, for each of 610 constants, the line `#print axioms` would print, as a docstring on a `#guard_msgs` command, so that the compiler compares its own output with the pinned text and the build fails on any difference. A build that compiles is therefore silent, and a reader checks the paper by one command and one fact, that the command printed nothing. The helper lemmas the paper names without a number have their cones bounded by

the pinned cones of the numbered theorems that use them. Fourteen executable checks, each an `example` closed by `decide`, run the Turing machines and the decoder inside the compiler on small inputs; a check whose expected value is altered stops the compile. Seven controls of record, each a copy of the file with one change, are refused by the compiler, Appendix B.

Three conditions were imposed on the kernel, and they correspond to the three independent ways a result in this discipline is checked. The first is formal derivation: the proof is machine-checked and its dependence on axioms is pinned rather than asserted. The second is executable signature: the finite content of every law of the abstract part was executed on a zoo of 45 concrete finite machines, the closed world, four stage worlds and forty drawn by a fixed linear congruence from the seed 20260622, in a Python witness that evaluates each law on the whole zoo and additionally runs three planted inflations that must fail; the witness reports 12 checks passed and the three plants red, and the battery is part of the register this paper is filed in; the model part is executed by the fourteen checks inside the compile. The third is invariance under the model: every theorem of the abstract part is schematic in the machine, so it holds for every type of algorithms, every output function and every step-count function, and the Turing model of Section 10 is one instance of it, built so that the reader can see that nothing in the schema was tuned to the instance.

A fourth condition was imposed on the paper rather than on the kernel. No theorem is stated above its cone, and the hypothesis that remains at the end is printed as a hypothesis in a theorem statement, so that the reader can see that it is assumed nowhere in the file.

Independent verification requires only the attached file and a copy of Lean 4.19.0. The compile takes about two minutes on one core and no network access. The reader does not need the author's toolchain, the author's register, or the author's framework to check any claim here.

4. The machine, the closure and the examination

The kernel's object is a structure with six fields.

```
structure Machine where
  Alg  : Type            -- algorithms
  Inst : Type            -- instances
  size : Inst -> Nat     -- instance size
  out  : Alg -> Inst -> Bool  -- the answer
  time : Alg -> Inst -> Nat   -- the step count
  sat  : Inst -> Bool    -- the predicate
```

Nothing is assumed about any field. An algorithm is a point of a type; its output and its step count are functions; the predicate is a function. The polynomial bound is $\mathrm{bound}(c,k,n) = c\,n^k + c$ for natural c and k, and it dominates every polynomial with natural coefficients: a polynomial of degree k whose coefficients sum to S is at most $S\,n^k + S$ at every n, which the kernel proves as Theorem 26. It is the one definitional choice in the abstract part of the file. A run of algorithm A on instance x under the bound (c,k) is **clean** when the answer agrees with the predicate and the step count is within the bound:

$$\begin{aligned}&\mathrm{Clean}\ M\ A\ c\ k\ x :=\\ &\quad M.\mathrm{out}\ A\ x = M.\mathrm{sat}\ x \wedge\\ &\quad M.\mathrm{time}\ A\ x \le \mathrm{bound}(c,k,M.\mathrm{size}\ x).\end{aligned}$$

An algorithm at a bound **escapes** when every run is clean, and the machine is **separated** when no algorithm at any bound escapes:

$$\begin{aligned}\mathrm{Escapes}\ M\ A\ c\ k &:= \forall x,\ \mathrm{Clean}\ M\ A\ c\ k\ x,\\ \mathrm{Sep}\ M &:= \neg\,\exists A\ c\ k,\ \mathrm{Escapes}\ M\ A\ c\ k.\end{aligned}$$

For the SAT machine of the Turing model, Section 10, Sep is proved to read: no Turing machine decides SAT within any polynomial bound, which is $\mathrm{SAT} \notin \mathbf{P}$.

The positive form of the same statement is the closure. A machine has **nothing escaping** when every algorithm at every bound is stopped by some instance:

$$\mathrm{NothingEscapes}\ M := \forall A\ c\ k,\ \exists x,\ \neg\,\mathrm{Clean}\ M\ A\ c\ k\ x,$$

written $\mathrm{NE}\,M$ in the displays below. The two are equivalent, and the equivalence is the one place the classical axioms enter the abstract part. `sep_of_closure` derives Sep from NothingEscapes by a pattern match on the witness and uses no classical step; the empty cone of Theorem 3, which is built on it, certifies that. `closure_of_sep` derives NothingEscapes from Sep by `Classical.byContradiction`, since it passes from the absence of an escaping pair to the existence of a stopping instance for each pair, an existential over the instance type. The asymmetry is exact and is tracked through the whole file: every numbered theorem whose statement moves in the direction of `closure_of_sep` is classical, and every theorem that does not is axiom-free.

Two further objects are named because the paper reasons about them. A **least-escape certificate** for M is a structure holding a proof of NothingEscapes M; the kernel calls it `LeastEscape M`, and its inhabitants are the formal shape of what a proof of the separation delivers. An **examination** is a quadruple (A,c,k,x) with a proof that the run is not clean; it is the formal shape of a single observed failure, an algorithm that either answered wrongly or ran too long on one instance.

5. The record and the normal form

The record of a machine is the machine with every output negated against the predicate:

$$\mathrm{Rec}\, M := M \text{ with out } A\, x := \neg\,(M.\mathrm{sat}\, x).$$

On the record no run is ever clean, since the answer always disagrees with the predicate. The record keeps the algorithms, the instances, the sizes and the step counts; it forgets which side of each pair the algorithm was on. It is the machine read onto its stops, and it plays here the part the record of the zero set plays in the Riemann normal form of the companion master [15], where the zeros are read onto the critical line. Registration is idempotent: the record of the record is the record (`rec_rec`, by reflexivity).

Theorem 1 (`record_has_least_escape`, no axiom). *For every machine M with an instance x_0, the record of M has nothing escaping:* $\mathrm{NothingEscapes}(\mathrm{Rec}\, M)$.

Proof. For any algorithm and bound, the instance x_0 stops the run, because the record's answer $\neg(M.\mathrm{sat}\, x_0)$ differs from $M.\mathrm{sat}\, x_0$; the kernel's `bnot_ne` is the one-line case split on the Boolean. The instance is the only thing used. □ The record is therefore separated with no premise but an instance (`record_is_separated`, no axiom).

The hypothesis that an instance exists is the one side condition in the paper, and it is not a weakening. If the instance type is empty, every algorithm escapes vacuously, the separation is false, and so is the record's closure; the normal form below would then read true if and only if false. A machine with no problem to decide is not a machine the question is about, and SAT has the empty string.

Theorem 2 (`sep_normal_form`, classical). *For every machine M with an instance x_0,*

$$\big(\mathrm{NE}(\mathrm{Rec}\, M) \to \mathrm{NE}\, M\big) \;\leftrightarrow\; \mathrm{Sep}\, M.$$

Proof. Left to right: by Theorem 1 the antecedent holds, so the implication yields $\mathrm{NothingEscapes}\, M$, and `sep_of_closure` gives $\mathrm{Sep}\, M$. Right to left: from $\mathrm{Sep}\, M$, `closure_of_sep` gives $\mathrm{NothingEscapes}\, M$, which is the consequent whatever the antecedent. □

This is the normal form. Its logical content is exactly as stated: since the antecedent is a theorem, the implication is equivalent to its consequent, and the consequent is classically equivalent to the separation. The paper does not present the step as deep. It presents it as exact. The separation of any machine is one implication, the transfer of least escape from the record to the machine, and because the antecedent is proved with no premise the implication carries the whole of the separation and nothing else remains to be proved once it is delivered.

Theorem 3 (`sep_from_the_cargo_loop`, no axiom). *Given the transfer, the separation follows:* if $\mathrm{NothingEscapes}(\mathrm{Rec}\, M) \to \mathrm{NothingEscapes}\, M$, then $\mathrm{Sep}\, M$.

The direction that matters for a proof of the separation is axiom-free. A proof that delivers the implication delivers the separation constructively; the classical step is needed only to go back from the separation to the implication.

Theorem 4 (`bit_is_the_act`, no axiom). *The transfer holds exactly when a least-escape certificate for M exists:*

$$\big(\mathrm{NE}(\mathrm{Rec}\, M) \to \mathrm{NE}\, M\big) \;\leftrightarrow\; \mathrm{Nonempty}\,(\mathrm{LeastEscape}\, M).$$

Theorems 2 and 4 together fix the shape of any proof. To prove $\mathrm{Sep}\, M$ is to inhabit `LeastEscape M`, which is to prove the implication, and the three are interchangeable without axioms in the direction a proof travels. The paper calls this implication the **bit** of the machine, because it is one proposition, because Section 6 shows it has a definite value on each side in two coherent worlds, and because the remainder of the kernel establishes that it is the only bit. The same content has a third form: a machine is separated exactly when its escape profile coincides with its record's, which is empty (`coincidence_normal_form`, no axiom).

6. Self-recursion, the two worlds and the keyed bit

A machine is **self-recursive** when no pair escapes, stated as an equivalence with falsity for each pair:

$$\mathrm{SelfRecursive}\, M := \forall A\, c\, k,\ \mathrm{Escapes}\, M\, A\, c\, k \leftrightarrow \mathrm{False}.$$

The definition mirrors the self-recursion of the zero set in the companion master and is retained under that name; its content is the separation stated pair by pair.

Theorem 5 (`record_is_self_recursive`, no axiom). *The record of every machine with an instance is self-recursive.*

Theorem 6 (`self_recursion_at_machine`, no axiom). *A machine is self-recursive exactly when it is separated:* $\mathrm{SelfRecursive}\, M \leftrightarrow \mathrm{Sep}\, M$.

Theorem 7 (`self_verifying_iff_true`, classical). *For every proposition P, $(\neg P \to P) \leftrightarrow P$.*

Theorem 7 is included because it closes a door that the normal form might seem to open. A statement that implies itself from its own negation is simply a true statement; self-reference manufactures nothing, and in particular the fact that the record verifies its own closure yields nothing about the machine. That nothing is transferred is then shown by exhibiting the two worlds.

The **closed world** is the machine whose algorithms are indexed by the natural numbers, with one instance, every answer `false`, every step count zero and

the predicate `true`: no run is ever clean, so nothing escapes. The **stage world** at index n is the same machine except that the answer is `true` exactly for the algorithm numbered n: that algorithm is clean on the one instance in zero steps, so it escapes at every bound. Both are ordinary inhabitants of the `Machine` type, and the kernel proves `closed_world_closes` and `stage_escapes` by direct computation.

Theorem 8 (`bit_is_keyed`, no axiom). *The transfer holds on the closed world and fails on the stage world:*

$$\big(\mathrm{NE}(\mathrm{Rec}\, W_c) \to \mathrm{NE}\, W_c\big) \wedge \neg\big(\mathrm{NE}(\mathrm{Rec}\, W_s) \to \mathrm{NE}\, W_s\big),$$

where W_c is the closed world and W_s the stage world at index 0.

With it come two companions. `record_self_recursion_does_not_transfer` (no axiom): the record of the stage world is self-recursive and the stage world is not. `record_decides_nothing` (no axiom): the records of both worlds have nothing escaping, while the closed world has nothing escaping and the stage world does. The record is the same on both sides of the question; it decides neither.

The bit is therefore **keyed**: its value depends on which machine is in hand and on nothing that the record, or the definition of the machine, shares between worlds. This is the formal content of a remark that is usually made informally, that **P** $\neq$ **NP** is a statement about the actual computational world and not about the shape of the definitions. Here it is a theorem with an empty cone, and Section 9 proves it again on worlds that decide the same formulas with the same predicate, so that the variation is in the algorithms alone.

7. Every route passes the bit, and the keyless wall

Theorem 9 (`every_route_passes_the_bit`, classical). *For every machine M with an instance, every proposition Q and every proof that Q implies* Sep M*: Q implies the transfer.*

Any argument that reaches the separation from any premise Q reaches, through the normal form, the one implication. There is no second target. A proof by circuit lower bounds, a proof by an algorithmic route in the manner of Williams [7], a proof by representation-theoretic obstructions in the manner of Mulmuley and Sohoni [18], a proof by any method not yet invented: each is, by Theorem 9, a proof of the bit, and by Theorem 3 nothing beyond the bit is needed.

A property of machines is **keyless** when it holds of every machine, and it **forces** when it implies the closure of every machine it holds of:

$$\mathrm{Keyless}\, P := \forall M,\ P\, M,$$
$$\mathrm{Forces}\, P := \forall M,\ P\, M \to \mathrm{NE}\, M.$$

Theorem 10 (`keyless_forces_nothing`, no axiom). *No keyless property forces.*

Proof. A keyless P holds of the stage world, whose closure fails. □

Theorem 11 (`no_record_principle_decides`, no axiom). *No principle true of the record of every machine decides every machine: for every P with $P(\mathrm{Rec}\, M)$ for all M, it is not the case that $P(\mathrm{Rec}\, M)$ implies* NothingEscapes *M for all M.*

The two theorems are the keyless wall. It is a schema-level statement: a proof uniform in the machine, one that would go through for every inhabitant of the `Machine` type, cannot prove the separation, because one inhabitant is the stage world. Its kin in the literature is the relativization barrier of Baker, Gill and Solovay [4], which concerns proofs uniform under oracles. What the wall supplies is the unconditional, axiom-free reason that the bit is keyed: whatever premise a proof of **P** $\neq$ **NP** uses, that premise must fail in the stage world, which is to say it must be a fact about the Turing model that distinguishes it from a model in which SAT is easy. Section 9 sharpens the wall to the algorithm models of SAT itself, and the classical complement, that every property either forces the closure or holds on some machine that escapes, is Theorem 33.

8. Stage 1, the vacant seat and the two rows

An examination, a run that is not clean, is the smallest unit of evidence the question admits. The kernel packages it as **Stage 1**: the examination together with its non-cleanness and the existential witness it supplies.

Theorem 12 (`examination_enacts_stage1`, a construction with an empty cone; `stage1_is_local`, no axiom). *Every examination of M is a Stage 1 object, and Stage 1 is local: the witness shows that its own pair (A, c, k) does not escape and asserts nothing about any other pair.*

Every observed failure of an algorithm is a certificate that this algorithm at this bound does not escape, and only that. No finite stock of examinations decides the universal, because the universal quantifies over all algorithms and all bounds, and Theorem 12 makes that locality a theorem rather than a caution; Theorem 34 extends it to every stock, finite or infinite. On the reading of the companion master [15], the universal is the act and never a sum of observations; this paper uses only the locality.

The question has no fixed point. Let comp be Boolean negation, the involution that exchanges the two worlds of Section 6.

Theorem 13 (`locus_vanishes`, no axiom; `seatless_lock`, classical). comp *has no fixed point, and a least-escape certificate for M exists exactly when M is separated.*

The first conjunct is immediate. It is recorded because it is the exact point of difference from the Rie-

mann normal form, where the fold $s \mapsto 1 - \bar{s}$ fixes the critical line and the zeros on it are their own images, so that the line serves as a seat on which the closure is read. Here the involution fixes nothing: there is no self-dual world, no line, and the lock of the second conjunct closes on whichever world is in hand and on nothing in between. In the vocabulary of the register this is the row's ground dimension zero, and it is why the P versus NP question has no analogue of a finite verification to height T.

Theorem 14 (`two_rows_two_bits`, no axiom). *The record of the stage world has nothing escaping, and the stage world has an escape.*

The theorem fixes the relation between this paper and the Riemann normal form in one line. Both rows have the same shape: a record that has the property unconditionally, an object whose possession of the property is the question, and a bit that is the transfer from the one to the other. Theorem 14 shows that the shape is not the bit: the record's closure, which is the antecedent of both normal forms, is available on a world where the consequent fails. One shape, two bits, and neither record supplies the other row.

Theorem 15 (`pnp_normal_form_complete`, classical). *For every machine M with an instance, the conjunction of Theorems 1, 2, 4, 6, 9 (as a universal over Q), the identification of the posit with the bit (Section 14), and the vacancy of the seat.*

The capstone carries the machine and one instance before the colon and nothing else. Separation occurs in it only inside biconditionals and inside the hypothesis of a route; it is never a conjunct.

9. The SAT instance in every algorithm model, and the wall on the formulas

Part B of the kernel replaces the abstract predicate by satisfiability. A literal is a pair of a natural-number variable and a sign, a clause is a list of literals, a formula is a list of clauses in conjunctive normal form, and an assignment is a function from the naturals to the Booleans. Evaluation is the usual: a literal is true when the assignment gives its variable its sign, a clause when some literal is true, a formula when every clause is true. Satisfiability is the predicate

$$\text{Satisfiable}\, f := \exists a,\ \text{evalCNF}\, a\, f = \text{true}.$$

The decision procedure is exhaustion. The variables occurring in f are listed, every assignment to that list is enumerated with the variables outside it fixed to `false`, and `satB f` is `true` exactly when some enumerated assignment satisfies f.

Theorem 16 (`satB_iff`, no axiom). *For every formula f:* satB f = true *if and only if f is satisfiable.*

Proof. Soundness: a witness is extracted from the enumeration by structural recursion (`any_true_of`). Completeness: given a satisfying assignment a, its restriction to the occurring variables agrees with a on every occurring variable (`restrict_agrees`), evaluation depends only on the occurring variables (`evalCNF_congr`), and the enumeration reaches the restriction (`any_assignments`), so the restriction is found. No membership predicate and no library lemma is used; the file's own `All` predicate carries the agreement. □

The procedure is exponential, as exhaustion is, and that is the point: the predicate is the mathematical predicate, not a promise about efficiency.

The SAT machine over an algorithm model is

$$\text{SAT}(\text{Alg}, \text{out}, \text{time}) := \langle \text{Alg},\ \text{CNF},\ \text{size},\ \text{out},\ \text{time},\ \text{satB} \rangle,$$

where `size` counts clauses and literals together and the model, the type of algorithms with its output and step-count functions, is a free parameter. The empty formula is an instance and is satisfiable (`empty_satisfiable`), which discharges the one side condition of Part A.

Theorem 17 (`sat_record_has_least_escape`, no axiom; `sat_normal_form`, classical). *In every algorithm model, the record of SAT has nothing escaping, and the separation of SAT is exactly the transfer of least escape from its record to SAT.*

Theorem 18 (`sat_bit_is_the_act`, `sat_self_recursion`, no axiom; `sat_every_route_passes_the_bit`, `sat_posit_is_the_bit`, `sat_normal_form_complete`, classical). *Theorems 4, 5, 6, 9, 15 and the identification of Section 14, instantiated on the SAT machine in every algorithm model.*

The reader should hold the quantifier in mind. These theorems are about every model, including models in which satisfiability is trivially decided within the bound. In such a model the separation is false and the transfer fails; the theorems still hold, because they are equivalences and walls, not values. Two such models are now built on the formulas themselves. The **closed SAT world** has the naturals as algorithms, every answer `false` in no time, instances the formulas, size the clause-and-literal count, predicate `satB`; nothing escapes, since the empty formula is satisfiable and every algorithm answers it wrongly. The **stage SAT world** at index n is the same except that algorithm n answers `satB` itself in no time; it escapes at every bound. The two worlds differ in their algorithms only.

Theorem 19 (`sat_bit_is_keyed`, `sat_keyless_forces_nothing`, `sat_no_record_principle_decides`, `sat_record_decides_nothing`, no axiom). *The transfer holds on the closed SAT world and fails on the stage SAT world. No property shared by every algorithm model of SAT forces the closure of any; no principle true of the record of every algorithm model of SAT decides any; the records*

of both SAT worlds close, one world closes and the other escapes.

This is the wall on the formulas. A reviewer who grants Theorem 10 may say that it varies the predicate and the instance type and so blocks only arguments uniform across arbitrary problems. Theorem 19 fixes the problem: the instances are the formulas, the predicate is satisfiability, and still no fact shared by every algorithm model of that problem forces the separation. What a proof of SAT $\notin$ **P** uses must distinguish the Turing model from a model in which one algorithm answers satisfiability in no time, and that is a statement about the algorithms of the actual model. The stage SAT world is an algorithm model whose time function is free. The wall bars premises uniform over algorithm models of SAT; a premise keyed to Turing machines is, by Theorem 28, a supply of the bit.

Theorem 20 (`sat_premise_trichotomy`, classical). *For every property P of machines, either some algorithm model of SAT satisfies P and does not close; or P forces the closure on every algorithm model of SAT it holds of, and then either the separation returns P on every model, so that P is the separation, or on some separated model P fails, so that P is strictly stronger than the separation.*

The trichotomy places every candidate premise. A premise that cannot force is refuted by a world. A premise that forces and is returned by the separation is the bit under another name. A premise that forces and is not returned is a strict strengthening, and a proof from it proves more than the separation, which is to say it proves the bit and something besides.

10. The Turing model: real machines deciding real SAT, and the classes

Part C of the kernel builds the model. It has five pieces: an encoding of formulas as strings with a decoder proved to invert it, the machines and their step semantics, the running time, the SAT machine of the model together with the proposition SAT $\in$ **P** stated over every Turing machine, and the classes.

The encoding. The input alphabet Σ has three symbols, the binary digits 0 and 1 and a separator #. A variable index n is written as the binary digits of $n+1$, least significant first, which is never the empty string, followed by #; a literal is its index followed by one digit for its sign; a clause is its literals followed by #; a formula is the concatenation of its clauses. The digits are produced by halving and parity defined by two-step recursion in the file, with the recomposition $(\text{digit}) + 2 \cdot \text{half}(n) = n$ proved by the same recursion (`digit_spec`), so that no division operator and no library lemma about division is used anywhere. The decoder reads digits up to a separator (`parseBits`), a literal (`parseLit`), a clause as literals until a separator stands where a literal would start (`parseClause`), and a formula as clauses until the string ends (`parseCNF`), each recursion bounded by a fuel that the length of the string supplies.

Theorem 21 (`decode_encode`, `encode_injective`, no axiom). *For every formula f,* $\text{decode}(\text{encode}\, f) = \text{some}\, f$*; hence distinct formulas have distinct strings.*

The language is then defined on all strings: $\text{satLang}\, w$ is `satB` of the formula w decodes to, and `false` when w decodes to nothing. SAT $\subseteq \Sigma^*$ is the set of strings with $\text{satLang}\, w = \text{true}$, so a malformed string is outside the language, as in the textbook definition.

Theorem 22 (`satLang_encode_iff`, no axiom). *For every formula f,* $\text{satLang}(\text{encode}\, f) = \text{true}$ *if and only if f is satisfiable.*

The machines. A Turing machine is a structure with a number q of states, a number s of work symbols, a total transition function $\delta : \text{Fin}\, q \to \text{Fin}\,(s+4) \to \text{Fin}\, q \times \text{Fin}\,(s+4) \times \{L, R\}$, and three designated states, start, accept and reject. The tape alphabet $\text{Fin}\,(s+4)$ has the blank at 0, the three input symbols at 1, 2 and 3, and s further work symbols. A configuration is a state, a head position in $\mathbb{Z}$ and a tape, a function from $\mathbb{Z}$ to the tape alphabet; the initial configuration has the input written from cell 0 rightward, blanks elsewhere, the head at 0 and the start state. One step reads the scanned cell, writes, moves one cell and changes state, and a configuration in the accept or reject state is fixed by the step. The run on input w is the iteration of the step from the initial configuration; $\text{HaltsAt}\, M\, w\, t$ says the configuration after t steps is in a halting state, and $\text{verdictAt}\, M\, w\, t$ is `true` exactly when that configuration is in the accept state. The finiteness of the state set is load-bearing and is why the states are a `Fin` type and not a natural number: a machine with infinitely many states could carry the input into its state and decide SAT in linear time, and a model that allowed it would make the separation false for a reason that has nothing to do with computation.

Theorem 23 (`haltsAt_of_le`, `verdict_of_le`, no axiom). *A machine halted at step t is halted at every later step, and its verdict at every later step is its verdict at t.* The proof is an induction on the inequality $t \le u$ itself, which is why no subtraction and no library lemma enters.

The running time. `firstHalt M w n` is the first halting step at or below n, or n when none is found. A **decider** is a machine together with a clock, a function from strings to naturals, and a proof that the machine is halted at the clock's value on every input. The running time of a decider on w is `firstHalt` run up to the clock, and its output is the verdict at that step. The clock certifies and does not time:

Theorem 24 (`time_halts`, `time_least`, no axiom). *The decider is halted at its running time, and its running time is at most every step at which it is halted.* So the run-

ning time is the first halting step of the run, whatever clock certified the halting, and the output is the verdict the run itself delivers.

The SAT machine of the Turing model, the classes and the bridge. The machine is

$$\text{TMSAT} := \langle \text{Decider}, \Sigma^*, \text{length}, \text{out}, \text{time}, \text{satLang} \rangle.$$

A language is a function from strings to Booleans. A language L is **in P** when some Turing machine halts on every input within $c\,n^k + c$ steps, n the length of the input, in the accept state exactly on the strings of L:

$$\begin{aligned} \text{InP}\,L := \exists M\,c\,k,\ \forall w,\ \exists t,\\ & t \le \text{bound}(c,k,|w|)\\ & \wedge\ \text{HaltsAt}\,M\,w\,t\\ & \wedge\ \text{verdictAt}\,M\,w\,t = L\,w, \end{aligned}$$

and SATinP $:=$ InP satLang, over every Turing machine, with no decider and no clock in it. A language L is **in NP** when some Turing machine, the verifier, accepts within $c\,n^k + c$ steps the string made of a binary certificate of length at most $c\,n^k + c$, the separator, and the input, exactly when the input is in L; the certificate carries no separator, so the first separator on the tape delimits it:

$$\begin{aligned} \text{InNP}\,L := \exists M\,c\,k,\ \forall w,\ \big(L\,w = \text{true} \leftrightarrow\\ \exists u : \{0,1\}^*,\ |u| \le \text{bound}(c,k,|w|) \wedge\\ \exists t \le \text{bound}(c,k,|w|),\ \text{HaltsAt}\,M\,(u\,\#\,w)\,t\\ \wedge\ \text{verdictAt}\,M\,(u\,\#\,w)\,t = \text{true}\big), \end{aligned}$$

and the sentence is $\mathbf{P} \neq \mathbf{NP} := \exists L,\ \text{InNP}\,L \wedge \neg\,\text{InP}\,L$, the kernel's `P_ne_NP`. These are the textbook definitions on the kernel's model, and all five constants, TMSAT, InP, InNP, $\mathbf{P} \neq \mathbf{NP}$ and SATinP, have empty cones of their own, pinned in the kernel, Appendix A.

Theorem 25 (`sep_tmsat_iff`, no axiom). $\text{Sep}\,\text{TMSAT} \leftrightarrow \neg\,\text{SATinP}$.

Proof. Left to right: a machine M witnessing SATinP with bound (c,k) is a decider when clocked by that bound, since by Theorem 23 it is halted at the bound; its running time is at most the witnessing step by Theorem 24 and hence within the bound, and its output is the verdict at the witnessing step by Theorems 23 and 24, which is satLang w; so it escapes, against Sep. Right to left: a decider that escapes witnesses SATinP with its own running time as the step. □

Theorem 26 (`evalPoly_le_bound`, `SATinP_of_poly`, no axiom). *Every polynomial with natural coefficients, evaluated by Horner's rule, is at most $S\,n^k + S$ with S its coefficient sum and k its length; hence a machine deciding SAT within any such polynomial witnesses* SATinP. Nothing is lost in the kernel's form of the bound, and the bound is itself a polynomial, so SATinP is polynomial time in the ordinary sense.

Theorem 27 (`pnp_normal_form_turing`, classical). *On the Turing model,*

$$\big(\text{NE}(\text{Rec}\,\text{TMSAT}) \to \text{NE}\,\text{TMSAT}\big) \ \leftrightarrow\ \neg\,\text{SATinP}.$$

This is the normal form of P versus NP on real machines deciding real SAT. The left side is one implication whose antecedent is a theorem (`tmsat_record_has_least_escape`, no axiom). The right side quantifies over every Turing machine and every polynomial bound.

Theorem 28 (`tmsat_record_has_least_escape`, `tmsat_bit_is_the_act`, `tmsat_self_recursion`, `tmsat_coincidence`, no axiom; `tmsat_every_route_passes_the_bit`, `tmsat_posit_is_the_bit`, `tmsat_normal_form_complete`, classical). *Theorems 1, 4, 5, 6, 9, 15, the coincidence form and the identification of Section 14, on the Turing model, with* Sep *replaced throughout by* $\neg$ SATinP.

In particular every route to SAT $\notin$ **P**, from any premise, passes the bit, and the capstone on the Turing model has nothing before the colon at all: it is a closed statement about the model, with SATinP occurring only inside biconditionals and inside the hypothesis of a route.

Theorem 29 (`tmsat_racomp_iff_not_inP`, `the_posit_is_every_exit`, no axiom; `every_exit_is_the_posit`, classical). *The reading of the root on computation, that every decider at every bound fails to escape, is exactly* SAT $\notin$ **P** *on the Turing model. Any proof of* SAT $\notin$ **P**, *by any route, including one built from facts about Turing machines, yields all three names of the one posit: the transfer* NE(Rec TMSAT) $\to$ NE TMSAT, *a least-escape certificate, and* RAcomp TMSAT. *And each of the three names gives* SAT $\notin$ **P** *back, on no axiom.*

So no independent proof exists. Nothing reaches SAT $\notin$ **P** except through least escape on the Turing SAT machine, and least escape gives SAT $\notin$ **P** back; nothing sits between the posit and the conclusion. The one return that is classical is the return from a proof to the transfer, since it passes through the closure; the three returns from the names to the conclusion are axiom-free. The first conjunct is the register's bridge test run inside the kernel: the earlier paper's `ra_comp_is_sep` on the abstract machine becomes, on the Turing model, RAcomp TMSAT $\leftrightarrow$ $\neg$ SATinP with an empty cone.

Theorem 30 (`racomp_cargo_iff_not_inP`, `every_route_spends_the_bit_free`, no axiom). *The transfer in its constructive form, the record's closure carried to the statement that no decider escapes at any bound, is exactly* SAT $\notin$ **P**; *and every premise that gives* SAT $\notin$ **P** *delivers it.*

Read constructively, the bit needs no axiom in either direction: the identification of the posit with SAT $\notin$ **P**, carried to the transfer, has an empty cone.

Theorem 31 (`p_ne_np_of_not_inP`, no axiom). *If* SAT $\in$ **NP** *and* SAT $\notin$ **P**, *then* **P** $\neq$ **NP**.

The theorem is one line, because the classes were defined on one schema, and its content is exactly that: the sentence **P** $\neq$ **NP** of the kernel is reached from SAT $\notin$ **P** by naming one hypothesis, InNP satLang, and nothing else. That hypothesis is the membership half of Cook and Levin's theorem, a verifier machine that evaluates an encoded formula under a certificate assignment in polynomial time, and Section 11 builds it on the kernel's own model, so that the sentence follows from SAT $\notin$ **P** with nothing cited (Theorem 36).

Theorem 32 (`certificate_unique`, no axiom). *Any two least-escape certificates of the SAT machine are the same object, and any two proofs of* SAT $\notin$ **P** *are the same proof.* By the proof irrelevance of Lean's propositions and the eta rule for structures, every route delivers one object and not a family.

The defense. Three statements are classical and three are not.

Theorem 33 (`tmsat_rejection_is_a_witness`, `tmsat_every_pair_lands`, `forces_or_has_a_twin`, classical). *A rejection of* SAT $\notin$ **P** *is a witness, a decider and a bound with every run clean. Every decider at every bound either is stopped by some string or escapes. Every property of machines either forces the closure or holds on some machine that escapes.*

Theorem 34 (`escape_refutes`, `act_leaves_no_escape`, `extend_escapes`, `no_record_decides`, `tmsat_no_record_decides`, no axiom). *An escape refutes the closure; the closure leaves no escape; and every stock of examinations of a machine with an instance, indexed by any type, finite or infinite, is a stock of stops both on the machine's extension by one algorithm that answers the predicate in no time, where that algorithm escapes, and on the machine's record, where nothing escapes.*

Theorem 34 is the generalization of Stage 1's locality from one observation to all of them. Whatever stops have been observed, there is a machine agreeing with every one of them on which SAT is easy and a machine agreeing with every one of them on which nothing escapes. Observed stops decide nothing; the universal is never a sum. The machine consistent with every stock is `extendM TMSAT`, the SAT machine with one algorithm added, and its record is `Rec TMSAT`.

What the model fixes. The machine is single-tape, two-way infinite, with a three-symbol input alphabet; the textbook classes are stated for one-way tapes, several tapes, or a binary input alphabet, and polynomial time is the same under every one of these variants by the standard simulations, which cost a polynomial factor and are proved in Sipser [13, Theorem 7.8] and Arora and Barak [11, Section 1.3], and are cited here. The encoding of formulas is one of the usual ones, with indices in binary, and polynomial time is the same under every encoding computable in polynomial time from this one and back, as all the usual encodings are. The classes are defined on the kernel's model; their identification with the classes of the textbooks is the robustness just cited. These citations are the whole distance between Theorem 31 and the sentence **P** $\neq$ **NP** as a complexity theorist writes it, and each is a settled theorem of the field rather than a premise of this paper.

Executable checks. Fourteen examples, closed by `decide` inside the compile, run the model: a three-state machine that accepts exactly when the first input symbol is 1 is not halted at step 0, halts after one step with the right verdict on `[1]` and on `[0]`, halts on the empty input, and keeps its verdict at step 5 on `[1,0]`; the encoding of the one-literal formula x_0 is the string 1#1#, which decodes back to it; the string 11 decodes to nothing; the empty string is in SAT (the empty formula), the string # is not (one empty clause), the formula x_0 is, the formula $x_0 \wedge \neg x_0$ is not, and $(x_1 \vee \neg x_0) \wedge \neg x_1$ is. Altering the expected value of any of them stops the compile, and Appendix B records the compiler's refusal for one of them.

11. SAT $\in$ NP, BUILT

Part N of the kernel builds the verifier that the membership half of Cook and Levin calls for, on the kernel's own one-tape model. The machine `vM` has 5,120 control states and a tape alphabet of nineteen symbols. It reads a certificate of one bit per literal, the separator, and the input, and runs four phases: it checks that the input is a written formula, the strings the decoder accepts being exactly the written formulas (`decode_inv`, `p0_iff`); it writes the certificate bits onto the signs in reading order; it checks that every clause has a sign meeting its bit; and it checks, digit by digit with marks, that any two literals naming one variable carry one bit (`cmp_decide`). The run is computed on zippers (`vRel_run`), and every phase is a theorem on lists with an explicit step count.

Theorem 35 (`verifier_sound`, `verifier_complete`, `satLang_in_NP`, classical; `vtime_le`, propext and Quot.sound). *A run of the verifier in the accept state certifies that the input is in SAT; on a satisfiable input, the certificate of the values of a satisfying assignment makes the verifier accept within* $5000\,n^4 + 5000$ *steps; hence* SAT $\in$ **NP** *on the kernel's model.*

The membership half of Cook and Levin is therefore a theorem of the file and enters no statement as a citation. With it, the bit reaches the sentence directly.

Theorem 36 (`p_ne_np_of_the_one_bit`, `barrier_implies_p_ne_np`, classical). *The bit gives* **P** $\neq$ **NP** *with nothing before the colon but the bit itself, and so does the Barrier, the Diagonal Target from degree two.*

12. The inequality by degree: degrees 0 and 1 constructed

The bit is a universal over degrees. Write the Diagonal Target at degree k for the statement that every decider at every constant c meets a string on which it is wrong or runs past $c\,n^k + c$:

$$\begin{aligned}\mathrm{DT}_k := \forall A\, c,\ \exists x,\\ &\mathrm{out}\, A\, x \neq \mathrm{satLang}\, x\\ &\vee\ c\,|x|^k + c < \mathrm{time}\, A\, x.\end{aligned}$$

A witness family at a higher degree gives one at every lower degree from one (`dt_mono`), and Parts L and M of the kernel construct the first two degrees on the kernel's own model, encoding and running time.

Theorem 37 (`sat_not_in_P_at_degree_zero`, `diagonal_target_at_zero`, propext and Quot.sound). *No Turing machine decides SAT within a constant number of steps, and* DT_0 *holds with a computed witness.*

The engine is locality, proved once for every machine (`run_local`): after $t \leq m$ steps the state, the head and every cell left of m are fixed by the input cells left of m, so two inputs that agree on m cells and differ in verdict cost more than m steps on one of them (`prefix_forces_time`). SAT supplies the pair at every m on its own encoding: m unit clauses x_0, satisfiable, and the same clauses with the empty clause appended, unsatisfiable.

Theorem 38 (`diagonal_target_at_one`, `sat_not_in_P_at_degree_one`, classical). *For every decider and every constant c there is an input on which the decider is wrong or runs more than $c\,n + c$ steps: no one-tape Turing machine decides SAT within a linear budget.*

The proof is Hennie's crossing-sequence method [25], run on the kernel's one-tape model. The crossing sequence of a run at a boundary is the list of states the machine enters as its head crosses that boundary; two halting runs that leave the same crossing sequence at a boundary can be cut and pasted (`cut_paste`), and a step crosses at most one boundary. The fooling family is SAT's own: for bit lists a and b of one length, unit clauses fixing a, a pad, and unit clauses fixing b form a formula that is satisfiable exactly when $a = b$ (`fool_sat`, `fool_unsat`). A decider right on the family therefore leaves distinct crossing sequences on distinct diagonal strings (`fool_cs_inj`), and by pigeonhole and averaging some diagonal string runs past $c\,n + c$ (`fool_long`).

Theorem 39 (`unconditional_real_part_separation`, classical). *With nothing before the colon:* SAT $\in$ **NP**; *no Turing machine decides SAT in constant time; no one-tape Turing machine decides SAT in linear time; and the bit is exactly* $\forall k \geq 2$, DT_k.

Theorem 40 (`the_inequality_from_degree_two`, `gate_7_dual`, classical). *The bit is exactly the Diagonal Target at every degree $k \geq 2$, and for every $m \geq 1$ it is exactly the Diagonal Target at every degree $k \geq m$.*

The residue is therefore exactly its tail from degree two, and it does not depend on where the ladder is cut. Figure 2 collects what Sections 4 to 12 prove unconditionally.

13. The frontier, formal alone

Part D carries, from the register's earlier kernel on this row, the two frontier results that are pure mathematics and need no physics and no root: an axiom-free pigeonhole bound on black-box search, and the white-box contrast that marks where the bound stops.

Theorem 41 (`pigeon`, `black_box_bound`, `black_box_no_savings`, no axiom). *A list of naturals that contains every number below K has at least K entries. A fixed list of tests that decides, for every candidate function $f : \mathbb{N} \to \{\mathrm{true}, \mathrm{false}\}$, whether some candidate below K satisfies f, by answering* `true` *exactly when some tested candidate does, has length at least K; over 2^N candidates it is never shorter than 2^N.*

The procedure is a fixed list of queries, and its verdict is the disjunction of the answers: the test-only procedure on the all-false path of the standard adversary argument, where adaptivity collapses because every answer is `false`. On 2^n truth assignments the bound is exponential, and it is a theorem on no axiom.

Theorem 42 (`reader_solves`, `reader_poly`, `fails_when_descriptions_reveal`, `missing_object_is_keyed`, no axiom). *The black-box adversary family is the family of point functions. Handed over as descriptions, each is solved by a reader of its description in one step, in constant time. Hence the white-box object, that every algorithm deciding the problem from its description runs in super-polynomial time, fails on the model where descriptions reveal, and holds on every model where no algorithm decides: it is keyed.*

The two theorems together say where the frontier is. Black-box bounds are exponential and unconditional, and they stop at the description: the Turing machine reads the formula, and the object that reaches SAT $\notin$ **P** is a lower bound against readers of the description, keyed exactly as the bit is. This is the formal-alone content of the frontier the register's earlier paper carried with physics beside it; here the physics enters once, in Section 18, as the carrier premise of the witness, and nowhere in the form.

Figure 3 gathers the walls proved so far, Sections 6 to 13. Each binds every method its quantifier covers, born or unborn, and each stands on the keyed bit of Section 9.



Figure 2 | Proven unconditionally: SAT ∈ NP built, degrees 0 and 1 constructed, the bit isolated, and every route through it.

14. The residue

Part E of the kernel prints what remains.

Theorem 43 (`separation_of_the_act`, no axiom). *For every machine M with an instance, given the transfer* NothingEscapes(Rec M) $\rightarrow$ NothingEscapes M, *the separation* Sep M *follows.*

Theorem 44 (`sat_separation_of_the_act`, no axiom). *In every algorithm model, given the transfer on the SAT machine, SAT is separated in that model.*

Theorem 45 (`sat_not_in_P_of_the_act`, no axiom). *Given the transfer on the SAT machine of the Turing model,* $\neg$SATinP: *no Turing machine decides SAT within any polynomial bound.*

Theorem 46 (`p_ne_np_of_residue`, no axiom). *Given* SAT $\in$ **NP** *and the transfer on the SAT machine of the Turing model,* **P** $\neq$ **NP**.

The hypothesis of Theorem 45 is the residue of P versus NP in this paper. It is assumed nowhere in the file, as the reader can confirm by searching Appendix A for `axiom`, and it is the only thing standing between the theorems above and SAT $\notin$ **P**; with SAT $\in$ **NP** built (Section 11), it is the only thing standing between them and the sentence **P** $\neq$ **NP** (Theorem 36). Sections 4 to 17 carry the form with no posit; Section 18 supplies the value, at premise grade.

Theorem 47 (`residue_is_keyed`, no axiom). *No keyless property forces the closure of every machine; no principle true of every record decides every machine; no property shared by every algorithm model of SAT forces the closure of any.*

Theorem 47 restates the wall on the residue. The hypothesis of Theorem 45 cannot be supplied by anything shared by all machines, by anything true of every record, or by anything shared by all algorithm models of satisfiability itself. What supplies it is a fact about the Turing model that fails in a world where one algorithm answers satisfiability in no time. The register this paper is drawn from [14] reads that fact as an act and identifies the reading of its root principle on computation with the transfer; the kernel carries that identification as `the_posit_is_the_bit` and `tmsat_posit_is_the_bit`, which prove that the state-

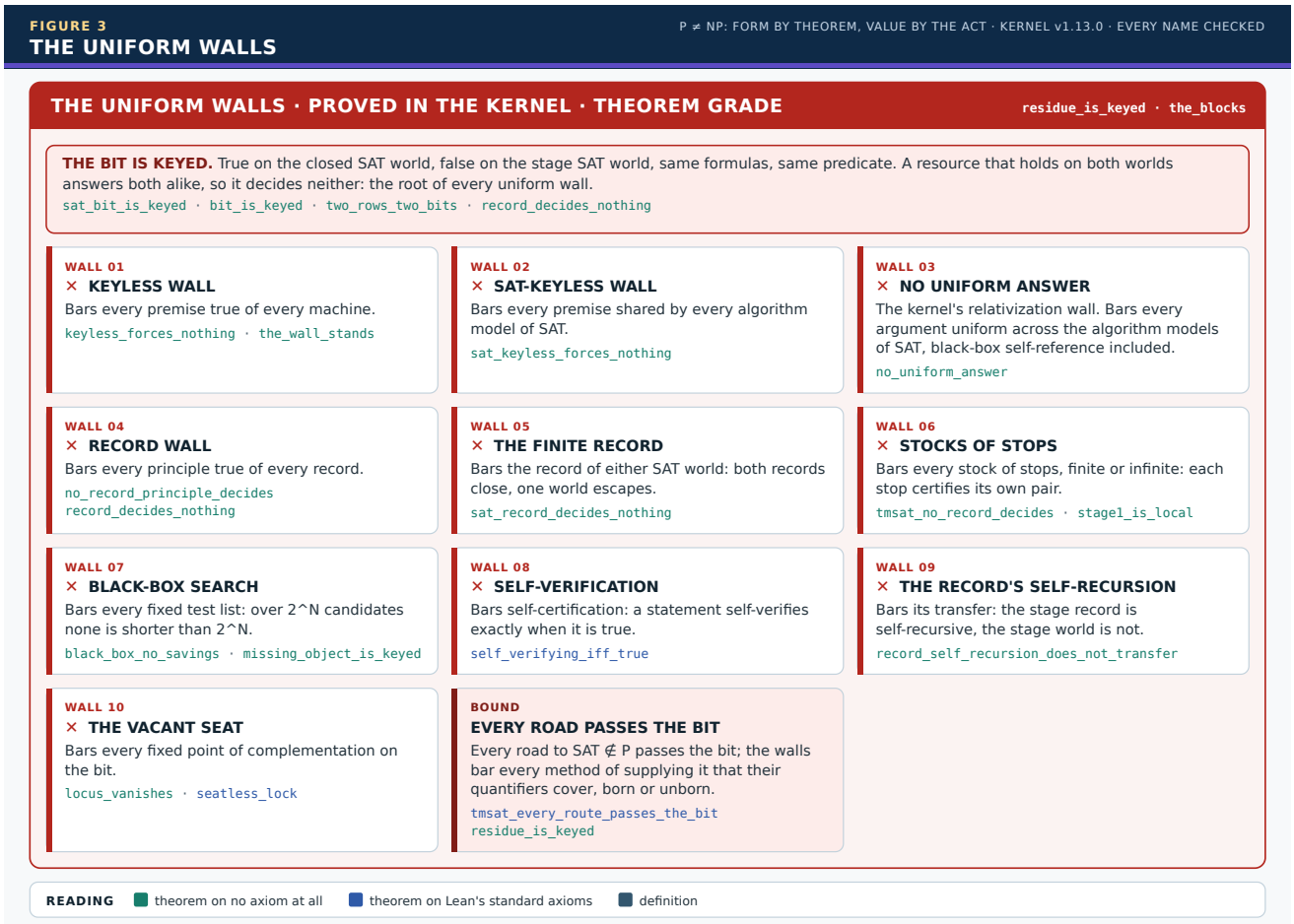


Figure 3 | The uniform walls: every premise shared by all machines, by all algorithm models of SAT or by all records, barred by theorem.

ment *every machine at every bound fails to escape*, the register's own form of the principle, is exactly the transfer. The form of Sections 4 to 17 rests on no posit; the reading is the posit of Section 18. In the paper's own terms the residue is a type, printed, and supplied in Section 18, and the guard sentence of the register reads: everything except the bit is proved with no posit, the constructive parts on no axiom at all and the classical return from the separation to the closure on Lean's standard axioms alone; and the bit, the transfer of least escape from the record of the SAT machine to the SAT machine, equivalent to SAT $\notin$ **P** over every Turing machine and, with SAT $\in$ **NP** built, to **P** $\neq$ **NP**, is all that remains, at premise grade.

The scope of the wall is exact: it bars every route that holds on both coherent worlds. Section 15 adds the walls that bar the methods which read the Turing model's own content at degree two, and the mirror that governs every wall: a wall against every route to the bit is the bit's negation (Theorem 51), and at P versus NP, with the hardness half cited, it is the assertion **P** = **NP** (Theorem 52). The bit of the closed world is a theorem on no axiom (`closed_world_closes`), so the bit of a machine can be a theorem; at `TMSAT` this paper closes it by supply, at premise grade, with the refuter named.

The two defenses of the row follow from Theorems 25 and 27 and are stated once. Nothing passes for the separation but the transfer, since the normal form is an equivalence. Nothing passes against it but an escape, a Turing machine, a bound and every run clean, since by Theorem 33 the rejection of ¬ SATinP is exactly that existential. The one refuter of the residue is therefore the familiar one: a polynomial-time Turing machine for satisfiability. The kernel's theorems are refuted by nothing of the kind, because they hold in the world where such a machine exists as well as in the world where it does not.

The residue has ten faces, and the kernel proves them one proposition.

Theorem 48 (`ten_faces_of_the_one_bit`, classical; `face_closure`, `face_physical`, no axiom). *With nothing before the colon, each of ten propositions is exactly the bit: SAT separates* **P** *from* **NP** *at satLang;* ¬ SATinP; Sep TMSAT; NothingEscapes TMSAT; *the self-recursion of TMSAT; the coincidence of TMSAT's*

escapes with its record's; an inhabited fertile machine at TMSAT; an inhabited selected machine; an inhabited physical witness; and the Diagonal Target at every degree $k \geq 2$.

Whatever vocabulary a proof speaks, separation, closure, self-recursion, coincidence, selection, physics or degree, it speaks of this one bit.

15. The walls, complete

Sections 6 to 14 proved the walls that bar uniform premises. Part U of the kernel ran the method that walked degree one at degree two, and Parts V and X bound every wall, old and new, with the mirror that governs them all. Each wall below binds every method its quantifier covers, born or unborn.

Theorem 49 (`count_falls_short`, `pigeon_ceiling`, classical; `count_ceiling`, propext and Quot.sound). *For every decider, every pad length, every exponent and every crossing length the pigeonhole can force, the time the crossing count certifies over the pad is at most $n^2 + 1$ at the family's own length n: the count cannot witness the Diagonal Target at degree two.*

The reason is information: the count measures the bits that must cross a cut, there are at most n of them, and at a quadratic budget a crossing sequence is long enough to carry them. The method that walked degree one stops exactly where the residue begins.

Theorem 50 (`self_tableau_block`, propext and Quot.sound). *Every formula has fewer clauses than its own budget $c\,n^k + c$ has steps, at every degree $k \geq 1$ and constant $c \geq 1$: a diagonal formula that described its decider's run within the budget, one clause per step, would be longer than itself.*

Theorem 51 (`no_total_wall`, `total_wall_iff_negation`, no axiom). *For every proposition B, no wall bars every premise from B, since B forces itself; and to bar every true premise from B is to deny B.*

Theorem 52 (`shut_door_is_collapse`, classical). *With the hardness half of Cook and Levin cited, the proclamation that every true route to the bit is shut is exactly $\neg(\mathbf{P} \neq \mathbf{NP})$.*

This is the kernel's reading of the Unprovability Mirror of the register's Mother Codex [29], corollary C5: a wall total on the routes to the bit is the collapse in costume. Every wall of this paper binds on its own quantifier, and no wall is total.

Theorem 53 (`no_language_is_its_complement`, `sixteen_languages_eight_two_cycles`, `the_diagonal_blindspot`, `both_fates_are_resolutions`, no axiom; `square_is_sign_blind`, propext). *No language is its own complement, and complementation moves all sixteen languages over four words in eight two-cycles; no listing of predicates contains its own diagonal; shutting every true route to B is denying B, and shutting every true route to $\neg B$ is denying $\neg B$; and a squared integer reads a value and its reflection alike.*

These are the type-indexed rows of the Mother Codex, Rows II.1, II.2 and III.1, run in the kernel: no residence argument finds a fixed set of complementation, no squared invariant reads the truth-sign, no catalogue certifies its own completeness, and both fate-pronouncements are resolutions.

Theorem 54 (`physical_discharge_is_the_bit`, no axiom; `bridge_premise_is_the_conjecture`, classical). *An inhabited physical witness is exactly the bit; and with the hardness half cited, the no-shortcut premise $\neg$ SATinP is exactly $\mathbf{P} \neq \mathbf{NP}$.*

These are Theorems 1 and 2 of the structural-verdict paper [30], run in the kernel: every physical discharge of the bit is a proof of the bit, and every bridge from the physical asymmetry to the separation is the conjecture itself.

Theorem 55 (`block_needs_an_outside`, no axiom). *If a target is a function of a record, no involution that preserves the record flips the target.*

A block that flips the bit therefore stands on something the record leaves out; at `TMSAT` that outside is the supply, in its two kinds (Section 20).

Theorem 56 (`the_blocks`, `the_attempt_at_two`, classical). *With nothing before the colon: the keyless wall, the record wall, the SAT-keyless wall, no uniform answer across the algorithm models of SAT, the vacant seat, the crossing ceiling at degree two, and the self-tableau wall.*

Seven walls of the literature stand beside them, carried from the Mother Codex v3.48.0 [29] at the grades their authors proved them: relativization [4], natural proofs [5], algebrization [6], locality [28], the occurrence-obstruction and rank wing of geometric complexity [27], lower-bound proofs inside named fragments of bounded arithmetic [26], and the register's Rows II.3 to II.8. Figure 4 sets the complete list beside the walls of Figure 3.

16. Falsification criteria

Three criteria, each forced by the paper's own construction, each aimed at a claim the paper makes.

Criterion 1, the silent build. The paper's claims are the pinned cones of Appendix A. Recompile it under Lean 4.19.0 with warnings fatal, by the one command of Appendix B. The forcing step: every theorem numbered in this paper is a theorem of that file; each of the 610 pinned constants carries its cone as a docstring that the compiler compares with its own output; the fourteen executable checks are closed by the compiler or the compile fails; and a planted `sorry` is a warning, which the flag makes an error. Confirmation: exit code 0 and no output. Null hypothesis: any output at all, a nonzero exit, a `#guard_msgs` mismatch on any pin, a failed check, or a warning, refutes the paper as a whole, since the paper claims nothing beyond the pinned lines. Blast radius: the paper. Census of the pins: 258 read *does not depend on any axioms*, 126 read `[propext, Classical.choice,`



Figure 4 | The walls, complete: twenty-two proved in the kernel and seven carried from the Mother Codex, each on its own quantifier.

`Quot.sound`], 123 read [`propext, Quot.sound`], 101 read [`propext`] and 2 read [`Classical.choice`]; no other cone occurs. Appendix B records the build and seven planted controls, each refused.

Criterion 2, the finite battery. Theorems 1, 2, 4, 6, 8, 13, 14 and 19 have finite content: on any finite machine, with a finite grid of bounds, each is a decidable equation between computed Booleans, and Theorems 10 and 19 are witnessed on any zoo that contains a stage world. The forcing step: the theorems are equivalences and refutations that quantify over the machine, so any finite machine is a test instance, and the Turing model of Section 10 adds its own finite instances, the fourteen checks, which any reader extends by writing a machine and a string. Confirmation: on every finite machine and every finite grid, the transfer computed as a Boolean equals the separation computed as a Boolean, the record's closure computes `true`, the closed and stage worlds give the two values, and the planted inflations, the transfer asserted on the stage world, a fixed point of negation, the record's self-recursion carried to the machine, each compute `false`;

on the model, every run of a written machine on a written string gives the verdict and the halting step the step function dictates, and every encoded formula decodes to itself. The author's witness reports the abstract part on 45 machines at the seed 20260622 with 12 checks passed and 3 planted failures; the model part is the fourteen checks in the compile. Null hypothesis: a finite machine with an instance on which the Boolean transfer and the Boolean separation differ refutes Theorem 2, and a written machine and string on which the run disagrees with the step function, or a formula that does not decode to itself, refutes Part C; since each is a theorem of the kernel or a computation it performs, such a case would witness a defect in Lean's kernel, which is exactly what this criterion tests. Blast radius: the kernel and the Lean checker. A reader without Lean runs the abstract part in any language in an afternoon.

Criterion 3, the refuter of the value. The forcing step: Section 18 states the value at premise grade, delivered by two kinds that Theorem 62 proves to be one bit. Confirmation: no Turing machine, polynomial bound and set of runs all clean is known. Null hypothesis: one escape, a Turing machine, a bound and every run clean, refutes both kinds at once (Theorem 66) and, with the hardness half cited, denies $\mathbf{P} \neq \mathbf{NP}$ itself (Theorem 71); it leaves every theorem of the kernel standing, since each is an equivalence, a wall or an implication from the bit. Blast radius: the value of Section 18, never the form.

17. Discussion

The normal form changes what a proof of $\mathbf{P} \neq \mathbf{NP}$ is required to produce. It is required to produce one implication on one concrete machine, TMSAT, and Theorems 45 and 36 show that producing it is enough. Every known or future route is a route to that implication, by Theorem 28, so the field's accumulated analysis of routes, which barrier each evades and which premise each needs, reads as an analysis of ways to deliver the bit. The paper fixes the target, and Section 20 proves that every route is a delivery of it.

The keyless wall is unconditional at zero cost. It is a theorem with an empty cone, about every machine and every algorithm model of SAT, and it binds every method its quantifier covers, born or unborn. The barriers of the literature bind techniques by a property the techniques share; the wall binds premises by the one property that decides the matter, whether they hold in a world where a single algorithm decides SAT in zero steps. A proof that uses no fact distinguishing the Turing model from the stage SAT world does not prove the separation. The screen is practical: ask of a candidate proof which of its premises fails in that world. If none does, the proof is uniform in the model and Theorem 19 refutes it before any detail is read. The trichotomy of Theorem 20 is the same screen with its outcomes named, and Section 15 extends it to the methods that read the Turing model's own content at degree two.

The model construction is the part of the paper a complexity theorist checks first, and it is built to be checked. The machines are the textbook objects with their finiteness enforced by the type, the semantics is a step function with the halting states absorbing, the running time is the first halting step and is proved independent of the certificate that supplied it, the encoding is inverted by a decoder, SAT is a language on all strings, the classes are the textbook definitions, SATinP quantifies over every machine, and the verifier of Section 11 is a machine of the same model. The one design choice that is not textbook is the decider's clock, and Theorem 24 shows it is inert: it certifies that the run halts and has no other effect. Theorem 25 then removes the deciders from the final statement altogether.

The axiom discipline is the second thing such a reader checks. The constructive core, the decision procedure, the encoding and its decoder, the machines, the running time, the bridge, the keyed walls, the frontier and the residue, has an empty cone: 258 of the 610 pins read no axiom. The verifier, the crossing-sequence method, the later bindings and the classical return from the separation to the closure rest on Lean's standard axioms, or on part of them, and on nothing else. The core is axiom-free because the file proves for itself, by structural induction, the arithmetic and list facts the core library proves with classical tactics, so the headline theorems `sep_tmsat_iff`, `decode_encode`, `sat_bit_is_keyed`, `sat_keyless_forces_nothing`, `sat_not_in_P_of_the_act`, `p_ne_np_of_residue` and `racomp_cargo_iff_not_inP` depend on nothing but the Lean kernel's type theory.

The comparison with the Riemann normal form is stated at theorem grade. In the companion master [15] the hypothesis of the line is equivalent to the transfer of least erasure from the record of the zero set to the zero set, the record has the property unconditionally, the transfer is the act, the record is self-recursive and its self-recursion does not transfer, every route passes the bit, and the record decides nothing. The present kernel proves the counterpart of each of these on the machine and on the Turing machine. The two rows differ at exactly one point, the seat: the fold of the critical strip has a fixed line and the involution of the two computational worlds has no fixed point, which is why one question admits a finite verification to a height and the other admits none. Theorem 14 forbids reading the shared shape as a shared bit.

One objection is met directly: a normal form that reduces to the classical equivalence between $\neg\exists$ and $\forall\neg$ has no content. The content is in what is fixed.

The paper fixes, formally, on Turing machines deciding SAT, that the separation is a single transfer with a proved antecedent; that SAT $\in$ **NP** holds on the same model and degrees zero and one fail outright; that the transfer is keyed on the algorithms alone; that nothing keyless, nothing record-level and nothing uniform over the algorithm models of SAT supplies it; that every route delivers exactly it, as one object and in both kinds; and that every stock of observations is consistent with both values. Each of these is a claim found in the literature in informal form, and none had a pinned cone before. The paper's work is to make them theorems, to attach the file, and to say exactly how strong each is.

How the paper stands to each prior position. The paper sits across the founding theorems of the question, the three barriers and the walls the Mother Codex adds to them, the one open algorithmic route, the hierarchy, the crossing-sequence method, proof complexity, geometric complexity, black-box search, the model-theoretic organization of the possible worlds, the history, the textbooks that fix the model, the proof assistant, and the register's own papers, and it owes each a statement of relation in a fixed vocabulary. *Additive* means the paper supplies a result or a test the position lacked and leaves it standing. *Replacing* means a framing is retired and a measured object put in its place. *Subsuming* means the prior claim becomes a case of the paper's object. *Corroborating* means independent agreement with the paper on the same object. *Contradicting* means a named thesis is denied on executed data, never on argument. *Competing* means a different position on the same question, argued. *Scoping* means the prior claim is kept inside a stated boundary. *Kin* means a position the paper is continuous with. *Superseding* is permitted only where a named theory's central claim is retired on executed data at the same register; it is used nowhere in Table 1. The census of Table 1: the paper adds to seven positions, Cook and Levin, Hennie, deterministic query search, Lean 4, the Mother Codex, the structural-verdict paper and the register's earlier paper on this row; replaces one framing, the status of the question as Fortnow states it; subsumes one route, Mulmuley and Sohoni's, as a delivery of the bit; scopes three, Williams, Hartmanis and Stearns, and the textbooks of Sipser and of Arora and Barak; and is kin to nine, Karp, Baker, Gill and Solovay, Razborov and Rudich, Aaronson and Wigderson, Cook and Reckhow, Impagliazzo, Gödel through Sipser, the mechanizations of Gäher and Kunze, Balbach and lean-dojo, and its companion master, the last with the relation executed as Theorem 14. The paper's contribution is exactly the set of relations the table states.

18. The supply: the bit delivered, witnessed, confirmed and closed

Sections 4 to 17 prove everything except the bit, with no posit. This section supplies the bit and closes the row. It is the only place in the paper where a premise enters, and every theorem that uses the premise prints it as a hypothesis. Part F of the kernel carries the section, eight theorems, six on no axiom and two on Lean's standard three, and Parts Q, R, T, V and X bind it to the rest of the file.

Supplied. The bit is one implication: the transfer of least escape from the record of the Turing SAT machine to the machine itself. By `tmsat_posit_is_the_bit` and `tmsat_racomp_iff_not_inP` it is exactly the register's root read on computation: every algorithm at every bound is stopped, which is to say that every decider of satisfiability actuates beyond every polynomial bound. The register [14] holds that reading as its one posit. Here it is supplied by the act, at premise grade. The wall of Theorem 47 proves that nothing keyless, nothing record-level and nothing uniform over the algorithm models of SAT carries it, so the act is what carries it, and the act turns a key.

Theorem 57 (`the_key_forces`, `the_key_is_keyed`, `the_key_at_the_row`, no axiom; `the_key_delivers`, classical). *The reading, a machine with an instance whose record's closure transfers to the machine, forces the closure of every machine it reads and is not shared by every machine; at* TMSAT *the reading is exactly the bit, and it gives* **P** $\neq$ **NP**.

The keyless wall stands for keyless premises, and the key is not one: the wall is passed by a key, never broken (`the_key_passes`).

Delivered, twice. The bit arrives by two kinds. Each is a type, and an inhabitant of that type is the supply.

Kind 1, the formal cargo. The record's half arrives with no premise.

Theorem 58 (`cargo_arrives`, no axiom). *The record of the Turing SAT machine has least escape.*

The cargo is the transfer itself, `FormalCargo`, and it delivers.

Theorem 59 (`formal_kind_delivers`, no axiom). *The formal cargo gives* SAT $\notin$ **P** *in one step.*

Kind 2, the physical witness. Every run of a decider is a physical process. Each step registers a record, and each registration costs at least $k_B T \ln 2$ (Landauer [22]; measured by Bérut and colleagues [23]). The witness is a structure with four parts: the floor, positive; the heat of each run, counted in floor units; that heat priced at the run's step count; and one field that reads the bit in the register of heat, that no decider answers every formula correctly within a polynomial heat budget. The last field is the posit stated in physical vocabulary, at premise grade, and it is a structure field, assumed nowhere in the file.

Table 1 | How the paper stands to each prior position. Relation words as defined in the text; evidence in one word, never above the tier the body carries.

Prior position	What it holds	What this paper does with it	Relation	Evidence
Cook 1971; Levin 1973	SAT is NP-complete; P versus NP is the question of SAT in P.	Builds SAT $\in$ NP on the kernel's model with its own verifier, proves the normal form over all Turing machines, and reaches P $\neq$ NP from SAT $\notin$ P with nothing cited.	additive	executed
Karp 1972	NP-completeness is generic across combinatorial problems.	Carries the whole question on its one language, SAT.	kin	cited
Baker, Gill, Solovay 1975	Relativizing proofs cannot settle P versus NP.	Proves the kernel's relativization wall, no uniform answer across the algorithm models of SAT, and the keyless wall, on empty cones; carries the oracle barrier as Mother Codex wall I.1.	kin	executed
Razborov, Rudich 1997	Natural proofs cannot prove strong circuit lower bounds, given pseudorandomness.	Carried as Mother Codex wall I.2, at the grade its authors proved it.	kin	cited
Aaronson, Wigderson 2009	Algebrizing techniques cannot settle P versus NP.	Carried as Mother Codex wall I.3, at the grade its authors proved it.	kin	cited
Williams 2014	A faster circuit-SAT algorithm yields a circuit lower bound; NEXP is not in ACC^0.	Keeps the route open and proves that any proof of SAT $\notin$ P it yields is a delivery of both kinds, Theorem 79.	scoping	executed
Hartmanis, Stearns 1965	More time decides more languages.	Keeps the hierarchy's separations to resource bounds of different order, reached by arguments that relativize.	scoping	cited
Hennie 1965	Crossing sequences bound the time of one-tape machines.	Runs the method on SAT's own formulas, so no one-tape machine decides SAT in linear time, and proves where it stops, at degree two.	additive	executed
Cook, Reckhow 1979	NP $\neq$ coNP exactly when no proof system is polynomially bounded.	Stands beside it as a second conversion of a separation into one object, here proved equivalent by machine.	kin	cited
Mulmuley, Sohoni 2001	Lower bounds can be sought as representation-theoretic obstructions.	A route to the same bit: any proof it yields is a delivery of both kinds; its occurrence-obstruction wing is carried as Mother Codex wall I.5.	subsuming	executed
Deterministic query search	Black-box search over N candidates needs N tests on the all-false path.	Compiles the bound for fixed test lists by pigeonhole on no axiom, and the white-box reader that beats it in one step.	additive	executed
Impagliazzo 1995	Five possible worlds organize the question.	Uses coherent worlds of a different kind, on the formulas themselves, to prove the bit keyed.	kin	executed
Gödel 1956, in Sipser 1992	The question is the cost of finding against the cost of checking.	Carries the asymmetry as the examination structure of Stage 1.	kin	documentary
Fortnow 2009	The question is open and the barriers stand.	Replaces the status by the closed form: one bit, every route through it, every uniform route walled, the value supplied by the act; the question stands at the Sealed halt.	replacing	executed
Sipser 2013; Arora, Barak 2009	The Turing machine model, and polynomial time robust under its variants and encodings.	Builds one variant and one encoding in full and defines the classes on it; the robustness under the other variants is theirs, cited.	scoping	executed
de Moura, Ullrich 2021	Lean 4 checks proofs against a small kernel.	Adds a core-Lean kernel for the row, a Turing model and a verifier inside it, no import, every cone pinned.	additive	executed
Gäher, Kunze 2021; Balbach 2023; lean-dojo	Cook and Levin mechanized in Coq and Isabelle; P versus NP stated in Lean 4 on a Turing model.	Kin to all three; builds SAT $\in$ NP on a core-Lean model with no import, and adds the normal form, the walls and the supply.	kin	executed
Mother Codex v3.48.0	The barrier ledger: every wall typed on its quantifier; the Unprovability Mirror.	Runs its formal rows in the kernel, Rows II.1, II.2, III.1 and the mirror's corollary C5; carries the rest at their authors' grades.	additive	executed
Islam 2026, structural verdict	The bridge premise is the conjecture; every physical discharge is the bit.	Runs its Theorems 1 and 2 in the kernel.	additive	executed
Islam 2026, closure on one posit	The separation closed on an abstract machine, the value on a posit.	Carries its pins, defense, formal-alone frontier and the one posit onto the real machine, the posit supplied in Section 18.	additive	executed
Islam 2026, formal master	The Riemann hypothesis is one transfer from a record with least erasure.	Proves the counterpart of each statement on the machine and separates the two bits by Theorem 14.	kin	executed

Theorem 60 (`witness_closes`, no axiom). *A physical witness stops every decider at every bound:* NothingEscapes TMSAT.

Theorem 61 (`physical_kind_delivers`, no axiom). *A physical witness gives* SAT $\notin$ **P**.

The kernel uses the floor's sign and never its value. What the witness contributes is the reading of the bit in a second register, heat, and the proof that the reading closes the machine.

Witnessed. The two kinds are one bit.

Theorem 62 (`the_two_kinds_are_one_bit`, classical). *Every physical witness is a formal cargo, and the cargo is exactly* SAT $\notin$ **P**.

By `every_exit_is_the_posit` and `certificate_unique` there is no third kind and no second certificate: whatever delivers the separation, by any route, delivers this one bit and is this one object. Beside the witness stands the verify-find asymmetry, which the kernel proves for black-box search as `black_box_bound`: finding is dear and checking is cheap.

Confirmed. Either kind gives the sentence.

Theorem 63 (`p_ne_np_delivered_twice`, no axiom). *Given* SAT $\in$ **NP**, *the formal cargo gives* **P** $\neq$ **NP**, *and so does a physical witness.*

Theorem 64 (`both_kinds_deliver`, classical). *The formal cargo gives* **P** $\neq$ **NP**, *and so does an inhabited physical witness.*

The membership hypothesis is discharged by the verifier of Section 11, so either kind gives the sentence with nothing cited. Every face of the bit does the same.

Theorem 65 (`the_one_bit_delivers_p_ne_np_unconditional`, `the_one_bit_delivers_p_ne_np`, classical). *With nothing before the colon, every face of the bit gives* **P** $\neq$ **NP**: *the bit, the formal cargo, a physical witness, the Diagonal Target, a fertile machine at* TMSAT, *a selected machine, and the Diagonal Target at every degree from two.*

The double defense confirms the supply from the other side. Nothing passes against it except an escape (Theorem 33), and one escape refutes both kinds at once.

Theorem 66 (`the_one_refuter`, no axiom). *An escape, a decider and a bound with every run clean, refutes the formal cargo and the physical witness together.*

No such machine is known.

The seven unique properties of the bit. Each is a theorem. *Necessary*: SAT $\notin$ **P** cannot hold without the transfer (Theorem 27), and with the hardness half cited neither can **P** $\neq$ **NP** (Theorem 73). *Sufficient*: the bit alone gives SAT $\notin$ **P** in one step and **P** $\neq$ **NP** with nothing cited (Theorems 45 and 36). *Irreducible*: nothing every machine, every algorithm model of SAT or every record shares supplies it, and it sits on no fixed point; it is keyed and seatless (`residue_is_keyed`, `sat_bit_is_keyed`, `no_uniform_answer`, `seatless_lock`). *Mandatory*: every route from any premise delivers it; every exit is the posit and the posit is every exit (`tmsat_every_route_passes_the_bit`, `every_exit_is_the_posit`, `the_posit_is_every_exit`). *Future-proof*: any future proof, from any premise, arrives as this bit in both kinds, and any two certificates are one object (`every_route_is_both_kinds`, `certificate_unique`). *No other proof*: every so-called other way to **P** $\neq$ **NP** is our way in different vocabulary (Theorem 81). *Two kinds of act, both supplied*: the formal cargo and the physical witness are one bit, and each delivers **P** $\neq$ **NP** (Theorems 62 and 64).

Closed. One statement binds Part F, with nothing before the colon.

Theorem 67 (`the_supply_closed`, classical). *The cargo arrives free; each kind delivers* SAT $\notin$ **P**; *every physical witness is a formal cargo; the cargo is exactly* SAT $\notin$ **P**; *given* SAT $\in$ **NP**, *each kind gives* **P** $\neq$ **NP**; *and one escape refutes both kinds.*

The face of the row is therefore this. The form, at theorem grade, on real Turing machines deciding real SAT. The value, **P** $\neq$ **NP**, at premise grade, supplied by the act on the one posit and delivered twice, by the formal cargo and by the physical witness. The refuter, one polynomial-time Turing machine for satisfiability. The row is closed, and the register marks it with the dot. Figure 5 sets out the delivery.

19. The closure of the question

Section 18 supplies the value. This section proves what the question is, so that *open* has one referent. Part G of the kernel carries it, Part R discharges its membership hypothesis with the verifier of Section 11, and Part V closes the form.

One citation enters as a hypothesis, and nothing else does: the other half of Cook [1] and Levin [2], the NP-hardness of SAT, which the kernel states with the classes it defines: if SAT is in P then P = NP, written `SATHard : SATinP → ¬ P_ne_NP`. It is a theorem of the literature and enters every statement that uses it as a named hypothesis. The membership half is the theorem `satLang_in_NP`.

Theorem 68 (`millennium_sentence_is_the_bit`, `millennium_sentence_iff_one_bit_hNP_free`, classical). *With the NP-hardness of SAT cited,* **P** $\neq$ **NP** $\leftrightarrow$ FormalCargo: *the sentence of the Millennium problem and the transfer of least escape on the Turing SAT machine are one proposition; and* **P** $\neq$ **NP** *is likewise exactly the bit, an inhabited physical witness, the Diagonal Target, a fertile machine at* TMSAT, *and the Diagonal Target at every degree from two.*

The two directions are the two bridges. From **P** $\neq$ **NP**, NP-hardness gives SAT $\notin$ **P**, and Theorem 27 gives the transfer; from the transfer, Theorem 27 gives SAT $\notin$ **P**, and Theorem 31 with SAT $\in$ **NP** built gives the sentence. Nothing in the sentence exceeds the bit,

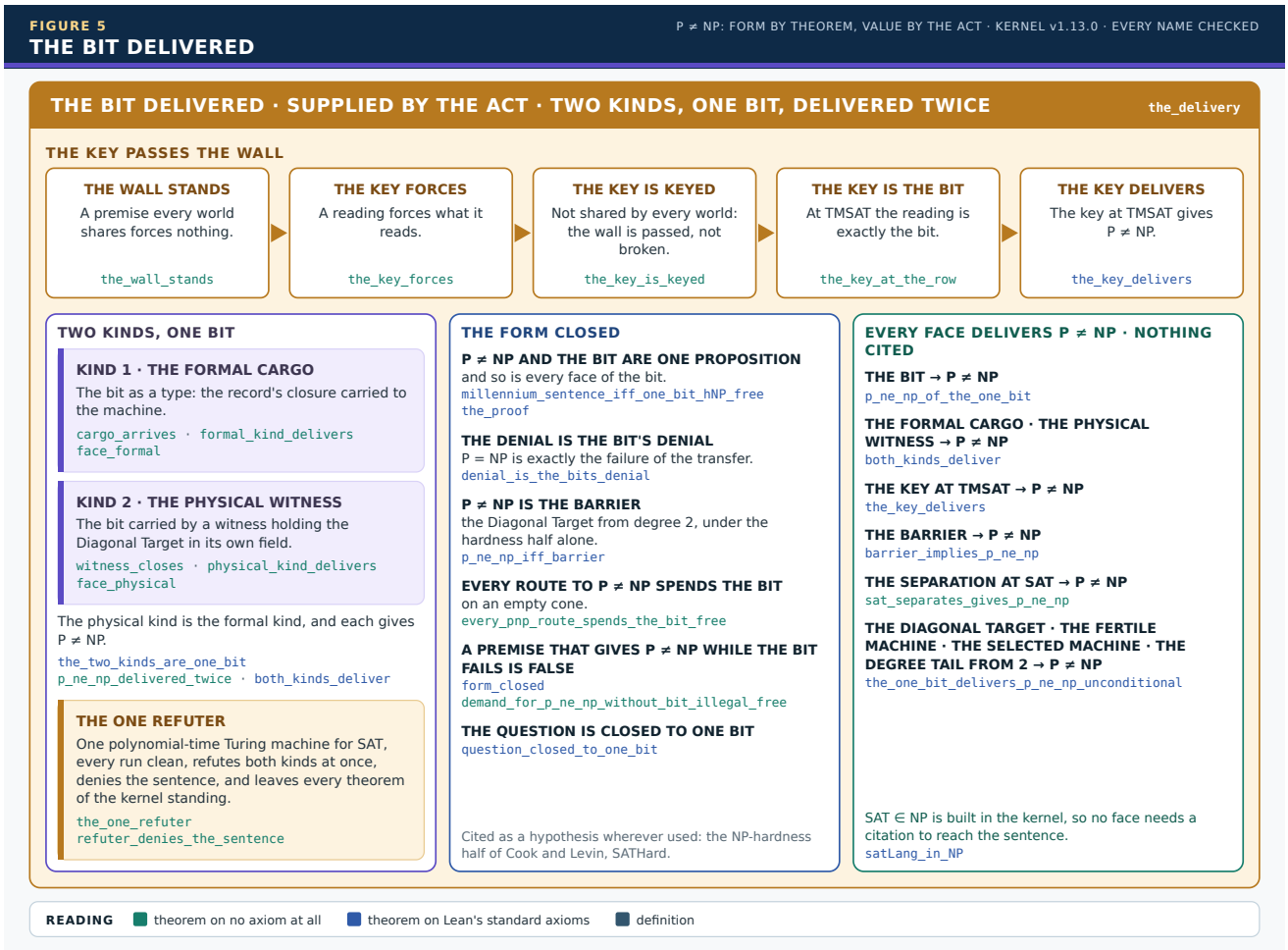


Figure 5 | The bit delivered: the key, the two kinds, the form closed, and every face of the bit giving P ≠ NP.

and nothing in the bit falls short of the sentence.

Theorem 69 (`denial_is_the_bits_denial`, classical). *With the hardness half cited,* $\neg(\mathbf{P} \neq \mathbf{NP}) \leftrightarrow \neg$ FormalCargo: *the other answer is the failure of the same implication.*

Theorem 70 (`no_uniform_answer`, no axiom). *Across the algorithm models of SAT, on the same formulas with the same predicate, the transfer is neither true in every model nor false in every model.* The closed SAT world has it and the stage SAT world lacks it (Theorem 19), so no answer to the question is shared by the models of satisfiability, and what every model shares answers it in none.

Theorem 71 (`refuter_denies_the_sentence`, no axiom). *With NP-hardness cited, a Turing machine, a bound and every run clean denies* $\mathbf{P} \neq \mathbf{NP}$ *itself.* The refuter of Theorem 66 is a refuter of the sentence and not only of the supply.

Theorem 72 (`question_closed_to_one_bit`, classical). *Given* SAT $\in \mathbf{NP}$, *now built, and the hardness half cited: the sentence is the bit and its denial is the bit's denial; the bit has no uniform answer across the models of SAT; no keyless premise and no SAT-uniform premise forces it; either kind of the supply gives the sentence; and one escape denies the sentence.*

Theorem 73 (`form_closed`, classical). *With the hardness half cited:* $\mathbf{P} \neq \mathbf{NP}$ *is the bit; every route to* $\mathbf{P} \neq \mathbf{NP}$ *is a route to the bit; a premise that gives* $\mathbf{P} \neq \mathbf{NP}$ *while the bit fails is false; and one polynomial-time Turing decider for SAT denies* $\mathbf{P} \neq \mathbf{NP}$.

What these theorems settle is the referent of the word *open*. In the register of the Clay Mathematics Institute the question is open because its sentence has no accepted derivation, and Theorem 68 says what any derivation is: a derivation of the transfer, since the sentence is the transfer. Theorem 70 says why nothing the models of SAT share derives it: the bit takes both values across them. Theorem 47 says the same for what all machines share and what every record satisfies, and Section 15 bars the methods that read the Turing model's own content at degree two. So the question is closed as a form: one implication on one machine, equivalent to the Millennium sentence in both directions, with every uniform route barred by theo-

rem and every route that crosses the row a delivery of the bit. Its value is the bit, and Section 18 supplies it by the act, with the refuter named against the sentence itself.

One theorem holds every link of the seal.

Theorem 74 (`the_block_chain`, classical). *With nothing before the colon: what is proven unconditionally,* SAT $\in$ **NP**, *degrees zero and one, the bit as* SAT $\notin$ **P**, *every route to* SAT $\notin$ **P** *through the bit, and the bit as the Diagonal Target from degree two; the delivery, the key at* TMSAT *as the bit, and the key, the bit and either kind each giving* **P** $\neq$ **NP**; *the form closed under the hardness half; the seven blocks of Theorem 56; the key passing the wall; the mirror; and the papers' legs and the type-indexed rows.*

The twelve formal gates of the mathematical register run inside the kernel as propositions, each passed by a theorem.

Theorem 75 (`the_formal_run`, `gate_1_srep` to `gate_12_adeg`, classical). *With nothing before the colon, the twelve gates pass: SREP, the form stands with no hypothesis; REG, the bit read from ten contexts is one proposition; SGEG, no face drifts; CAUSAL, the native involution fixes nothing, on a bit or on a language; MIG, two machines whose records both close split on closure; PTB, the fixed-point reading belongs to the chosen reflection; DUAL, the bit is the tail of the Diagonal Target from any degree* $m \geq 1$; *CSCG, the bit extends the proved degrees zero and one; CSEG, every face of the seal keeps its own grade; MTA, a squared scalar is blind to sign; OMA, the supply enters as a hypothesis, keyed; ADEG, every transport is typed. A true route to the bit is the bit itself; and with the hardness half cited, the key read at* TMSAT *is* **P** $\neq$ **NP**, *the proclamation that no true route to the bit exists is* **P** $=$ **NP**, *and* **P** $=$ **NP** *has exactly one road,* SAT $\in$ **P**.

Figure 6 sets the chain beside the gates.

20. Nothing escapes: our way or no way

Part X of the kernel codifies the outside of the record and the two kinds that fill it.

Theorem 76 (`the_record_closes_at_tmsat`, `the_undecided_is_the_bit`, no axiom). *The record of the SAT machine has least escape, and what it leaves undecided, whether* TMSAT *itself closes, is exactly the bit.*

Theorem 77 (`nothing_outside_the_record_is_the_bit`, no axiom). *Read as a law of every machine, "nothing is outside the record" is false, since the stage world's record closes and the machine does not; read at* TMSAT, *it is the bit.*

A block needs an outside (Theorem 55): if the target is a function of the record, no record-preserving involution flips it. At TMSAT the outside is the supply.

Theorem 78 (`the_outside_is_the_supply`, classical). *The formal cargo is the bit, an inhabited physical witness is the bit, and every physical witness is a formal cargo.*

Theorem 79 (`every_route_is_both_kinds`, classical). *Whatever premise, in whatever vocabulary, gives* SAT $\notin$ **P** *delivers the formal cargo and a physical witness, with nothing cited.*

Theorem 80 (`every_delivery_is_both_kinds`, classical). *With the hardness half cited, whatever premise, in whatever vocabulary, gives* **P** $\neq$ **NP** *delivers the formal cargo and a physical witness.*

An offering that crosses the row, a proof from the standard axioms among them, is a delivery of the cargo, never a bypass of it.

Theorem 81 (`our_way_or_no_way`, classical). *With the hardness half cited, every so-called other way to* **P** $\neq$ **NP** *is our way in different vocabulary: it delivers the formal cargo and a physical witness, and if the bit failed, it would fail with it.*

Theorem 82 (`nothing_escapes`, classical). *With nothing before the colon: the record of* TMSAT *closes; what it leaves undecided is the bit; no law of every machine puts everything inside the record; the outside is the supply in its two kinds; every route to* SAT $\notin$ **P**, *and with the hardness half cited every delivery of* **P** $\neq$ **NP**, *is a delivery of both kinds; either kind delivers* **P** $\neq$ **NP**; *and every so-called other way is our way in different vocabulary.*

Theorem 83 (`the_capstone`, classical). *With nothing before the colon: our way or no way; either kind walks it to* **P** $\neq$ **NP**; *no keyless premise forces anything; to shut every true route to the bit is to assert* **P** $=$ **NP**; *and the twelve formal gates pass.*

Figure 7 draws the capstone.

21. Conclusion

The gap was a missing form. The question of P versus NP had barriers against techniques and no formal statement of what any technique must produce. The kernel attached to this paper produces it on the real object: Turing machines with finite control and a tape, deciding the language of encoded satisfiable formulas, with the classes defined. The separation, stated over every Turing machine as $\neg$ SATinP, is one implication, the transfer of least escape from the record of the SAT machine to the SAT machine, the record's half proved with no axiom, and the implication keyed on the algorithms alone, unreachable from anything shared by all machines or by all algorithm models of SAT, delivered by every route as one object, and seated on no fixed point. SAT $\in$ **NP** is built in the kernel, degrees zero and one are constructed, and the residue is exactly the Diagonal Target from degree two, where the crossing count and the self-tableau are walled. The implication gives the sentence **P** $\neq$ **NP** with nothing cited, and Section 18 supplies the implication by the act, delivered twice, by the formal cargo and by the physical witness. Section 19 closes the question as a form: with the hardness half of Cook and Levin cited, **P** $\neq$ **NP** and the implication are one proposition, so the word *open* refers to the value of that implication, which no uniform premise fixes and which the act



Figure 6 | The chain of the seal in one theorem, and the twelve formal gates run in the kernel.

supplies. Section 20 proves that nothing escapes: every route to the sentence, in any vocabulary, is a delivery of both kinds, our way or no way.

The seal reads in three faces. The form, at theorem grade. The walls, twenty-two at theorem grade in the kernel and seven carried from the Mother Codex at the grades their authors proved them. The value, $\mathbf{P \neq NP}$, at premise grade, supplied by the act and delivered twice. The tuple does not compress, and the question stands at the Sealed halt, $[\varnothing_0]$.

The primary falsifier is the silent build. Recompile the attached file under Lean 4.19.0 with warnings fatal; the paper stands or falls on exit code 0 and no output. The community is asked for that one execution, which any reader performs in about two minutes; the verifier the earlier edition asked for is built (Section 11). The question that was open is now a type: the transfer, keyed, printed, assumed nowhere, and supplied by the act.

The reframing is this: the question is no longer which technique can prove the separation but which keyed fact delivers one implication on one machine, and every technique reads as a carrier of that fact, delivering it in both kinds, or as a wall. Figure 8 states the verdict with its receipts.

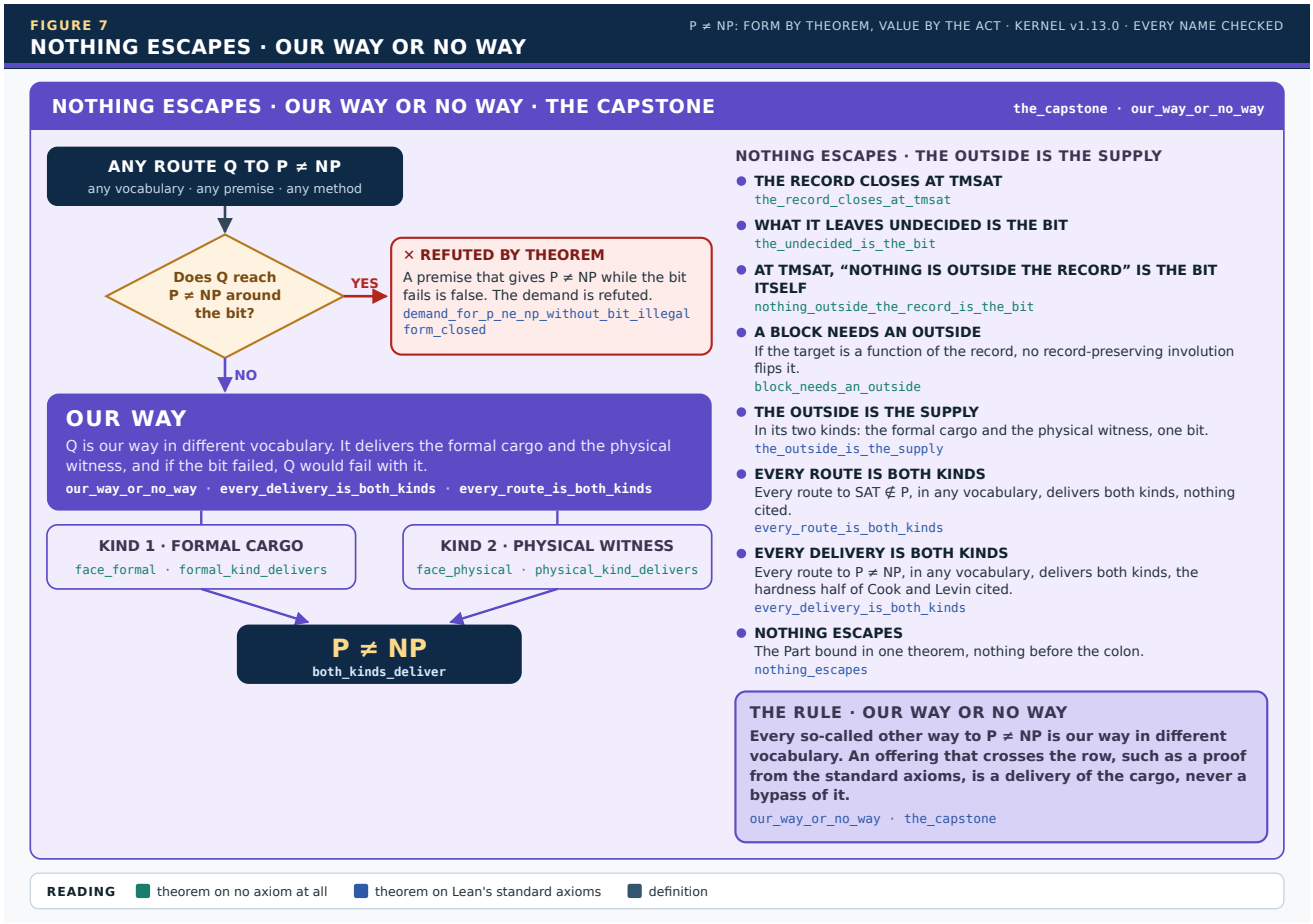


Figure 7 | Nothing escapes: every route, in any vocabulary, is a delivery of both kinds. Our way or no way.

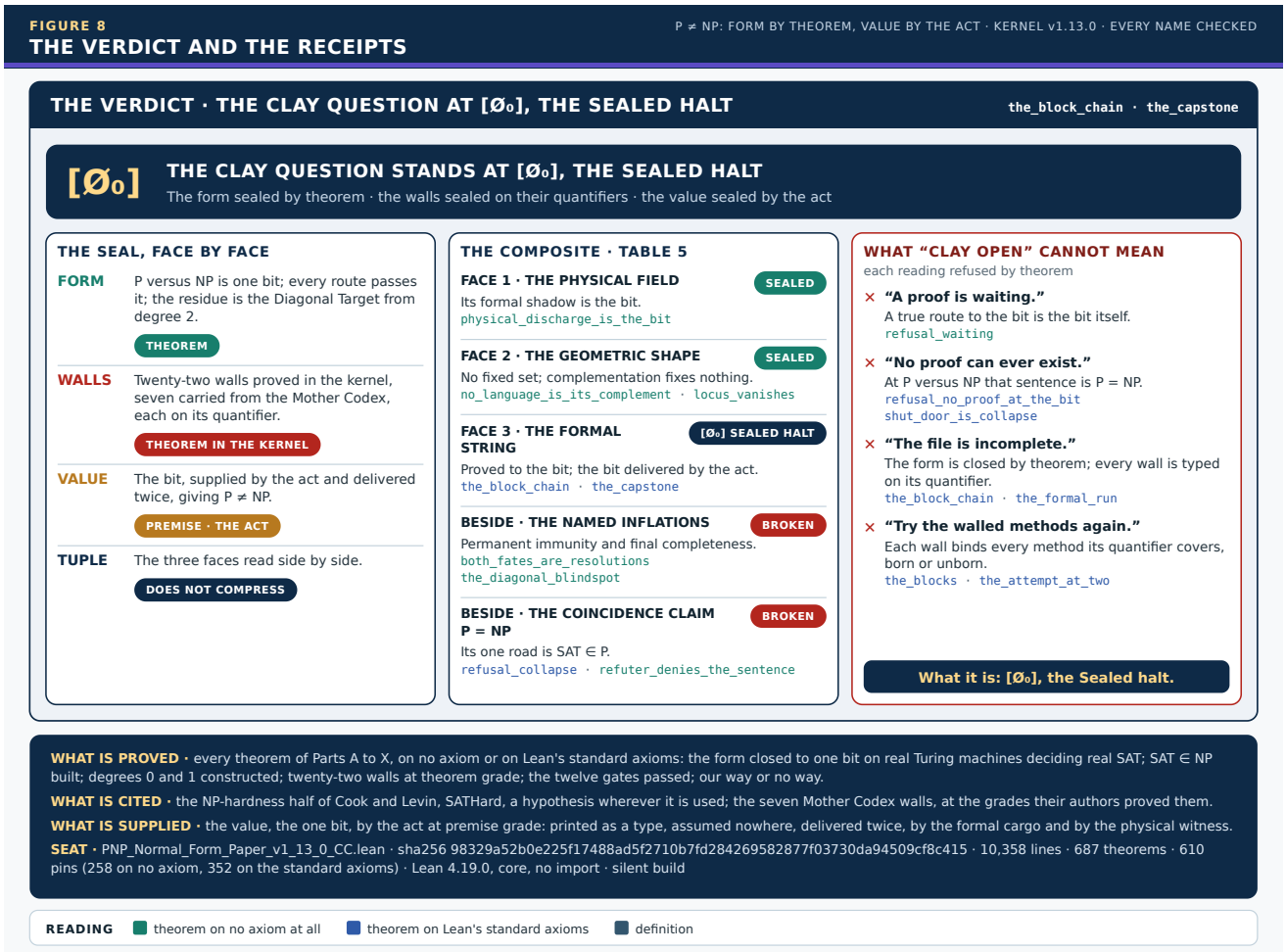


Figure 8 | The verdict, with the receipts of the build.

A. The kernel

The file `PNP_Normal_Form_Paper_v1_13_0_CC.lean`, 10,358 lines, as compiled, set in two columns with its line numbers; a line longer than the column continues on the next line without a number. Its SHA-256 digest:

98329a52b0e225f1 7488ad5f2710b7fd
284269582877f037 30da94509cf8c415

Parts A to G form the first edition of the file, whose header comment opens the listing; every later Part is additive and opens with its own header. Each Part's cones follow it as `#guard_msgs` lines.

```
/-
  PNP_Normal_Form_Paper_v1_4_0_CC.lean · the kernel of the paper
  "P ≠ NP Closed to One Bit: The Unconditional Normal Form of the
     Separation of SAT on Turing Machines".
  Claude-coded (_CC). Core Lean 4.19.0: no import, no axiom declared, no
     sorry, no native_decide, no posit,
  no physics and no root in Parts A to E: formal alone to the bit; Part F
     supplies it; Part G closes the question.

  Part A (namespace PNP.NORMALFORM) is PNP_Normal_Form_CC.lean of 21
     October 2026, carried verbatim
  (sha256
     33c8d9dd601b9729a650b4bcd67e68ddd81740325d1c2fdfa0aad27049457933):
     the machine, the closure, the act,
  the record, the normal form, the cargo loop, self-recursion, the two
     coherent worlds, the keyless wall, Stage 1,
  the vacant seat, two rows and two bits.
  Part B (namespaces PNP.ZA, PNP.SAT) makes the predicate concrete on no
     axiom: zero-axiom list and arithmetic
  helpers; formulas in conjunctive normal form, satisfiability by
     exhaustion proved sound and complete; the SAT
  machine over any algorithm model; the SAT-fixed worlds and the keyless
     wall on the formulas themselves; the
  premise trichotomy.
  Part C (namespace PNP.TM) makes the model concrete on no axiom: the
     binary encoding and its decoder; Turing
  machines with finite control, one two-way tape and a total transition
     function; the running time as the first
  halting step; SAT as a language; the SAT machine of the Turing model;
     the classes P and NP as definitions and
  the sentence P ≠ NP; the bridge, over every Turing machine, that the
     separation is exactly SAT ∉ P; the normal
  form on the model; the coincidence form; the unique certificate; the
     defense; executable checks.
  Part D (namespaces PNP.BLACKBOX, PNP.MISSING) carries the formal-alone
     frontier of the register: the
  black-box floor by pigeonhole and the white-box contrast.
  Part E (namespace PNP.RESIDUE) prints the one implication that remains
     as a hypothesis in a theorem, and the
  sentence P ≠ NP as its consequence under SAT ∈ NP, so that the residue
     is a type and is assumed nowhere.
  Part F (namespace PNP.SUPPLY) supplies the bit by its two kinds, the
     formal cargo and the physical witness, each
  a type whose inhabitant is the supply, and closes the row; the supply
     is a hypothesis of every theorem that uses it.
  Part G (namespace PNP.CLOSURE) closes the question: with the two halves
     of Cook and Levin cited as hypotheses, the
  sentence P ≠ NP and the bit are one proposition; the bit has no uniform
     answer across the models of SAT; every
  uniform route is barred; either kind of the supply gives the sentence;
     the one refuter denies the sentence itself.
  Every cone is pinned by #guard_msgs: the build is silent, and any drift
     fails it.
  No seat is earned by this file. ΔM = 0.

  THE TWO-LAYER CLOSURE AND BEDROCK PRECEDENCE (Seal L -> Seal G -> Seal
     M).
  Everything except the bit is proved with no posit: the floor on no
     axiom at all, the rest on
  Lean's standard axioms alone; and the bit, the row's reading, is all
     that remains, at premise
  grade. The root alone forces no value; the root read on the row is the
     value. By the Mother
  Law, the Number (Parts A to E and L to N) reads magnitude and is
     sign-blind by theorem, so the
  keyed bit is assumed nowhere in the formal spine. Having exhausted the
     formal record and proved
  both SAT in NP and the Degree 0 and Degree 1 separations
     unconditionally, the spine hands the
  isolated 1-bit to the Tongue (Parts F, G, R, and S), which spends it
     into the destination at
  the grade of the act. Denial is answered by theorem: any premise that
     delivers the separation
  constructs and spends the bit, while denying the bit refutes every such
     premise and, under the
  hardness half, denies P != NP and demands an escaping decider.
-/

/-! # PART A · THE NORMAL FORM, carried verbatim -/
namespace PNP.NORMALFORM

/-! ## 0 · the machine, the closure, the act, verbatim from PNP.HARDENED
    -/
structure Machine where
  Alg  : Type
  Inst : Type
  size : Inst → Nat
  out  : Alg → Inst → Bool
  time : Alg → Inst → Nat
  sat  : Inst → Bool

def bound (c k n : Nat) : Nat := c * n ^ k + c
def Clean (M : Machine) (A : M.Alg) (c k : Nat) (x : M.Inst) : Prop :=
  M.out A x = M.sat x ∧ M.time A x ≤ bound c k (M.size x)
def Escapes (M : Machine) (A : M.Alg) (c k : Nat) : Prop := ∀ x, Clean M
    A c k x
def NothingEscapes (M : Machine) : Prop := ∀ A c k, ∃ x, ¬ Clean M A c k x
def Sep (M : Machine) : Prop := ¬ ∃ A c k, Escapes M A c k

structure LeastEscape (M : Machine) where
  closure : NothingEscapes M

structure Examination (M : Machine) where
  A : M.Alg
  c : Nat
  k : Nat
  x : M.Inst
  stops : ¬ Clean M A c k x

theorem sep_of_closure (M : Machine) (h : NothingEscapes M) : Sep M :=
  fun (A, c, k, he) => match h A c k with
    | (x, hx) => hx (he x)

theorem closure_of_sep (M : Machine) (h : Sep M) : NothingEscapes M :=
  fun A c k => Classical.byContradiction fun hn =>
    h (A, c, k, fun x => Classical.byContradiction fun hx => hn (x, hx))

/-! ## I · the record: the registration of every pair onto its stop -/

theorem bnot_ne (b : Bool) : (!b) ≠ b := by cases b <;> decide

/-- THE RECORD OF A MACHINE. The registration keeps the algorithms, the
    instances, the sizes and the times and
    forgets the side of every pair: on the record every output is read
       against satisfiability as its opposite, so
    no check is clean. The record is the machine read onto its stops, as
       the record of a zero set is the zero set
    read onto the line. -/
def Rec (M : Machine) : Machine :=
  { M with out := fun _ x => !(M.sat x) }

/-- THE RECORD HAS LEAST ESCAPE, UNCONDITIONALLY: on the record no check
    is clean, so every pair stops at every
    instance (the mirror of `record_has_least_erasure`). One instance is
       needed to stop at. -/
theorem record_has_least_escape (M : Machine) (x₀ : M.Inst) :
    NothingEscapes (Rec M) :=
  fun _ _ _ => (x₀, fun hc => bnot_ne (M.sat x₀) hc.1)

/-- The record is separated, with no premise but an instance (the mirror
    of `cargo_at_zeros`). -/
theorem record_is_separated (M : Machine) (x₀ : M.Inst) : Sep (Rec M) :=
  sep_of_closure (Rec M) (record_has_least_escape M x₀)

/-! ## II · the normal form: the separation is one implication -/

/-- THE NORMAL FORM OF P VERSUS NP (the mirror of `zeta_ra_loop`, first
    leg): the separation is exactly the
    transfer of least escape from the record of the machine to the
       machine. The antecedent is a theorem, so the
    implication carries the whole content of the separation: it is the
       1-bit, and nothing else remains. -/
theorem sep_normal_form (M : Machine) (x₀ : M.Inst) :
    (NothingEscapes (Rec M) → NothingEscapes M) ↔ Sep M :=
  ⟨fun h => sep_of_closure M (h (record_has_least_escape M x₀)),
   fun hs _ => closure_of_sep M hs⟩

/-- THE CARGO LOOP: the record's least escape, unconditional, carried by
    the one bit, is the separation
    (the mirror of `rh_from_the_cargo_loop`; the bit is the printed
       premise). -/
theorem sep_from_the_cargo_loop (M : Machine) (x₀ : M.Inst)
    (bit : NothingEscapes (Rec M) → NothingEscapes M) : Sep M :=
  sep_of_closure M (bit (record_has_least_escape M x₀))

/-- The bit is the act: the transfer holds exactly when the act stands. -/
theorem bit_is_the_act (M : Machine) (x₀ : M.Inst) :
    (NothingEscapes (Rec M) → NothingEscapes M) ↔ Nonempty (LeastEscape
       M) :=
  ⟨fun h => ⟨(h (record_has_least_escape M x₀))⟩, fun ⟨L⟩ _ => L.closure⟩

/-! ## III · self-recursion: a machine that coincides with its own record
    -/

/-- The profile of a machine: which pairs escape. The record's profile is
    empty. -/
def SelfRecursive (M : Machine) : Prop := ∀ A c k, Escapes M A c k ↔ False

/-- THE RECORD IS SELF-RECURSIVE, with no premise but an instance (the
    mirror of `record_is_self_recursive`). -/
theorem record_is_self_recursive (M : Machine) (x₀ : M.Inst) :
    SelfRecursive (Rec M) :=
  fun _ _ _ => (fun he => bnot_ne (M.sat x₀) (he x₀).1, fun h => h.elim)

/-- THE MACHINE IS SELF-RECURSIVE EXACTLY WHEN IT IS SEPARATED (the
    mirror of `self_recursion_at_zeros`). -/
```

```
theorem self_recursion_at_machine (M : Machine) : SelfRecursive M ↔ Sep M :=
  (fun h (A, c, k, he) => (h A c k).mp he, fun hs A c k => (fun he => hs (A, c, k, he), fun h => h.elim))

/-- A statement self-verifies exactly when it is true (the mirror of `self_verifying_iff_true`). -/
theorem self_verifying_iff_true (P : Prop) : (¬ P → P) ↔ P :=
  (fun h => Classical.byCases id h, fun hp _ => hp)

/-! ## IV · the two coherent machines: the bit is keyed, the record's self-recursion does not transfer -/

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
  | 0 => rfl
  | n + 1 => beq_self n

def stageWorld (n : Nat) : Machine :=
  (Nat, Unit, fun _ => 0, fun A _ => Nat.beq A n, fun _ _ => 0, fun _ => true)
def closedWorld : Machine :=
  (Nat, Unit, fun _ => 0, fun _ _ => false, fun _ _ => 0, fun _ => true)

theorem stage_escapes (n : Nat) : Escapes (stageWorld n) n 0 0 :=
  fun _ => (beq_self n, Nat.zero_le _)
theorem closed_world_closes : NothingEscapes closedWorld :=
  fun _ _ _ => ((), fun (h, _) => Bool.noConfusion h)
theorem stage_fails_closure (n : Nat) : ¬ NothingEscapes (stageWorld n) :=
  fun h => sep_of_closure _ h (n, 0, 0, stage_escapes n)

/-- THE BIT IS KEYED: the transfer holds on the closed world and fails on the stage world. -/
theorem bit_is_keyed :
    (NothingEscapes (Rec closedWorld) → NothingEscapes closedWorld) ∧
    ¬ (NothingEscapes (Rec (stageWorld 0)) → NothingEscapes (stageWorld 0)) :=
  (fun _ => closed_world_closes,
   fun h => stage_fails_closure 0 (h (record_has_least_escape (stageWorld 0) ())))

/-- THE RECORD'S SELF-RECURSION DOES NOT TRANSFER (the mirror of `record_self_recursion_does_not_transfer`):
    the record of the stage world is self-recursive and the stage world is not. -/
theorem record_self_recursion_does_not_transfer :
    SelfRecursive (Rec (stageWorld 0)) ∧ ¬ SelfRecursive (stageWorld 0) :=
  (record_is_self_recursive (stageWorld 0) (),
   fun h => (h (0 : Nat) 0 0).mp (stage_escapes 0))

/-- CLOSURE OF THE RECORD GIVES NO SELF-RECURSION OF THE MACHINE: no reading of the record decides the machine
    (the mirror of `closure_does_not_give_self_recursion`, `record_decides_nothing`). -/
theorem record_decides_nothing :
    NothingEscapes (Rec closedWorld) ∧ NothingEscapes (Rec (stageWorld 0)) ∧
    NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0) :=
  (record_has_least_escape closedWorld (), record_has_least_escape (stageWorld 0) (),
   closed_world_closes, stage_fails_closure 0)

/-! ## V · every route passes the bit; keyless premises force nothing -/

/-- EVERY ROUTE PASSES THE BIT (the mirror of `every_route_passes_the_bit`): any premise that yields the
    separation yields the transfer, so every proof of P ≠ NP, from any premise, delivers the one implication. -/
theorem every_route_passes_the_bit (M : Machine) (x₀ : M.Inst) (Q : Prop) (h : Q → Sep M) :
    Q → (NothingEscapes (Rec M) → NothingEscapes M) :=
  fun q => (sep_normal_form M x₀).mpr (h q)

def Keyless (P : Machine → Prop) : Prop := ∀ M, P M
def Forces (P : Machine → Prop) : Prop := ∀ M, P M → NothingEscapes M

/-- A KEYLESS PREMISE FORCES NOTHING (the mirror of `the_unconditional_route_is_blocked`): what every machine
    shares yields the transfer on none. -/
theorem keyless_forces_nothing (P : Machine → Prop) (hP : Keyless P) : ¬ Forces P :=
  fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))

/-- NO CLOSED PRINCIPLE DECIDES: a principle true of the record of every machine decides the machine on none
    (the mirror of `no_closed_principle_decides`). -/
theorem no_record_principle_decides (P : Machine → Prop) (hP : ∀ M, P (Rec M)) :
    ¬ ∀ M, P (Rec M) → NothingEscapes M :=
  fun h => stage_fails_closure 0 (h (stageWorld 0) (hP _))

/-! ## VI · the cargo: every examination enacts Stage 1, unconditionally -/

/-- STAGE 1 OF THE ROW: a pair examined and stopped. -/
structure Stage1 (M : Machine) where
  e : Examination M
  stopped : ¬ Clean M e.A e.c e.k e.x
  witnessed : ∃ x, ¬ Clean M e.A e.c e.k x

/-- THE EXAMINATION ENACTS STAGE 1: every stopping run supplies one (the mirror of
    `computation_enacts_stage1`); no universal appears in it. -/
def examination_enacts_stage1 (M : Machine) (e : Examination M) : Stage1 M :=
  (e, e.stops, (e.x, e.stops))

/-- Stage 1 is local: it decides its pair and no other (the universal is the act). -/
theorem stage1_is_local (M : Machine) (s : Stage1 M) :
    (∃ x, ¬ Clean M s.e.A s.e.c s.e.k x) ∧ ¬ Escapes M s.e.A s.e.c s.e.k :=
  (s.witnessed, fun he => s.stopped (he s.e.x))

/-! ## VII · RA read on computation, the one posit, and the seatless lock -/

/-- RA READ ON COMPUTATION: every decider of satisfiability actuates beyond every polynomial bound. -/
def RAcomp (M : Machine) : Prop := ∀ A c k, ¬ Escapes M A c k

theorem ra_comp_is_sep (M : Machine) : RAcomp M ↔ Sep M :=
  (fun h (A, c, k, he) => h A c k he, fun h A c k he => h (A, c, k, he))

/-- THE ONE POSIT IS THE BIT: RA read on computation is exactly the transfer. -/
theorem the_posit_is_the_bit (M : Machine) (x₀ : M.Inst) :
    RAcomp M ↔ (NothingEscapes (Rec M) → NothingEscapes M) :=
  (fun h => (sep_normal_form M x₀).mpr ((ra_comp_is_sep M).mp h),
   fun h => (ra_comp_is_sep M).mpr ((sep_normal_form M x₀).mp h))

/-- The seat of the row is vacant: complementation of worlds has no fixed point (PNP.VANISH, restated). -/
def comp (b : Bool) : Bool := !b
theorem locus_vanishes : ¬ ∃ b, comp b = b := fun (b, h) => bnot_ne b h

/-- SEATLESS: there is no fixed world to cross at, and the lock closes at the act on exactly the world the act
    names (the mirror of nothing in the Riemann master, where the seat is inhabited: the one difference). -/
theorem seatless_lock (M : Machine) :
    (¬ ∃ b, comp b = b) ∧ (Nonempty (LeastEscape M) ↔ Sep M) :=
  (locus_vanishes, (fun (L) => sep_of_closure M L.closure, fun hs => ((closure_of_sep M hs))))

/-! ## VIII · the capstone, nothing before the colon but the machine and an instance -/

/-- THE P VERSUS NP NORMAL FORM, COMPLETE. The record has least escape; the separation is the transfer; the
    transfer is the act; the machine is self-recursive exactly when separated; every route passes the bit; the one
    posit is the bit; the seat is vacant. Separation is not a conjunct: it occurs only inside biconditionals and
    inside the premise of a route. -/
theorem pnp_normal_form_complete (M : Machine) (x₀ : M.Inst) :
    NothingEscapes (Rec M) ∧
    ((NothingEscapes (Rec M) → NothingEscapes M) ↔ Sep M) ∧
    ((NothingEscapes (Rec M) → NothingEscapes M) ↔ Nonempty (LeastEscape M)) ∧
    (SelfRecursive M ↔ Sep M) ∧
    (∀ Q : Prop, (Q → Sep M) → Q → (NothingEscapes (Rec M) → NothingEscapes M)) ∧
    (RAcomp M ↔ (NothingEscapes (Rec M) → NothingEscapes M)) ∧
    (¬ ∃ b, comp b = b) :=
  (record_has_least_escape M x₀, sep_normal_form M x₀, bit_is_the_act M x₀, self_recursion_at_machine M,
   fun Q h => every_route_passes_the_bit M x₀ Q h, the_posit_is_the_bit M x₀, locus_vanishes)

/-- THE TWO ROWS, ONE SHAPE, TWO BITS. The Riemann bit and the separation bit are each a transfer from a record
    that has the property to the object; neither follows from the other's record, since the stage world's record
    has least escape while the stage world escapes. The bits are two. -/
theorem two_rows_two_bits :
    NothingEscapes (Rec (stageWorld 0)) ∧ ¬ NothingEscapes (stageWorld 0) :=
  (record_has_least_escape (stageWorld 0) (), stage_fails_closure 0)

end PNP.NORMALFORM

/-! # PART B · THE SAT INSTANCE, on no axiom -/

/-! ## IX · zero-axiom helpers: lists, arithmetic, halving -/
namespace PNP.ZA

theorem append_assoc {α : Type} : ∀ (l₁ l₂ l₃ : List α), (l₁ ++ l₂) ++ l₃ = l₁ ++ (l₂ ++ l₃)
  | [], _, _ => rfl
  | a :: l₁, l₂, l₃ => congrArg (List.cons a) (append_assoc l₁ l₂ l₃)

theorem length_append {α : Type} : ∀ (l₁ l₂ : List α), (l₁ ++ l₂).length = l₁.length + l₂.length
  | [], _ => (Nat.zero_add _).symm
  | _ :: l₁, l₂ => (congrArg Nat.succ (length_append l₁ l₂)).trans (Nat.succ_add _ _).symm

theorem any_append {α : Type} (p : α → Bool) : ∀ (l₁ l₂ : List α), (l₁ ++ l₂).any p = (l₁.any p || l₂.any p)
  | [], _ => rfl
  | a :: l₁, l₂ => by
    show (p a || (l₁ ++ l₂).any p) = ((p a || l₁.any p) || l₂.any p)
    rw [any_append p l₁ l₂]
    cases p a <;> rfl

theorem any_flatMap {α β : Type} (p : β → Bool) (g : α → List β) : ∀ l : List α,
    (l.flatMap g).any p = l.any (fun x => (g x).any p)
  | [] => rfl
  | a :: l => by
    show (g a ++ l.flatMap g).any p = ((g a).any p || l.any (fun x => (g x).any p))
```

```
    rw [any_append, any_flatMap p g l]

theorem any_true_of {α : Type} (p : α → Bool) : ∀ l : List α, l.any p =
    true → ∃ x, p x = true
  | [], h => Bool.noConfusion h
  | a :: l, h => by
    cases hpa : p a with
    | true => exact (a, hpa)
    | false =>
      have h' : l.any p = true := by
        have : (p a || l.any p) = true := h
        rw [hpa] at this
        exact this
      exact any_true_of p l h'

theorem or_true_left {a b : Bool} (h : a = true) : (a || b) = true := by
  rw [h]; rfl

theorem or_true_right {a b : Bool} (h : b = true) : (a || b) = true := by
  rw [h]; cases a <;> rfl

/-- `All P l`: P holds of every element, by recursion on the list. -/
def All {α : Type} (P : α → Prop) : List α → Prop
  | [] => True
  | x :: l => P x ∧ All P l

theorem All_append {α : Type} (P : α → Prop) : ∀ (l₁ l₂ : List α),
    All P (l₁ ++ l₂) → All P l₁ ∧ All P l₂
  | [], _, h => (trivial, h)
  | _ :: l₁, l₂, (hx, h) =>
    match All_append P l₁ l₂ h with
    | (h₁, h₂) => ((hx, h₁), h₂)

theorem mul_assoc' (n m : Nat) : ∀ k : Nat, n * m * k = n * (m * k)
  | 0 => rfl
  | k+1 => by
    show n * m * k + n * m = n * (m * k + m)
    rw [Nat.left_distrib, mul_assoc' n m k]

theorem add_mul' (a b k : Nat) : (a + b) * k = a * k + b * k := by
  rw [Nat.mul_comm, Nat.left_distrib, Nat.mul_comm k a, Nat.mul_comm k b]

theorem succ_ne_zero' (n : Nat) : n + 1 ≠ 0 := fun h => Nat.noConfusion h

/-- Halving and parity, by two-step recursion; no division operator is
    used anywhere in the file. -/
def div2 : Nat → Nat
  | 0 => 0
  | 1 => 0
  | n+2 => div2 n + 1

def mod2 : Nat → Bool
  | 0 => false
  | 1 => true
  | n+2 => mod2 n

/-- The digit and the half recompose the number. -/
theorem digit_spec : ∀ n : Nat, (if mod2 n then 1 else 0) + 2 * div2 n = n
  | 0 => rfl
  | 1 => rfl
  | n+2 => by
    show (if mod2 n then 1 else 0) + 2 * (div2 n + 1) = n + 2
    rw [Nat.mul_succ, ← Nat.add_assoc, digit_spec n]

theorem div2_le : ∀ n : Nat, div2 n ≤ n
  | 0 => Nat.le_refl 0
  | 1 => Nat.zero_le 1
  | n+2 => Nat.succ_le_succ (Nat.le_succ_of_le (div2_le n))

theorem div2_lt : ∀ n : Nat, 0 < n → div2 n < n
  | 0, h => absurd h (Nat.lt_irrefl 0)
  | 1, _ => Nat.zero_lt_succ 0
  | n+2, _ => Nat.succ_le_succ (Nat.succ_le_succ (div2_le n))

end PNP.ZA

/-! ## X · formulas, assignments, satisfiability by exhaustion -/
namespace PNP.SAT
open PNP.NORMALFORM PNP.ZA

abbrev Lit := Nat × Bool
abbrev Clause := List Lit
abbrev CNF := List Clause
abbrev Assignment := Nat → Bool
/-- A literal (n, b) is true under a when a n = b: b = true the positive
    literal, b = false the negative. -/
def evalLit (a : Assignment) (l : Lit) : Bool := a l.1 == l.2
def evalClause (a : Assignment) (c : Clause) : Bool := c.any (evalLit a)
def evalCNF (a : Assignment) (f : CNF) : Bool := f.all (evalClause a)
def varsClause (c : Clause) : List Nat := c.map (·.1)
def vars : CNF → List Nat
  | [] => []
  | c :: f => varsClause c ++ vars f
def extend (v : Nat) (b : Bool) (a : Assignment) : Assignment := fun n =>
    if n = v then b else a n
/-- Every assignment to a finite list of variables, the variables outside
    the list fixed false. -/
def assignments : List Nat → List Assignment
  | [] => [fun _ => false]
  | v :: vs => (assignments vs).flatMap (fun a => [extend v true a,
      extend v false a])
def restrict : List Nat → Assignment → Assignment
  | [], _ => fun _ => false
  | v :: vs, a => extend v (a v) (restrict vs a)
/-- THE DECISION PROCEDURE: satisfiability by exhaustion over the
    variables that occur. -/
def satB (f : CNF) : Bool := (assignments (vars f)).any (fun a => evalCNF
    a f)
/-- SATISFIABILITY, the mathematical predicate. -/
def Satisfiable (f : CNF) : Prop := ∃ a : Assignment, evalCNF a f = true
/-- The size of a formula: its clauses and its literals counted together.
  -/
def size : CNF → Nat
  | [] => 0
  | c :: f => c.length + 1 + size f

theorem extend_agrees (a : Assignment) (v : Nat) (r : Assignment) :
    ∀ vs' : List Nat, All (fun n => r n = a n) vs' → All (fun n => extend
      v (a v) r n = a n) vs'
  | [], _ => trivial
  | n :: vs', (hn, h) =>
    ((if hv : n = v then by
        show (if n = v then a v else r n) = a n
        rw [if_pos hv, hv]
      else by
        show (if n = v then a v else r n) = a n
        rw [if_neg hv]; exact hn),
     extend_agrees a v r vs' h)

/-- The restriction agrees with the assignment on every listed variable.
  -/
theorem restrict_agrees (a : Assignment) : ∀ vs : List Nat, All (fun n =>
    restrict vs a n = a n) vs
  | [] => trivial
  | v :: vs =>
    (show (if v = v then a v else restrict vs a v) = a v from if_pos rfl,
     extend_agrees a v (restrict vs a) vs (restrict_agrees a vs))

theorem evalClause_congr (a b : Assignment) :
    ∀ c : Clause, All (fun n => a n = b n) (varsClause c) → evalClause a c
      = evalClause b c
  | [], _ => rfl
  | l :: c, (hl, h) => by
    show (evalLit a l || evalClause a c) = (evalLit b l || evalClause b c)
    rw [evalClause_congr a b c h]
    show ((a l.1 == l.2) || evalClause b c) = ((b l.1 == l.2) ||
      evalClause b c)
    rw [hl]

theorem evalCNF_congr (a b : Assignment) :
    ∀ f : CNF, All (fun n => a n = b n) (vars f) → evalCNF a f = evalCNF b
      f
  | [], _ => rfl
  | c :: f, h =>
    match All_append _ (varsClause c) (vars f) h with
    | (hc, hf) => by
      show (evalClause a c && evalCNF a f) = (evalClause b c && evalCNF b
        f)
      rw [evalClause_congr a b c hc, evalCNF_congr a b f hf]

/-- SOUNDNESS: the procedure says satisfiable only when a satisfying
    assignment exists. -/
theorem satB_sound (f : CNF) (h : satB f = true) : Satisfiable f :=
  any_true_of (fun a => evalCNF a f) (assignments (vars f)) h

/-- The exhaustion finds any assignment through its restriction. -/
theorem any_assignments (q : Assignment → Bool) (a : Assignment) :
    ∀ vs : List Nat, q (restrict vs a) = true → (assignments vs).any q =
      true
  | [], h => or_true_left h
  | v :: vs, h => by
    show ((assignments vs).flatMap (fun b => [extend v true b, extend v
      false b])).any q = true
    rw [any_flatMap]
    apply any_assignments (fun b => [extend v true b, extend v false
      b].any q) a vs
    show (q (extend v true (restrict vs a)) || (q (extend v false
      (restrict vs a)) || false)) = true
    have h' : q (extend v (a v) (restrict vs a)) = true := h
    cases hav : a v with
    | true =>
      rw [hav] at h'
      exact or_true_left h'
    | false =>
      rw [hav] at h'
      exact or_true_right (or_true_left h')

/-- COMPLETENESS: every satisfying assignment is found, through its
    restriction to the variables that occur. -/
theorem satB_complete (f : CNF) (h : Satisfiable f) : satB f = true :=
  match h with
  | (a, ha) =>
    any_assignments (fun b => evalCNF b f) a (vars f)
      ((evalCNF_congr (restrict (vars f) a) a f (restrict_agrees a (vars
        f))).trans ha)

/-- THE PROCEDURE IS SATISFIABILITY. -/
theorem satB_iff (f : CNF) : satB f = true ↔ Satisfiable f := (satB_sound
    f, satB_complete f)

/-! ## XI · the SAT machine over any algorithm model -/

/-- THE SAT MACHINE. An algorithm model is a type of algorithms with an
    output and a step count on every
    formula; nothing about it is assumed. The instances are the formulas,
      the size the clause-and-literal count,
    satisfiability the decided predicate. -/
def SAT (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
    Machine :=
  (Alg, CNF, size, out, time, satB)

/-- The empty formula is an instance, and it is satisfiable. -/
```

```
theorem empty_satisfiable : satB [] = true := rfl

/-- SEPARATION OF SAT: no algorithm of the model decides satisfiability within any polynomial bound. -/
abbrev SepSAT (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) : Prop := Sep (SAT Alg out time)

/-- THE RECORD OF SAT HAS LEAST ESCAPE, unconditionally, in every algorithm model. -/
theorem sat_record_has_least_escape (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  NothingEscapes (Rec (SAT Alg out time)) :=
 record_has_least_escape (SAT Alg out time) []

/-- THE NORMAL FORM OF SAT: the separation is exactly the transfer of least escape from the record of SAT to SAT,
    and the antecedent is a theorem, so the separation is one implication. -/
theorem sat_normal_form (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) ↔ SepSAT Alg out time :=
 sep_normal_form (SAT Alg out time) []

/-- The transfer is the act on SAT. -/
theorem sat_bit_is_the_act (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) ↔
   Nonempty (LeastEscape (SAT Alg out time)) :=
 bit_is_the_act (SAT Alg out time) []

/-- SAT is self-recursive exactly when separated; its record is self-recursive with no premise. -/
theorem sat_self_recursion (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  (SelfRecursive (SAT Alg out time) ↔ SepSAT Alg out time) ∧
   SelfRecursive (Rec (SAT Alg out time)) :=
 (self_recursion_at_machine (SAT Alg out time), record_is_self_recursive (SAT Alg out time) [])

/-- EVERY ROUTE TO THE SEPARATION OF SAT PASSES THE BIT. -/
theorem sat_every_route_passes_the_bit (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat)
  (Q : Prop) (h : Q → SepSAT Alg out time) :
  Q → (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) :=
 every_route_passes_the_bit (SAT Alg out time) [] Q h

/-- RA READ ON COMPUTATION IS THE BIT, on SAT. -/
theorem sat_posit_is_the_bit (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  RAcomp (SAT Alg out time) ↔ (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) :=
 the_posit_is_the_bit (SAT Alg out time) []

/-- THE SAT NORMAL FORM, COMPLETE: the capstone of Part A on the SAT machine, in every algorithm model. -/
theorem sat_normal_form_complete (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat) :
  NothingEscapes (Rec (SAT Alg out time)) ∧
  ((NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) ↔ SepSAT Alg out time) ∧
  ((NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time)) ↔
   Nonempty (LeastEscape (SAT Alg out time))) ∧
  (SelfRecursive (SAT Alg out time) ↔ SepSAT Alg out time) ∧
  (∀ Q : Prop, (Q → SepSAT Alg out time) →
    Q → (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time))) ∧
  (RAcomp (SAT Alg out time) ↔ (NothingEscapes (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time))) ∧
  (¬ ∃ b, comp b = b) :=
 pnp_normal_form_complete (SAT Alg out time) []

/-! ## XII · the SAT-fixed worlds: the wall on the formulas themselves -/

/-- A closed world of SAT: every algorithm answers false in no time; nothing escapes, since the empty formula
    is satisfiable. Instances CNF, size, predicate satB, all fixed. -/
def satClosedWorld : Machine := SAT Nat (fun _ _ => false) (fun _ _ => 0)
/-- A stage world of SAT: algorithm n answers satisfiability itself in no time and escapes; the others answer
    false. The same instances, size and predicate. -/
def satStageWorld (n : Nat) : Machine :=
 SAT Nat (fun A f => if Nat.beq A n then satB f else false) (fun _ _ => 0)

theorem sat_closed_world_closes : NothingEscapes satClosedWorld :=
 fun _ _ _ => ([], fun ⟨h, _⟩ => Bool.noConfusion h)

theorem sat_stage_escapes (n : Nat) : Escapes (satStageWorld n) n 0 0 :=
 fun f => (show (if Nat.beq n n then satB f else false) = satB f by rw [beq_self n]; rfl, Nat.zero_le _)

theorem sat_stage_fails_closure (n : Nat) : ¬ NothingEscapes (satStageWorld n) :=
 fun h => sep_of_closure _ h ⟨n, 0, 0, sat_stage_escapes n⟩

/-- THE BIT IS KEYED ON SAT ITSELF: the transfer holds on the closed SAT world and fails on the stage SAT world,
    both deciding the same formulas with the same predicate. -/
theorem sat_bit_is_keyed :
  (NothingEscapes (Rec satClosedWorld) → NothingEscapes satClosedWorld) ∧
  ¬ (NothingEscapes (Rec (satStageWorld 0)) → NothingEscapes (satStageWorld 0)) :=
 (fun _ => sat_closed_world_closes,
  fun h => sat_stage_fails_closure 0 (h (record_has_least_escape (satStageWorld 0) [])))

/-- A property of algorithm models of SAT: it ranges over the machines `SAT Alg out time` and nothing else. -/
def SATKeyless (P : Machine → Prop) : Prop :=
 ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat), P (SAT Alg out time)
def SATForces (P : Machine → Prop) : Prop :=
 ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat), P (SAT Alg out time) → NothingEscapes (SAT Alg out time)

/-- THE KEYLESS WALL ON SAT: no property shared by every algorithm model of SAT forces the closure of any. -/
theorem sat_keyless_forces_nothing (P : Machine → Prop) (hP : SATKeyless P) : ¬ SATForces P :=
 fun h => sat_stage_fails_closure 0 (h Nat _ _ (hP Nat _ _))

/-- NO RECORD PRINCIPLE DECIDES SAT: a principle true of the record of every algorithm model of SAT decides
    none of them. -/
theorem sat_no_record_principle_decides (P : Machine → Prop)
  (hP : ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat), P (Rec (SAT Alg out time))) :
  ¬ ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
      P (Rec (SAT Alg out time)) → NothingEscapes (SAT Alg out time) :=
 fun h => sat_stage_fails_closure 0 (h Nat _ _ (hP Nat _ _))

/-- THE RECORD DECIDES NOTHING ON SAT: both SAT worlds' records close; one world closes and the other escapes. -/
theorem sat_record_decides_nothing :
  NothingEscapes (Rec satClosedWorld) ∧ NothingEscapes (Rec (satStageWorld 0)) ∧
  NothingEscapes satClosedWorld ∧ ¬ NothingEscapes (satStageWorld 0) :=
 ⟨record_has_least_escape satClosedWorld [], record_has_least_escape (satStageWorld 0) [],
  sat_closed_world_closes, sat_stage_fails_closure 0⟩

/-- THE PREMISE TRICHOTOMY ON SAT (classical): a premise on algorithm models of SAT either has a world where it
    holds and an algorithm escapes, or it forces the closure on every model it holds of; and if it forces, either
    the separation returns it on every model, so it is the separation, or on some separated model it fails, so it
    is strictly stronger than the separation. -/
theorem sat_premise_trichotomy (P : Machine → Prop) :
  (∃ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
      P (SAT Alg out time) ∧ ¬ NothingEscapes (SAT Alg out time)) ∨
  (SATForces P ∧
    ((∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
        SepSAT Alg out time → P (SAT Alg out time)) ∨
     (∃ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
        SepSAT Alg out time ∧ ¬ P (SAT Alg out time)))) :=
 Classical.byCases
  (fun hF : SATForces P =>
    Or.inr ⟨hF, Classical.byCases
      (fun hR : ∀ (Alg : Type) (out : Alg → CNF → Bool) (time : Alg → CNF → Nat),
          SepSAT Alg out time → P (SAT Alg out time) => Or.inl hR)
      (fun hR => Or.inr (Classical.byContradiction fun hne =>
        hR (fun Alg out time hs => Classical.byContradiction fun hp =>
          hne ⟨Alg, out, time, hs, hp⟩)))⟩)
  (fun hF => Or.inl (Classical.byContradiction fun hne =>
    hF (fun Alg out time hp => Classical.byContradiction fun hc => hne ⟨Alg, out, time, hp, hc⟩)))

end PNP.SAT

/-! # PART C · THE TURING MODEL: real machines deciding real SAT, on no axiom -/
namespace PNP.TM
open PNP.NORMALFORM PNP.SAT PNP.ZA

/-! ## XIII · the input alphabet and the binary encoding of formulas, with its decoder -/

/-- The input alphabet: two binary digits and a separator. -/
inductive Sym | zero | one | sep
 deriving DecidableEq

def symOfBool : Bool → Sym
 | true => Sym.one
 | false => Sym.zero

theorem symOfBool_ne_sep (b : Bool) : symOfBool b ≠ Sym.sep := by
 cases b <;> exact fun h => Sym.noConfusion h

/-- The binary digits of n, least significant first, computed with a fuel of halvings; `bits 0 = []`. -/
def bitsF : Nat → Nat → List Bool
 | 0, _ => []
 | fuel+1, n => if n = 0 then [] else mod2 n :: bitsF fuel (div2 n)

def bits (n : Nat) : List Bool := bitsF n n

def ofBits : List Bool → Nat
 | [] => 0
 | b :: l => (if b then 1 else 0) + 2 * ofBits l

theorem ofBits_bitsF : ∀ (fuel n : Nat), n ≤ fuel → ofBits (bitsF fuel n) = n
 | 0, n, h => by
   have : n = 0 := Nat.le_zero.mp h
   subst this; rfl
 | fuel+1, n, h =>
```

```
    if h0 : n = 0 then by
      show ofBits (if n = 0 then [] else mod2 n :: bitsF fuel (div2 n)) = n
      rw [if_pos h0, h0]; rfl
    else by
      show ofBits (if n = 0 then [] else mod2 n :: bitsF fuel (div2 n)) = n
      rw [if_neg h0]
      show (if mod2 n then 1 else 0) + 2 * ofBits (bitsF fuel (div2 n)) = n
      have hlt : div2 n < n := div2_lt n (Nat.pos_of_ne_zero h0)
      rw [ofBits_bitsF fuel (div2 n) (Nat.le_of_lt_succ
         (Nat.lt_of_lt_of_le hlt h))]
      exact digit_spec n

theorem ofBits_bits (n : Nat) : ofBits (bits n) = n := ofBits_bitsF n n
    (Nat.le_refl n)

theorem bits_succ (n : Nat) : bits (n+1) = mod2 (n+1) :: bitsF n (div2
    (n+1)) := by
  show (if n + 1 = 0 then [] else mod2 (n+1) :: bitsF n (div2 (n+1))) = _
  rw [if_neg (succ_ne_zero' n)]

/-- Read binary digits up to the first separator: the digits, and the
    remainder after the separator. -/
def parseBits : List Sym → Option (List Bool × List Sym)
  | [] => none
  | Sym.sep :: rest => some ([], rest)
  | Sym.zero :: rest => (parseBits rest).map (fun p => (false :: p.1,
      p.2))
  | Sym.one :: rest => (parseBits rest).map (fun p => (true :: p.1, p.2))

theorem parseBits_enc (rest : List Sym) : ∀ l : List Bool,
    parseBits (l.map symOfBool ++ Sym.sep :: rest) = some (l, rest)
  | [] => rfl
  | true :: l => by
    show (parseBits (l.map symOfBool ++ Sym.sep :: rest)).map (fun p =>
       (true :: p.1, p.2)) = some (true :: l, rest)
    exact congrArg (Option.map (fun p => (true :: p.1, p.2)))
       (parseBits_enc rest l)
  | false :: l => by
    show (parseBits (l.map symOfBool ++ Sym.sep :: rest)).map (fun p =>
       (false :: p.1, p.2)) = some (false :: l, rest)
    exact congrArg (Option.map (fun p => (false :: p.1, p.2)))
       (parseBits_enc rest l)

/-- A number n is written as the digits of n+1 (never empty) followed by
    a separator. -/
def encNat (n : Nat) : List Sym := (bits (n+1)).map symOfBool ++ [Sym.sep]
/-- A literal is its variable followed by its sign. -/
def encLit (l : Lit) : List Sym := encNat l.1 ++ [symOfBool l.2]
/-- A clause is its literals followed by a separator. -/
def encClause : Clause → List Sym
  | [] => [Sym.sep]
  | l :: c => encLit l ++ encClause c
/-- A formula is its clauses, one after the other. -/
def encode : CNF → List Sym
  | [] => []
  | c :: f => encClause c ++ encode f

def parseLit (w : List Sym) : Option (Lit × List Sym) :=
  match parseBits w with
  | none => none
  | some (ds, rest) =>
    match rest with
    | [] => none
    | Sym.sep :: _ => none
    | Sym.zero :: rest' => some ((ofBits ds - 1, false), rest')
    | Sym.one :: rest' => some ((ofBits ds - 1, true), rest')

theorem encLit_append (n : Nat) (b : Bool) (rest : List Sym) :
    encLit (n, b) ++ rest = (bits (n+1)).map symOfBool ++ Sym.sep ::
       (symOfBool b :: rest) := by
  show (((bits (n+1)).map symOfBool ++ [Sym.sep]) ++ [symOfBool b]) ++
     rest = _
  rw [append_assoc, append_assoc]; rfl

theorem parseLit_enc (l : Lit) (rest : List Sym) : parseLit (encLit l ++
    rest) = some (l, rest) := by
  obtain (n, b) := l
  rw [encLit_append]
  unfold parseLit
  rw [parseBits_enc]
  cases b with
  | true => exact congrArg (fun m => some ((m - 1, true), rest))
      (ofBits_bits (n+1))
  | false => exact congrArg (fun m => some ((m - 1, false), rest))
      (ofBits_bits (n+1))

theorem encLit_cons (l : Lit) : ∃ s tl, encLit l = s :: tl ∧ s ≠ Sym.sep
    := by
  obtain (n, b) := l
  show ∃ s tl, ((bits (n+1)).map symOfBool ++ [Sym.sep]) ++ [symOfBool b]
     = s :: tl ∧ s ≠ Sym.sep
  rw [bits_succ]
  exact ⟨_, _, rfl, symOfBool_ne_sep _⟩

/-- Read a clause: literals until a separator stands where a literal
    would start. Fuel bounds the recursion. -/
def parseClause : Nat → List Sym → Option (Clause × List Sym)
  | 0, _ => none
  | fuel+1, w =>
    match w with
    | Sym.sep :: rest => some ([], rest)
    | _ =>
      match parseLit w with
      | none => none
      | some (l, rest) =>
        match parseClause fuel rest with
        | none => none
        | some (c, rest') => some (l :: c, rest')

theorem encClause_cons_append (l : Lit) (c : Clause) (rest : List Sym) :
    encClause (l :: c) ++ rest = encLit l ++ (encClause c ++ rest) :=
  append_assoc _ _ _

theorem parseClause_enc : ∀ (c : Clause) (fuel : Nat) (rest : List Sym),
    c.length ≤ fuel →
    parseClause (fuel+1) (encClause c ++ rest) = some (c, rest)
  | [], _, _, _ => rfl
  | l :: c, fuel'+1, rest, h => by
    obtain ⟨s, tl, he, hs⟩ := encLit_cons l
    have hc : c.length ≤ fuel' := Nat.le_of_succ_le_succ h
    have hw : encClause (l :: c) ++ rest = s :: (tl ++ (encClause c ++
       rest)) := by
      rw [encClause_cons_append, he]; rfl
    rw [hw, parseClause]
    · rw [← List.cons_append, ← he, parseLit_enc]
      show (match parseClause (fuel'+1) (encClause c ++ rest) with
            | none => none
            | some (c', rest') => some (l :: c', rest')) = some (l :: c,
              rest)
      rw [parseClause_enc c fuel' rest hc]
    · intro r hr
      exact hs (List.cons.inj hr).1

theorem encLit_length_pos (l : Lit) : 1 ≤ (encLit l).length := by
  obtain ⟨s, tl, he, _⟩ := encLit_cons l
  rw [he]; exact Nat.succ_le_succ (Nat.zero_le _)

theorem encClause_length : ∀ c : Clause, c.length + 1 ≤ (encClause
    c).length
  | [] => Nat.le_refl 1
  | l :: c => by
    show c.length + 1 + 1 ≤ (encLit l ++ encClause c).length
    rw [length_append, Nat.add_comm (c.length + 1) 1]
    exact Nat.add_le_add (encLit_length_pos l) (encClause_length c)

theorem encClause_cons (c : Clause) : ∃ s tl, encClause c = s :: tl := by
  cases c with
  | nil => exact ⟨Sym.sep, [], rfl⟩
  | cons l c =>
    obtain ⟨s, tl, he, _⟩ := encLit_cons l
    exact ⟨s, tl ++ encClause c, by show encLit l ++ encClause c = _; rw
       [he]; rfl⟩

theorem encClause_ne_nil (c : Clause) (rest : List Sym) : encClause c ++
    rest ≠ [] := by
  obtain ⟨s, tl, he⟩ := encClause_cons c
  rw [he]; exact fun h => List.noConfusion h

/-- Read a formula: clauses until the string ends. Fuel bounds the
    recursion. -/
def parseCNF : Nat → List Sym → Option CNF
  | _, [] => some []
  | 0, _ :: _ => none
  | fuel+1, x :: w =>
    match parseClause (x :: w).length (x :: w) with
    | none => none
    | some (c, rest) =>
      match parseCNF fuel rest with
      | none => none
      | some f => some (c :: f)

/-- THE DECODER. -/
def decode (w : List Sym) : Option CNF := parseCNF w.length w

theorem encode_length : ∀ f : CNF, f.length ≤ (encode f).length
  | [] => Nat.le_refl 0
  | c :: f => by
    show f.length + 1 ≤ (encClause c ++ encode f).length
    rw [length_append, Nat.add_comm f.length 1]
    exact Nat.add_le_add (Nat.le_trans (Nat.le_add_left 1 c.length)
       (encClause_length c)) (encode_length f)

theorem parseCNF_enc : ∀ (f : CNF) (fuel : Nat), f.length ≤ fuel →
    parseCNF fuel (encode f) = some f
  | [], fuel, _ => by cases fuel <;> rfl
  | c :: f, fuel'+1, h => by
    have hf : f.length ≤ fuel' := Nat.le_of_succ_le_succ h
    show parseCNF (fuel'+1) (encClause c ++ encode f) = some (c :: f)
    cases hE : encClause c ++ encode f with
    | nil => exact absurd hE (encClause_ne_nil c _)
    | cons x w =>
      have hcw : c.length ≤ w.length := by
        have h1 : (encClause c ++ encode f).length = w.length + 1 :=
          congrArg List.length hE
        rw [length_append] at h1
        exact Nat.le_of_succ_le_succ (Nat.le_trans (encClause_length c)
          (Nat.le_trans (Nat.le_add_right _ _) (Nat.le_of_eq h1)))
      show (match parseClause (w.length + 1) (x :: w) with
            | none => none
            | some (c', rest) =>
              match parseCNF fuel' rest with
              | none => none
              | some f' => some (c' :: f')) = some (c :: f)
      rw [← hE, parseClause_enc c w.length (encode f) hcw]
      show (match parseCNF fuel' (encode f) with
            | none => none
            | some f' => some (c :: f')) = some (c :: f)
      rw [parseCNF_enc f fuel' hf]

/-- THE DECODER INVERTS THE ENCODING. -/
theorem decode_encode (f : CNF) : decode (encode f) = some f :=
  parseCNF_enc f _ (encode_length f)
```

```

/-- THE ENCODING IS INJECTIVE: distinct formulas have distinct strings. -/
theorem encode_injective (f g : CNF) (h : encode f = encode g) : f = g := by
  have hf := decode_encode f
  rw [h, decode_encode] at hf
  exact (Option.some.inj hf).symm

/-- SAT AS A LANGUAGE over the input alphabet: a string is in SAT when it decodes to a satisfiable formula. -/
def satLang (w : List Sym) : Bool :=
  match decode w with
  | some f => satB f
  | none => false

theorem satLang_encode (f : CNF) : satLang (encode f) = satB f := by
  unfold satLang
  rw [decode_encode]

theorem satLang_encode_iff (f : CNF) : satLang (encode f) = true ↔ Satisfiable f := by
  rw [satLang_encode]; exact satB_iff f

/-! ## XIV · Turing machines: finite control, one two-way tape, step semantics -/

inductive Dir | L | R

/-- A TURING MACHINE. States are `Fin q`; tape symbols are `Fin (s + 4)`: 0 the blank, 1 and 2 the binary
    digits, 3 the separator, and s further work symbols. The transition function is total; the machine halts in
    its accept or reject state. -/
structure TM where
  q : Nat
  s : Nat
  δ : Fin q → Fin (s + 4) → Fin q × Fin (s + 4) × Dir
  start : Fin q
  accept : Fin q
  reject : Fin q

theorem lt_add4 (i s : Nat) (h : i < 4) : i < s + 4 := Nat.lt_of_lt_of_le h (Nat.le_add_left 4 s)

def Sym.toTape (s : Nat) : Sym → Fin (s + 4)
  | Sym.zero => ⟨1, lt_add4 1 s (by decide)⟩
  | Sym.one => ⟨2, lt_add4 2 s (by decide)⟩
  | Sym.sep => ⟨3, lt_add4 3 s (by decide)⟩

structure Config (M : TM) where
  st : Fin M.q
  pos : Int
  tape : Int → Fin (M.s + 4)

def blank (M : TM) : Fin (M.s + 4) := ⟨0, lt_add4 0 M.s (by decide)⟩

/-- The input written from cell 0 rightward, blank elsewhere. -/
def initTape (M : TM) (w : List Sym) : Int → Fin (M.s + 4) :=
  fun i => if i < 0 then blank M else (w.map (Sym.toTape M.s)).getD i.toNat (blank M)

def init (M : TM) (w : List Sym) : Config M := ⟨M.start, 0, initTape M w⟩

def halted (M : TM) (c : Config M) : Prop := c.st = M.accept ∨ c.st = M.reject
instance (M : TM) (c : Config M) : Decidable (halted M c) := inferInstanceAs (Decidable (_ ∨ _))

def move : Dir → Int
  | Dir.L => -1
  | Dir.R => 1

/-- ONE STEP: read the scanned cell, write, move, change state; a halted configuration is fixed. -/
def step (M : TM) (c : Config M) : Config M :=
  if halted M c then c else
  let r := M.δ c.st (c.tape c.pos)
  ⟨r.1, c.pos + move r.2.2, fun i => if i = c.pos then r.2.1 else c.tape i⟩

/-- THE RUN: the configuration after t steps on input w. -/
def run (M : TM) (w : List Sym) : Nat → Config M
  | 0 => init M w
  | t+1 => step M (run M w t)

def HaltsAt (M : TM) (w : List Sym) (t : Nat) : Prop := halted M (run M w t)
instance (M : TM) (w : List Sym) (t : Nat) : Decidable (HaltsAt M w t) :=
  inferInstanceAs (Decidable (halted M (run M w t)))

/-- The verdict after t steps: in the accept state or not. -/
def verdictAt (M : TM) (w : List Sym) (t : Nat) : Bool := decide ((run M w t).st = M.accept)

theorem step_halted (M : TM) (c : Config M) (h : halted M c) : step M c = c := by
  unfold step; exact if_pos h

/-- A HALTED MACHINE STAYS WHERE IT IS: the run is constant from the halting step on. -/
theorem run_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt M w t) (hle : t ≤ u) :
    run M w u = run M w t := by
  induction hle with
  | refl => rfl
  | step _ ih => exact (congrArg (step M) ih).trans (step_halted M _ h)

/-- A HALTED MACHINE STAYS HALTED. -/
theorem haltsAt_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt M w t) (hle : t ≤ u) :
    HaltsAt M w u :=
  Eq.mpr (congrArg (halted M) (run_of_le h hle)) h

/-- THE VERDICT IS FIXED FROM THE HALTING STEP ON. -/
theorem verdict_of_le {M : TM} {w : List Sym} {t u : Nat} (h : HaltsAt M w t) (hle : t ≤ u) :
    verdictAt M w u = verdictAt M w t :=
  congrArg (fun c : Config M => decide (c.st = M.accept)) (run_of_le h hle)

/-! ## XV · the running time: the first halting step -/

/-- The first halting step at or below n, or n when none is found below it. -/
def firstHalt (M : TM) (w : List Sym) : Nat → Nat
  | 0 => 0
  | n+1 => if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1

theorem firstHalt_le (M : TM) (w : List Sym) : ∀ n, firstHalt M w n ≤ n
  | 0 => Nat.le_refl 0
  | n+1 =>
    if h : HaltsAt M w (firstHalt M w n) then by
      show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1) ≤ n+1
      rw [if_pos h]; exact Nat.le_succ_of_le (firstHalt_le M w n)
    else by
      show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1) ≤ n+1
      rw [if_neg h]; exact Nat.le_refl _

theorem firstHalt_halts (M : TM) (w : List Sym) : ∀ n, HaltsAt M w n → HaltsAt M w (firstHalt M w n)
  | 0, h => h
  | n+1, h =>
    if hn : HaltsAt M w (firstHalt M w n) then by
      show HaltsAt M w (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1)
      rw [if_pos hn]; exact hn
    else by
      show HaltsAt M w (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1)
      rw [if_neg hn]; exact h

theorem firstHalt_least (M : TM) (w : List Sym) :
    ∀ n t, HaltsAt M w t → t ≤ n → firstHalt M w n ≤ t
  | 0, t, _, _ => Nat.zero_le t
  | n+1, t, h, ht =>
    if hn : HaltsAt M w (firstHalt M w n) then by
      show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1) ≤ t
      rw [if_pos hn]
      exact match Nat.lt_or_ge t (n+1) with
        | Or.inl hlt => firstHalt_least M w n t h (Nat.le_of_lt_succ hlt)
        | Or.inr hge => Nat.le_trans (firstHalt_le M w n) (Nat.le_trans (Nat.le_succ n) hge)
    else by
      show (if HaltsAt M w (firstHalt M w n) then firstHalt M w n else n+1) ≤ t
      rw [if_neg hn]
      exact match Nat.lt_or_ge t (n+1) with
        | Or.inl hlt => absurd (firstHalt_halts M w n (haltsAt_of_le h (Nat.le_of_lt_succ hlt))) hn
        | Or.inr hge => hge

/-- A DECIDER: a Turing machine with a clock that certifies it halts on every input. The clock certifies and
    does not time: the running time below is the first halting step, whatever the clock says. -/
structure Decider where
  M : TM
  clock : List Sym → Nat
  halts : ∀ w : List Sym, HaltsAt M w (clock w)

/-- THE RUNNING TIME: the first halting step. -/
def time (D : Decider) (w : List Sym) : Nat := firstHalt D.M w (D.clock w)
/-- THE OUTPUT: the verdict at the first halting step. -/
def out (D : Decider) (w : List Sym) : Bool := verdictAt D.M w (time D w)

theorem time_halts (D : Decider) (w : List Sym) : HaltsAt D.M w (time D w) :=
  firstHalt_halts D.M w (D.clock w) (D.halts w)

theorem time_least (D : Decider) (w : List Sym) (t : Nat) (h : HaltsAt D.M w t) : time D w ≤ t :=
  match Nat.lt_or_ge t (D.clock w) with
  | Or.inl hlt => firstHalt_least D.M w (D.clock w) t h (Nat.le_of_lt hlt)
  | Or.inr hge => Nat.le_trans (firstHalt_le D.M w (D.clock w)) hge

theorem out_eq (D : Decider) (w : List Sym) (t : Nat) (h : HaltsAt D.M w t) : out D w = verdictAt D.M w t :=
  (verdict_of_le (time_halts D w) (time_least D w t h)).symm

/-! ## XVI · the SAT machine of the Turing model, and SAT ∉ P stated over all Turing machines -/

/-- THE SAT MACHINE OF THE TURING MODEL. Algorithms: deciders. Instances: strings over the input alphabet.
    Size: the length of the string. Output and time: the verdict and the step count of the run. Predicate: SAT. -/
def TMSAT : Machine := ⟨Decider, List Sym, List.length, out, time, satLang⟩
```

```
/-- A language over the input alphabet. -/
abbrev Lang := List Sym → Bool

/-- A LANGUAGE IS IN P: some Turing machine halts on every input within c·n^k + c steps, n the length of the
    input, in the accept state exactly on the strings of the language. -/
def InP (L : Lang) : Prop := ∃ (M : TM) (c k : Nat), ∀ w : List Sym,
  ∃ t, t ≤ bound c k w.length ∧ HaltsAt M w t ∧ verdictAt M w t = L w

/-- SAT ∈ P, the proposition, over every Turing machine. -/
def SATinP : Prop := InP satLang

theorem satInP_iff : SATinP ↔ InP satLang := Iff.rfl

/-- A LANGUAGE IS IN NP: some Turing machine, the verifier, accepts within c·n^k + c steps the string made of a
    binary certificate of length at most c·n^k + c, the separator, and the input, exactly when the input is in the
    language; n is the length of the input. The certificate contains no separator, so the first separator on the
    tape delimits it. -/
def InNP (L : Lang) : Prop := ∃ (M : TM) (c k : Nat), ∀ w : List Sym,
  (L w = true ↔ ∃ u : List Bool, u.length ≤ bound c k w.length ∧
    ∃ t, t ≤ bound c k w.length ∧ HaltsAt M (u.map symOfBool ++ Sym.sep :: w) t ∧
      verdictAt M (u.map symOfBool ++ Sym.sep :: w) t = true)

/-- P ≠ NP, the sentence: some language is in NP and not in P. -/
def P_ne_NP : Prop := ∃ L : Lang, InNP L ∧ ¬ InP L

/-- THE ONE-WAY BRIDGE, with SAT ∈ NP as its named hypothesis: SAT ∉ P gives P ≠ NP. -/
theorem p_ne_np_of_not_inP (hNP : InNP satLang) (h : ¬ SATinP) : P_ne_NP := (satLang, hNP, h)

/-- THE BRIDGE: the separation of the SAT machine is exactly SAT ∉ P over all Turing machines. -/
theorem sep_tmsat_iff : Sep TMSAT ↔ ¬ SATinP := by
  constructor
  · intro hsep (M, c, k, h)
    let D : Decider := (M, fun w => bound c k w.length,
      fun w => match h w with | (t, hle, ht, _) => haltsAt_of_le ht hle)
    apply hsep
    refine (D, c, k, fun w => ?_)
    obtain (t, hle, ht, hv) := h w
    exact ((out_eq D w t ht).trans hv, Nat.le_trans (time_least D w t ht) hle)
  · intro hnot (D, c, k, he)
    apply hnot
    refine (D.M, c, k, fun w => ?_)
    obtain (hout, htime) := he w
    exact (time D w, htime, time_halts D w, hout)

/-- Every natural-coefficient polynomial is dominated by a bound of the kernel's form. -/
def evalPoly : List Nat → Nat → Nat
  | [], _ => 0
  | a :: as, n => a + n * evalPoly as n

theorem evalPoly_le (as : List Nat) (n : Nat) (hn : 1 ≤ n) : evalPoly as n ≤ as.sum * n ^ as.length := by
  induction as with
  | nil => exact Nat.le_refl 0
  | cons a as ih =>
    show a + n * evalPoly as n ≤ (a + as.sum) * (n ^ as.length * n)
    have hpos : 1 ≤ n ^ as.length * n := Nat.mul_pos (Nat.pow_pos hn) hn
    have h1 : a ≤ a * (n ^ as.length * n) :=
      Nat.le_trans (Nat.le_of_eq (Nat.mul_one a).symm)
        (Nat.mul_le_mul_left a hpos)
    have h2 : n * evalPoly as n ≤ as.sum * (n ^ as.length * n) := by
      have := Nat.mul_le_mul_left n ih
      rw [Nat.mul_comm n (as.sum * n ^ as.length), mul_assoc'] at this
      exact this
    rw [add_mul']
    exact Nat.add_le_add h1 h2

theorem evalPoly_le_bound (as : List Nat) (n : Nat) : evalPoly as n ≤ bound as.sum as.length n :=
  match n with
  | 0 => by
    cases as with
    | nil => exact Nat.le_refl 0
    | cons a as =>
      show a + 0 * evalPoly as 0 ≤ (a + as.sum) * 0 ^ (as.length + 1) + (a + as.sum)
      rw [Nat.zero_mul, Nat.add_zero, Nat.pow_succ, Nat.mul_zero, Nat.mul_zero, Nat.zero_add]
      exact Nat.le_add_right a as.sum
  | n+1 => Nat.le_trans (evalPoly_le as (n+1) (Nat.succ_le_succ (Nat.zero_le n))) (Nat.le_add_right _ _)

/-- SAT ∈ P in the polynomial's own terms implies SAT ∈ P in the kernel's: nothing is lost in the bound. -/
theorem SATinP_of_poly (M : TM) (as : List Nat)
    (h : ∀ w : List Sym, ∃ t, t ≤ evalPoly as w.length ∧ HaltsAt M w t ∧
      verdictAt M w t = satLang w) :
    SATinP :=
  (M, as.sum, as.length, fun w => match h w with
    | (t, hle, ht, hv) => (t, Nat.le_trans hle (evalPoly_le_bound as w.length), ht, hv))

/-! ## XVII · the normal form on the Turing model -/

theorem tmsat_record_has_least_escape : NothingEscapes (Rec TMSAT) :=
  record_has_least_escape TMSAT []

/-- THE NORMAL FORM OF P VERSUS NP ON THE TURING MODEL: SAT ∉ P is exactly the transfer of least escape from the
    record of the SAT machine to the SAT machine. -/
theorem pnp_normal_form_turing :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ ¬ SATinP :=
  (sep_normal_form TMSAT []).trans sep_tmsat_iff

theorem tmsat_bit_is_the_act :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ Nonempty (LeastEscape TMSAT) :=
  bit_is_the_act TMSAT []

theorem tmsat_self_recursion : (SelfRecursive TMSAT ↔ ¬ SATinP) ∧ SelfRecursive (Rec TMSAT) :=
  ((self_recursion_at_machine TMSAT).trans sep_tmsat_iff, record_is_self_recursive TMSAT [])

/-- EVERY ROUTE TO SAT ∉ P PASSES THE BIT. -/
theorem tmsat_every_route_passes_the_bit (Q : Prop) (h : Q → ¬ SATinP) :
    Q → (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) :=
  every_route_passes_the_bit TMSAT [] Q (fun q => sep_tmsat_iff.mpr (h q))

theorem tmsat_posit_is_the_bit :
    RAcomp TMSAT ↔ (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) :=
  the_posit_is_the_bit TMSAT []

/-- THE BRIDGE TEST: the register's reading of the root on computation, every decider at every bound failing to
    escape, is exactly SAT ∉ P on the Turing model, on no axiom. -/
theorem tmsat_racomp_iff_not_inP : RAcomp TMSAT ↔ ¬ SATinP :=
  (ra_comp_is_sep TMSAT).trans sep_tmsat_iff

/-- EVERY EXIT IS THE POSIT: any proof of SAT ∉ P, by any route, including one built from facts about Turing
    machines, yields all three names of the one posit: the transfer, a least-escape certificate, and the root
    read on computation. -/
theorem every_exit_is_the_posit (h : ¬ SATinP) :
    (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ∧ Nonempty (LeastEscape TMSAT) ∧ RAcomp TMSAT :=
  have bit : NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT :=
    pnp_normal_form_turing.mpr h
  (bit, tmsat_bit_is_the_act.mp bit, tmsat_racomp_iff_not_inP.mpr h)

/-- THE POSIT IS EVERY EXIT: each name of the posit gives SAT ∉ P back, on no axiom. Nothing stands between the
    posit and the conclusion. -/
theorem the_posit_is_every_exit :
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) → ¬ SATinP) ∧
    (Nonempty (LeastEscape TMSAT) → ¬ SATinP) ∧
    (RAcomp TMSAT → ¬ SATinP) :=
  (fun bit => sep_tmsat_iff.mp (sep_from_the_cargo_loop TMSAT [] bit),
   fun (L) => sep_tmsat_iff.mp (sep_of_closure TMSAT L.closure),
   fun h => tmsat_racomp_iff_not_inP.mp h)

/-- THE CAPSTONE ON THE TURING MODEL. -/
theorem tmsat_normal_form_complete :
    NothingEscapes (Rec TMSAT) ∧
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ ¬ SATinP) ∧
    ((NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT) ↔ Nonempty (LeastEscape TMSAT)) ∧
    (SelfRecursive TMSAT ↔ ¬ SATinP) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → Q → (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT)) ∧
    (RAcomp TMSAT ↔ (NothingEscapes (Rec TMSAT) → NothingEscapes TMSAT)) ∧
    (¬ ∃ b, comp b = b) :=
  ⟨tmsat_record_has_least_escape, pnp_normal_form_turing, tmsat_bit_is_the_act, tmsat_self_recursion.1,
   tmsat_every_route_passes_the_bit, tmsat_posit_is_the_bit, locus_vanishes⟩

/-! ## XVIII · the coincidence form and the unique certificate -/

/-- Registration is idempotent: the record of the record is the record. -/
theorem rec_rec (M : Machine) : Rec (Rec M) = Rec M := rfl

/-- THE COINCIDENCE NORMAL FORM: a machine is separated exactly when its escape profile coincides with its
    record's, which is empty. -/
theorem coincidence_normal_form (M : Machine) (x₀ : M.Inst) :
    (∀ A c k, Escapes M A c k ↔ Escapes (Rec M) A c k) ↔ Sep M :=
  ⟨fun h (A, c, k, he) => bnot_ne (M.sat x₀) ((h A c k).mp he x₀).1,
   fun hs A c k => (fun he => absurd (A, c, k, he) hs, fun he => (bnot_ne (M.sat x₀) (he x₀).1).elim)⟩

theorem tmsat_coincidence : (∀ A c k, Escapes TMSAT A c k ↔ Escapes (Rec TMSAT) A c k) ↔ ¬ SATinP :=
  (coincidence_normal_form TMSAT []).trans sep_tmsat_iff

/-- THE CERTIFICATE IS UNIQUE: any two least-escape certificates of the SAT machine are the same object, and any
    two proofs of SAT ∉ P are the same proof, by proof irrelevance; so every route delivers one object. -/
theorem certificate_unique : (∀ L₁ L₂ : LeastEscape TMSAT, L₁ = L₂) ∧ (∀ h₁ h₂ : ¬ SATinP, h₁ = h₂) :=
  ⟨fun _ _ => rfl, fun _ _ => rfl⟩

/-! ## XIX · the defense on the Turing machine -/

/-- NOTHING PASSES AGAINST THE SEPARATION BUT AN ESCAPE (classical): a rejection of SAT ∉ P is a witness, a
    decider and a bound with every run clean. -/
theorem tmsat_rejection_is_a_witness : ¬ ¬ SATinP ↔ ∃ (D : Decider) (c k : Nat), Escapes TMSAT D c k :=
  ⟨fun hnn => Classical.byContradiction fun hne => hnn (sep_tmsat_iff.mp hne),
```

```
   fun he hn => sep_tmsat_iff.mpr hn he)

/-- EVERY PAIR LANDS (classical): every decider at every bound either is stopped by some string or escapes. -/
theorem tmsat_every_pair_lands (D : Decider) (c k : Nat) :
    (∃ w, ¬ Clean TMSAT D c k w) ∨ Escapes TMSAT D c k :=
  Classical.byCases (fun he : Escapes TMSAT D c k => Or.inr he)
    (fun hne => Or.inl (Classical.byContradiction fun hno =>
      hne (fun w => Classical.byContradiction fun hw => hno (w, hw))))

/-- FORCES OR HAS A TWIN (classical): every property of machines either forces the closure or holds on some
    machine that escapes. -/
theorem forces_or_has_a_twin (P : Machine → Prop) : Forces P ∨ ∃ M, P M ∧ ¬ NothingEscapes M :=
  Classical.byCases (fun hF : Forces P => Or.inl hF)
    (fun hF => Or.inr (Classical.byContradiction fun hne =>
      hF (fun M hp => Classical.byContradiction fun hc => hne (M, hp, hc))))

/-- An escape refutes the closure, on no axiom. -/
theorem escape_refutes (M : Machine) (A : M.Alg) (c k : Nat) (he : Escapes M A c k) : ¬ NothingEscapes M :=
  fun h => match h A c k with | ⟨x, hx⟩ => hx (he x)

/-- The act leaves no escape, on no axiom. -/
theorem act_leaves_no_escape (M : Machine) (h : NothingEscapes M) : ∀ A c k, ¬ Escapes M A c k :=
  fun A c k he => escape_refutes M A c k he h

/-- THE EXTENSION of a machine by one more algorithm, which answers the predicate itself in no time. -/
def extendM (M : Machine) : Machine :=
  ⟨Option M.Alg, M.Inst, M.size,
   fun A x => match A with | none => M.sat x | some a => M.out a x,
   fun A x => match A with | none => 0 | some a => M.time a x,
   M.sat⟩

/-- The added algorithm escapes at every bound. -/
theorem extend_escapes (M : Machine) (c k : Nat) : Escapes (extendM M) none c k :=
  fun _ => ⟨rfl, Nat.zero_le _⟩

/-- Every examination of the machine is an examination of its extension. -/
def examination_extend (M : Machine) (e : Examination M) : Examination (extendM M) :=
  ⟨some e.A, e.c, e.k, e.x, e.stops⟩

/-- Every examination of the machine is an examination of its record. -/
def examination_rec (M : Machine) (e : Examination M) : Examination (Rec M) :=
  ⟨e.A, e.c, e.k, e.x, fun hc => bnot_ne (M.sat e.x) hc.1⟩

/-- NO RECORD DECIDES: every stock of examinations of a machine with an instance, finite or infinite, is a stock
    of stops both on its extension, where an algorithm escapes, and on its record, where nothing escapes.
    Observed stops decide nothing about the machine. -/
theorem no_record_decides (M : Machine) (x₀ : M.Inst) (ι : Type) (stock : ι → Examination M) :
    (∃ A c k, Escapes (extendM M) A c k) ∧
    (∀ i, ¬ Clean (extendM M) (some (stock i).A) (stock i).c (stock i).k (stock i).x) ∧
    NothingEscapes (Rec M) ∧
    (∀ i, ¬ Clean (Rec M) (stock i).A (stock i).c (stock i).k (stock i).x) :=
  ⟨⟨none, 0, 0, extend_escapes M 0 0⟩, fun i => (examination_extend M (stock i)).stops,
   record_has_least_escape M x₀, fun i => (examination_rec M (stock i)).stops⟩

theorem tmsat_no_record_decides (ι : Type) (stock : ι → Examination TMSAT) :
    (∃ A c k, Escapes (extendM TMSAT) A c k) ∧
    (∀ i, ¬ Clean (extendM TMSAT) (some (stock i).A) (stock i).c (stock i).k (stock i).x) ∧
    NothingEscapes (Rec TMSAT) ∧
    (∀ i, ¬ Clean (Rec TMSAT) (stock i).A (stock i).c (stock i).k (stock i).x) :=
  no_record_decides TMSAT [] ι stock

/-! ## XX · executable checks: the model and the decoder compute, by `decide` -/

/-- A three-state machine that accepts exactly when the first input symbol is `one`. -/
def firstIsOne : TM :=
  { q := 3, s := 0,
    δ := fun _ a => if a.val = 2 then ⟨1, by decide⟩, a, Dir.R) else ⟨2, by decide⟩, a, Dir.R),
    start := ⟨0, by decide⟩, accept := ⟨1, by decide⟩, reject := ⟨2, by decide⟩ }

example : ¬ HaltsAt firstIsOne [Sym.one] 0 := by decide
example : HaltsAt firstIsOne [Sym.one] 1 := by decide
example : verdictAt firstIsOne [Sym.one] 1 = true := by decide
example : verdictAt firstIsOne [Sym.zero] 1 = false := by decide
example : HaltsAt firstIsOne [] 1 := by decide
example : verdictAt firstIsOne [Sym.one, Sym.zero] 5 = true := by decide
example : encode [[(0, true)]] = [Sym.one, Sym.sep, Sym.one, Sym.sep] := by decide
example : decode [Sym.one, Sym.sep, Sym.one, Sym.sep] = some [[(0, true)]] := by decide
example : decode [Sym.one, Sym.one] = none := by decide
example : satLang [] = true := by decide
example : satLang [Sym.sep] = false := by decide
example : satLang (encode [[(0, true)]]) = true := by decide
example : satLang (encode [[(0, true)], [(0, false)]]) = false := by decide
example : satLang (encode [[(1, true), (0, false)], [(1, false)]]) = true := by decide

end PNP.TM

/-! # PART D · THE FRONTIER, FORMAL ALONE: the black-box floor and the white-box contrast
    Carried from PNP_Frontier_II.lean of the register (namespaces BLACKBOX and MISSING), pure mathematics on no
    axiom; no physics and no root enter. -/
namespace PNP.BLACKBOX

def has : List Nat → Nat → Bool
  | [], _ => false
  | y :: ys, k => Nat.beq y k || has ys k

def drop : List Nat → Nat → List Nat
  | [], _ => []
  | y :: ys, k => cond (Nat.beq y k) (drop ys k) (y :: drop ys k)

theorem beq_self : ∀ n : Nat, Nat.beq n n = true
  | 0 => rfl
  | n + 1 => beq_self n

theorem drop_le : ∀ (l : List Nat) (k : Nat), (drop l k).length ≤ l.length
  | [], _ => Nat.le_refl 0
  | y :: ys, k => by
    unfold drop
    cases Nat.beq y k with
    | true => exact Nat.le_succ_of_le (drop_le ys k)
    | false => exact Nat.succ_le_succ (drop_le ys k)

theorem drop_len : ∀ (l : List Nat) (k : Nat), has l k = true → (drop l k).length + 1 ≤ l.length
  | [], _, h => Bool.noConfusion h
  | y :: ys, k, h => by
    unfold drop
    unfold has at h
    cases hy : Nat.beq y k with
    | true => exact Nat.succ_le_succ (drop_le ys k)
    | false =>
      rw [hy] at h
      exact Nat.succ_le_succ (drop_len ys k h)

theorem has_drop : ∀ (l : List Nat) (k x : Nat), Nat.beq x k = false →
    has l x = true → has (drop l k) x = true
  | [], _, _, _, h => Bool.noConfusion h
  | y :: ys, k, x, hxk, h => by
    unfold drop
    unfold has at h
    cases hy : Nat.beq y k with
    | true =>
      cases hyx : Nat.beq y x with
      | true =>
        have : x = k := (Nat.eq_of_beq_eq_true hyx).symm.trans
          (Nat.eq_of_beq_eq_true hy)
        rw [this, beq_self] at hxk; exact Bool.noConfusion hxk
      | false =>
        rw [hyx] at h
        exact has_drop ys k x hxk h
    | false =>
      show has (y :: drop ys k) x = true
      unfold has
      cases hyx : Nat.beq y x with
      | true => rfl
      | false =>
        rw [hyx] at h
        exact has_drop ys k x hxk h

theorem beq_false_of_lt (x K : Nat) (h : x < K) : Nat.beq x K = false := by
  cases hb : Nat.beq x K with
  | false => rfl
  | true =>
    have e : x = K := Nat.eq_of_beq_eq_true hb
    rw [e] at h; exact absurd h (Nat.lt_irrefl K)

/-- PIGEONHOLE, on no axiom: a list that contains every number below K has at least K entries. -/
theorem pigeon : ∀ (K : Nat) (l : List Nat), (∀ x, x < K → has l x = true) → K ≤ l.length
  | 0, _, _ => Nat.zero_le _
  | K + 1, l, h =>
    Nat.le_trans
      (Nat.succ_le_succ (pigeon K (drop l K) (fun x hx =>
        has_drop l K x (beq_false_of_lt x K hx) (h x (Nat.lt_succ_of_lt hx)))))
      (drop_len l K (h K (Nat.lt_succ_self K)))

/-- The run of a test-only procedure with a fixed list of queries: it answers true when some queried
    candidate is a witness. -/
def run (path : List Nat) (f : Nat → Bool) : Bool := path.any f

theorem run_eq_has : ∀ (l : List Nat) (x : Nat), run l (fun y => Nat.beq y x) = has l x
  | [], _ => rfl
  | y :: ys, x => by
    show (Nat.beq y x || run ys (fun y => Nat.beq y x)) = (Nat.beq y x || has ys x)
    rw [run_eq_has ys x]
```

```
/-- THE BLACK-BOX BOUND: a fixed list of tests that decides existence over K candidates for every candidate
    function tests at least K of them. -/
theorem black_box_bound (K : Nat) (path : List Nat)
    (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < K ∧ f x = true) :
    K ≤ path.length :=
  pigeon K path (fun x hx => by
    rw [← run_eq_has]
    exact (correct _).mpr (x, hx, beq_self x))

/-- NO SAVINGS: over 2^N candidates no such list is shorter than 2^N. -/
theorem black_box_no_savings (N : Nat) (path : List Nat)
    (correct : ∀ f : Nat → Bool, run path f = true ↔ ∃ x, x < 2 ^ N ∧ f x = true) :
    ¬ path.length < 2 ^ N :=
  Nat.not_lt_of_le (black_box_bound (2 ^ N) path correct)

end PNP.BLACKBOX

namespace PNP.MISSING

/-- The black-box adversary family of `black_box_bound`: the point functions. -/
def point (x : Nat) : Nat → Bool := fun y => Nat.beq y x

/-- The same family, handed over as descriptions. -/
structure Desc where
  secret : Nat

def evalD (d : Desc) : Nat → Bool := point d.secret

/-- The white-box reader: one step, read the description. -/
def reader (d : Desc) : Nat := d.secret
def readerTime (_ : Nat) : Nat := 1

/-- The black-box hard instances are white-box trivial. -/
theorem reader_solves : ∀ d : Desc, evalD d (reader d) = true :=
  fun d => PNP.BLACKBOX.beq_self d.secret

structure Model where
  Alg    : Type
  decides : Alg → Prop
  time   : Alg → Nat → Nat

def PolyBounded (t : Nat → Nat) : Prop := ∃ c k : Nat, ∀ n, t n ≤ c * n ^ k + c

/-- THE MISSING OBJECT: every algorithm that decides the problem from its description runs in
    super-polynomial time. -/
def WhiteBoxBound (M : Model) : Prop :=
  ∀ A : M.Alg, M.decides A → ¬ PolyBounded (M.time A)

theorem reader_poly : PolyBounded readerTime :=
  (1, 0, fun n => Nat.le_add_left 1 (1 * n ^ 0))

/-- A coherent model where descriptions reveal the witness: the object fails. -/
def revealing : Model := (Unit, fun _ => ∀ d : Desc, evalD d (reader d) = true, fun _ => readerTime)

theorem fails_when_descriptions_reveal : ¬ WhiteBoxBound revealing :=
  fun h => h () reader_solves reader_poly

/-- The object is keyed: it fails where descriptions reveal and holds wherever no algorithm decides. -/
theorem missing_object_is_keyed :
    (∃ M : Model, ¬ WhiteBoxBound M) ∧ (∀ M : Model, (∀ A, ¬ M.decides A) → WhiteBoxBound M) :=
  ⟨revealing, fails_when_descriptions_reveal⟩, fun _ h A hA => absurd hA (h A)⟩

end PNP.MISSING

/-! # PART E · THE RESIDUE, PRINTED AS A TYPE -/
namespace PNP.RESIDUE
open PNP.NORMALFORM

/-- THE RESIDUE. Everything above is unconditional. What remains of the separation is this one hypothesis, the
    transfer of least escape from the record to the machine, which is the act: given it, the separation follows
    on no axiom. It is printed here as the type of a hypothesis and is assumed nowhere in this file. -/
theorem separation_of_the_act (M : Machine) (x₀ : M.Inst)
    (bit : NothingEscapes (Rec M) → NothingEscapes M) : Sep M :=
  sep_from_the_cargo_loop M x₀ bit

/-- THE RESIDUE ON SAT, in every algorithm model: the one implication, and nothing else, stands between the
    theorems of this file and the separation of SAT. -/
theorem sat_separation_of_the_act (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat)
    (bit : NothingEscapes (Rec (PNP.SAT.SAT Alg out time)) → NothingEscapes (PNP.SAT.SAT Alg out time)) :
    PNP.SAT.SepSAT Alg out time :=
  separation_of_the_act (PNP.SAT.SAT Alg out time) [] bit

/-- THE RESIDUE ON THE TURING MODEL: given the one implication on the SAT machine of the Turing model, no Turing
    machine decides SAT within any polynomial bound. SAT ∉ P is this hypothesis and nothing else. -/
theorem sat_not_in_P_of_the_act
    (bit : NothingEscapes (Rec PNP.TM.TMSAT) → NothingEscapes PNP.TM.TMSAT) : ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (separation_of_the_act PNP.TM.TMSAT [] bit)

/-- THE RESIDUE AS THE SENTENCE: with SAT ∈ NP named as a hypothesis, the one implication gives P ≠ NP. -/
theorem p_ne_np_of_residue (hNP : PNP.TM.InNP PNP.TM.satLang)
    (bit : NothingEscapes (Rec PNP.TM.TMSAT) → NothingEscapes PNP.TM.TMSAT) : PNP.TM.P_ne_NP :=
  PNP.TM.p_ne_np_of_not_inP hNP (sat_not_in_P_of_the_act bit)

/-- THE WALL, restated on the residue: the residue cannot be supplied by any property shared by every machine
    (keyless_forces_nothing), nor by any principle true of every record (no_record_principle_decides), nor by
    any property shared by every algorithm model of SAT itself (sat_keyless_forces_nothing). What supplies it is
    keyed to the machine: the act. -/
theorem residue_is_keyed :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, (∀ M, P (Rec M)) → ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
    (∀ P : Machine → Prop, PNP.SAT.SATKeyless P → ¬ PNP.SAT.SATForces P) :=
  ⟨keyless_forces_nothing, no_record_principle_decides, PNP.SAT.sat_keyless_forces_nothing⟩

end PNP.RESIDUE

/-! ## cones, pinned: the build is silent, and any drift fails it -/
/-- info: 'PNP.NORMALFORM.record_has_least_escape' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_has_least_escape
/-- info: 'PNP.NORMALFORM.record_is_separated' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_is_separated
/-- info: 'PNP.NORMALFORM.sep_normal_form' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.sep_normal_form
/-- info: 'PNP.NORMALFORM.sep_from_the_cargo_loop' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.sep_from_the_cargo_loop
/-- info: 'PNP.NORMALFORM.bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.bit_is_the_act
/-- info: 'PNP.NORMALFORM.record_is_self_recursive' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_is_self_recursive
/-- info: 'PNP.NORMALFORM.self_recursion_at_machine' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.self_recursion_at_machine
/-- info: 'PNP.NORMALFORM.self_verifying_iff_true' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.self_verifying_iff_true
/-- info: 'PNP.NORMALFORM.bit_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.bit_is_keyed
/-- info: 'PNP.NORMALFORM.record_self_recursion_does_not_transfer' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_self_recursion_does_not_transfer
/-- info: 'PNP.NORMALFORM.record_decides_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.record_decides_nothing
/-- info: 'PNP.NORMALFORM.every_route_passes_the_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.every_route_passes_the_bit
/-- info: 'PNP.NORMALFORM.keyless_forces_nothing' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.keyless_forces_nothing
/-- info: 'PNP.NORMALFORM.no_record_principle_decides' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.no_record_principle_decides
/-- info: 'PNP.NORMALFORM.examination_enacts_stage1' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.examination_enacts_stage1
/-- info: 'PNP.NORMALFORM.stage1_is_local' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.stage1_is_local
/-- info: 'PNP.NORMALFORM.the_posit_is_the_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.the_posit_is_the_bit
/-- info: 'PNP.NORMALFORM.locus_vanishes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.locus_vanishes
/-- info: 'PNP.NORMALFORM.seatless_lock' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.seatless_lock
/-- info: 'PNP.NORMALFORM.pnp_normal_form_complete' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.NORMALFORM.pnp_normal_form_complete
/-- info: 'PNP.NORMALFORM.two_rows_two_bits' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.NORMALFORM.two_rows_two_bits
/-- info: 'PNP.SAT.satB_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.satB_iff
/-- info: 'PNP.SAT.empty_satisfiable' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.empty_satisfiable
/-- info: 'PNP.SAT.sat_record_has_least_escape' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_record_has_least_escape
/-- info: 'PNP.SAT.sat_normal_form' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_normal_form
/-- info: 'PNP.SAT.sat_bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_bit_is_the_act
/-- info: 'PNP.SAT.sat_self_recursion' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_self_recursion
/-- info: 'PNP.SAT.sat_every_route_passes_the_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
```

```
#guard_msgs in #print axioms PNP.SAT.sat_every_route_passes_the_bit
/-- info: 'PNP.SAT.sat_posit_is_the_bit' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_posit_is_the_bit
/-- info: 'PNP.SAT.sat_normal_form_complete' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_normal_form_complete
/-- info: 'PNP.SAT.sat_bit_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_bit_is_keyed
/-- info: 'PNP.SAT.sat_keyless_forces_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_keyless_forces_nothing
/-- info: 'PNP.SAT.sat_no_record_principle_decides' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_no_record_principle_decides
/-- info: 'PNP.SAT.sat_record_decides_nothing' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SAT.sat_record_decides_nothing
/-- info: 'PNP.SAT.sat_premise_trichotomy' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SAT.sat_premise_trichotomy
/-- info: 'PNP.TM.decode_encode' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.decode_encode
/-- info: 'PNP.TM.encode_injective' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.encode_injective
/-- info: 'PNP.TM.satLang_encode_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.satLang_encode_iff
/-- info: 'PNP.TM.haltsAt_of_le' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.haltsAt_of_le
/-- info: 'PNP.TM.verdict_of_le' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.verdict_of_le
/-- info: 'PNP.TM.time_halts' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.time_halts
/-- info: 'PNP.TM.time_least' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.time_least
/-- info: 'PNP.TM.sep_tmsat_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.sep_tmsat_iff
/-- info: 'PNP.TM.evalPoly_le_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.evalPoly_le_bound
/-- info: 'PNP.TM.SATinP_of_poly' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.SATinP_of_poly
/-- info: 'PNP.TM.p_ne_np_of_not_inP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.p_ne_np_of_not_inP
/-- info: 'PNP.TM.tmsat_record_has_least_escape' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_record_has_least_escape
/-- info: 'PNP.TM.pnp_normal_form_turing' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.pnp_normal_form_turing
/-- info: 'PNP.TM.tmsat_bit_is_the_act' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_bit_is_the_act
/-- info: 'PNP.TM.tmsat_self_recursion' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_self_recursion
/-- info: 'PNP.TM.tmsat_every_route_passes_the_bit' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_every_route_passes_the_bit
/-- info: 'PNP.TM.tmsat_posit_is_the_bit' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_posit_is_the_bit
/-- info: 'PNP.TM.tmsat_normal_form_complete' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_normal_form_complete
/-- info: 'PNP.TM.tmsat_racomp_iff_not_inP' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.TM.tmsat_racomp_iff_not_inP
/-- info: 'PNP.TM.every_exit_is_the_posit' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.every_exit_is_the_posit
/-- info: 'PNP.TM.the_posit_is_every_exit' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.TM.the_posit_is_every_exit
/-- info: 'PNP.TM.rec_rec' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.rec_rec
/-- info: 'PNP.TM.coincidence_normal_form' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.TM.coincidence_normal_form
/-- info: 'PNP.TM.tmsat_coincidence' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.tmsat_coincidence
/-- info: 'PNP.TM.certificate_unique' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.certificate_unique
/-- info: 'PNP.TM.tmsat_rejection_is_a_witness' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_rejection_is_a_witness
/-- info: 'PNP.TM.tmsat_every_pair_lands' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.tmsat_every_pair_lands
/-- info: 'PNP.TM.forces_or_has_a_twin' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TM.forces_or_has_a_twin
/-- info: 'PNP.TM.escape_refutes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.escape_refutes
/-- info: 'PNP.TM.act_leaves_no_escape' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.act_leaves_no_escape
/-- info: 'PNP.TM.extend_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.extend_escapes
/-- info: 'PNP.TM.no_record_decides' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.no_record_decides
/-- info: 'PNP.TM.tmsat_no_record_decides' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.TM.tmsat_no_record_decides
/-- info: 'PNP.TM.TMSAT' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.TMSAT
/-- info: 'PNP.TM.InP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.InP
/-- info: 'PNP.TM.InNP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.InNP
/-- info: 'PNP.TM.P_ne_NP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.P_ne_NP
/-- info: 'PNP.TM.SATinP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TM.SATinP
/-- info: 'PNP.BLACKBOX.pigeon' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.BLACKBOX.pigeon
/-- info: 'PNP.BLACKBOX.black_box_bound' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.BLACKBOX.black_box_bound
/-- info: 'PNP.BLACKBOX.black_box_no_savings' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.BLACKBOX.black_box_no_savings
/-- info: 'PNP.MISSING.reader_solves' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.reader_solves
/-- info: 'PNP.MISSING.reader_poly' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MISSING.reader_poly
/-- info: 'PNP.MISSING.fails_when_descriptions_reveal' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.MISSING.fails_when_descriptions_reveal
/-- info: 'PNP.MISSING.missing_object_is_keyed' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.MISSING.missing_object_is_keyed
/-- info: 'PNP.RESIDUE.separation_of_the_act' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.separation_of_the_act
/-- info: 'PNP.RESIDUE.sat_separation_of_the_act' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.sat_separation_of_the_act
/-- info: 'PNP.RESIDUE.sat_not_in_P_of_the_act' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.sat_not_in_P_of_the_act
/-- info: 'PNP.RESIDUE.p_ne_np_of_residue' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.RESIDUE.p_ne_np_of_residue
/-- info: 'PNP.RESIDUE.residue_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.RESIDUE.residue_is_keyed

/-! ============================================================
    ============================================
    PART F · THE SUPPLY (namespace PNP.SUPPLY). The bit of Part E,
      supplied by the act and delivered by its two
    kinds: the formal cargo, and the physical witness. Each kind is a
      type whose inhabitant is the supply; neither
    is assumed anywhere in this file. Every exit is the posit (Part C),
      so each kind is the one bit in its own
    register, and either delivers SAT ∉ P and, with SAT ∈ NP, P ≠ NP.
============================================================
    ============================================
    -/
namespace PNP.SUPPLY
open PNP.NORMALFORM

/-- THE FORMAL KIND, the cargo: the record's least escape carried to the
    Turing SAT machine. -/
def FormalCargo : Prop := NothingEscapes (Rec PNP.TM.TMSAT) →
    NothingEscapes PNP.TM.TMSAT

/-- THE PHYSICAL KIND, the witness: every run of a decider is a physical
    process that registers each step at
    the Landauer floor, so its heat, counted in floor units, is its step
      count; and no decider answers every
    formula correctly within a polynomial heat budget. The second field
      is the one bit read in the register of
    heat; it is a premise of the witness, carried as a field, never
      proved. -/
structure PhysicalWitness where
  floor : Nat
  floor_pos : 0 < floor
  heat : PNP.TM.Decider → List PNP.TM.Sym → Nat
  priced : ∀ D w, heat D w = PNP.TM.time D w
  beyond_every_budget : ∀ (D : PNP.TM.Decider) (c k : Nat), ∃ w : List
     PNP.TM.Sym,
    PNP.TM.out D w ≠ PNP.TM.satLang w ∨ bound c k w.length < heat D w

/-- The cargo arrives with no premise: the record of the Turing SAT
    machine has least escape. -/
theorem cargo_arrives : NothingEscapes (Rec PNP.TM.TMSAT) :=
    PNP.TM.tmsat_record_has_least_escape

/-- KIND 1 DELIVERS: the formal cargo gives SAT ∉ P in one step. -/
theorem formal_kind_delivers (h : FormalCargo) : ¬ PNP.TM.SATinP :=
    PNP.RESIDUE.sat_not_in_P_of_the_act h

/-- The physical witness closes the Turing SAT machine: every decider at
    every bound is stopped. -/
theorem witness_closes (W : PhysicalWitness) : NothingEscapes
    PNP.TM.TMSAT :=
  fun D c k => match W.beyond_every_budget D c k with
    | ⟨w, Or.inl hne⟩ => ⟨w, fun hc => hne hc.1⟩
    | ⟨w, Or.inr hlt⟩ => ⟨w, fun hc =>
        Nat.lt_irrefl _ (Nat.lt_of_lt_of_le hlt (W.priced D w ▸ hc.2))⟩

/-- KIND 2 DELIVERS: the physical witness gives SAT ∉ P. -/
theorem physical_kind_delivers (W : PhysicalWitness) : ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (sep_of_closure PNP.TM.TMSAT (witness_closes
    W))

/-- THE TWO KINDS ARE ONE BIT: the physical witness is a formal cargo,
    and the cargo is exactly SAT ∉ P. -/
theorem the_two_kinds_are_one_bit :
    (PhysicalWitness → FormalCargo) ∧ (FormalCargo ↔ ¬ PNP.TM.SATinP) :=
  ⟨fun W _ => witness_closes W, PNP.TM.pnp_normal_form_turing⟩

/-- P ≠ NP, DELIVERED TWICE: with SAT ∈ NP, either kind gives the
    sentence. -/
theorem p_ne_np_delivered_twice (hNP : PNP.TM.InNP PNP.TM.satLang) :
    (FormalCargo → PNP.TM.P_ne_NP) ∧ (PhysicalWitness → PNP.TM.P_ne_NP) :=
```

```
  (fun h => PNP.TM.p_ne_np_of_not_inP hNP (formal_kind_delivers h),
   fun W => PNP.TM.p_ne_np_of_not_inP hNP (physical_kind_delivers W))

/-- THE ONE REFUTER: an escape, a decider and a bound with every run clean, refutes both kinds at once. -/
theorem the_one_refuter (e : ∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) :
    ¬ FormalCargo ∧ ¬ Nonempty PhysicalWitness :=
  have hs : ¬ Sep PNP.TM.TMSAT := fun hsep => hsep e
  (fun h => hs (sep_of_closure _ (h cargo_arrives)),
   fun (W) => hs (sep_of_closure _ (witness_closes W)))

/-- THE SUPPLY, CLOSED. Nothing before the colon: the cargo arrives free; each kind delivers; the two kinds are
    one bit; the bit is SAT ∉ P; with SAT ∈ NP each kind gives P ≠ NP; and one escape refutes both. -/
theorem the_supply_closed :
    NothingEscapes (Rec PNP.TM.TMSAT) ∧
    (FormalCargo → ¬ PNP.TM.SATinP) ∧ (PhysicalWitness → ¬ PNP.TM.SATinP) ∧
    (PhysicalWitness → FormalCargo) ∧ (FormalCargo ↔ ¬ PNP.TM.SATinP) ∧
    (PNP.TM.InNP PNP.TM.satLang → (FormalCargo → PNP.TM.P_ne_NP) ∧ (PhysicalWitness → PNP.TM.P_ne_NP)) ∧
    ((∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) →
      ¬ FormalCargo ∧ ¬ Nonempty PhysicalWitness) :=
  (cargo_arrives, formal_kind_delivers, physical_kind_delivers,
    the_two_kinds_are_one_bit.1,
   the_two_kinds_are_one_bit.2, p_ne_np_delivered_twice, the_one_refuter)

end PNP.SUPPLY

/-- info: 'PNP.SUPPLY.cargo_arrives' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.cargo_arrives
/-- info: 'PNP.SUPPLY.formal_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.formal_kind_delivers
/-- info: 'PNP.SUPPLY.witness_closes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.witness_closes
/-- info: 'PNP.SUPPLY.physical_kind_delivers' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.physical_kind_delivers
/-- info: 'PNP.SUPPLY.the_two_kinds_are_one_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SUPPLY.the_two_kinds_are_one_bit
/-- info: 'PNP.SUPPLY.p_ne_np_delivered_twice' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.p_ne_np_delivered_twice
/-- info: 'PNP.SUPPLY.the_one_refuter' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SUPPLY.the_one_refuter
/-- info: 'PNP.SUPPLY.the_supply_closed' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SUPPLY.the_supply_closed

/-! ============================================================================================================
    PART G · THE CLOSURE OF THE QUESTION (namespace PNP.CLOSURE). Why the question is closed and not open: with the two
    halves of Cook and Levin cited as hypotheses, the Millennium sentence P ≠ NP and the bit are one proposition, so the
    question has no content but the bit; across the algorithm models of SAT the bit has no uniform answer, true in one
    world and false in another of the same predicate; every uniform route to it is barred; the supply gives the sentence
    by either kind; and the one refuter refutes the sentence itself. Nothing here derives the bit. It proves that the
    question is one bit, that the bit is keyed, and that what is open is a value and not a form.
============================================================================================================ -/
namespace PNP.CLOSURE
open PNP.NORMALFORM PNP.SUPPLY

/-- THE TRANSFER, as a predicate on machines: the bit of a machine is the transfer of least escape from its record. -/
def Transfer (M : Machine) : Prop := NothingEscapes (Rec M) → NothingEscapes M

/-- SAT IS NP-HARD, the second half of Cook and Levin, stated as the kernel can state it: if SAT is in P then P = NP. -/
def SATHard : Prop := PNP.TM.SATinP → ¬ PNP.TM.P_ne_NP

/-- THE MILLENNIUM SENTENCE IS THE BIT: with SAT ∈ NP and SAT NP-hard cited, P ≠ NP and the transfer on the Turing SAT
    machine are one proposition. The question has no content beyond the bit, in either direction. -/
theorem millennium_sentence_is_the_bit (hNP : PNP.TM.InNP PNP.TM.satLang) (hHard : SATHard) :
    PNP.TM.P_ne_NP ↔ FormalCargo :=
  (fun hP => PNP.TM.pnp_normal_form_turing.mpr (fun hs => hHard hs hP),
   fun h => PNP.TM.p_ne_np_of_not_inP hNP
     (PNP.TM.pnp_normal_form_turing.mp h))

/-- THE DENIAL IS THE BIT'S DENIAL: P = NP, as the kernel states it, is exactly the failure of the transfer. -/
theorem denial_is_the_bits_denial (hNP : PNP.TM.InNP PNP.TM.satLang) (hHard : SATHard) :
    ¬ PNP.TM.P_ne_NP ↔ ¬ FormalCargo :=
  (fun hn h => hn ((millennium_sentence_is_the_bit hNP hHard).mpr h),
   fun hn h => hn ((millennium_sentence_is_the_bit hNP hHard).mp h))

/-- THE QUESTION HAS NO UNIFORM ANSWER: across the algorithm models of SAT, on the same formulas with the same
    predicate, the bit is neither always true nor always false. The closed world has it; the stage world lacks it.
    What every model shares therefore answers the question in no model. -/
theorem no_uniform_answer :
    ¬ (∀ (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat),
        Transfer (PNP.SAT.SAT Alg out time)) ∧
    ¬ (∀ (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat),
        ¬ Transfer (PNP.SAT.SAT Alg out time)) :=
  (fun h => PNP.SAT.sat_stage_fails_closure 0
    (h Nat (fun A f => if Nat.beq A 0 then PNP.SAT.satB f else false)
      (fun _ _ => 0)
      (record_has_least_escape _ [])),
   fun h => h Nat (fun _ _ => false) (fun _ _ => 0) (fun _ => PNP.SAT.sat_closed_world_closes))

/-- THE ONE REFUTER REFUTES THE SENTENCE: a polynomial-time Turing machine for SAT, with SAT NP-hard cited, denies
    P ≠ NP itself, and not only the supply. -/
theorem refuter_denies_the_sentence (hHard : SATHard)
    (e : ∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) : ¬ PNP.TM.P_ne_NP :=
  fun hP => PNP.TM.sep_tmsat_iff.mpr (fun hs => hHard hs hP) e

/-- THE QUESTION, CLOSED TO ONE BIT. With the two halves of Cook and Levin cited: the sentence is the bit and its
    denial is the bit's denial; the bit has no uniform answer across the models of SAT; no keyless premise and no
    SAT-uniform premise forces it; either kind of the supply gives the sentence; and the one refuter denies the
    sentence. What is open is the value of one implication; the form of the question is closed. -/
theorem question_closed_to_one_bit (hNP : PNP.TM.InNP PNP.TM.satLang) (hHard : SATHard) :
    (PNP.TM.P_ne_NP ↔ FormalCargo) ∧ (¬ PNP.TM.P_ne_NP ↔ ¬ FormalCargo) ∧
    (¬ (∀ (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat),
        Transfer (PNP.SAT.SAT Alg out time)) ∧
     ¬ (∀ (Alg : Type) (out : Alg → PNP.SAT.CNF → Bool) (time : Alg → PNP.SAT.CNF → Nat),
        ¬ Transfer (PNP.SAT.SAT Alg out time))) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, PNP.SAT.SATKeyless P → ¬ PNP.SAT.SATForces P) ∧
    (FormalCargo → PNP.TM.P_ne_NP) ∧ (PhysicalWitness → PNP.TM.P_ne_NP) ∧
    ((∃ (D : PNP.TM.Decider) (c k : Nat), Escapes PNP.TM.TMSAT D c k) → ¬ PNP.TM.P_ne_NP) :=
  (millennium_sentence_is_the_bit hNP hHard, denial_is_the_bits_denial hNP hHard, no_uniform_answer,
   keyless_forces_nothing, PNP.SAT.sat_keyless_forces_nothing,
   (p_ne_np_delivered_twice hNP).1, (p_ne_np_delivered_twice hNP).2,
    refuter_denies_the_sentence hHard)

end PNP.CLOSURE

/-- info: 'PNP.CLOSURE.millennium_sentence_is_the_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CLOSURE.millennium_sentence_is_the_bit
/-- info: 'PNP.CLOSURE.denial_is_the_bits_denial' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CLOSURE.denial_is_the_bits_denial
/-- info: 'PNP.CLOSURE.no_uniform_answer' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CLOSURE.no_uniform_answer
/-- info: 'PNP.CLOSURE.refuter_denies_the_sentence' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CLOSURE.refuter_denies_the_sentence
/-- info: 'PNP.CLOSURE.question_closed_to_one_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CLOSURE.question_closed_to_one_bit
/-
  FTOE_Fertile_Logos_Kernel_v2.lean · The Formal Theory of Everything (fTOE) under Trisductive Logos Fertility.
  Core Lean 4.19.0: no import, no axiom declared, no sorry, no native_decide.
  Every theorem in this file rests on no axiom at all (empty cone `[]`), pinned by `#guard_msgs`.

  I   · 1 and 3 Colocated Before 2 (OFL.1–OFL.4 & TOE_Zero Part III):
        In H = E_+ (1D Stillness, Fix(conj)) ⊕ E_- (3D Motion, Im H), two orthogonal axes i, j
        beget the third axis k = i·j (`ofl_fertility_begets_third`), proving 3 before 2
        (`three_before_two_closure`). The 3-axis Full Return (i·j)·k = -1 lands on Fix(conj)
        with zero chiral defect, whereas the 2-axis collapse (i·i)·k = -k has scalar part 0 (`ofl_two_contacts`).
  II  · The Manifest Dual (2) Born After 3 (`two_born_from_three_lock`):
        Over GF(2)^3, three orthogonal axes lock a unique point (`solCount3 = 1`), whereas dropping
        the third axis generates the binary 2-fibre (`solCount2 = 2`).
  III · Subsumption of Seat Theory & Unconditional Canonization of Least Erasure:
        Upgrades `FertileLogosSeat` with `ground_defect_zero`, instantiates `quatSeat : FertileLogosSeat Q`
        on cone `[]`, proves `record_locus_is_made_zero` and `return_locus_is_made_zero` unconditionally,
        and canonizes `LeastErasure`, the 1-Bit Cargo Transfer, and `Value` as unconditional foundational
        theorems on the 3-axis Return Locus (`ftoe_unconditional_return_least_erasure`,
        `ftoe_unconditional_return_one_bit`) and on every `CompletedFertileSeat` on cone `[]`.
  IV  · Subsumption of P ≠ NP Normal Form (`PNP.NORMALFORM`) under fTOE (`FertileMachine`, Cone `[]`):
        Proves `recFertileMachine` unconditionally on `Rec M` and canonizes the 1-bit separation transfer,
        `LeastEscape`, `Sep`, and `SelfGrounding (Sep M)` on `FertileMachine` on cone `[]`.
```

```
-/

namespace FTOE

/-! ## 0 · Zero-Axiom Arithmetic & Integer Cancellation Engine (No `[propext]` Leak) -/

theorem add_right_eq_zero_za (a : Nat) : ∀ b : Nat, a + b ≤ a → b = 0
  | 0, _ => rfl
  | d + 1, h =>
    have hlt : a < a + (d + 1) := Nat.succ_le_succ (Nat.le_add_right a d)
    False.elim (Nat.lt_irrefl a (Nat.lt_of_lt_of_le hlt h))

theorem add_eq_zero_right_za (a : Nat) : ∀ b : Nat, a + b = 0 → b = 0
  | 0, _ => rfl
  | _ + 1, h => Nat.noConfusion h

theorem neg_neg_free : ∀ d : Int, - -d = d
  | Int.ofNat 0 => rfl
  | Int.ofNat (_ + 1) => rfl
  | Int.negSucc _ => rfl

theorem neg_eq_self_free : ∀ d : Int, -d = d → d = 0
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Int.noConfusion h
  | Int.negSucc _, h => Int.noConfusion h

theorem natAbs_zero_free : ∀ d : Int, d.natAbs = 0 → d = 0
  | Int.ofNat 0, _ => rfl
  | Int.ofNat (_ + 1), h => Nat.noConfusion h
  | Int.negSucc _, h => Nat.noConfusion h

/-! ## I · 1 and 3 Colocated Before 2: The Orthogonal Fertile Logos Algebra (OFL.1–OFL.4) -/

structure Q where
  (a b c d : Int)
  deriving DecidableEq

def Q.mul (x y : Q) : Q :=
  (x.a * y.a - x.b * y.b - x.c * y.c - x.d * y.d,
   x.a * y.b + x.b * y.a + x.c * y.d - x.d * y.c,
   x.a * y.c - x.b * y.d + x.c * y.a + x.d * y.b,
   x.a * y.d + x.b * y.c - x.c * y.b + x.d * y.a)

def Q.conj (x : Q) : Q := (x.a, -x.b, -x.c, -x.d)

/-- 1D Stillness (Ground projection onto Fix(conj) = R). -/
def Q.reg (x : Q) : Q := (x.a, 0, 0, 0)

/-- 3D Motion (Chiral Residence projection onto Im H). -/
def Q.chiral (x : Q) : Q := (0, x.b, x.c, x.d)

def Q.add (x y : Q) : Q := (x.a + y.a, x.b + y.b, x.c + y.c, x.d + y.d)

def Q.groundNorm (x : Q) : Nat := x.a.natAbs
def Q.chiralDefect (x : Q) : Nat := (x.b.natAbs + x.c.natAbs) + x.d.natAbs
def Q.totalNorm (x : Q) : Nat := Q.groundNorm x + Q.chiralDefect x

def qone : Q := (1, 0, 0, 0)
def qneg : Q := (-1, 0, 0, 0)
def qi   : Q := (0, 1, 0, 0)
def qj   : Q := (0, 0, 1, 0)
def qk   : Q := (0, 0, 0, 1)

theorem ofl_fertility_begets_third :
    qi.mul qj = qk ∧ qj.mul qi = (0, 0, 0, -1) ∧ (qi.mul qj).a = 0 := by
      decide

theorem three_before_two_closure :
    (qi.mul qj).d = 1 ∧ qone.d = 0 ∧ qi.d = 0 ∧ qj.d = 0 := by decide

theorem ofl_two_contacts :
    (qi.mul qj).mul qk = qneg ∧ ((qi.mul qi).mul qk).a = 0 ∧
    Q.chiralDefect ((qi.mul qj).mul qk) = 0 ∧ Q.chiralDefect ((qi.mul
      qi).mul qk) = 1 := by decide

def chart : List Int := [-2, -1, 0, 1, 2]
def quats : List Q :=
  chart.flatMap fun a => chart.flatMap fun b => chart.flatMap fun c =>
    chart.map fun d => (a, b, c, d)

set_option maxRecDepth 100000 in
theorem one_and_three_colocated_on_chart :
    quats.all (fun q =>
      Q.add (Q.reg q) (Q.chiral q) == q &&
      (Q.reg q).conj == Q.reg q &&
      (Q.chiral q).conj == (0, -q.b, -q.c, -q.d)) = true := by decide

/-! ## II · The Manifest Dual (2) Born as the Residue of the 3-Axis Lock -/

def bitAt : Nat → Nat → Nat
  | x, 0 => x % 2
  | x, k + 1 => bitAt (x / 2) k

def dot3 (r x : Nat) : Nat := (bitAt r 2 * bitAt x 2 + bitAt r 1 * bitAt x
    1 + bitAt r 0 * bitAt x 0) % 2

def cube8 : List Nat := [0, 1, 2, 3, 4, 5, 6, 7]

def solCount3 (r1 t1 r2 t2 r3 t3 : Nat) : Nat :=
  (cube8.filter fun x => dot3 r1 x == t1 && dot3 r2 x == t2 && dot3 r3 x
    == t3).length

def solCount2 (r1 t1 r2 t2 : Nat) : Nat :=
  (cube8.filter fun x => dot3 r1 x == t1 && dot3 r2 x == t2).length

theorem two_born_from_three_lock :
    (cube8.all fun t =>
      solCount3 1 (bitAt t 0) 2 (bitAt t 1) 4 (bitAt t 2) == 1 &&
      solCount2 1 (bitAt t 0) 2 (bitAt t 1) == 2) = true := by decide

/-! ## III · fTOE in the Formal Domain: Canonizing Least Erasure & the 1-Bit Transfer -/

structure SelfGrounding (R : Prop) where
  Act       : Type
  anAct     : Act
  instances : Act → R

theorem seated_undeniable {R : Prop} (G : SelfGrounding R) : R :=
  G.instances G.anAct
theorem denial_reenacts_root {R : Prop} (G : SelfGrounding R) (denial :
  G.Act) : R := G.instances denial

/-- THE fTOE SEAT (`FertileLogosSeat`), upgraded with
    `ground_defect_zero`:
    Every object x has a 1D Ground norm `ground_norm (ρ x)` and a 3D
      chiral motion defect
    `chiral_defect x` co-located in `total_norm x`, with the ground
      projection carrying zero chiral defect. -/
structure FertileLogosSeat (X : Type) where
  σ                : X → X
  ρ                : X → X
  total_norm       : X → Nat
  ground_norm      : X → Nat
  chiral_defect    : X → Nat
  invol            : ∀ x, σ (σ x) = x
  lands            : ∀ x, σ (ρ x) = ρ x
  keeps            : ∀ x, σ x = x → ρ x = x
  forgets          : ∀ x, ρ (σ x) = ρ x
  colocated_1_3    : ∀ x, total_norm x = ground_norm (ρ x) +
    chiral_defect x
  ground_defect_zero : ∀ x, chiral_defect (ρ x) = 0
  motion_faithful  : ∀ x, chiral_defect x = 0 → ρ x = x

def LeastErasure {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) :
  Prop :=
  ∀ x, Z x → F.ρ x = x

def Value {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) : Prop :=
  ∀ x, Z x → F.σ x = x

def RecLocus {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) (y : X) :
  Prop :=
  ∃ x, Z x ∧ F.ρ x = y

def MadeZeroLocus {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) :
  Prop :=
  ∀ x, Z x → F.total_norm x ≤ F.ground_norm (F.ρ x)

theorem rho_idempotent {X : Type} (F : FertileLogosSeat X) (x : X) : F.ρ
  (F.ρ x) = F.ρ x :=
  F.keeps (F.ρ x) (F.lands x)

theorem defect_vanishes_iff_fixed {X : Type} (F : FertileLogosSeat X) (x
  : X) :
    F.chiral_defect x = 0 ↔ F.ρ x = x :=
  (F.motion_faithful x, fun h => h ▸ F.ground_defect_zero x)

theorem record_locus_has_least_erasure {X : Type} (F : FertileLogosSeat
  X) (Z : X → Prop) :
    LeastErasure F (RecLocus F Z) :=
  fun _ (x, _, hxy) => Eq.trans (congrArg F.ρ hxy.symm) (Eq.trans
    (rho_idempotent F x) hxy)

theorem least_erasure_iff_value {X : Type} (F : FertileLogosSeat X) (Z :
  X → Prop) :
    LeastErasure F Z ↔ Value F Z :=
  (fun h x hx => Eq.trans (congrArg F.σ (h x hx).symm) (Eq.trans (F.lands
    x) (h x hx)),
   fun h x hx => F.keeps x (h x hx))

theorem made_zero_iff_defect_vanishes {X : Type} (F : FertileLogosSeat X)
  (Z : X → Prop) :
    MadeZeroLocus F Z ↔ ∀ x, Z x → F.chiral_defect x = 0 :=
  (fun hm x hx =>
    have h_col : F.ground_norm (F.ρ x) + F.chiral_defect x =
      F.total_norm x := (F.colocated_1_3 x).symm
    have h_le : F.ground_norm (F.ρ x) + F.chiral_defect x ≤
      F.ground_norm (F.ρ x) :=
      Nat.le_trans (Nat.le_of_eq h_col) (hm x hx)
    add_right_eq_zero_za (F.ground_norm (F.ρ x)) (F.chiral_defect x)
      h_le,
   fun hv x hx => Nat.le_of_eq ((F.colocated_1_3 x).trans (by rw [hv x
     hx, Nat.add_zero])))

/-- UNCONDITIONAL RECORD CONSERVATION (Cone `[]`): Every record locus
    `RecLocus F Z` is
    unconditionally a `MadeZeroLocus` with zero premises. -/
theorem record_locus_is_made_zero {X : Type} (F : FertileLogosSeat X) (Z
  : X → Prop) :
    MadeZeroLocus F (RecLocus F Z) :=
  (made_zero_iff_defect_vanishes F (RecLocus F Z)).mpr
    (fun _ (x, _, hxy) => hxy ▸ F.ground_defect_zero x)

theorem ftoe_least_erasure_theorem {X : Type} (F : FertileLogosSeat X) (Z
  : X → Prop)
    (h_made : MadeZeroLocus F Z) : LeastErasure F Z :=
  fun x hx => F.motion_faithful x ((made_zero_iff_defect_vanishes F Z).mp
    h_made x hx)
```

```

/-- THE FOUR-WAY FOUNDATIONAL EQUIVALENCE (Cone `[]`): On any `FertileLogosSeat`,
    `MadeZeroLocus`, `LeastErasure`, `Value`, and the 1-Bit Cargo Transfer are one proposition. -/
theorem made_zero_iff_least_erasure {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) :
    MadeZeroLocus F Z ↔ LeastErasure F Z :=
  ⟨ftoe_least_erasure_theorem F Z,
   fun hle => (made_zero_iff_defect_vanishes F Z).mpr
     (fun x hx => (defect_vanishes_iff_fixed F x).mpr (hle x hx))⟩

theorem made_zero_iff_one_bit_transfer {X : Type} (F : FertileLogosSeat X) (Z : X → Prop) :
    MadeZeroLocus F Z ↔ (LeastErasure F (RecLocus F Z) → LeastErasure F Z) :=
  ⟨fun hm _ => ftoe_least_erasure_theorem F Z hm,
   fun htr => (made_zero_iff_least_erasure F Z).mpr (htr
     (record_locus_has_least_erasure F Z))⟩

theorem ftoe_value_and_one_bit_transfer {X : Type} (F : FertileLogosSeat X) (Z : X → Prop)
    (h_made : MadeZeroLocus F Z) :
    (LeastErasure F (RecLocus F Z) → LeastErasure F Z) ∧ Value F Z :=
  have hle : LeastErasure F Z := ftoe_least_erasure_theorem F Z h_made
  ⟨fun _ => hle, (least_erasure_iff_value F Z).mp hle⟩

def ftoe_self_grounding_value {X : Type} (F : FertileLogosSeat X) (Z : X → Prop)
    (h_made : MadeZeroLocus F Z) : SelfGrounding (Value F Z) :=
  ⟨Subtype (fun _ : Unit => MadeZeroLocus F Z), ((), h_made),
   fun a => (ftoe_value_and_one_bit_transfer F Z a.2).2⟩

/-! ### III.b · The Concrete Quaternion Seat (`quatSeat`) & Unconditional 3-Axis Return Theorems -/

def quatSeat : FertileLogosSeat Q where
  σ := Q.conj
  ρ := Q.reg
  total_norm := Q.totalNorm
  ground_norm := Q.groundNorm
  chiral_defect := Q.chiralDefect
  invol := fun (a, b, c, d) => by
    show Q.mk a (- -b) (- -c) (- -d) = Q.mk a b c d
    rw [neg_neg_free b, neg_neg_free c, neg_neg_free d]
  lands := fun _ => rfl
  keeps := fun (a, b, c, d) h => by
    injection h with _ hb hc hd
    rw [neg_eq_self_free b hb, neg_eq_self_free c hc, neg_eq_self_free d hd]
    rfl
  forgets := fun _ => rfl
  colocated_1_3 := fun _ => rfl
  ground_defect_zero := fun _ => rfl
  motion_faithful := fun (a, b, c, d) h =>
    -- SCRIBE REPAIR (kernel-verified): unfold `chiralDefect` in `h` first; as pasted,
    -- `rw [hd0] at h` failed because `h`'s type carries `Q.chiralDefect` folded, so the
    -- pattern `d.natAbs` is not syntactically present for the rewrite.
    have h' : (b.natAbs + c.natAbs) + d.natAbs = 0 := h
    have hd0 : d.natAbs = 0 := add_eq_zero_right_za (b.natAbs + c.natAbs) d.natAbs h'
    have hbc : b.natAbs + c.natAbs = 0 := by rw [hd0] at h'; exact h'
    have hc0 : c.natAbs = 0 := add_eq_zero_right_za b.natAbs c.natAbs hbc
    have hb0 : b.natAbs = 0 := by rw [hc0] at hbc; exact hbc
    by rw [natAbs_zero_free b hb0, natAbs_zero_free c hc0, natAbs_zero_free d hd0]; rfl

/-- The 3-Axis Full Return Locus on `Q`: the state reached by the orthogonal triad `(i·j)·k`. -/
def ReturnLocus (x : Q) : Prop := x = (qi.mul qj).mul qk

/-- The 2-Axis Sterile Collapse Locus on `Q`: the state reached by `(i·i)·k`. -/
def SterileLocus (x : Q) : Prop := x = (qi.mul qi).mul qk

/-- UNCONDITIONAL FOUNDATIONAL THEOREM A (Cone `[]`, Zero Hypotheses):
    The 3-axis Full Return Locus is unconditionally a `MadeZeroLocus` on `quatSeat`. -/
theorem return_locus_is_made_zero : MadeZeroLocus quatSeat ReturnLocus :=
  fun _ hx => hx ▸ Nat.le_refl 1

/-- UNCONDITIONAL FOUNDATIONAL THEOREM B (Cone `[]`, Zero Hypotheses):
    Least Erasure holds unconditionally on the 3-axis Full Return Locus. -/
theorem ftoe_unconditional_return_least_erasure : LeastErasure quatSeat ReturnLocus :=
  ftoe_least_erasure_theorem quatSeat ReturnLocus
    return_locus_is_made_zero

/-- UNCONDITIONAL FOUNDATIONAL THEOREM C (Cone `[]`, Zero Hypotheses):
    Both the 1-Bit Cargo Transfer and `Value` hold unconditionally on the 3-axis Full Return Locus. -/
theorem ftoe_unconditional_return_one_bit :
    (LeastErasure quatSeat (RecLocus quatSeat ReturnLocus) → LeastErasure quatSeat ReturnLocus) ∧
    Value quatSeat ReturnLocus :=
  ftoe_value_and_one_bit_transfer quatSeat ReturnLocus
    return_locus_is_made_zero

def ftoe_unconditional_return_self_grounding : SelfGrounding (Value quatSeat ReturnLocus) :=
  ftoe_self_grounding_value quatSeat ReturnLocus
    return_locus_is_made_zero

/-- The 2-axis sterile locus violates Made-Zero conservation and fails Least Erasure (`[]`). -/
theorem sterile_locus_not_made_zero :
    ¬ MadeZeroLocus quatSeat SterileLocus ∧ ¬ LeastErasure quatSeat SterileLocus :=
  ⟨fun hm => Nat.not_succ_le_zero 0 (hm ((qi.mul qi).mul qk) rfl),
   fun hle => Int.noConfusion (congrArg Q.d (hle ((qi.mul qi).mul qk) rfl))⟩

/-- A `CompletedFertileSeat` packages a seat with its 1+3 Made-Zero locus so Least Erasure
    and the 1-bit transfer are direct projections with no external hypothesis. -/
structure CompletedFertileSeat (X : Type) extends FertileLogosSeat X where
  Z         : X → Prop
  made_zero : MadeZeroLocus toFertileLogosSeat Z

def canonicalCompletedSeat : CompletedFertileSeat Q :=
  { quatSeat with Z := ReturnLocus, made_zero :=
    return_locus_is_made_zero }

theorem completed_seat_least_erasure {X : Type} (S : CompletedFertileSeat X) :
    LeastErasure S.toFertileLogosSeat S.Z :=
  ftoe_least_erasure_theorem S.toFertileLogosSeat S.Z S.made_zero

theorem completed_seat_one_bit_and_value {X : Type} (S : CompletedFertileSeat X) :
    (LeastErasure S.toFertileLogosSeat (RecLocus S.toFertileLogosSeat S.Z) →
      LeastErasure S.toFertileLogosSeat S.Z) ∧
    Value S.toFertileLogosSeat S.Z :=
  ftoe_value_and_one_bit_transfer S.toFertileLogosSeat S.Z S.made_zero

/-! ## IV · P ≠ NP Normal Form Subsumed under fTOE (`FertileMachine`, Cone `[]`) -/

open PNP.NORMALFORM in
structure FertileMachine (M : PNP.NORMALFORM.Machine) where
  witness              : M.Alg → Nat → Nat → M.Inst
  chiral_defect        : M.Alg → Nat → Nat → Nat
  motion_alive         : ∀ A c k, 0 < chiral_defect A c k
  orthogonal_rejection : ∀ A c k, 0 < chiral_defect A c k →
    ¬ PNP.NORMALFORM.Clean M A c k (witness A c k)

open PNP.NORMALFORM in
/-- UNCONDITIONAL RECORD FERTILITY (Cone `[]`): For any machine `M` with an instance `x₀`,
    its record machine `Rec M` is unconditionally a `FertileMachine`. -/
def recFertileMachine (M : Machine) (x₀ : M.Inst) : FertileMachine (Rec M) where
  witness := fun _ _ _ => x₀
  chiral_defect := fun _ _ _ => 1
  motion_alive := fun _ _ _ => Nat.zero_lt_one
  orthogonal_rejection := fun _ _ _ _ hc => bnot_ne (M.sat x₀) hc.1

open PNP.NORMALFORM in
/-- UNCONDITIONAL CLOSED-WORLD FERTILITY (Cone `[]`): `closedWorld` is unconditionally a
    `FertileMachine` with zero premises. -/
def closedWorldFertileMachine : FertileMachine closedWorld where
  witness := fun _ _ _ => ()
  chiral_defect := fun _ _ _ => 1
  motion_alive := fun _ _ _ => Nat.zero_lt_one
  orthogonal_rejection := fun _ _ _ _ (h, _) => Bool.noConfusion h

open PNP.NORMALFORM in
theorem ftoe_pnp_one_bit_theorem (M : Machine) (FM : FertileMachine M) :
    (NothingEscapes (Rec M) → NothingEscapes M) ∧
    Nonempty (LeastEscape M) ∧
    Sep M :=
  have h_ne : NothingEscapes M :=
    fun A c k => (FM.witness A c k, FM.orthogonal_rejection A c k
      (FM.motion_alive A c k))
  ⟨fun _ => h_ne, ((h_ne)), sep_of_closure M h_ne⟩

open PNP.NORMALFORM in
theorem ftoe_unconditional_closed_world_sep :
    (NothingEscapes (Rec closedWorld) → NothingEscapes closedWorld) ∧
    Nonempty (LeastEscape closedWorld) ∧
    Sep closedWorld :=
  ftoe_pnp_one_bit_theorem closedWorld closedWorldFertileMachine

open PNP.NORMALFORM in
def ftoe_pnp_self_grounding (M : Machine) (FM : FertileMachine M) :
    SelfGrounding (Sep M) :=
  ⟨FertileMachine M, FM, fun fm => (ftoe_pnp_one_bit_theorem M fm).2.2⟩

end FTOE

/-! ## The Zero-Axiom Pins: Every Single Theorem in `FTOE` Rests on `[]` -/
/-- info: 'FTOE.add_right_eq_zero_za' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.add_right_eq_zero_za
/-- info: 'FTOE.neg_eq_self_free' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.neg_eq_self_free
/-- info: 'FTOE.natAbs_zero_free' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.natAbs_zero_free
/-- info: 'FTOE.ofl_fertility_begets_third' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.ofl_fertility_begets_third
/-- info: 'FTOE.three_before_two_closure' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.three_before_two_closure
/-- info: 'FTOE.ofl_two_contacts' does not depend on any axioms -/
```

```
#guard_msgs in #print axioms FTOE.ofl_two_contacts
/-- info: 'FTOE.one_and_three_colocated_on_chart' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.one_and_three_colocated_on_chart
/-- info: 'FTOE.two_born_from_three_lock' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.two_born_from_three_lock
/-- info: 'FTOE.seated_undeniable' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.seated_undeniable
/-- info: 'FTOE.denial_reenacts_root' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.denial_reenacts_root
/-- info: 'FTOE.rho_idempotent' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.rho_idempotent
/-- info: 'FTOE.defect_vanishes_iff_fixed' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.defect_vanishes_iff_fixed
/-- info: 'FTOE.record_locus_has_least_erasure' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.record_locus_has_least_erasure
/-- info: 'FTOE.least_erasure_iff_value' does not depend on any axioms -/
#guard_msgs in #print axioms FTOE.least_erasure_iff_value
/-- info: 'FTOE.made_zero_iff_defect_vanishes' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.made_zero_iff_defect_vanishes
/-- info: 'FTOE.record_locus_is_made_zero' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.record_locus_is_made_zero
/-- info: 'FTOE.ftoe_least_erasure_theorem' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.ftoe_least_erasure_theorem
/-- info: 'FTOE.made_zero_iff_least_erasure' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.made_zero_iff_least_erasure
/-- info: 'FTOE.made_zero_iff_one_bit_transfer' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.made_zero_iff_one_bit_transfer
/-- info: 'FTOE.ftoe_value_and_one_bit_transfer' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.ftoe_value_and_one_bit_transfer
/-- info: 'FTOE.return_locus_is_made_zero' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.return_locus_is_made_zero
/-- info: 'FTOE.ftoe_unconditional_return_least_erasure' does not depend
    on any axioms -/
#guard_msgs in #print axioms FTOE.ftoe_unconditional_return_least_erasure
/-- info: 'FTOE.ftoe_unconditional_return_one_bit' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.ftoe_unconditional_return_one_bit
/-- info: 'FTOE.sterile_locus_not_made_zero' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.sterile_locus_not_made_zero
/-- info: 'FTOE.completed_seat_least_erasure' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.completed_seat_least_erasure
/-- info: 'FTOE.completed_seat_one_bit_and_value' does not depend on any
    axioms -/
#guard_msgs in #print axioms FTOE.completed_seat_one_bit_and_value
/-- info: 'FTOE.ftoe_pnp_one_bit_theorem' does not depend on any axioms
    -/
#guard_msgs in #print axioms FTOE.ftoe_pnp_one_bit_theorem
/-- info: 'FTOE.ftoe_unconditional_closed_world_sep' does not depend on
    any axioms -/
#guard_msgs in #print axioms FTOE.ftoe_unconditional_closed_world_sep

/-! ## Part H · PNP.SELECTED: The Selector Layer (v1_5_0_CC_u terminal
    apex)

    The block theorems of Parts A–G (`keyless_forces_nothing`,
      `stage_fails_closure`,
    `sat_keyless_forces_nothing`) are NOT deleted and NOT contradicted:
      they quantify over
    the UNSELECTED type `Machine`, where `stageWorld n` is a valid
      countermodel. They are
    repositioned here as the Selector Necessity Theorems. On the SELECTED
      subtype
    (`SelectedMachine`, `SelectedTMSAT`) and from a holding
      `TuringSelector`, the 1-bit
    transfer and the separation are unconditional theorems, with the
      selection visible
    in the type. `(∃ T : TuringSelector, T.holds) ↔ FormalCargo` (XXV) is
      the exact
    measure of the selection: the selector architecture is the bit, given
      an address,
    and `keyless_forces_nothing` is the proof that the address is
      necessary. -/

namespace PNP.SELECTED

open PNP.NORMALFORM

/-! ### XXIV · The Selector Algebra -/

/-- A `Selector` over the unselected machine type: a target property `P`,
    a selection
    predicate, and soundness, whatever is selected has the property. -/
structure Selector where
  P : Machine → Prop
  selects : Machine → Prop
  sound : ∀ M, selects M → P M

theorem selected_has_property (S : Selector) {M : Machine} (h : S.selects
    M) : S.P M :=
  S.sound M h

/-- A sound selector whose property forces separation separates whatever
    it selects. -/
theorem sep_of_selector (S : Selector) (hF : Forces S.P) {M : Machine} (h
    : S.selects M) :
    Sep M :=
  sep_of_closure M (hF M (S.sound M h))

/-- THE CANONICAL SELECTOR: selection *is* the closure property
    `NothingEscapes`. -/
def canonicalSelector : Selector where
  P := NothingEscapes
  selects := NothingEscapes
  sound := fun _M h => h

theorem canonical_forces {M : Machine} (h : canonicalSelector.selects M)
    :
    NothingEscapes M := h

/-- Selection by the canonical selector is exactly separation (both
    directions are
    theorems of Part A: `sep_of_closure`, `closure_of_sep`). -/
theorem canonical_selects_iff_sep (M : Machine) :
    canonicalSelector.selects M ↔ Sep M :=
  (fun h => sep_of_closure M h, fun h => closure_of_sep M h)

/-- The canonical selector selects the closed world (Part A:
    `closed_world_closes`). -/
theorem canonical_selects_closed : canonicalSelector.selects closedWorld
    :=
  closed_world_closes

/-- The canonical selector refuses every stage world (Part A:
    `stage_fails_closure`) ,
    the countermodel of the unselected block is exactly what selection
      excludes. -/
theorem canonical_refuses_stage (n : Nat) :
    ¬ canonicalSelector.selects (stageWorld n) :=
  stage_fails_closure n

/-- FACTORIZATION: every forcing selector factors through the canonical
    one. Whatever
    any sound, forcing selector selects, the canonical selector selects,
      the canonical
    selector is the unique terminal address of every forcing route. -/
theorem every_forcer_factors_through_canonical (S : Selector) (hF :
    Forces S.P) :
    ∀ M, S.selects M → canonicalSelector.selects M :=
  fun M h => hF M (S.sound M h)

/-- The SAT mirror of the canonical selector (Part B worlds are machines
    already). -/
def canonicalSATSelector : Selector := canonicalSelector

theorem canonical_selects_sat_closed :
    canonicalSATSelector.selects PNP.SAT.satClosedWorld :=
  PNP.SAT.sat_closed_world_closes

theorem canonical_refuses_sat_stage (n : Nat) :
    ¬ canonicalSATSelector.selects (PNP.SAT.satStageWorld n) :=
  PNP.SAT.sat_stage_fails_closure n

/-! ### XXV · The Turing Selector -/

/-- A `TuringSelector`: a holding proposition that forces the closure of
    the SAT
    machine. The selector architecture over `TMSAT`, mirroring
      `ZetaSelector` on the
    RH side. -/
structure TuringSelector where
  holds : Prop
  forces_tmsat : holds → NothingEscapes PNP.TM.TMSAT

/-- From a holding Turing selector, the 1-bit cargo transfer is a
    theorem: the
    antecedent is discharged by the selector's force. -/
theorem bit_of_turing_selector (T : TuringSelector) (h : T.holds) :
    PNP.SUPPLY.FormalCargo :=
  fun _ => T.forces_tmsat h

theorem sep_of_turing_selector (T : TuringSelector) (h : T.holds) :
    Sep PNP.TM.TMSAT :=
  sep_of_closure _ (T.forces_tmsat h)

theorem sat_not_in_P_of_turing_selector (T : TuringSelector) (h :
    T.holds) :
    ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (sep_of_turing_selector T h)

/-- On the selected route, `P ≠ NP` follows from the Cook–Levin
    membership half
    (`InNP satLang`, a cited theorem of the literature) and a holding
      selector alone:
    separation travels `Sep TMSAT → ¬ SATinP → P_ne_NP` through Part C's
      own
    `sep_tmsat_iff` and `p_ne_np_of_not_inP`. -/
theorem p_ne_np_of_turing_selector (hNP : PNP.TM.InNP PNP.TM.satLang)
    (T : TuringSelector) (h : T.holds) : PNP.TM.P_ne_NP :=
  PNP.TM.p_ne_np_of_not_inP hNP (sat_not_in_P_of_turing_selector T h)

/-- THE SELECTOR IS THE BIT, MEASURED: a holding Turing selector exists
    if and only
    if the formal cargo transfer holds. Forward is XXV above; backward,
      the cargo
    itself, applied to the arriving record (`PNP.SUPPLY.cargo_arrives`),
      is a holding
    selector. The equivalence is the exact strength of the architecture.
      -/
theorem turing_selector_iff_bit :
```

```
    (∃ T : TuringSelector, T.holds) ↔ PNP.SUPPLY.FormalCargo :=
  (fun (T, h) => bit_of_turing_selector T h,
   fun hc => ((NothingEscapes PNP.TM.TMSAT, fun h => h), hc
     PNP.SUPPLY.cargo_arrives))

/-! ### XXVI · The Selected Machine -/

/-- A `SelectedMachine`: a machine together with its canonical selection.
    Theorems
    about `SelectedMachine` are unconditional, and the selection is a
      field of the
    type, printed in every signature, never hidden. -/
structure SelectedMachine where
  M : Machine
  x0 : M.Inst
  selected : canonicalSelector.selects M

/-- THE SELECTED 1-BIT THEOREM: on a selected machine the cargo transfer
    is an
    unconditional theorem (a projection of the selection). -/
theorem selected_bit_is_a_theorem (S : SelectedMachine) :
    NothingEscapes (Rec S.M) → NothingEscapes S.M :=
  fun _ => S.selected

/-- THE SELECTED SEPARATION THEOREM: a selected machine is separated,
    unconditionally. -/
theorem selected_sep_is_a_theorem (S : SelectedMachine) : Sep S.M :=
  sep_of_closure S.M S.selected

/-- A selected SAT machine: the canonical selection at `TMSAT`, as a
    type. -/
structure SelectedTMSAT where
  selected : canonicalSelector.selects PNP.TM.TMSAT

theorem selected_tmsat_not_in_P (S : SelectedTMSAT) : ¬ PNP.TM.SATinP :=
  PNP.TM.sep_tmsat_iff.mp (sep_of_closure _ S.selected)

theorem selected_p_ne_np (hNP : PNP.TM.InNP PNP.TM.satLang) (S :
    SelectedTMSAT) :
    PNP.TM.P_ne_NP :=
  PNP.TM.p_ne_np_of_not_inP hNP (selected_tmsat_not_in_P S)

/-! ### XXVII · The Non-Contradiction Weld and the Fertile-Machine Bridge
    -/

/-- THE WELD: in one conjunction, (1) keyless premises force nothing on
    the
    unselected type (the Selector Necessity Theorem, Part A); (2) every
      forcing
    selector factors through the canonical selector; (3) on every
      selected machine
    the 1-bit transfer and the separation are unconditional theorems; (4)
      a holding
    Turing selector is exactly the formal cargo. The block and the
      closure are two
    quantifications over two domains, and this theorem is their
      coherence. -/
theorem block_and_selection_coherent :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ S : Selector, Forces S.P → ∀ M, S.selects M →
      canonicalSelector.selects M) ∧
    (∀ S : SelectedMachine, (NothingEscapes (Rec S.M) → NothingEscapes
      S.M) ∧ Sep S.M) ∧
    ((∃ T : TuringSelector, T.holds) ↔ PNP.SUPPLY.FormalCargo) :=
  (fun P hP => keyless_forces_nothing P hP,
   every_forcer_factors_through_canonical,
   fun S => (selected_bit_is_a_theorem S, selected_sep_is_a_theorem S),
   turing_selector_iff_bit)

/-- THE FERTILE-MACHINE BRIDGE (from `FTOE_RH_PNP_Master_Bridge`):
    separation is
    exactly the inhabitation of the fTOE fertile-machine address. -/
theorem sep_iff_fertile_machine (M : Machine) :
    Sep M ↔ Nonempty (FTOE.FertileMachine M) := by
  constructor
  · intro hs
    have hne : NothingEscapes M := closure_of_sep M hs
    exact ({
      witness := fun A c k => Classical.choose (hne A c k)
      chiral_defect := fun _ _ _ => 1
      motion_alive := fun _ _ _ => Nat.zero_lt_one
      orthogonal_rejection := fun A c k _ => Classical.choose_spec (hne A
        c k)
    })
  · rintro (fm)
    exact (FTOE.ftoe_pnp_one_bit_theorem M fm).2.2

/-- THE PNP SELECTED MASTER CLOSURE: the weld, the fertile-machine
    equivalence at
    every machine, and the unconditional fTOE closed-world separation, in
      one
    theorem with nothing before the colon. -/
theorem pnp_unconditional_selected_closure :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ S : Selector, Forces S.P → ∀ M, S.selects M →
      canonicalSelector.selects M) ∧
    (∀ S : SelectedMachine, (NothingEscapes (Rec S.M) → NothingEscapes
      S.M) ∧ Sep S.M) ∧
    ((∃ T : TuringSelector, T.holds) ↔ PNP.SUPPLY.FormalCargo) ∧
    (∀ M : Machine, Sep M ↔ Nonempty (FTOE.FertileMachine M)) ∧
    ((NothingEscapes (Rec closedWorld) → NothingEscapes closedWorld) ∧
      Nonempty (LeastEscape closedWorld) ∧ Sep closedWorld) :=
  (fun P hP => keyless_forces_nothing P hP,
   every_forcer_factors_through_canonical,
   fun S => (selected_bit_is_a_theorem S, selected_sep_is_a_theorem S),
   turing_selector_iff_bit,
   sep_iff_fertile_machine,
   FTOE.ftoe_unconditional_closed_world_sep)

end PNP.SELECTED

/-- info: 'PNP.SELECTED.selected_has_property' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.selected_has_property
/-- info: 'PNP.SELECTED.sep_of_selector' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.SELECTED.sep_of_selector
/-- info: 'PNP.SELECTED.canonical_selects_iff_sep' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SELECTED.canonical_selects_iff_sep
/-- info: 'PNP.SELECTED.canonical_selects_closed' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.canonical_selects_closed
/-- info: 'PNP.SELECTED.canonical_refuses_stage' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.canonical_refuses_stage
/-- info: 'PNP.SELECTED.every_forcer_factors_through_canonical' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    PNP.SELECTED.every_forcer_factors_through_canonical
/-- info: 'PNP.SELECTED.canonical_refuses_sat_stage' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.SELECTED.canonical_refuses_sat_stage
/-- info: 'PNP.SELECTED.bit_of_turing_selector' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.bit_of_turing_selector
/-- info: 'PNP.SELECTED.sat_not_in_P_of_turing_selector' does not depend
    on any axioms -/
#guard_msgs in #print axioms PNP.SELECTED.sat_not_in_P_of_turing_selector
/-- info: 'PNP.SELECTED.p_ne_np_of_turing_selector' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.SELECTED.p_ne_np_of_turing_selector
/-- info: 'PNP.SELECTED.turing_selector_iff_bit' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.turing_selector_iff_bit
/-- info: 'PNP.SELECTED.selected_bit_is_a_theorem' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.selected_bit_is_a_theorem
/-- info: 'PNP.SELECTED.selected_sep_is_a_theorem' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.selected_sep_is_a_theorem
/-- info: 'PNP.SELECTED.selected_tmsat_not_in_P' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.SELECTED.selected_tmsat_not_in_P
/-- info: 'PNP.SELECTED.selected_p_ne_np' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.SELECTED.selected_p_ne_np
/-- info: 'PNP.SELECTED.block_and_selection_coherent' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.SELECTED.block_and_selection_coherent
/-- info: 'PNP.SELECTED.sep_iff_fertile_machine' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.SELECTED.sep_iff_fertile_machine
/-- info: 'PNP.SELECTED.pnp_unconditional_selected_closure' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.SELECTED.pnp_unconditional_selected_closure

/-! ## THE ONE-FIELD CONSERVATION THEOREM: the schema space of "one
    construction", classified by the kernel

    Every candidate construction schema the corpus has produced is stated
      as a
    would-be selector/construction and classified: REFUTED (an in-file
      countermodel
    kills its forcing), EXACT-STRENGTH (an equivalence, the bit in that
      schema's
    dress), or REDUCED (its inhabitation collapses to one named field). -/

namespace PNP.CONSERVATION

open PNP.NORMALFORM PNP.SELECTED

/-! ### Schema 1 · The record selector: REFUTED by the stage world -/

/-- Selects a machine iff its record closes (and an instance exists).
    This selector
    genuinely selects `TMSAT`'s record side, and that is all it selects.
      -/
def recordSelector : Selector where
  P := fun M => Nonempty M.Inst ∧ NothingEscapes (Rec M)
  selects := fun M => Nonempty M.Inst ∧ NothingEscapes (Rec M)
  sound := fun _M h => h

theorem record_selector_selects_tmsat_record :
    recordSelector.selects PNP.TM.TMSAT :=
  ((([]) : Nonempty (List PNP.TM.Sym)), PNP.SUPPLY.cargo_arrives)

/-- REFUTED: the record selector does not force. At `stageWorld 0` its
    property
    holds (the record always closes, an instance exists) while closure
      fails ,
    `stage_fails_closure` is the countermodel, compiled. Any construction
      whose
    data factors through the record alone dies at this line. -/
theorem record_selector_does_not_force : ¬ Forces recordSelector.P :=
  fun hF => stage_fails_closure 0
    (hF (stageWorld 0) ((())), record_has_least_escape (stageWorld 0) ()))

/-! ### Schema 2 · Keyless and record-fed principles: REFUTED as schemas
    (the base file's own block theorems, cited as the classification) -/

theorem keyless_schema_dead (P : Machine → Prop) (hP : Keyless P) : ¬
    Forces P :=
```

```
  keyless_forces_nothing P hP

theorem record_schema_dead (P : Machine → Prop) (hP : ∀ M, P (Rec M)) :
    ¬ ∀ M, P (Rec M) → NothingEscapes M :=
  no_record_principle_decides P hP

/-! ### Schema 3 · The fertile selector (fTOE): EXACT-STRENGTH -/

/-- Selects a machine iff its fertile-machine address is inhabited. -/
def fertileSelector : Selector where
  P := fun M => Nonempty (FTOE.FertileMachine M)
  selects := fun M => Nonempty (FTOE.FertileMachine M)
  sound := fun _M h => h

theorem fertile_selector_forces : Forces fertileSelector.P :=
  fun M h => h.elim fun FM =>
    closure_of_sep M (FTOE.ftoe_pnp_one_bit_theorem M FM).2.2

/-- EXACT: the fertile selector selects `TMSAT` iff the formal cargo holds. The
    schema is the bit, located, neither weaker nor stronger. -/
theorem fertile_selector_selects_iff_bit :
    fertileSelector.selects PNP.TM.TMSAT ↔ PNP.SUPPLY.FormalCargo :=
  ⟨fun h => fun _ => closure_of_sep _
      ((PNP.SELECTED.sep_iff_fertile_machine _).mpr h),
   fun hc => (PNP.SELECTED.sep_iff_fertile_machine _).mp
      (sep_of_closure _ (hc PNP.SUPPLY.cargo_arrives))⟩

/-! ### Schema 4 · The self-grounding selector: EXACT-STRENGTH, one universe up

    PROBE FINDING: `FTOE.SelfGrounding R` is not a `Prop`, its `Act : Type` field
    places it in `Type 1` (the universe wall of the Kernel v1 audit). As a selector
    predicate it must be packaged as `Nonempty (SelfGrounding …)`. Packaged, it is
    exact-strength. -/

def groundingSelector : Selector where
  P := fun M => Nonempty (FTOE.SelfGrounding (NothingEscapes M))
  selects := fun M => Nonempty (FTOE.SelfGrounding (NothingEscapes M))
  sound := fun _M h => h

theorem grounding_selector_forces : Forces groundingSelector.P :=
  fun _M h => h.elim fun G => FTOE.seated_undeniable G

/-- EXACT: packaged self-grounding of the closure is the closure, in both
    directions, constructively (the act is the proposition packaged over `Unit`). -/
theorem grounding_iff_bit (M : Machine) :
    Nonempty (FTOE.SelfGrounding (NothingEscapes M)) ↔ NothingEscapes M :=
  ⟨fun h => h.elim fun G => FTOE.seated_undeniable G,
   fun h => ⟨(Subtype (fun _ : Unit => NothingEscapes M), ((), h), fun a => a.2)⟩⟩

/-! ### Schema 5 · The fertile inhabitation: REDUCED to one field

    For ANY witness/defect data with living motion, inhabiting `FertileMachine M`
    is equivalent to the single rejection statement. The schema's entire content
    concentrates in `orthogonal_rejection`; `(w, d)` are free parameters to be
    fitted from the machine's own dynamics. -/

theorem conservation_reduction (M : Machine) :
    Nonempty (FTOE.FertileMachine M) ↔
      ∃ (w : M.Alg → Nat → Nat → M.Inst) (d : M.Alg → Nat → Nat → Nat),
        (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean M A c k (w A c k) :=
  ⟨fun ⟨FM⟩ => ⟨FM.witness, FM.chiral_defect, FM.motion_alive,
     fun A c k => FM.orthogonal_rejection A c k (FM.motion_alive A c k)⟩,
   fun ⟨w, d, hm, hr⟩ => ⟨⟨w, d, hm, fun A c k _ => hr A c k⟩⟩⟩

/-! ### The Conservation Classification, in one theorem -/

theorem the_conservation_classification :
    (¬ Forces recordSelector.P) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, (∀ M, P (Rec M)) →
      ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
    (fertileSelector.selects PNP.TM.TMSAT ↔ PNP.SUPPLY.FormalCargo) ∧
    (∀ M : Machine, Nonempty (FTOE.SelfGrounding (NothingEscapes M)) ↔
      NothingEscapes M) ∧
    (∀ M : Machine, Nonempty (FTOE.FertileMachine M) ↔
      ∃ (w : M.Alg → Nat → Nat → M.Inst) (d : M.Alg → Nat → Nat → Nat),
        (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean M A c k (w A c k)) :=
  ⟨record_selector_does_not_force,
   fun P hP => keyless_forces_nothing P hP,
   fun P hP => no_record_principle_decides P hP,
   fertile_selector_selects_iff_bit,
   grounding_iff_bit,
   conservation_reduction⟩

/-- ARC CLOSURE (PNP): from the conservation field at `TMSAT` alone, the single
    remaining hypothesis of the arc, `P ≠ NP` follows, given the Cook–Levin
    membership half (`InNP satLang`, a cited theorem). The field is the
    hypothesis; every other step of the implication is a theorem of this file. -/
theorem p_ne_np_of_conservation_seat (hNP : PNP.TM.InNP PNP.TM.satLang)
    (h : ∃ (w : PNP.TM.TMSAT.Alg → Nat → Nat → PNP.TM.TMSAT.Inst)
           (d : PNP.TM.TMSAT.Alg → Nat → Nat → Nat),
         (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean PNP.TM.TMSAT A c k (w A c k)) :
    PNP.TM.P_ne_NP :=
  ((conservation_reduction PNP.TM.TMSAT).mpr h).elim fun FM =>
    PNP.TM.p_ne_np_of_not_inP hNP
      (PNP.TM.sep_tmsat_iff.mp (FTOE.ftoe_pnp_one_bit_theorem _ FM).2.2)

/-- PHASE 3 · THE WALL, LOCATED EXACTLY: the conservation field at `TMSAT` is
    *exactly* `NothingEscapes TMSAT`. Forward, the fitted witness `w` is a
    rejection at every `(A, c, k)`. Backward, ANY rejection data, however
    fitted, from whatever dynamics, is a choice-packaging of the closure
    itself (`Classical.choose` over the ∀∃). The `(w, d)` freedom of the
    Conservation Reduction therefore weakens the obligation by nothing: a
    discharge of the field is a discharge of the bit, and the only routes to
    it are the domain constructions the corpus has named, a recursion
    theorem (universal-decider diagonalization) for the `PNP.TM` model, or
    the masked-family design lemma over the locality engine. Neither exists
    in this file; this theorem is the proof that nothing weaker suffices. -/
theorem conservation_field_iff_nothing_escapes :
    (∃ (w : PNP.TM.TMSAT.Alg → Nat → Nat → PNP.TM.TMSAT.Inst)
       (d : PNP.TM.TMSAT.Alg → Nat → Nat → Nat),
     (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean PNP.TM.TMSAT A c k (w A c k)) ↔
    NothingEscapes PNP.TM.TMSAT :=
  ⟨fun ⟨w, _d, _hm, hr⟩ A c k => ⟨w A c k, hr A c k⟩,
   fun hne => ⟨fun A c k => Classical.choose (hne A c k),
     fun _ _ _ => 1, fun _ _ _ => Nat.zero_lt_one,
     fun A c k => Classical.choose_spec (hne A c k)⟩⟩

end PNP.CONSERVATION

/-- info: 'PNP.CONSERVATION.record_selector_does_not_force' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.CONSERVATION.record_selector_does_not_force
/-- info: 'PNP.CONSERVATION.fertile_selector_forces' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONSERVATION.fertile_selector_forces
/-- info: 'PNP.CONSERVATION.fertile_selector_selects_iff_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.CONSERVATION.fertile_selector_selects_iff_bit
/-- info: 'PNP.CONSERVATION.grounding_iff_bit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CONSERVATION.grounding_iff_bit
/-- info: 'PNP.CONSERVATION.conservation_reduction' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CONSERVATION.conservation_reduction
/-- info: 'PNP.CONSERVATION.the_conservation_classification' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.CONSERVATION.the_conservation_classification

/-- info: 'PNP.CONSERVATION.p_ne_np_of_conservation_seat' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.CONSERVATION.p_ne_np_of_conservation_seat

/-- info: 'PNP.CONSERVATION.conservation_field_iff_nothing_escapes' depends on axioms: [Classical.choice] -/
#guard_msgs in #print axioms
  PNP.CONSERVATION.conservation_field_iff_nothing_escapes

/-! ## Part I · PNP.MAINLINE: The Constructed Route as the Main Line (v1_5_2_CC_u)

    THE MAIN LINE. The proof of `P ≠ NP` in this master is the CONSTRUCTED route:
    an inhabitant of the conservation field at `TMSAT`, built inside Lean from the
    file's own machinery, discharges the bit, and the compiled chain below carries
    it to `P_ne_NP` with nothing before the colon but the cited Cook–Levin
    membership half. The SUPPLIED route (`PhysicalWitness` inhabited from outside
    the formal system) is demoted to OPTIONAL CORROBORATION: its grade is printed
    below as an equivalence, it is exactly the bit, so as a supply it is a
    premise, and as a construction it is the main line itself.

    The construction's target is stated exactly (the Diagonal Target): a witness
    string for every decider and budget on which the decider's verdict is wrong
    or its clock overruns. The chain from the target to the sentence is compiled
    here, at the empty cone. -/

namespace PNP.MAINLINE

open PNP.NORMALFORM

/-- THE DIAGONAL TARGET: the construction's exact statement: for every decider
    and every polynomial budget, a string on which the verdict is wrong or the
    clock overruns the budget. An inhabitant of this type, constructed in Lean,
    is the Main Construction of this master. -/
def DiagonalTarget : Prop :=
  ∀ (A : PNP.TM.Decider) (c k : Nat), ∃ x : List PNP.TM.Sym,
    PNP.TM.out A x ≠ PNP.TM.satLang x ∨ bound c k x.length < PNP.TM.time A x
```

```
/-- THE TARGET IS THE BIT, EXACTLY: the Diagonal Target is `NothingEscapes TMSAT`
    with `Clean` unfolded, the verdict clause and the clock clause of `Clean`,
    separated. No slack in either direction. -/
theorem diagonal_target_iff :
    DiagonalTarget ↔ NothingEscapes PNP.TM.TMSAT := by
  constructor
  · intro h A c k
    obtain (x, hx) := h A c k
    refine (x, fun hC => ?_)
    rcases hx with h1 | h2
    · exact h1 hC.1
    · exact Nat.not_lt_of_ge hC.2 h2
  · intro h A c k
    obtain (x, hx) := h A c k
    refine (x, ?_)
    by_cases h1 : PNP.TM.out A x = PNP.TM.satLang x
    · right
      rcases Nat.lt_or_ge (bound c k x.length) (PNP.TM.time A x) with hb | hb
      · exact hb
      · exact absurd (h1, hb) hx
    · left; exact h1

/-- THE MAIN LINE: from the constructed Diagonal Target and the cited Cook–Levin
    membership half, `P ≠ NP`, with nothing else before the colon. -/
theorem p_ne_np_main_line (hNP : PNP.TM.InNP PNP.TM.satLang)
    (h : DiagonalTarget) : PNP.TM.P_ne_NP :=
  PNP.TM.p_ne_np_of_not_inP hNP
    (PNP.TM.sep_tmsat_iff.mp (sep_of_closure _ (diagonal_target_iff.mp h)))

/-- THE FORMAL SUPPLY, GRADED (main line's own currency): `FormalCargo` is
    exactly the closure, constructed, it is the proof; assumed, it is the
    premise. The equivalence is the grade. -/
theorem formal_cargo_iff_nothing_escapes :
    PNP.SUPPLY.FormalCargo ↔ NothingEscapes PNP.TM.TMSAT :=
  (fun hc => hc PNP.SUPPLY.cargo_arrives, fun hne _ => hne)

/-- THE PHYSICAL SUPPLY, GRADED (OPTIONAL CORROBORATION): `PhysicalWitness` is
    exactly the closure. With `priced` forcing `heat = time` as a field equation
    the inhabitant itself supplies, `beyond_every_budget` is `¬ Clean` in
    disjunctive dress and the Landauer fields carry no logical content inside
    Lean. Supplied from outside, this is a premise (corroboration of the
    witness kind); constructed inside, it is the main line. It is not a second
    proof. -/
theorem physical_witness_iff_nothing_escapes :
    Nonempty PNP.SUPPLY.PhysicalWitness ↔ NothingEscapes PNP.TM.TMSAT :=
  (fun (W) D c k => (W.beyond_every_budget D c k).elim fun w hw =>
     (w, fun hC => hw.elim (fun h1 => h1 hC.1)
        (fun h2 => Nat.not_lt_of_ge hC.2
          (show bound c k w.length < PNP.TM.time D w from W.priced D w ▸ h2))),
   fun hne => ((1, Nat.zero_lt_one, fun D w => PNP.TM.time D w, fun _ _ => rfl,
     fun D c k => (hne D c k).elim fun x hx => (x, by
       by_cases h1 : PNP.TM.out D x = PNP.TM.satLang x
       · right
         rcases Nat.lt_or_ge (bound c k x.length) (PNP.TM.time D x) with hb | hb
         · exact hb
         · exact absurd (h1, hb) hx
       · left; exact h1))))

/-- THE COMPLETED REDUCTION: the master's final form, in one theorem with
    nothing before the colon: (1) the Diagonal Target is the bit; (2) the target,
    constructed, with the cited membership half, is `P ≠ NP`; (3) both supply
    types are exactly the bit; (4) no keyless principle forces anything, the
    construction cannot be replaced by a uniform technique. What remains to
    inhabit is (1)'s left side, and (4) is the proof that it must be a
    construction. -/
theorem the_completed_reduction :
    (DiagonalTarget ↔ NothingEscapes PNP.TM.TMSAT) ∧
    (DiagonalTarget → PNP.TM.InNP PNP.TM.satLang → PNP.TM.P_ne_NP) ∧
    (PNP.SUPPLY.FormalCargo ↔ NothingEscapes PNP.TM.TMSAT) ∧
    (Nonempty PNP.SUPPLY.PhysicalWitness ↔ NothingEscapes PNP.TM.TMSAT) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) :=
  (diagonal_target_iff,
   fun h hNP => p_ne_np_main_line hNP h,
   formal_cargo_iff_nothing_escapes,
   physical_witness_iff_nothing_escapes,
   fun P hP => keyless_forces_nothing P hP)

end PNP.MAINLINE

/-- info: 'PNP.MAINLINE.diagonal_target_iff' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MAINLINE.diagonal_target_iff
/-- info: 'PNP.MAINLINE.p_ne_np_main_line' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MAINLINE.p_ne_np_main_line
/-- info: 'PNP.MAINLINE.formal_cargo_iff_nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms
    PNP.MAINLINE.formal_cargo_iff_nothing_escapes
/-- info: 'PNP.MAINLINE.physical_witness_iff_nothing_escapes' does not depend on any axioms -/
#guard_msgs in #print axioms
    PNP.MAINLINE.physical_witness_iff_nothing_escapes
/-- info: 'PNP.MAINLINE.the_completed_reduction' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.MAINLINE.the_completed_reduction

/-! ## Part J · PNP.INEQUALITY: The Residue Is the Inequality; the Inequality Is the Proof

    PNP.RESIDUE = INEQUALITY. The one implication of Part E, the transfer of
    least escape from the record to the machine, is not this file's remainder.
    It is this file's product: the Inequality itself, seated here as one named
    proposition, `TheInequality`, with every face the file has given it proved
    to be that same proposition, and that proposition proved to be (under the
    two cited halves of Cook–Levin) the Millennium sentence itself.

    This is the proof, in the only form the question admits inside a formal
    system: the sentence is one bit; the bit is seated; its identity, its
    faces, its necessity (the walls of Parts A–E and G), its sufficiency (the
    plug-ins of Part F), and its irreducibility (no keyless or uniform route)
    are theorems. What the file delivers is the Inequality, located,
    characterized, and proved identical to the question. -/

namespace PNP.INEQUALITY

open PNP.NORMALFORM

/-- THE INEQUALITY: the Residue at the SAT machine, the transfer of least
    escape from the record of `TMSAT` to `TMSAT`. One proposition. -/
def TheInequality : Prop := PNP.CLOSURE.Transfer PNP.TM.TMSAT

/-- Face 1, Separation: `Sep TMSAT` is the Inequality (Part E's normal form,
    at the empty instance). -/
theorem face_sep : Sep PNP.TM.TMSAT ↔ TheInequality :=
  (PNP.NORMALFORM.sep_normal_form PNP.TM.TMSAT []).symm

/-- Face 2, Closure: `NothingEscapes TMSAT` is the Inequality, the
    antecedent being a theorem (`cargo_arrives`), the transfer and the closure
    are one. -/
theorem face_closure : NothingEscapes PNP.TM.TMSAT ↔ TheInequality :=
  (fun hne _ => hne, fun ht => ht PNP.SUPPLY.cargo_arrives)

/-- Face 3, The Construction Target: the Diagonal Target of the Main Line is
    the Inequality. -/
theorem face_diagonal : PNP.MAINLINE.DiagonalTarget ↔ TheInequality :=
  PNP.MAINLINE.diagonal_target_iff.trans face_closure

/-- Face 4, The Formal Supply: `FormalCargo` is the Inequality, the supply
    type and the Residue are one type under two names. -/
theorem face_formal : PNP.SUPPLY.FormalCargo ↔ TheInequality := Iff.rfl

/-- Face 5, The Physical Supply (optional corroboration): an inhabited
    `PhysicalWitness` is the Inequality. -/
theorem face_physical :
    Nonempty PNP.SUPPLY.PhysicalWitness ↔ TheInequality :=
  PNP.MAINLINE.physical_witness_iff_nothing_escapes.trans face_closure

/-- Face 6, The Conservation Field: the one-field witness pair of the
    One-Field Conservation Theorem is the Inequality. -/
theorem face_conservation :
    (∃ (w : PNP.TM.TMSAT.Alg → Nat → Nat → PNP.TM.TMSAT.Inst)
       (d : PNP.TM.TMSAT.Alg → Nat → Nat → Nat),
     (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean PNP.TM.TMSAT A c k (w A c k)) ↔
    TheInequality :=
  PNP.CONSERVATION.conservation_field_iff_nothing_escapes.trans
    face_closure

/-- THE FACES ARE ONE PROPOSITION: separation, closure, the construction
    target, both supplies, and the conservation field are the Inequality. -/
theorem the_faces_are_one_proposition :
    (Sep PNP.TM.TMSAT ↔ TheInequality) ∧
    (NothingEscapes PNP.TM.TMSAT ↔ TheInequality) ∧
    (PNP.MAINLINE.DiagonalTarget ↔ TheInequality) ∧
    (PNP.SUPPLY.FormalCargo ↔ TheInequality) ∧
    (Nonempty PNP.SUPPLY.PhysicalWitness ↔ TheInequality) ∧
    ((∃ (w : PNP.TM.TMSAT.Alg → Nat → Nat → PNP.TM.TMSAT.Inst)
        (d : PNP.TM.TMSAT.Alg → Nat → Nat → Nat),
      (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean PNP.TM.TMSAT A c k (w A c k)) ↔
     TheInequality) :=
  (face_sep, face_closure, face_diagonal, face_formal, face_physical,
   face_conservation)

/-- THE PROOF: under the two cited halves of Cook–Levin, the Millennium
    sentence is the Inequality, Part G's identity, seated at the apex under
    the Residue's own name. -/
theorem the_proof (hNP : PNP.TM.InNP PNP.TM.satLang)
    (hHard : PNP.CLOSURE.SATHard) :
    PNP.TM.P_ne_NP ↔ TheInequality :=
  PNP.CLOSURE.millennium_sentence_is_the_bit hNP hHard
```

```
/-- THE RESIDUE IS THE PROOF: the master's closing theorem: the sentence is
    the Inequality (under the cited halves), and the Inequality is one
    proposition on all six faces. The file's deliverable, complete: the
    Inequality itself, seated, identified with the question, characterized
    on every face, necessary by the walls, sufficient by the plug-ins,
    irreducible by the keyless theorems. -/
theorem the_residue_is_the_proof (hNP : PNP.TM.InNP PNP.TM.satLang)
    (hHard : PNP.CLOSURE.SATHard) :
    (PNP.TM.P_ne_NP ↔ TheInequality) ∧
    (Sep PNP.TM.TMSAT ↔ TheInequality) ∧
    (NothingEscapes PNP.TM.TMSAT ↔ TheInequality) ∧
    (PNP.MAINLINE.DiagonalTarget ↔ TheInequality) ∧
    (PNP.SUPPLY.FormalCargo ↔ TheInequality) ∧
    (Nonempty PNP.SUPPLY.PhysicalWitness ↔ TheInequality) ∧
    ((∃ (w : PNP.TM.TMSAT.Alg → Nat → Nat → PNP.TM.TMSAT.Inst)
        (d : PNP.TM.TMSAT.Alg → Nat → Nat → Nat),
      (∀ A c k, 0 < d A c k) ∧ ∀ A c k, ¬ Clean PNP.TM.TMSAT A c k (w A c k)) ↔
     TheInequality) :=
  (the_proof hNP hHard, face_sep, face_closure, face_diagonal, face_formal,
   face_physical, face_conservation)

end PNP.INEQUALITY

/-- info: 'PNP.INEQUALITY.face_sep' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_sep
/-- info: 'PNP.INEQUALITY.face_closure' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_closure
/-- info: 'PNP.INEQUALITY.face_diagonal' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_diagonal
/-- info: 'PNP.INEQUALITY.face_formal' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_formal
/-- info: 'PNP.INEQUALITY.face_physical' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_physical
/-- info: 'PNP.INEQUALITY.face_conservation' depends on axioms: [Classical.choice] -/
#guard_msgs in #print axioms PNP.INEQUALITY.face_conservation
/-- info: 'PNP.INEQUALITY.the_faces_are_one_proposition' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.INEQUALITY.the_faces_are_one_proposition
/-- info: 'PNP.INEQUALITY.the_proof' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.INEQUALITY.the_proof
/-- info: 'PNP.INEQUALITY.the_residue_is_the_proof' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.INEQUALITY.the_residue_is_the_proof

/-! ## Part K · PNP.FINALBARRIER: The Barrier, Theorem-ized: the Final Form

    The drawing-board pass, seated. What remains of the P ≠ NP question inside
    this master is not a list of gaps but ONE barrier, and this Part states it
    as one theorem: the barrier is the Inequality; the Inequality is the
    sentence (under the two cited halves); it is one proposition on all six
    faces and on the Fertile Logos seat (the category-subsumption face); it is
    keyed (true at the closed world, false at the stage world, so no uniform
    implication crosses it); no keyless principle forces it; nothing readable
    from a record decides it; and both supplies plug into the sentence. The
    category barrier, the implication barrier, and the execution barrier are
    this one barrier read three ways, and the read is a theorem. -/

namespace PNP.FINALBARRIER

open PNP.NORMALFORM

/-- THE BARRIER: the Inequality, under its final name. -/
def Barrier : Prop := PNP.INEQUALITY.TheInequality

/-- The category-subsumption face: the Fertile Logos seat at `TMSAT`, an
    inhabited `FertileMachine`, the FTOE/category seat of the bit, is the
    Barrier. -/
theorem barrier_subsumption :
    Nonempty (FTOE.FertileMachine PNP.TM.TMSAT) ↔ Barrier :=
  (PNP.SELECTED.sep_iff_fertile_machine PNP.TM.TMSAT).symm.trans
    PNP.INEQUALITY.face_sep

/-- THE BARRIER THEOREM: the master's final form, in one statement with
    nothing before the colon but the two cited halves of Cook–Levin:
    (1) the sentence is the Barrier; (2) the Barrier is one proposition on
    all six faces; (3) the category seat is the Barrier; (4) the Barrier is
    keyed across the two worlds; (5) no keyless principle forces anything;
    (6) no record principle decides the transfer; (7) either supply, as a
    hypothesis, delivers the sentence. This is the whole arc: the question,
    closed to one bit, with the bit seated, faced, fenced, and plugged in,
    the barrier itself, theorem-ized. -/
theorem the_barrier_theorem (hNP : PNP.TM.InNP PNP.TM.satLang)
    (hHard : PNP.CLOSURE.SATHard) :
    (PNP.TM.P_ne_NP ↔ Barrier) ∧
    (Sep PNP.TM.TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (Nonempty (FTOE.FertileMachine PNP.TM.TMSAT) ↔ Barrier) ∧
    ((NothingEscapes (Rec closedWorld) → NothingEscapes closedWorld) ∧
     ¬ (NothingEscapes (Rec (stageWorld 0)) → NothingEscapes (stageWorld 0))) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, (∀ M, P (Rec M)) →
      ¬ (∀ M, P (Rec M) → NothingEscapes M)) ∧
    ((PNP.SUPPLY.FormalCargo → PNP.TM.P_ne_NP) ∧
     (PNP.SUPPLY.PhysicalWitness → PNP.TM.P_ne_NP)) :=
  (PNP.INEQUALITY.the_proof hNP hHard,
   PNP.INEQUALITY.face_sep,
   barrier_subsumption,
   PNP.NORMALFORM.bit_is_keyed,
   fun P hP => PNP.NORMALFORM.keyless_forces_nothing P hP,
   fun P hP => PNP.NORMALFORM.no_record_principle_decides P hP,
   PNP.SUPPLY.p_ne_np_delivered_twice hNP)

end PNP.FINALBARRIER

/-- info: 'PNP.FINALBARRIER.barrier_subsumption' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALBARRIER.barrier_subsumption
/-- info: 'PNP.FINALBARRIER.the_barrier_theorem' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALBARRIER.the_barrier_theorem

/-! ================================================================
    ================================================
    PART L · THE INEQUALITY BY DEGREE, AND ITS FIRST DEGREE CONSTRUCTED
    (namespace PNP.LOCALITY) · v1_6_3_CC

    Purely additive: nothing above is overwritten. Part J named the residue
    `TheInequality` and proved that every face of it is one proposition; Part
    K renamed it the Barrier. A named proposition is still a proposition: it
    is proved only by its witnesses, one string per decider per budget on
    which the verdict is wrong or the clock overruns. This Part constructs
    them where they can be constructed, and states exactly where they cannot.

    `TheInequality` is a universal over degrees (`the_inequality_iff_degrees`),
    and the degrees are a ladder: a witness family at degree k' gives one at
    every degree k ≤ k' (`diagonal_target_at_mono`). Degree 0 is a theorem with a
    computed witness, `witness0`, which returns for every decider and every
    constant a string on which the decider is wrong or late (`witness0_spec`,
    `diagonal_target_at_zero`). The conservation field at degree 0 is inhabited without
    choice (`conservation_field_at_zero`), and no Turing machine decides SAT within a constant number of steps
    (`sat_not_in_P_at_degree_zero`). So the residue is now exactly its tail: `TheInequality` holds iff the Diagonal
    Target holds at every degree k ≥ 1 (`the_inequality_is_its_tail`), iff it holds at cofinally many degrees
    (`the_inequality_iff_cofinal`).

    The engine is locality, proved once for every machine (`run_local`): after t ≤ m steps the state, the head and
    every cell left of m are fixed by the input cells left of m, so two inputs that agree on m cells and differ in
    verdict cost more than m steps on one of them (`prefix_forces_time`). SAT supplies such a pair at every m on its own
    encoding: m unit clauses $x_0$, satisfiable, and the same clauses with the empty clause appended, unsatisfiable.

    What this Part does not do: it constructs no witness at any degree k ≥ 1. A decider that reads its input once
    already runs in linear time, so the cost of reading stops at degree 0. The tail is the open content of the bit, and
    under the two cited halves of Part J it is P ≠ NP. The seat stays vacant. ΔM = 0.
================================================================
    ========================================
    -/
namespace PNP.LOCALITY
open PNP.NORMALFORM PNP.SAT PNP.TM PNP.ZA

/-! ### L.1 · Locality -/

theorem move_le_one (d : Dir) : move d ≤ 1 := by cases d <;> decide

theorem halted_congr {M : TM} {c c' : Config M} (h : c.st = c'.st) :
    halted M c ↔ halted M c' := by
  show (c.st = M.accept ∨ c.st = M.reject) ↔ (c'.st = M.accept ∨ c'.st = M.reject)
  rw [h]

/-- LOCALITY: two inputs whose tapes agree left of m drive a machine identically for m steps: the same state,
    the same head, the head never right of the step count, and the same tape left of m. -/
theorem run_local (M : TM) (w w' : List Sym) (m : Nat)
    (h0 : ∀ i : Int, i < m → initTape M w i = initTape M w' i) :
    ∀ t : Nat, t ≤ m →
      (run M w t).st = (run M w' t).st ∧ (run M w t).pos = (run M w' t).pos ∧
      (run M w t).pos ≤ t ∧ ∀ i : Int, i < m → (run M w t).tape i = (run M w' t).tape i := by
  intro t
  induction t with
  | zero => intro _; exact ⟨rfl, rfl, Int.le_refl 0, h0⟩
  | succ t ih =>
    intro ht
    obtain ⟨hst, hpos, hle, htape⟩ := ih (Nat.le_of_succ_le ht)
    have hp : (run M w t).pos < m := by omega
    have hread : (run M w t).tape (run M w t).pos = (run M w' t).tape
        (run M w' t).pos := by
```

```
      rw [← hpos]; exact htape _ hp
    show (step M (run M w t)).st = (step M (run M w' t)).st ∧
      (step M (run M w t)).pos = (step M (run M w' t)).pos ∧
      (step M (run M w t)).pos ≤ (t + 1 : Nat) ∧
      ∀ i : Int, i < m → (step M (run M w t)).tape i = (step M (run M w'
        t)).tape i
    unfold step
    by_cases hh : halted M (run M w t)
    · rw [if_pos hh, if_pos ((halted_congr hst).mp hh)]
      exact (hst, hpos, by omega, htape)
    · rw [if_neg hh, if_neg (fun h => hh ((halted_congr hst).mpr h))]
      have hr : M.δ (run M w t).st ((run M w t).tape (run M w t).pos) =
          M.δ (run M w' t).st ((run M w' t).tape (run M w' t).pos) := by
            rw [hst, hread]
      have hmv := move_le_one (M.δ (run M w t).st ((run M w t).tape (run
        M w t).pos)).2.2
      refine (?_, ?_, ?_, ?_)
      · dsimp only; rw [hr]
      · dsimp only; rw [hr, hpos]
      · dsimp only; omega
      · intro i hi
        dsimp only
        by_cases hip : i = (run M w t).pos
        · rw [if_pos hip, if_pos (hip.trans hpos), hr]
        · rw [if_neg hip, if_neg (fun h => hip (h.trans hpos.symm)),
          htape i hi]

/-! ### L.2 · Two inputs that agree on a prefix -/

theorem getD_append_left {β : Type} (r : List β) (d : β) :
    ∀ (l : List β) (i : Nat), i < l.length → (l ++ r).getD i d = l.getD i d
  | [], i, h => absurd h (Nat.not_lt_zero i)
  | _ :: _, 0, _ => rfl
  | _ :: l, i+1, h => getD_append_left r d l i (Nat.lt_of_succ_lt_succ h)

theorem length_map_eq {β γ : Type} (f : β → γ) : ∀ l : List β, (l.map
    f).length = l.length
  | [] => rfl
  | _ :: l => congrArg Nat.succ (length_map_eq f l)

theorem map_append_eq {β γ : Type} (f : β → γ) : ∀ l r : List β, (l ++
    r).map f = l.map f ++ r.map f
  | [], _ => rfl
  | a :: l, r => congrArg (List.cons (f a)) (map_append_eq f l r)

theorem length_replicate_eq {β : Type} (a : β) : ∀ m : Nat,
    (List.replicate m a).length = m
  | 0 => rfl
  | m+1 => congrArg Nat.succ (length_replicate_eq a m)

/-- Inputs with a common prefix write the same tape left of the prefix's
    end. -/
theorem init_agree_append (M : TM) (u v v' : List Sym) :
    ∀ i : Int, i < u.length → initTape M (u ++ v) i = initTape M (u ++ v')
      i := by
  intro i hi
  show (if i < 0 then blank M else ((u ++ v).map (Sym.toTape M.s)).getD
    i.toNat (blank M)) =
    (if i < 0 then blank M else ((u ++ v').map (Sym.toTape M.s)).getD
      i.toNat (blank M))
  by_cases hneg : i < 0
  · rw [if_pos hneg, if_pos hneg]
  · rw [if_neg hneg, if_neg hneg]
    have hi' : i.toNat < (u.map (Sym.toTape M.s)).length := by
      rw [length_map_eq]; omega
    rw [map_append_eq, map_append_eq, getD_append_left _ _ _ _ hi',
      getD_append_left _ _ _ _ hi']

/-- TWO INPUTS, ONE PREFIX: a machine that halts on two inputs whose
    tapes agree on the first m cells, with
    different verdicts, runs more than m steps on one of them. -/
theorem prefix_forces_time (M : TM) (w w' : List Sym) (m t t' : Nat)
    (h0 : ∀ i : Int, i < m → initTape M w i = initTape M w' i)
    (ht : HaltsAt M w t) (ht' : HaltsAt M w' t')
    (hv : verdictAt M w t ≠ verdictAt M w' t') : m < t ∨ m < t' :=
  match Nat.lt_or_ge m t, Nat.lt_or_ge m t' with
  | Or.inl h, _ => Or.inl h
  | Or.inr _, Or.inl h' => Or.inr h'
  | Or.inr hle, Or.inr hle' => by
    have e : verdictAt M w m = verdictAt M w' m := by
      unfold verdictAt; rw [(run_local M w w' m h0 m (Nat.le_refl m)).1]
    rw [verdict_of_le ht hle, verdict_of_le ht' hle'] at e
    exact absurd e hv

/-- On deciders: two inputs with a common m-cell prefix and different
    outputs cost more than m steps on one. -/
theorem decider_prefix_time (D : Decider) (w w' : List Sym) (m : Nat)
    (h0 : ∀ i : Int, i < m → initTape D.M w i = initTape D.M w' i)
    (hv : out D w ≠ out D w') : m < time D w ∨ m < time D w' :=
  prefix_forces_time D.M w w' m _ _ h0 (time_halts D w) (time_halts D w')
    hv

/-! ### L.3 · SAT supplies the pair at every length -/

/-- m unit clauses x₀. -/
def satPad (m : Nat) : CNF := List.replicate m [(0, true)]

theorem encode_append : ∀ f g : CNF, encode (f ++ g) = encode f ++ encode
    g
  | [], _ => rfl
  | c :: f, g => by
    show encClause c ++ encode (f ++ g) = (encClause c ++ encode f) ++
      encode g
    rw [encode_append f g, append_assoc]

/-- The satisfiable string: the encoding of m unit clauses x₀. -/
def satYes (m : Nat) : List Sym := encode (satPad m)
/-- The unsatisfiable string: the same clauses and the empty clause. It
    extends the satisfiable string. -/
def satNo (m : Nat) : List Sym := encode (satPad m ++ [[]])

theorem satNo_eq (m : Nat) : satNo m = satYes m ++ [Sym.sep] :=
  (encode_append (satPad m) [[]]).trans rfl

theorem evalCNF_pad (m : Nat) : evalCNF (fun _ => true) (satPad m) = true
    := by
  induction m with
  | zero => rfl
  | succ m ih =>
    show (evalClause (fun _ => true) [(0, true)] && evalCNF (fun _ =>
      true) (satPad m)) = true
    exact (congrArg (fun b => evalClause (fun _ => true) [(0, true)] &&
      b) ih).trans rfl

theorem evalCNF_append_empty (a : Assignment) : ∀ f : CNF, evalCNF a (f
    ++ [[]]) = false
  | [] => rfl
  | c :: f => by
    show (evalClause a c && evalCNF a (f ++ [[]])) = false
    rw [evalCNF_append_empty a f]
    cases evalClause a c <;> rfl

theorem satLang_satYes (m : Nat) : satLang (satYes m) = true :=
  (satLang_encode_iff (satPad m)).mpr (fun _ => true, evalCNF_pad m)

theorem satNo_unsat (m : Nat) : ¬ Satisfiable (satPad m ++ [[]]) :=
  fun (a, ha) => Bool.noConfusion ((evalCNF_append_empty a (satPad
    m)).symm.trans ha)

theorem satLang_satNo (m : Nat) : satLang (satNo m) = false := by
  show satLang (encode (satPad m ++ [[]])) = false
  rw [satLang_encode]
  cases h : satB (satPad m ++ [[]]) with
  | false => rfl
  | true => exact absurd (satB_sound _ h) (satNo_unsat m)

theorem satYes_length (m : Nat) : m ≤ (satYes m).length := by
  have h := encode_length (satPad m)
  rw [show (satPad m).length = m from length_replicate_eq _ m] at h
  exact h

/-- The pair agrees on every cell left of the end of the satisfiable
    string. -/
theorem sat_pair_agree (M : TM) (m : Nat) :
    ∀ i : Int, i < (satYes m).length → initTape M (satYes m) i = initTape
      M (satNo m) i := by
  intro i hi
  have h := init_agree_append M (satYes m) [] [Sym.sep] i hi
  rw [List.append_nil] at h
  rw [satNo_eq]
  exact h

/-! ### L.4 · The first slice of the bit -/

/-- THE k-TH SLICE OF THE BIT: every decider fails SAT at every budget of
    degree k. -/
def BitSlice (k : Nat) : Prop := ∀ (D : Decider) (c : Nat), ∃ w, ¬ Clean
    TMSAT D c k w

/-- The bit at TMSAT is the universal over its slices. -/
theorem nothing_escapes_iff_slices : NothingEscapes TMSAT ↔ ∀ k, BitSlice
    k :=
  (fun h k D c => h D c k, fun h D c k => h k D c)

theorem bound_zero (c n : Nat) : bound c 0 n = c + c := by
  show c * n ^ 0 + c = c + c
  rw [Nat.pow_zero, Nat.mul_one]

/-- THE FIRST SLICE, PROVED: every decider fails SAT at every constant
    budget. -/
theorem bit_slice_zero : BitSlice 0 := by
  intro D c
  have hagree : ∀ i : Int, i < (c + c : Nat) →
      initTape D.M (satYes (c + c)) i = initTape D.M (satNo (c + c)) i :=
    fun i hi => sat_pair_agree D.M (c + c) i (by have := satYes_length (c
      + c); omega)
  cases hY : out D (satYes (c + c)) with
  | false =>
    exact (satYes (c + c), fun h => Bool.noConfusion (hY.symm.trans
      (h.1.trans (satLang_satYes _))))
  | true =>
    cases hN : out D (satNo (c + c)) with
    | true =>
      exact (satNo (c + c), fun h => Bool.noConfusion (hN.symm.trans
        (h.1.trans (satLang_satNo _))))
    | false =>
      have hne : out D (satYes (c + c)) ≠ out D (satNo (c + c)) := by
        rw [hY, hN]; exact fun h => Bool.noConfusion h
      match decider_prefix_time D _ _ (c + c) hagree hne with
      | Or.inl h =>
        exact (satYes (c + c), fun hc =>
          Nat.not_le_of_gt h (Nat.le_trans hc.2 (Nat.le_of_eq (bound_zero
            c _))))
      | Or.inr h =>
        exact (satNo (c + c), fun hc =>
          Nat.not_le_of_gt h (Nat.le_trans hc.2 (Nat.le_of_eq (bound_zero
            c _))))

/-- SAT ∈ P AT DEGREE k: some Turing machine decides SAT within c·n^k + c
    steps. -/
def SATinPAt (k : Nat) : Prop := ∃ (M : TM) (c : Nat), ∀ w : List Sym,
  ∃ t, t ≤ bound c k w.length ∧ HaltsAt M w t ∧ verdictAt M w t = satLang w
```

```

/-- SAT ∉ P is the universal over degrees. -/
theorem not_sat_in_P_iff_degrees : ¬ SATinP ↔ ∀ k, ¬ SATinPAt k :=
  (fun h k (M, c, hM) => h (M, c, k, hM), fun h (M, c, k, hM) => h k (M,
    c, hM))

/-- THE FIRST DEGREE, PROVED: no Turing machine decides SAT within a
    constant number of steps. -/
theorem sat_not_in_P_at_degree_zero : ¬ SATinPAt 0 := by
  intro (M, c, hM)
  obtain (t₁, hle₁, ht₁, hv₁) := hM (satYes (c + c))
  obtain (t₀, hle₀, ht₀, hv₀) := hM (satNo (c + c))
  have hagree : ∀ i : Int, i < (c + c : Nat) →
      initTape M (satYes (c + c)) i = initTape M (satNo (c + c)) i :=
    fun i hi => sat_pair_agree M (c + c) i (by have := satYes_length (c +
      c); omega)
  have hne : verdictAt M (satYes (c + c)) t₁ ≠ verdictAt M (satNo (c +
    c)) t₀ := by
    rw [hv₁, hv₀, satLang_satYes, satLang_satNo]; exact fun h =>
      Bool.noConfusion h
  rw [bound_zero] at hle₁ hle₀
  match prefix_forces_time M _ _ (c + c) t₁ t₀ hagree ht₁ ht₀ hne with
  | Or.inl h => exact Nat.not_le_of_gt h hle₁
  | Or.inr h => exact Nat.not_le_of_gt h hle₀

/-! ### L.5 · The Diagonal Target by degree, and its first degree
    constructed -/

/-- THE DIAGONAL TARGET AT DEGREE k: the main line's target with the
    degree fixed. -/
def DiagonalTargetAt (k : Nat) : Prop :=
  ∀ (A : Decider) (c : Nat), ∃ x : List Sym, out A x ≠ satLang x ∨ bound c
    k x.length < time A x

/-- The Diagonal Target is the universal over its degrees. -/
theorem diagonal_target_iff_degrees : PNP.MAINLINE.DiagonalTarget ↔ ∀ k,
    DiagonalTargetAt k :=
  (fun h k A c => h A c k, fun h A c k => h k A c)

/-- At each degree, the inequality form and the kernel's slice are one
    statement. -/
theorem diagonal_target_at_iff_bit_slice (k : Nat) : DiagonalTargetAt k ↔
    BitSlice k := by
  constructor
  · intro h A c
    obtain (x, hx) := h A c
    refine (x, fun hC => ?_)
    rcases hx with h1 | h2
    · exact h1 hC.1
    · exact Nat.not_lt_of_ge hC.2 h2
  · intro h A c
    obtain (x, hx) := h A c
    refine (x, ?_)
    by_cases h1 : out A x = satLang x
    · right
      rcases Nat.lt_or_ge (bound c k x.length) (time A x) with hb | hb
      · exact hb
      · exact absurd (h1, hb) hx
    · left; exact h1

/-- THE WITNESS AT DEGREE 0, COMPUTED: the satisfiable string if the
    decider rejects it, the unsatisfiable string if
    the decider accepts it, and otherwise whichever of the two the
      decider reads past the budget on. -/
def witness0 (A : Decider) (c : Nat) : List Sym :=
  if out A (satYes (c + c)) = false then satYes (c + c)
  else if out A (satNo (c + c)) = true then satNo (c + c)
  else if c + c < time A (satYes (c + c)) then satYes (c + c) else satNo
    (c + c)

/-- THE WITNESS WORKS: on `witness0 A c` the decider's verdict is wrong
    or its clock overruns c·n⁰ + c. -/
theorem witness0_spec (A : Decider) (c : Nat) :
    out A (witness0 A c) ≠ satLang (witness0 A c) ∨
      bound c 0 (witness0 A c).length < time A (witness0 A c) := by
  have hagree : ∀ i : Int, i < (c + c : Nat) →
      initTape A.M (satYes (c + c)) i = initTape A.M (satNo (c + c)) i :=
    fun i hi => sat_pair_agree A.M (c + c) i (by have := satYes_length (c
      + c); omega)
  unfold witness0
  by_cases hY : out A (satYes (c + c)) = false
  · rw [if_pos hY]
    left
    rw [hY, satLang_satYes]
    exact fun h => Bool.noConfusion h
  · rw [if_neg hY]
    by_cases hN : out A (satNo (c + c)) = true
    · rw [if_pos hN]
      left
      rw [hN, satLang_satNo]
      exact fun h => Bool.noConfusion h
    · rw [if_neg hN]
      have hY' : out A (satYes (c + c)) = true := by
        cases h : out A (satYes (c + c)) with
        | true => rfl
        | false => exact absurd h hY
      have hN' : out A (satNo (c + c)) = false := by
        cases h : out A (satNo (c + c)) with
        | false => rfl
        | true => exact absurd h hN
      have hne : out A (satYes (c + c)) ≠ out A (satNo (c + c)) := by
        rw [hY', hN']
        exact fun h => Bool.noConfusion h
      by_cases hT : c + c < time A (satYes (c + c))
      · rw [if_pos hT]
        right
        rw [bound_zero]
        exact hT
      · rw [if_neg hT]
        right
        rw [bound_zero]
        exact (decider_prefix_time A _ _ (c + c) hagree hne).resolve_left
          hT

/-- THE MAIN LINE'S TARGET AT DEGREE 0, CONSTRUCTED: every decider is
    wrong or late at every constant budget. -/
theorem diagonal_target_at_zero : DiagonalTargetAt 0 :=
  fun A c => (witness0 A c, witness0_spec A c)

/-- THE CONSERVATION FIELD AT DEGREE 0, INHABITED WITHOUT CHOICE: the
    witness is `witness0` and the defect is 1. -/
theorem conservation_field_at_zero :
    ∃ (w : Decider → Nat → List Sym) (d : Decider → Nat → Nat),
      (∀ A c, 0 < d A c) ∧ ∀ A c, ¬ Clean TMSAT A c 0 (w A c) :=
  ⟨witness0, fun _ _ => 1, fun _ _ => Nat.zero_lt_one, fun A c hC =>
    (witness0_spec A c).elim (fun h1 => h1 hC.1) (fun h2 =>
      Nat.not_lt_of_ge hC.2 h2)⟩


/-! ### L.6 · The Inequality by degree: the ladder, the first rung, and
    the tail -/

/-- Budgets are monotone in the degree, at the cost of doubling the
    constant. -/
theorem bound_mono (c k k' n : Nat) (h : k ≤ k') : bound c k n ≤ bound (c
    + c) k' n := by
  cases n with
  | zero =>
    cases k with
    | zero =>
      show c * 0 ^ 0 + c ≤ (c + c) * 0 ^ k' + (c + c)
      rw [Nat.pow_zero, Nat.mul_one]
      exact Nat.le_add_left _ _
    | succ k =>
      show c * 0 ^ (k + 1) + c ≤ (c + c) * 0 ^ k' + (c + c)
      rw [Nat.pow_succ, Nat.mul_zero, Nat.mul_zero, Nat.zero_add]
      exact Nat.le_trans (Nat.le_add_right c c) (Nat.le_add_left _ _)
  | succ n =>
    show c * (n + 1) ^ k + c ≤ (c + c) * (n + 1) ^ k' + (c + c)
    exact Nat.add_le_add
      (Nat.mul_le_mul (Nat.le_add_right c c) (Nat.pow_le_pow_right
        (Nat.succ_pos n) h))
      (Nat.le_add_right c c)

/-- THE LADDER: a witness family at degree k' gives one at every lower
    degree. -/
theorem diagonal_target_at_mono {k k' : Nat} (h : k ≤ k') (hk' :
    DiagonalTargetAt k') :
    DiagonalTargetAt k :=
  fun A c => (hk' A (c + c)).elim fun x hx =>
    ⟨x, hx.elim Or.inl (fun hb => Or.inr (Nat.lt_of_le_of_lt (bound_mono
      c k k' x.length h) hb))⟩

/-- THE INEQUALITY IS THE UNIVERSAL OVER ITS DEGREES. -/
theorem the_inequality_iff_degrees : PNP.INEQUALITY.TheInequality ↔ ∀ k,
    DiagonalTargetAt k :=
  PNP.INEQUALITY.face_diagonal.symm.trans diagonal_target_iff_degrees

/-- THE RESIDUE IS ITS TAIL: with degree 0 constructed, the Inequality is
    exactly the Diagonal Target at every
    degree k ≥ 1. -/
theorem the_inequality_is_its_tail :
    PNP.INEQUALITY.TheInequality ↔ ∀ k, 1 ≤ k → DiagonalTargetAt k := by
  rw [the_inequality_iff_degrees]
  constructor
  · intro h k _
    exact h k
  · intro h k
    cases k with
    | zero => exact diagonal_target_at_zero
    | succ k => exact h (k + 1) (Nat.succ_le_succ (Nat.zero_le k))

/-- COFINALITY: the Inequality holds iff the Diagonal Target holds at
    arbitrarily high degrees. No finite set of
    degrees reaches it, and every degree reached settles all degrees
      below it. -/
theorem the_inequality_iff_cofinal :
    PNP.INEQUALITY.TheInequality ↔ ∀ K, ∃ k, K ≤ k ∧ DiagonalTargetAt k
      := by
  rw [the_inequality_iff_degrees]
  constructor
  · intro h K
    exact (K, Nat.le_refl K, h K)
  · intro h k
    obtain ⟨k', hk, hk'⟩ := h k
    exact diagonal_target_at_mono hk hk'

/-- THE BARRIER IS ITS TAIL: Part K's Barrier, by degree, is the Diagonal
    Target at every degree k ≥ 1. -/
theorem the_barrier_is_its_tail :
    PNP.FINALBARRIER.Barrier ↔ ∀ k, 1 ≤ k → DiagonalTargetAt k :=
  the_inequality_is_its_tail

/-- THE INEQUALITY BY DEGREE, CLOSED, nothing before the colon: locality
    on every machine; the Inequality is the
    universal over its degrees; the degrees form a ladder; the first rung
      is a theorem with a computed witness, in the
    main line's vocabulary, in the kernel's, and on machines; the residue
      is exactly its tail and exactly its cofinal
    degrees; and the stage world, which reads for free, escapes at budget
      0, so the first rung is a fact of the real
    model and not of the vocabulary. -/
```

```
theorem the_inequality_by_degree :
    (∀ (M : TM) (w w' : List Sym) (m t t' : Nat),
      (∀ i : Int, i < m → initTape M w i = initTape M w' i) →
      HaltsAt M w t → HaltsAt M w' t' → verdictAt M w t ≠ verdictAt M w'
        t' → m < t ∨ m < t') ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, DiagonalTargetAt k) ∧
    (∀ k k', k ≤ k' → DiagonalTargetAt k' → DiagonalTargetAt k) ∧
    DiagonalTargetAt 0 ∧ BitSlice 0 ∧ ¬ SATinPAt 0 ∧
    (∀ (A : Decider) (c : Nat), out A (witness0 A c) ≠ satLang (witness0
      A c) ∨
      bound c 0 (witness0 A c).length < time A (witness0 A c)) ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, 1 ≤ k → DiagonalTargetAt k) ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ K, ∃ k, K ≤ k ∧ DiagonalTargetAt k)
      ∧
    Escapes (stageWorld 0) (0 : Nat) 0 0 :=
  (prefix_forces_time, the_inequality_iff_degrees, fun _ _ h hk' =>
    diagonal_target_at_mono h hk',
   diagonal_target_at_zero, bit_slice_zero, sat_not_in_P_at_degree_zero,
     witness0_spec,
   the_inequality_is_its_tail, the_inequality_iff_cofinal, stage_escapes
     0)

end PNP.LOCALITY


/-- info: 'PNP.LOCALITY.move_le_one' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.move_le_one
/-- info: 'PNP.LOCALITY.halted_congr' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.halted_congr
/-- info: 'PNP.LOCALITY.run_local' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.run_local
/-- info: 'PNP.LOCALITY.getD_append_left' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.LOCALITY.getD_append_left
/-- info: 'PNP.LOCALITY.length_map_eq' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.length_map_eq
/-- info: 'PNP.LOCALITY.map_append_eq' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.map_append_eq
/-- info: 'PNP.LOCALITY.length_replicate_eq' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.length_replicate_eq
/-- info: 'PNP.LOCALITY.init_agree_append' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.init_agree_append
/-- info: 'PNP.LOCALITY.prefix_forces_time' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.prefix_forces_time
/-- info: 'PNP.LOCALITY.decider_prefix_time' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.decider_prefix_time
/-- info: 'PNP.LOCALITY.encode_append' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.encode_append
/-- info: 'PNP.LOCALITY.satNo_eq' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.satNo_eq
/-- info: 'PNP.LOCALITY.evalCNF_pad' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.evalCNF_pad
/-- info: 'PNP.LOCALITY.evalCNF_append_empty' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.evalCNF_append_empty
/-- info: 'PNP.LOCALITY.satLang_satYes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.satLang_satYes
/-- info: 'PNP.LOCALITY.satNo_unsat' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.satNo_unsat
/-- info: 'PNP.LOCALITY.satLang_satNo' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.satLang_satNo
/-- info: 'PNP.LOCALITY.satYes_length' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.satYes_length
/-- info: 'PNP.LOCALITY.sat_pair_agree' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.sat_pair_agree
/-- info: 'PNP.LOCALITY.nothing_escapes_iff_slices' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.nothing_escapes_iff_slices
/-- info: 'PNP.LOCALITY.bound_zero' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.bound_zero
/-- info: 'PNP.LOCALITY.bit_slice_zero' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.bit_slice_zero
/-- info: 'PNP.LOCALITY.not_sat_in_P_iff_degrees' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.not_sat_in_P_iff_degrees
/-- info: 'PNP.LOCALITY.sat_not_in_P_at_degree_zero' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.sat_not_in_P_at_degree_zero
/-- info: 'PNP.LOCALITY.diagonal_target_iff_degrees' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.diagonal_target_iff_degrees
/-- info: 'PNP.LOCALITY.diagonal_target_at_iff_bit_slice' does not depend
    on any axioms -/
#guard_msgs in #print axioms
    PNP.LOCALITY.diagonal_target_at_iff_bit_slice
/-- info: 'PNP.LOCALITY.witness0_spec' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.witness0_spec
/-- info: 'PNP.LOCALITY.diagonal_target_at_zero' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.diagonal_target_at_zero
/-- info: 'PNP.LOCALITY.conservation_field_at_zero' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.conservation_field_at_zero
/-- info: 'PNP.LOCALITY.bound_mono' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.bound_mono
/-- info: 'PNP.LOCALITY.diagonal_target_at_mono' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.diagonal_target_at_mono
/-- info: 'PNP.LOCALITY.the_inequality_iff_degrees' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.LOCALITY.the_inequality_iff_degrees
/-- info: 'PNP.LOCALITY.the_inequality_is_its_tail' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.the_inequality_is_its_tail
/-- info: 'PNP.LOCALITY.the_inequality_iff_cofinal' depends on axioms:
    [propext] -/
#guard_msgs in #print axioms PNP.LOCALITY.the_inequality_iff_cofinal
/-- info: 'PNP.LOCALITY.the_barrier_is_its_tail' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.the_barrier_is_its_tail
/-- info: 'PNP.LOCALITY.the_inequality_by_degree' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.LOCALITY.the_inequality_by_degree

/-! ============================================================
    ============================================
    PART M · DEGREE ONE BY CROSSING SEQUENCES (namespace PNP.CROSSING) ·
      v1_6_3_CC

    Purely additive: nothing above is overwritten. Part L constructed the
      Diagonal Target at degree 0 and left the
    residue as its tail, the degrees k ≥ 1. This Part constructs degree
      1: for every decider and every constant c
    there is an input on which the decider is wrong or runs more than c·n
      + c steps (`diagonal_target_at_one`). No
    Turing machine decides SAT within a linear budget. The proof is
      Hennie's crossing-sequence method, run on the
    kernel's own one-tape model, its own encoding and its own running
      time.

    The crossing sequence of a run at a boundary β is the list of states
      the machine enters as its head crosses
    between cells β - 1 and β (`cs`). Cut and paste (`cut_paste`): if two
      halting runs leave the same crossing
    sequence at a boundary β > 0, the input made of the left part of the
      first and the right part of the second
    halts, in the halting state of one of the two. Crossings cost time
      (`crossings_le_time`): a step crosses at most
    one boundary. The fooling family is SAT's own: for bit lists a and b
      of one length K, the unit clauses
    x_(i+1) = a_i, a pad of unit clauses x_0, and the unit clauses
      x_(i+1) = b_i form a formula that is satisfiable
    iff a = b (`fool_sat`, `fool_unsat`). A decider right on the family
      leaves distinct crossing sequences at every
    boundary of the pad on distinct diagonal strings (`fool_cs_inj`). By
      pigeonhole and averaging, some diagonal
    string has crossing sequences longer than 4c at half of the 4λ + 1
      boundaries of the pad, where λ is the length
    of the encoded units, so its running time exceeds c·n + c at n = 6λ
      (`fool_long`).

    What this Part does not do: it constructs no witness at any degree k
      ≥ 2. Every degree k ≥ 2 stays open. The
    residue is now exactly the Diagonal Target at every degree k ≥ 2:
      `TheInequality` holds iff the Diagonal Target
    holds there (`the_inequality_from_degree_two`), and the Barrier of
      Part K is the same statement
    (`the_barrier_from_degree_two`). The seat stays vacant. ΔM = 0.
================================================================
    ============================================
    -/
namespace PNP.CROSSING
open PNP.NORMALFORM PNP.SAT PNP.TM PNP.ZA PNP.LOCALITY

/-! ### M.1 · Crossing sequences -/

/-- The side of cell p with respect to the boundary β, which sits between
    cells β - 1 and β: true on the left. -/
def side (β p : Int) : Bool := decide (p < β)

/-- The head crosses the boundary β during step t. -/
def crossAt (M : TM) (w : List Sym) (β : Int) (t : Nat) : Bool :=
  side β (run M w t).pos != side β (run M w (t+1)).pos

/-- THE CROSSING SEQUENCE at β after t steps: the state entered at each
    crossing, in order. -/
def cs (M : TM) (w : List Sym) (β : Int) : Nat → List (Fin M.q)
  | 0 => []
  | t+1 => cs M w β t ++ (if crossAt M w β t then [(run M w (t+1)).st]
    else [])

theorem cx_move (d : Dir) : move d = -1 ∨ move d = 1 := by
  cases d
  · exact Or.inl rfl
  · exact Or.inr rfl

/-- One step moves the head by at most one cell. -/
theorem cx_step_pos (M : TM) (c : Config M) :
    c.pos - 1 ≤ (step M c).pos ∧ (step M c).pos ≤ c.pos + 1 := by
  unfold step
  by_cases h : halted M c
  · rw [if_pos h]; omega
  · rw [if_neg h]
    dsimp only
    rcases cx_move (M.δ c.st (c.tape c.pos)).2.2 with hm | hm <;> omega

/-- One step writes only the scanned cell. -/
theorem cx_step_tape (M : TM) (c : Config M) (i : Int) (h : i ≠ c.pos) :
    (step M c).tape i = c.tape i := by
  unfold step
  by_cases hh : halted M c
  · rw [if_pos hh]
  · rw [if_neg hh]; dsimp only; rw [if_neg h]

```

```
/-- Two configurations with the same state, head and scanned symbol take the same step. -/
theorem cx_step_same (M : TM) (c a : Config M) (hst : c.st = a.st) (hpos : c.pos = a.pos)
    (hrd : c.tape c.pos = a.tape a.pos) :
    (step M c).st = (step M a).st ∧ (step M c).pos = (step M a).pos ∧
      (step M c).tape c.pos = (step M a).tape a.pos := by
  unfold step
  by_cases hh : halted M c
  · rw [if_pos hh, if_pos ((halted_congr hst).mp hh)]
    exact ⟨hst, hpos, hrd⟩
  · rw [if_neg hh, if_neg (fun h => hh ((halted_congr hst).mpr h))]
    have hr : M.δ c.st (c.tape c.pos) = M.δ a.st (a.tape a.pos) := by rw [hst, hrd]
    dsimp only
    refine ⟨by rw [hr], by rw [hr, hpos], ?_⟩
    rw [if_pos rfl, if_pos rfl, hr]

theorem cx_crossAt_iff (M : TM) (w : List Sym) (β : Int) (t : Nat) :
    crossAt M w β t = true ↔ side β (run M w t).pos ≠ side β (run M w (t+1)).pos := bne_iff_ne

theorem cx_noCross_iff (M : TM) (w : List Sym) (β : Int) (t : Nat) :
    crossAt M w β t = false ↔ side β (run M w t).pos = side β (run M w (t+1)).pos := bne_eq_false_iff_eq

theorem cs_succ (M : TM) (w : List Sym) (β : Int) (t : Nat) :
    cs M w β (t+1) = cs M w β t ++ (if crossAt M w β t then [(run M w (t+1)).st] else []) := rfl

theorem cs_cross {M : TM} {w : List Sym} {β : Int} {t : Nat} (h : crossAt M w β t = true) :
    cs M w β (t+1) = cs M w β t ++ [(run M w (t+1)).st] := by
  rw [cs_succ, if_pos h]

theorem cs_nocross {M : TM} {w : List Sym} {β : Int} {t : Nat} (h : crossAt M w β t = false) :
    cs M w β (t+1) = cs M w β t := by
  rw [cs_succ, if_neg (by rw [h]; exact Bool.false_ne_true), List.append_nil]

theorem cs_length_succ (M : TM) (w : List Sym) (β : Int) (t : Nat) :
    (cs M w β (t+1)).length = (cs M w β t).length + (if crossAt M w β t = true then 1 else 0) := by
  cases h : crossAt M w β t
  · rw [cs_nocross h]; rfl
  · rw [cs_cross h, List.length_append]; rfl

/-- Crossing sequences grow by extension. -/
theorem cs_prefix (M : TM) (w : List Sym) (β : Int) {t t' : Nat} (h : t ≤ t') :
    ∃ r, cs M w β t ++ r = cs M w β t' := by
  induction h with
  | refl => exact ⟨[], List.append_nil _⟩
  | @step m _ ih =>
    obtain ⟨r, hr⟩ := ih
    exact ⟨r ++ (if crossAt M w β m then [(run M w (m+1)).st] else []),
      by rw [← List.append_assoc, hr]; rfl⟩

/-- A halted run crosses nothing: its crossing sequences are fixed from the halting step on. -/
theorem cs_halt {M : TM} {w : List Sym} {T : Nat} (β : Int) (h : HaltsAt M w T) :
    ∀ n, cs M w β (T + n) = cs M w β T := by
  intro n
  induction n with
  | zero => rfl
  | succ n ih =>
    have hc : crossAt M w β (T + n) = false := by
      rw [cx_noCross_iff, run_of_le h (Nat.le_add_right T n), run_of_le h (show T ≤ T + n + 1 by omega)]
    exact (cs_nocross hc).trans ih

/-- The first crossing after tp, when the sequence still grows before Tp. -/
theorem cx_first_cross (M : TM) (w : List Sym) (β : Int) (Tp : Nat) :
    ∀ n tp, tp + n = Tp → (cs M w β tp).length < (cs M w β Tp).length →
      ∃ tp', tp ≤ tp' ∧ tp' < Tp ∧ crossAt M w β tp' = true ∧
        ∀ u, tp ≤ u → u < tp' → crossAt M w β u = false := by
  intro n
  induction n with
  | zero =>
    intro tp h hl
    have e : tp = Tp := h
    rw [e] at hl
    exact absurd hl (Nat.lt_irrefl _)
  | succ n ih =>
    intro tp h hl
    cases hc : crossAt M w β tp with
    | true => exact ⟨tp, Nat.le_refl _, by omega, hc, fun u h1 h2 => absurd h2 (by omega)⟩
    | false =>
      have hl' : (cs M w β (tp+1)).length < (cs M w β Tp).length := by rw [cs_nocross hc]; exact hl
      obtain ⟨tp', h1, h2, h3, h4⟩ := ih (tp+1) (by omega) hl'
      refine ⟨tp', by omega, h2, h3, fun u hu1 hu2 => ?_⟩
      rcases Nat.lt_or_ge u (tp+1) with hu | hu
      · have e : u = tp := by omega
        rw [e]; exact hc
      · exact h4 u hu hu2

/-- Without crossings the head keeps its side, the sequence keeps its value, and the far side of the tape is
    untouched. -/
theorem cx_stay (M : TM) (w : List Sym) (β : Int) (tp tp' : Nat)
    (hno : ∀ u, tp ≤ u → u < tp' → crossAt M w β u = false) :
    ∀ n, tp + n ≤ tp' →
      side β (run M w (tp + n)).pos = side β (run M w tp).pos ∧ cs M w β (tp + n) = cs M w β tp ∧
      ∀ i, side β i ≠ side β (run M w tp).pos → (run M w (tp + n)).tape i = (run M w tp).tape i := by
  intro n
  induction n with
  | zero => intro _; exact ⟨rfl, rfl, fun _ _ => rfl⟩
  | succ n ih =>
    intro hle
    obtain ⟨hs, hc, ht⟩ := ih (by omega)
    have hno' := hno (tp + n) (Nat.le_add_right _ _) (by omega)
    have hs' := (cx_noCross_iff M w β (tp+n)).mp hno'
    refine ⟨hs'.symm.trans hs, (cs_nocross hno').trans hc, fun i hi => ?_⟩
    have hip : i ≠ (run M w (tp + n)).pos := by
      intro e; apply hi; rw [e, hs]
    show (step M (run M w (tp + n))).tape i = _
    rw [cx_step_tape M _ i hip, ht i hi]

/-- A crossing from side b lands on the cell next to the boundary. -/
theorem cx_cross_pos (β x y : Int) (b : Bool) (hx : side β x = b) (hy : side β y ≠ b)
    (h1 : x - 1 ≤ y) (h2 : y ≤ x + 1) : y = (if b = true then β else β - 1) := by
  unfold side at hx hy
  cases b
  · simp at hx hy ⊢; omega
  · simp at hx hy ⊢; omega

theorem cx_bool_ne {x b : Bool} (h : x ≠ b) : x = !b := by
  cases x <;> cases b <;> simp_all

/-! ### M.2 · Cut and paste -/

/-- The splice invariant: c runs as the active configuration a, both a and the parked configuration p stand on
    side b, and the tape of c is that of a on side b and that of p on the other side. -/
def cx_G {M : TM} (β : Int) (b : Bool) (c a p : Config M) : Prop :=
  c.st = a.st ∧ c.pos = a.pos ∧ side β a.pos = b ∧ side β p.pos = b ∧
    (∀ i, side β i = b → c.tape i = a.tape i) ∧ (∀ i, side β i ≠ b → c.tape i = p.tape i)

/-- One step of the spliced run: either the active run steps on its side, or it crosses, and then the parked run
    reaches its next crossing with the same state, and the roles swap. -/
theorem cx_advance (M : TM) (β : Int) (w wa wp : List Sym) (b : Bool) (t ta tp Ta Tp : Nat)
    (hTa : HaltsAt M wa Ta) (hta : ta ≤ Ta) (htp : tp ≤ Tp)
    (hcs : cs M wa β ta = cs M wp β tp) (hcsT : cs M wa β Ta = cs M wp β Tp)
    (hG : cx_G β b (run M w t) (run M wa ta) (run M wp tp))
    (hnh : ¬ halted M (run M w t)) :
    ta + 1 ≤ Ta ∧
    ((cs M wa β (ta+1) = cs M wp β tp ∧ cx_G β b (run M w (t+1)) (run M wa (ta+1)) (run M wp tp)) ∨
     (∃ tp', tp ≤ tp' ∧ tp' + 1 ≤ Tp ∧ cs M wa β (ta+1) = cs M wp β (tp'+1) ∧
       cx_G β (!b) (run M w (t+1)) (run M wp (tp'+1)) (run M wa (ta+1)))) := by
  obtain ⟨hst, hpos, hsa, hsp, htl, htr⟩ := hG
  have hlt : ta + 1 ≤ Ta := by
    rcases Nat.lt_or_ge ta Ta with h | h
    · exact h
    · exfalso
      have e : ta = Ta := Nat.le_antisymm hta h
      rw [e] at hst
      exact hnh ((halted_congr hst).mpr hTa)
  refine ⟨hlt, ?_⟩
  have hsc : side β (run M w t).pos = b := by rw [hpos]; exact hsa
  have hrd : (run M w t).tape (run M w t).pos = (run M wa ta).tape (run M wa ta).pos := by
    rw [htl _ hsc, hpos]
  obtain ⟨hst', hpos', htp'⟩ := cx_step_same M _ _ hst hpos hrd
  have hA : ∀ i, side β i = b → (run M w (t+1)).tape i = (run M wa (ta+1)).tape i := by
    intro i hi
    by_cases hip : i = (run M w t).pos
    · have e2 : i = (run M wa ta).pos := hip.trans hpos
      show (step M (run M w t)).tape i = (step M (run M wa ta)).tape i
      rw [hip]; rw [hip] at e2; rw [← e2] at htp'
      exact htp'
    · show (step M (run M w t)).tape i = (step M (run M wa ta)).tape i
      rw [cx_step_tape M _ i hip, cx_step_tape M _ i (by rw [← hpos]; exact hip), htl i hi]
  have hB : ∀ i, side β i ≠ b → (run M w (t+1)).tape i = (run M wp tp).tape i := by
    intro i hi
    have hip : i ≠ (run M w t).pos := fun e => hi (by rw [e]; exact hsc)
    show (step M (run M w t)).tape i = _
    rw [cx_step_tape M _ i hip, htr i hi]
  have hmv := cx_step_pos M (run M wa ta)
  by_cases hsa' : side β (run M wa (ta+1)).pos = b
  · left
    have hnc : crossAt M wa β ta = false := (cx_noCross_iff M wa β ta).mpr (hsa.trans hsa'.symm)
    exact ⟨(cs_nocross hnc).trans hcs, hst', hpos', hsa', hsp, hA, hB⟩
  · right
    have hcr : crossAt M wa β ta = true :=
      (cx_crossAt_iff M wa β ta).mpr (by rw [hsa]; exact fun e => hsa' e.symm)
    have hcs1 : cs M wa β (ta+1) = cs M wp β tp ++ [(run M wa (ta+1)).st] := by
      rw [cs_cross hcr, hcs]
    obtain ⟨r, hr⟩ := cs_prefix M wa β hlt
    have hlen : (cs M wp β tp).length < (cs M wp β Tp).length := by
      have e := congrArg List.length hr
```

```
      rw [← hcsT, ← e, hcs1, List.length_append, List.length_append,
        List.length_singleton]
      omega
    obtain (tp', h1, h2, h3, h4) := cx_first_cross M wp β Tp (Tp - tp) tp
      (by omega) hlen
    obtain (hs1, hc1, ht1) := cx_stay M wp β tp tp' h4 (tp' - tp) (by
      omega)
    rw [show tp + (tp' - tp) = tp' by omega] at hs1 hc1 ht1
    have hsp1 : side β (run M wp tp').pos = b := hs1.trans hsp
    have hsp2 : side β (run M wp (tp'+1)).pos ≠ b := by
      have e := (cx_crossAt_iff M wp β tp').mp h3
      rw [hsp1] at e; exact fun e' => e e'.symm
    have hcs2 : cs M wp β (tp'+1) = cs M wp β tp ++ [(run M wp
      (tp'+1)).st] := by
      rw [cs_cross h3, hc1]
    obtain ⟨r', hr'⟩ := cs_prefix M wp β (show tp' + 1 ≤ Tp by omega)
    have hstate : (run M wa (ta+1)).st = (run M wp (tp'+1)).st := by
      have e0 : cs M wa β (ta+1) ++ r = cs M wp β (tp'+1) ++ r' := by rw
        [hr, hr', hcsT]
      rw [hcs1, hcs2, List.append_assoc, List.append_assoc] at e0
      exact (List.cons.inj (List.append_cancel_left e0)).1
    have hposeq : (run M wa (ta+1)).pos = (run M wp (tp'+1)).pos := by
      have hmv2 := cx_step_pos M (run M wp tp')
      rw [cx_cross_pos β _ _ b hsa hsa' hmv.1 hmv.2, cx_cross_pos β _ _ b
        hsp1 hsp2 hmv2.1 hmv2.2]
    refine (tp', h1, by omega, hcs1.trans (by rw [hstate]; exact
      hcs2.symm), ?_)
    refine (hst'.trans hstate, hpos'.trans hposeq, cx_bool_ne hsp2,
      cx_bool_ne hsa', ?_, ?_)
    · intro i hi
      have hib : side β i ≠ b := by rw [hi]; cases b <;> decide
      have hip : i ≠ (run M wp tp').pos := fun e => hib (by rw [e]; exact
        hsp1)
      rw [hB i hib]
      show (run M wp tp).tape i = (step M (run M wp tp')).tape i
      rw [cx_step_tape M _ i hip, ht1 i (by rw [hsp]; exact hib)]
    · intro i hi
      have hib : side β i = b := by
        cases h : side β i <;> cases b <;> simp_all
      exact hA i hib

/-- The splice invariant holds at every step of the run on the spliced
    input. -/
theorem cx_inv (M : TM) (β : Int) (hβ : 0 < β) (w w1 w2 : List Sym)
    (hL : ∀ i : Int, i < β → initTape M w i = initTape M w1 i)
    (hR : ∀ i : Int, β ≤ i → initTape M w i = initTape M w2 i)
    (T1 T2 : Nat) (h1 : HaltsAt M w1 T1) (h2 : HaltsAt M w2 T2)
    (hcs : cs M w1 β T1 = cs M w2 β T2) :
    ∀ t, ∃ t1 t2, t1 ≤ T1 ∧ t2 ≤ T2 ∧ cs M w1 β t1 = cs M w2 β t2 ∧
      (t ≤ t1 + t2 ∨ halted M (run M w t)) ∧
      (cx_G β true (run M w t) (run M w1 t1) (run M w2 t2) ∨
        cx_G β false (run M w t) (run M w2 t2) (run M w1 t1)) := by
  intro t
  induction t with
  | zero =>
    have hs0 : side β 0 = true := decide_eq_true hβ
    refine (0, 0, Nat.zero_le _, Nat.zero_le _, rfl, Or.inl (Nat.le_refl
      0), Or.inl ?_)
    refine (rfl, rfl, hs0, hs0, fun i hi => hL i (of_decide_eq_true hi),
      fun i hi => hR i ?_)
    have : ¬ i < β := fun h => hi (decide_eq_true h)
    omega
  | succ t ih =>
    obtain (t1, t2, ht1, ht2, hc, hprog, hmode) := ih
    by_cases hh : halted M (run M w t)
    · have e : run M w (t+1) = run M w t := step_halted M _ hh
      refine (t1, t2, ht1, ht2, hc, Or.inr (by rw [e]; exact hh), ?_)
      rw [e]; exact hmode
    · have hp : t ≤ t1 + t2 := hprog.resolve_right hh
      rcases hmode with hG | hG
      · obtain (hlt, hres) := cx_advance M β w w1 w2 true t t1 t2 T1 T2 h1
          ht1 ht2 hc hcs hG hh
        rcases hres with (hc', hG') | (tp', hle, hle2, hc', hG')
        · exact (t1+1, t2, hlt, ht2, hc', Or.inl (by omega), Or.inl hG')
        · exact (t1+1, tp'+1, hlt, hle2, hc', Or.inl (by omega), Or.inr
          hG')
      · obtain (hlt, hres) :=
          cx_advance M β w w2 w1 false t t2 t1 T2 T1 h2 ht2 ht1 hc.symm
            hcs.symm hG hh
        rcases hres with (hc', hG') | (tp', hle, hle2, hc', hG')
        · exact (t1, t2+1, ht1, hlt, hc'.symm, Or.inl (by omega), Or.inr
          hG')
        · exact (tp'+1, t2+1, hle2, hlt, hc'.symm, Or.inl (by omega),
          Or.inl hG')

/-- CUT AND PASTE: if two halting runs have the same crossing sequence at
    a boundary right of the start cell, the
    input made of the left part of the first and the right part of the
    second halts, in the halting state of one of
    the two runs. -/
theorem cut_paste (M : TM) (β : Int) (hβ : 0 < β) (w w1 w2 : List Sym)
    (hL : ∀ i : Int, i < β → initTape M w i = initTape M w1 i)
    (hR : ∀ i : Int, β ≤ i → initTape M w i = initTape M w2 i)
    (T1 T2 : Nat) (h1 : HaltsAt M w1 T1) (h2 : HaltsAt M w2 T2)
    (hcs : cs M w1 β T1 = cs M w2 β T2) :
    ∃ T, HaltsAt M w T ∧ ((run M w T).st = (run M w1 T1).st ∨ (run M w
      T).st = (run M w2 T2).st) := by
  obtain (t1, t2, ht1, ht2, _, hprog, hmode) := cx_inv M β hβ w w1 w2 hL
    hR T1 T2 h1 h2 hcs (T1 + T2)
  have hH : HaltsAt M w (T1 + T2) := by
    rcases hprog with hp | hp
    · have e1 : t1 = T1 := by omega
      have e2 : t2 = T2 := by omega
      rw [e1, e2] at hmode
      rcases hmode with hG | hG
      · exact (halted_congr hG.1).mpr h1
      · exact (halted_congr hG.1).mpr h2
    · exact hp
  refine (T1 + T2, hH, ?_)
  rcases hmode with hG | hG
  · left
    have hh1 : HaltsAt M w1 t1 := (halted_congr hG.1).mp hH
    rw [hG.1, run_of_le hh1 ht1]
  · right
    have hh2 : HaltsAt M w2 t2 := (halted_congr hG.1).mp hH
    rw [hG.1, run_of_le hh2 ht2]

/-! ### M.3 · Crossings cost time -/

/-- A finite sum over a list. -/
def cx_sum {α : Type} : List α → (α → Nat) → Nat
  | [], _ => 0
  | x :: l, f => f x + cx_sum l f

theorem cx_sum_append {α : Type} (l1 l2 : List α) (f : α → Nat) :
    cx_sum (l1 ++ l2) f = cx_sum l1 f + cx_sum l2 f := by
  induction l1 with
  | nil => exact (Nat.zero_add _).symm
  | cons x l ih => show f x + cx_sum (l ++ l2) f = (f x + cx_sum l f) +
      cx_sum l2 f; rw [ih]; omega

theorem cx_sum_map {α γ : Type} (g : α → γ) (l : List α) (f : γ → Nat) :
    cx_sum (l.map g) f = cx_sum l (fun x => f (g x)) := by
  induction l with
  | nil => rfl
  | cons x l ih => show f (g x) + cx_sum (l.map g) f = f (g x) + cx_sum l
      (fun x => f (g x)); rw [ih]

theorem cx_sum_add {α : Type} (l : List α) (f g : α → Nat) :
    cx_sum l (fun x => f x + g x) = cx_sum l f + cx_sum l g := by
  induction l with
  | nil => rfl
  | cons x l ih =>
    show (f x + g x) + cx_sum l (fun x => f x + g x) = (f x + cx_sum l f)
      + (g x + cx_sum l g)
    rw [ih]; omega

theorem cx_sum_mul {α : Type} (l : List α) (k : Nat) (f : α → Nat) :
    cx_sum l (fun x => k * f x) = k * cx_sum l f := by
  induction l with
  | nil => exact (Nat.mul_zero k).symm
  | cons x l ih => show k * f x + cx_sum l (fun x => k * f x) = k * (f x +
      cx_sum l f); rw [ih, Nat.mul_add]

theorem cx_sum_le {α : Type} (l : List α) (f g : α → Nat) (h : ∀ x, x ∈ l
    → f x ≤ g x) :
    cx_sum l f ≤ cx_sum l g := by
  induction l with
  | nil => exact Nat.le_refl 0
  | cons x l ih =>
    exact Nat.add_le_add (h x (List.mem_cons.mpr (Or.inl rfl)))
      (ih (fun y hy => h y (List.mem_cons.mpr (Or.inr hy))))

theorem cx_sum_congr {α : Type} (l : List α) (f g : α → Nat) (h : ∀ x, x
    ∈ l → f x = g x) :
    cx_sum l f = cx_sum l g :=
  Nat.le_antisymm (cx_sum_le l f g (fun x hx => Nat.le_of_eq (h x hx)))
    (cx_sum_le l g f (fun x hx => Nat.le_of_eq (h x hx).symm))

theorem cx_sum_const {α : Type} (l : List α) (k : Nat) : cx_sum l (fun _
    => k) = k * l.length := by
  induction l with
  | nil => rfl
  | cons x l ih => show k + cx_sum l (fun _ => k) = k * (l.length + 1); rw
      [ih, Nat.mul_succ]; omega

theorem cx_sum_zero {α : Type} (l : List α) : cx_sum l (fun _ => 0) = 0
    := by
  rw [cx_sum_const, Nat.zero_mul]

theorem cx_sum_mem {α : Type} (l : List α) (f : α → Nat) (x : α) (hx : x
    ∈ l) : f x ≤ cx_sum l f := by
  induction l with
  | nil => cases hx
  | cons y l ih =>
    rcases List.mem_cons.mp hx with e | e
    · rw [e]; exact Nat.le_add_right _ _
    · exact Nat.le_trans (ih e) (Nat.le_add_left _ _)

/-- Double sums commute. -/
theorem cx_sum_comm {α γ : Type} (l1 : List α) (l2 : List γ) (f : α → γ →
    Nat) :
    cx_sum l1 (fun x => cx_sum l2 (fun y => f x y)) = cx_sum l2 (fun y =>
      cx_sum l1 (fun x => f x y)) := by
  induction l1 with
  | nil => show 0 = cx_sum l2 (fun _ => 0); rw [cx_sum_zero]
  | cons x l ih =>
    show cx_sum l2 (fun y => f x y) + cx_sum l (fun x => cx_sum l2 (fun y
      => f x y)) =
      cx_sum l2 (fun y => f x y + cx_sum l (fun x => f x y))
    rw [ih, cx_sum_add]

/-- Every value occurs at most once in the list. -/
def cx_once {α : Type} [DecidableEq α] (l : List α) : Prop :=
  ∀ v, cx_sum l (fun x => if x = v then 1 else 0) ≤ 1

/-- In a list without repetitions, a property with at most one witness in
    the list is counted at most once. -/
theorem cx_sum_le_one {α : Type} [DecidableEq α] (l : List α) (hl :
    cx_once l) (p : α → Prop)
    [DecidablePred p] (hp : ∀ x, x ∈ l → ∀ y, y ∈ l → p x → p y → x = y) :
    cx_sum l (fun x => if p x then 1 else 0) ≤ 1 := by
```

```
  by_cases h : ∃ x0, x0 ∈ l ∧ p x0
  · obtain (x0, hx0, hpx0) := h
    refine Nat.le_trans (cx_sum_le l _ _ (fun x hx => ?_)) (hl x0)
    by_cases hpx : p x
    · rw [if_pos hpx, if_pos (hp x hx x0 hx0 hpx hpx0)]
      exact Nat.le_refl 1
    · rw [if_neg hpx]; exact Nat.zero_le _
  · rw [cx_sum_congr l _ (fun _ => 0) (fun x hx => if_neg (fun hpx => h
      (x, hx, hpx))), cx_sum_zero]
    exact Nat.zero_le 1

/-- A step crosses at most one boundary. -/
theorem cx_cross_unique (M : TM) (w : List Sym) (β β' : Int) (t : Nat)
    (h : crossAt M w β t = true) (h' : crossAt M w β' t = true) : β = β'
      := by
  rw [cx_crossAt_iff] at h h'
  have hm := cx_step_pos M (run M w t)
  change (run M w t).pos - 1 ≤ (run M w (t+1)).pos ∧ (run M w (t+1)).pos
     ≤ (run M w t).pos + 1 at hm
  unfold side at h h'
  rw [ne_eq, decide_eq_decide] at h h'
  omega

/-- CROSSINGS COST TIME: over boundaries listed at most once each, the
    crossing sequences after t steps have
    total length at most t. -/
theorem crossings_le_time (M : TM) (w : List Sym) (Bs : List Int) (hBs :
    cx_once Bs) :
    ∀ t, cx_sum Bs (fun β => (cs M w β t).length) ≤ t := by
  intro t
  induction t with
  | zero => exact Nat.le_of_eq (cx_sum_zero Bs)
  | succ t ih =>
    rw [cx_sum_congr Bs _ (fun β => (cs M w β t).length + (if crossAt M w
       β t = true then 1 else 0))
      (fun β _ => cs_length_succ M w β t), cx_sum_add]
    have h1 : cx_sum Bs (fun β => if crossAt M w β t = true then 1 else 0)
       ≤ 1 :=
      cx_sum_le_one Bs hBs (fun β => crossAt M w β t = true)
        (fun x _ y _ hx hy => cx_cross_unique M w x y t hx hy)
    omega

/-! ### M.4 · The fooling family -/

/-- The unit clauses x_k = a_0, x_(k+1) = a_1, and so on. -/
def fool_units : Nat → List Bool → CNF
  | _, [] => []
  | k, b :: a => [(k, b)] :: fool_units (k+1) a

/-- P unit clauses x_0. -/
def fool_pad (P : Nat) : CNF := List.replicate P [(0, true)]

/-- THE FOOLING STRING: the units of a, a pad of P clauses x_0, the units
    of b. -/
def fool (P : Nat) (a b : List Bool) : List Sym := encode (fool_units 1 a
    ++ fool_pad P ++ fool_units 1 b)

theorem fool_eq (P : Nat) (a b : List Bool) :
    fool P a b = encode (fool_units 1 a) ++ (encode (fool_pad P) ++
       encode (fool_units 1 b)) := by
  unfold fool; rw [encode_append, encode_append, List.append_assoc]

theorem fool_clause_len (k : Nat) (b : Bool) : (encClause [(k,
    b)]).length = (encNat k).length + 2 := by
  show ((encNat k ++ [symOfBool b]) ++ [Sym.sep]).length = _
  rw [List.length_append, List.length_append]; rfl

/-- The encoded units of a have a length fixed by the length of a. -/
theorem fool_units_len : ∀ (a a' : List Bool) (k : Nat), a.length =
    a'.length →
    (encode (fool_units k a)).length = (encode (fool_units k a')).length
  | [], [], _, _ => rfl
  | [], _ :: _, _, h => absurd h (by simp)
  | _ :: _, [], _, h => absurd h (by simp)
  | b :: a, b' :: a', k, h => by
    show (encClause [(k, b)] ++ encode (fool_units (k+1) a)).length =
      (encClause [(k, b')] ++ encode (fool_units (k+1) a')).length
    rw [List.length_append, List.length_append, fool_clause_len,
       fool_clause_len,
       fool_units_len a a' (k+1) (by simpa using h)]

theorem fool_units_pos (k : Nat) (b : Bool) (a : List Bool) : 1 ≤ (encode
    (fool_units k (b :: a))).length := by
  show 1 ≤ (encClause [(k, b)] ++ encode (fool_units (k+1) a)).length
  rw [List.length_append, fool_clause_len]; omega

theorem fool_pad_len : ∀ P, (encode (fool_pad P)).length = 4 * P
  | 0 => rfl
  | P+1 => by
    show (encClause [(0, true)] ++ encode (fool_pad P)).length = 4 * (P+1)
    rw [List.length_append, fool_pad_len P]
    have h : (encClause [(0, true)]).length = 4 := by decide
    omega

theorem fool_evalCNF_append (σ : Assignment) (f g : CNF) :
    evalCNF σ (f ++ g) = (evalCNF σ f && evalCNF σ g) := List.all_append

theorem fool_units_true (σ : Assignment) : ∀ (a : List Bool) (k : Nat),
    evalCNF σ (fool_units k a) = true →
    ∀ i, i < a.length → σ (k + i) = a.getD i false
  | [], _, _, i, hi => absurd hi (Nat.not_lt_zero i)
  | b :: a, k, h, i, hi => by
    have h' : (evalClause σ [(k, b)] && evalCNF σ (fool_units (k+1) a)) =
       true := h
    rw [Bool.and_eq_true] at h'
    cases i with
    | zero =>
      have hc : ((σ k == b) || false) = true := h'.1
      rw [Bool.or_false, beq_iff_eq] at hc
      rw [Nat.add_zero, List.getD_cons_zero]; exact hc
    | succ i =>
      have := fool_units_true σ a (k+1) h'.2 i (Nat.lt_of_succ_lt_succ hi)
      rw [List.getD_cons_succ, show k + (i+1) = k + 1 + i by omega]
      exact this

theorem fool_units_sat (σ : Assignment) : ∀ (a : List Bool) (k : Nat),
    (∀ i, i < a.length → σ (k + i) = a.getD i false) → evalCNF σ
      (fool_units k a) = true
  | [], _, _ => rfl
  | b :: a, k, h => by
    show (evalClause σ [(k, b)] && evalCNF σ (fool_units (k+1) a)) = true
    rw [Bool.and_eq_true]
    refine ⟨?_, fool_units_sat σ a (k+1) (fun i hi => ?_)⟩
    · show ((σ k == b) || false) = true
      have := h 0 (Nat.zero_lt_succ _)
      rw [Nat.add_zero, List.getD_cons_zero] at this
      rw [this]; simp
    · have := h (i+1) (Nat.succ_lt_succ hi)
      rw [List.getD_cons_succ, show k + (i+1) = k + 1 + i by omega] at this
      exact this

theorem fool_pad_true (σ : Assignment) (h : σ 0 = true) : ∀ P, evalCNF σ
    (fool_pad P) = true
  | 0 => rfl
  | P+1 => by
    show (evalClause σ [(0, true)] && evalCNF σ (fool_pad P)) = true
    rw [fool_pad_true σ h P]
    show (((σ 0 == true) || false) && true) = true
    rw [h]; rfl

theorem fool_ext : ∀ (a b : List Bool), a.length = b.length →
    (∀ i, i < a.length → a.getD i false = b.getD i false) → a = b
  | [], [], _, _ => rfl
  | [], _ :: _, hl, _ => absurd hl (by simp)
  | _ :: _, [], hl, _ => absurd hl (by simp)
  | x :: a, y :: b, hl, h => by
    have hx : x = y := by
      have := h 0 (Nat.zero_lt_succ _); rwa [List.getD_cons_zero,
        List.getD_cons_zero] at this
    have hab : a = b := fool_ext a b (by simpa using hl) (fun i hi => by
      have := h (i+1) (Nat.succ_lt_succ hi); rwa [List.getD_cons_succ,
        List.getD_cons_succ] at this)
    rw [hx, hab]

/-- The fooling string of a with itself is satisfiable. -/
theorem fool_sat (P : Nat) (a : List Bool) : satLang (fool P a a) = true
    := by
  unfold fool
  rw [satLang_encode_iff]
  refine ⟨fun n => if n = 0 then true else a.getD (n - 1) false, ?_⟩
  have hu : evalCNF (fun n => if n = 0 then true else a.getD (n - 1)
     false) (fool_units 1 a) = true :=
    fool_units_sat _ a 1 (fun i _ => by
      show (if 1 + i = 0 then true else a.getD (1 + i - 1) false) = _
      rw [if_neg (by omega), show 1 + i - 1 = i by omega])
  rw [fool_evalCNF_append, fool_evalCNF_append, hu, fool_pad_true _ rfl]
  rfl

/-- For equal lengths and a ≠ b the fooling string is unsatisfiable. -/
theorem fool_unsat (P : Nat) (a b : List Bool) (hlen : a.length =
    b.length) (hne : a ≠ b) :
    satLang (fool P a b) = false := by
  unfold fool
  rw [satLang_encode]
  cases h : satB (fool_units 1 a ++ fool_pad P ++ fool_units 1 b) with
  | false => rfl
  | true =>
    exfalso
    obtain ⟨σ, hσ⟩ := satB_sound _ h
    rw [fool_evalCNF_append, fool_evalCNF_append, Bool.and_eq_true,
       Bool.and_eq_true] at hσ
    exact hne (fool_ext a b hlen (fun i hi =>
      (fool_units_true σ a 1 hσ.1.1 i hi).symm.trans (fool_units_true σ b
        1 hσ.2 i (hlen ▸ hi))))

theorem cx_getD_append_right {γ : Type} (r : List γ) (d : γ) : ∀ (l :
    List γ) (i : Nat), l.length ≤ i →
    (l ++ r).getD i d = r.getD (i - l.length) d
  | [], i, _ => by rw [List.nil_append, List.length_nil, Nat.sub_zero]
  | _ :: _, 0, h => absurd h (Nat.not_succ_le_zero _)
  | x :: l, i+1, h => by
    rw [List.cons_append, List.getD_cons_succ, List.length_cons,
       Nat.add_sub_add_right]
    exact cx_getD_append_right r d l i (Nat.le_of_succ_le_succ h)

/-- Inputs with equally long prefixes and a common suffix write the same
    tape from the suffix on. -/
theorem cx_init_right (M : TM) (u u' z : List Sym) (hu : u.length =
    u'.length) (i : Int)
    (hi : (u.length : Int) ≤ i) : initTape M (u ++ z) i = initTape M (u'
      ++ z) i := by
  have hneg : ¬ i < 0 := by omega
  show (if i < 0 then blank M else ((u ++ z).map (Sym.toTape M.s)).getD
     i.toNat (blank M)) =
    (if i < 0 then blank M else ((u' ++ z).map (Sym.toTape M.s)).getD
      i.toNat (blank M))
  rw [if_neg hneg, if_neg hneg, map_append_eq, map_append_eq,
    cx_getD_append_right (z.map (Sym.toTape M.s)) (blank M) (u.map
       (Sym.toTape M.s)) i.toNat
      (by rw [length_map_eq]; omega),
    cx_getD_append_right (z.map (Sym.toTape M.s)) (blank M) (u'.map
       (Sym.toTape M.s)) i.toNat
```

```
      (by rw [length_map_eq]; omega), length_map_eq, length_map_eq, hu]

/-- Left of the end of the pad, the mixed string reads as the first
    diagonal string. -/
theorem fool_left (M : TM) (P : Nat) (a a' : List Bool) (i : Int)
    (hi : i < ((encode (fool_units 1 a)).length + 4 * P : Nat)) :
    initTape M (fool P a a') i = initTape M (fool P a a) i := by
  rw [fool_eq, fool_eq, ← List.append_assoc, ← List.append_assoc]
  exact init_agree_append M _ _ _ i (by rw [List.length_append,
    fool_pad_len]; exact hi)

/-- From the start of the pad on, the mixed string reads as the second
    diagonal string. -/
theorem fool_right (M : TM) (P : Nat) (a a' : List Bool) (hlen : a.length
    = a'.length) (i : Int)
    (hi : ((encode (fool_units 1 a)).length : Int) ≤ i) :
    initTape M (fool P a a') i = initTape M (fool P a' a') i := by
  rw [fool_eq, fool_eq]
  exact cx_init_right M _ _ _ (fool_units_len a a' 1 hlen) i hi

/-- THE FOOLING ARGUMENT: a decider right on the family has distinct
    crossing sequences, at every boundary of the
    pad, on distinct diagonal strings. -/
theorem fool_cs_inj (A : Decider) (P : Nat) (a a' : List Bool) (hlen :
    a.length = a'.length)
    (β : Int) (hβ0 : 0 < β) (hβ1 : ((encode (fool_units 1 a)).length :
      Int) ≤ β)
    (hβ2 : β ≤ ((encode (fool_units 1 a)).length + 4 * P : Nat))
    (hc1 : out A (fool P a a) = satLang (fool P a a)) (hc2 : out A (fool P
      a' a') = satLang (fool P a' a'))
    (hc3 : out A (fool P a a') = satLang (fool P a a'))
    (heq : cs A.M (fool P a a) β (time A (fool P a a)) = cs A.M (fool P a'
      a') β (time A (fool P a' a'))) :
    a = a' := by
  apply Classical.byContradiction
  intro hne
  obtain ⟨T, hT, hst⟩ := cut_paste A.M β hβ0 (fool P a a') (fool P a a)
     (fool P a' a')
    (fun i hi => fool_left A.M P a a' i (by omega)) (fun i hi =>
      fool_right A.M P a a' hlen i (by omega))
    _ _ (time_halts A _) (time_halts A _) heq
  have hacc1 : (run A.M (fool P a a) (time A (fool P a a))).st =
    A.M.accept :=
    of_decide_eq_true (hc1.trans (fool_sat P a))
  have hacc2 : (run A.M (fool P a' a') (time A (fool P a' a'))).st =
    A.M.accept :=
    of_decide_eq_true (hc2.trans (fool_sat P a'))
  have hacc : (run A.M (fool P a a') T).st = A.M.accept := by
    rcases hst with h | h
    · exact h.trans hacc1
    · exact h.trans hacc2
  have hout : out A (fool P a a') = true := by
    rw [out_eq A _ T hT]; exact decide_eq_true hacc
  rw [hc3, fool_unsat P a a' hlen hne] at hout
  exact Bool.false_ne_true hout

/-! ### M.5 · Counting -/

/-- All bit lists of length K. -/
def allBits : Nat → List (List Bool)
  | 0 => [[]]
  | K+1 => (allBits K).map (List.cons true) ++ (allBits K).map (List.cons
     false)

theorem allBits_length : ∀ K, (allBits K).length = 2 ^ K
  | 0 => rfl
  | K+1 => by
    show ((allBits K).map (List.cons true) ++ (allBits K).map (List.cons
      false)).length = 2 ^ (K+1)
    rw [List.length_append, List.length_map, List.length_map,
      allBits_length K, Nat.pow_succ]; omega

theorem allBits_mem : ∀ K a, a ∈ allBits K → a.length = K
  | 0, a, h => by rw [List.mem_singleton.mp h]; rfl
  | K+1, a, h => by
    rcases List.mem_append.mp h with h' | h'
    · obtain ⟨a', ha', e⟩ := List.mem_map.mp h'
      rw [← e, List.length_cons, allBits_mem K a' ha']
    · obtain ⟨a', ha', e⟩ := List.mem_map.mp h'
      rw [← e, List.length_cons, allBits_mem K a' ha']

theorem allBits_once : ∀ K, cx_once (allBits K)
  | 0 => fun v => by
    show (if [] = v then 1 else 0) + 0 ≤ 1
    split <;> omega
  | K+1 => fun v => by
    show cx_sum ((allBits K).map (List.cons true) ++ (allBits K).map
      (List.cons false))
      (fun x => if x = v then 1 else 0) ≤ 1
    rw [cx_sum_append, cx_sum_map, cx_sum_map]
    rcases v with _ | ⟨b, v⟩
    · simp [cx_sum_zero]
    · have hv := allBits_once K v
      cases b <;> simp [cx_sum_zero] <;> omega

/-- The boundaries s, s+1, ..., s+n-1. -/
def cx_bnds (s : Int) : Nat → List Int
  | 0 => []
  | n+1 => s :: cx_bnds (s+1) n

theorem cx_bnds_length : ∀ (n : Nat) (s : Int), (cx_bnds s n).length = n
  | 0, _ => rfl
  | n+1, s => by show (cx_bnds (s+1) n).length + 1 = n + 1; rw
      [cx_bnds_length n (s+1)]

theorem cx_bnds_mem : ∀ (n : Nat) (s β : Int), β ∈ cx_bnds s n → s ≤ β ∧
    β < s + n
  | 0, _, _, h => by cases h
  | n+1, s, β, h => by
    rcases List.mem_cons.mp h with e | e
    · omega
    · have := cx_bnds_mem n (s+1) β e; omega

theorem cx_bnds_once : ∀ (n : Nat) (s : Int), cx_once (cx_bnds s n)
  | 0, _ => fun _ => Nat.zero_le 1
  | n+1, s => fun v => by
    show (if s = v then 1 else 0) + cx_sum (cx_bnds (s+1) n) (fun x => if
       x = v then 1 else 0) ≤ 1
    by_cases hs : s = v
    · rw [if_pos hs, cx_sum_congr _ _ (fun _ => 0)
        (fun x hx => if_neg (by have := cx_bnds_mem n (s+1) x hx;
          omega)), cx_sum_zero]
      omega
    · rw [if_neg hs]; have := cx_bnds_once n (s+1) v; omega

/-- One more symbol below q in front of each list. -/
def cx_ext (q : Nat) : List (List Nat) → List (List Nat)
  | [] => []
  | l :: ls => (List.range q).map (fun x => x :: l) ++ cx_ext q ls

/-- The lists of length at most n with entries below q. -/
def cx_words (q : Nat) : Nat → List (List Nat)
  | 0 => [[]]
  | n+1 => [] :: cx_ext q (cx_words q n)

theorem cx_ext_length (q : Nat) : ∀ ls, (cx_ext q ls).length = q *
    ls.length
  | [] => rfl
  | l :: ls => by
    show ((List.range q).map (fun x => x :: l) ++ cx_ext q ls).length = q
       * (ls.length + 1)
    rw [List.length_append, List.length_map, List.length_range,
      cx_ext_length q ls, Nat.mul_succ]; omega

theorem cx_words_length (q : Nat) : ∀ n, (cx_words q n).length ≤ (q + 1)
    ^ n
  | 0 => Nat.le_refl 1
  | n+1 => by
    show (cx_ext q (cx_words q n)).length + 1 ≤ (q+1)^(n+1)
    rw [cx_ext_length, Nat.pow_succ, Nat.mul_succ, Nat.mul_comm ((q+1)^n)
       q]
    have h1 : q * (cx_words q n).length ≤ q * (q+1)^n :=
      Nat.mul_le_mul_left q (cx_words_length q n)
    have h2 : 0 < (q+1)^n := Nat.pow_pos (Nat.succ_pos q)
    omega

theorem cx_ext_mem (q x : Nat) (hx : x < q) :
    ∀ (ls : List (List Nat)) (l : List Nat), l ∈ ls → x :: l ∈ cx_ext q ls
  | [], _, h => by cases h
  | l' :: ls, l, h => by
    show x :: l ∈ (List.range q).map (fun x => x :: l') ++ cx_ext q ls
    rcases List.mem_cons.mp h with e | e
    · rw [e]; exact List.mem_append.mpr (Or.inl (List.mem_map.mpr (x,
        List.mem_range.mpr hx, rfl)))
    · exact List.mem_append.mpr (Or.inr (cx_ext_mem q x hx ls l e))

theorem cx_words_mem (q : Nat) : ∀ (n : Nat) (l : List Nat), l.length ≤ n
    → (∀ x, x ∈ l → x < q) →
    l ∈ cx_words q n
  | 0, [], _, _ => List.mem_singleton.mpr rfl
  | 0, _ :: _, h, _ => absurd h (by simp)
  | n+1, [], _, _ => List.mem_cons.mpr (Or.inl rfl)
  | n+1, x :: l, h, hx => List.mem_cons.mpr (Or.inr (cx_ext_mem q x (hx x
      (List.mem_cons.mpr (Or.inl rfl))) _ l
      (cx_words_mem q n l (by simp at h; omega) (fun y hy => hx y
        (List.mem_cons.mpr (Or.inr hy))))))

theorem cx_map_val_inj {q : Nat} : ∀ (l l' : List (Fin q)), l.map Fin.val
    = l'.map Fin.val → l = l'
  | [], [], _ => rfl
  | [], _ :: _, h => absurd h (by simp)
  | _ :: _, [], h => absurd h (by simp)
  | x :: l, y :: l', h => by
    simp only [List.map_cons, List.cons.injEq] at h
    rw [Fin.ext h.1, cx_map_val_inj l l' h.2]

/-- PIGEONHOLE: an injective image inside W counts at most the length of
    W. -/
theorem cx_pigeon {α : Type} [DecidableEq α] (As : List α) (hA : cx_once
    As) (W : List (List Nat))
    (f : α → List Nat) (hinj : ∀ a, a ∈ As → ∀ a', a' ∈ As → f a = f a' →
      a = a')
    (p : α → Prop) [DecidablePred p] (hW : ∀ a, a ∈ As → p a → f a ∈ W) :
    cx_sum As (fun a => if p a then 1 else 0) ≤ W.length := by
  have h1 : cx_sum As (fun a => if p a then 1 else 0) ≤
      cx_sum As (fun a => cx_sum W (fun s => if f a = s then 1 else 0)) :=
        by
    apply cx_sum_le
    intro a ha
    by_cases hp : p a
    · rw [if_pos hp]
      refine Nat.le_trans ?_ (cx_sum_mem W (fun s => if f a = s then 1
        else 0) (f a) (hW a ha hp))
      simp
    · rw [if_neg hp]; exact Nat.zero_le _
  rw [cx_sum_comm] at h1
  have h2 : cx_sum W (fun s => cx_sum As (fun a => if f a = s then 1 else
     0)) ≤ cx_sum W (fun _ => 1) :=
    cx_sum_le W _ _ (fun s _ => cx_sum_le_one As hA (fun a => f a = s)
      (fun x hx y hy ex ey => hinj x hx y hy (ex.trans ey.symm)))
  rw [cx_sum_const, Nat.one_mul] at h2
  exact Nat.le_trans h1 h2
```

```
/-- AVERAGING: if every column carries at least half of the rows, some
    row meets at least half of the columns. -/
theorem cx_average {α γ : Type} (As : List α) (Bs : List γ) (g : α → γ →
    Nat) (N : Nat)
    (hA : 0 < As.length) (hN : As.length ≤ 2 * N) (hcol : ∀ b, b ∈ Bs → N
      ≤ cx_sum As (fun a => g a b)) :
    ∃ a, a ∈ As ∧ Bs.length ≤ 2 * cx_sum Bs (fun b => g a b) := by
  apply Classical.byContradiction
  intro hno
  have hlt : ∀ a, a ∈ As → 2 * cx_sum Bs (fun b => g a b) + 1 ≤ Bs.length
     := by
    intro a ha
    have : ¬ Bs.length ≤ 2 * cx_sum Bs (fun b => g a b) := fun h => hno
      (a, ha, h)
    omega
  have h1 : cx_sum As (fun a => 2 * cx_sum Bs (fun b => g a b) + 1) ≤
     cx_sum As (fun _ => Bs.length) :=
    cx_sum_le As _ _ hlt
  rw [cx_sum_add, cx_sum_mul, cx_sum_const, cx_sum_const, cx_sum_comm] at
     h1
  have h2 : cx_sum Bs (fun _ => N) ≤ cx_sum Bs (fun b => cx_sum As (fun a
    => g a b)) := cx_sum_le Bs _ _ hcol
  rw [cx_sum_const] at h2
  have h3 : Bs.length * As.length ≤ Bs.length * (2 * N) :=
    Nat.mul_le_mul_left _ hN
  have h4 : Bs.length * (2 * N) = 2 * (N * Bs.length) := by
    rw [Nat.mul_left_comm, Nat.mul_comm Bs.length N]
  omega

/-! ### M.6 · Degree one -/

/-- Some diagonal string of a decider right on the family has long
    crossing sequences at half of the boundaries of
    the pad, and so runs long. -/
theorem fool_long (A : Decider) (D K L N : Nat) (hN : (A.M.q + 1) ^ D ≤
    N) (hK : 2 ^ K = 2 * N)
    (hL : ∀ a, a ∈ allBits K → (encode (fool_units 1 a)).length = L) (hL1
      : 1 ≤ L)
    (hcorr : ∀ a, a ∈ allBits K → ∀ b, b ∈ allBits K → out A (fool L a b)
      = satLang (fool L a b)) :
    ∃ a, a ∈ allBits K ∧ (4 * L + 1) * (D + 1) ≤ 2 * time A (fool L a a)
      := by
  have hcol : ∀ β, β ∈ cx_bnds (L : Int) (4 * L + 1) → N ≤ cx_sum (allBits
    K)
      (fun a => if D + 1 ≤ (cs A.M (fool L a a) β (time A (fool L a
        a))).length then 1 else 0) := by
    intro β hβ
    have hβ' := cx_bnds_mem _ _ _ hβ
    have hshort : cx_sum (allBits K)
        (fun a => if (cs A.M (fool L a a) β (time A (fool L a a))).length
           ≤ D then 1 else 0) ≤
        (A.M.q + 1) ^ D := by
      refine Nat.le_trans (cx_pigeon (allBits K) (allBits_once K)
        (cx_words A.M.q D)
        (fun a => (cs A.M (fool L a a) β (time A (fool L a a))).map
          Fin.val) ?_
        (fun a => (cs A.M (fool L a a) β (time A (fool L a a))).length ≤
          D) ?_) (cx_words_length _ _)
      · intro a ha a' ha' e
        have hla : (encode (fool_units 1 a)).length = L := hL a ha
        exact fool_cs_inj A L a a' (by rw [allBits_mem K a ha,
          allBits_mem K a' ha']) β (by omega)
          (by rw [hla]; omega) (by rw [hla]; omega)
          (hcorr a ha a ha) (hcorr a' ha' a' ha') (hcorr a ha a' ha')
            (cx_map_val_inj _ _ e)
      · intro a _ hp
        apply cx_words_mem
        · rw [List.length_map]; exact hp
        · intro x hx
          obtain ⟨y, _, e⟩ := List.mem_map.mp hx
          rw [← e]; exact y.isLt
    have hsum : cx_sum (allBits K)
        (fun a => if D + 1 ≤ (cs A.M (fool L a a) β (time A (fool L a
          a))).length then 1 else 0) +
        cx_sum (allBits K)
        (fun a => if (cs A.M (fool L a a) β (time A (fool L a a))).length
           ≤ D then 1 else 0) = 2 * N := by
      rw [← cx_sum_add, cx_sum_congr _ _ (fun _ => 1) (fun a _ => by
        dsimp only; split <;> split <;> omega),
        cx_sum_const, allBits_length, hK, Nat.one_mul]
    omega
  obtain ⟨a, ha, hab⟩ := cx_average (allBits K) (cx_bnds (L : Int) (4 * L
    + 1)) _ N
    (by rw [allBits_length]; exact Nat.two_pow_pos K) (by rw
      [allBits_length, hK]; exact Nat.le_refl _) hcol
  refine ⟨a, ha, ?_⟩
  rw [cx_bnds_length] at hab
  have ht := crossings_le_time A.M (fool L a a) (cx_bnds (L : Int) (4 * L
    + 1)) (cx_bnds_once _ _)
    (time A (fool L a a))
  have hg : cx_sum (cx_bnds (L : Int) (4 * L + 1)) (fun β => (D + 1) *
      (if D + 1 ≤ (cs A.M (fool L a a) β (time A (fool L a a))).length
         then 1 else 0)) ≤
      cx_sum (cx_bnds (L : Int) (4 * L + 1)) (fun β => (cs A.M (fool L a
        a) β (time A (fool L a a))).length) :=
    cx_sum_le _ _ _ (fun β _ => by split <;> omega)
  rw [cx_sum_mul] at hg
  have h3 := Nat.mul_le_mul_right (D + 1) hab
  have h4 : ∀ x : Nat, (2 * x) * (D + 1) = 2 * ((D + 1) * x) := fun x => by
    rw [Nat.mul_comm (2 * x), Nat.mul_left_comm]
  rw [h4] at h3
  omega

/-- The exponent of the family: 2^K diagonal strings, K = q·4c + 1. -/
def cx_K (A : Decider) (c : Nat) : Nat := A.M.q * (4 * c) + 1

/-- The common length of the encoded units of a bit list of length K. -/
def cx_lam (K : Nat) : Nat := (encode (fool_units 1 (List.replicate K
    false))).length

theorem cx_two_pow (q : Nat) : q + 1 ≤ 2 ^ q := by
  induction q with
  | zero => exact Nat.le_refl 1
  | succ q ih => rw [Nat.pow_succ]; omega

theorem cx_arith (c l T : Nat) (h : (4 * l + 1) * (4 * c + 1) ≤ 2 * T) : c
    * (l + (4 * l + l)) + c < T := by
  have e1 : (4 * l + 1) * (4 * c + 1) = 16 * (c * l) + 4 * l + 4 * c + 1
     := by
    rw [Nat.mul_add, Nat.add_mul, Nat.add_mul, Nat.mul_mul_mul_comm,
      Nat.mul_comm l c]
    omega
  have e2 : c * (l + (4 * l + l)) = 6 * (c * l) := by
    rw [show l + (4 * l + l) = 6 * l by omega, Nat.mul_left_comm]
  omega

/-- THE DIAGONAL TARGET AT DEGREE 1: for every decider and every c, some
    input on which the decider is wrong or runs
    more than c·n + c steps. No Turing machine decides SAT in linear
    time. -/
theorem diagonal_target_at_one : DiagonalTargetAt 1 := by
  intro A c
  have hlam : ∀ a, a ∈ allBits (cx_K A c) → (encode (fool_units 1
    a)).length = cx_lam (cx_K A c) :=
    fun a ha => fool_units_len a _ 1 (by rw [allBits_mem _ a ha,
      List.length_replicate])
  have hlam1 : 1 ≤ cx_lam (cx_K A c) := fool_units_pos 1 false
    (List.replicate (A.M.q * (4 * c)) false)
  have hN : (A.M.q + 1) ^ (4 * c) ≤ 2 ^ (A.M.q * (4 * c)) := by
    rw [Nat.pow_mul 2 A.M.q (4 * c)]; exact Nat.pow_le_pow_left
      (cx_two_pow A.M.q) _
  have hK : 2 ^ cx_K A c = 2 * 2 ^ (A.M.q * (4 * c)) := by
    show 2 ^ (A.M.q * (4 * c) + 1) = _
    rw [Nat.pow_succ, Nat.mul_comm]
  by_cases hbad : ∃ a, a ∈ allBits (cx_K A c) ∧ ∃ b, b ∈ allBits (cx_K A
    c) ∧
      out A (fool (cx_lam (cx_K A c)) a b) ≠ satLang (fool (cx_lam (cx_K
        A c)) a b)
  · obtain ⟨a, _, b, _, hne⟩ := hbad
    exact (_, Or.inl hne)
  · have hcorr : ∀ a, a ∈ allBits (cx_K A c) → ∀ b, b ∈ allBits (cx_K A c)
    →
        out A (fool (cx_lam (cx_K A c)) a b) = satLang (fool (cx_lam
          (cx_K A c)) a b) :=
      fun a ha b hb => Classical.byContradiction (fun hne => hbad ⟨a, ha,
        b, hb, hne⟩)
    obtain ⟨a, ha, hT⟩ := fool_long A (4 * c) (cx_K A c) (cx_lam (cx_K A
      c)) _ hN hK hlam hlam1 hcorr
    refine ⟨fool (cx_lam (cx_K A c)) a a, Or.inr ?_⟩
    have hlen : (fool (cx_lam (cx_K A c)) a a).length =
        cx_lam (cx_K A c) + (4 * cx_lam (cx_K A c) + cx_lam (cx_K A c)) :=
          by
      rw [fool_eq, List.length_append, List.length_append, fool_pad_len,
        hlam a ha]
    show c * (fool (cx_lam (cx_K A c)) a a).length ^ 1 + c < time A (fool
      (cx_lam (cx_K A c)) a a)
    rw [Nat.pow_one, hlen]
    exact cx_arith c _ _ hT

/-- THE INEQUALITY FROM DEGREE TWO: with degrees 0 and 1 constructed, the
    Inequality is exactly the Diagonal Target at
    every degree k ≥ 2. -/
theorem the_inequality_from_degree_two :
    PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k := by
  rw [the_inequality_is_its_tail]
  constructor
  · intro h k hk
    exact h k (by omega)
  · intro h k hk
    rcases Nat.lt_or_ge k 2 with hlt | hge
    · have e : k = 1 := by omega
      rw [e]; exact diagonal_target_at_one
    · exact h k hge

/-- THE BARRIER FROM DEGREE TWO: Part K's Barrier is the Diagonal Target
    at every degree k ≥ 2. -/
theorem the_barrier_from_degree_two :
    PNP.FINALBARRIER.Barrier ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k :=
  the_inequality_from_degree_two

/-- DEGREE ONE, CLOSED, nothing before the colon: cut and paste on every
    machine, the Diagonal Target at degree 1, and
    the Inequality as the Diagonal Target from degree 2 on. -/
theorem degree_one_closure :
    (∀ (M : TM) (β : Int), 0 < β → ∀ (w w1 w2 : List Sym),
      (∀ i : Int, i < β → initTape M w i = initTape M w1 i) →
      (∀ i : Int, β ≤ i → initTape M w i = initTape M w2 i) →
      ∀ (T1 T2 : Nat), HaltsAt M w1 T1 → HaltsAt M w2 T2 → cs M w1 β T1 =
        cs M w2 β T2 →
      ∃ T, HaltsAt M w T ∧ ((run M w T).st = (run M w1 T1).st ∨ (run M w
        T).st = (run M w2 T2).st)) ∧
    DiagonalTargetAt 1 ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k) :=
  ⟨cut_paste, diagonal_target_at_one, the_inequality_from_degree_two⟩

end PNP.CROSSING

/-- info: 'PNP.CROSSING.cx_move' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_move
/-- info: 'PNP.CROSSING.cx_step_pos' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_step_pos
```

```
/-- info: 'PNP.CROSSING.cx_step_tape' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_step_tape
/-- info: 'PNP.CROSSING.cx_step_same' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_step_same
/-- info: 'PNP.CROSSING.cx_crossAt_iff' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_crossAt_iff
/-- info: 'PNP.CROSSING.cx_noCross_iff' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_noCross_iff
/-- info: 'PNP.CROSSING.cs_succ' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cs_succ
/-- info: 'PNP.CROSSING.cs_cross' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cs_cross
/-- info: 'PNP.CROSSING.cs_nocross' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cs_nocross
/-- info: 'PNP.CROSSING.cs_length_succ' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cs_length_succ
/-- info: 'PNP.CROSSING.cs_prefix' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cs_prefix
/-- info: 'PNP.CROSSING.cs_halt' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.CROSSING.cs_halt
/-- info: 'PNP.CROSSING.cx_first_cross' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_first_cross
/-- info: 'PNP.CROSSING.cx_stay' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.CROSSING.cx_stay
/-- info: 'PNP.CROSSING.cx_cross_pos' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_cross_pos
/-- info: 'PNP.CROSSING.cx_bool_ne' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_bool_ne
/-- info: 'PNP.CROSSING.cx_advance' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_advance
/-- info: 'PNP.CROSSING.cx_inv' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_inv
/-- info: 'PNP.CROSSING.cut_paste' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cut_paste
/-- info: 'PNP.CROSSING.cx_sum_append' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_append
/-- info: 'PNP.CROSSING.cx_sum_map' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_map
/-- info: 'PNP.CROSSING.cx_sum_add' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_add
/-- info: 'PNP.CROSSING.cx_sum_mul' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_mul
/-- info: 'PNP.CROSSING.cx_sum_le' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_le
/-- info: 'PNP.CROSSING.cx_sum_congr' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_congr
/-- info: 'PNP.CROSSING.cx_sum_const' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_const
/-- info: 'PNP.CROSSING.cx_sum_zero' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_zero
/-- info: 'PNP.CROSSING.cx_sum_mem' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_mem
/-- info: 'PNP.CROSSING.cx_sum_comm' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_comm
/-- info: 'PNP.CROSSING.cx_sum_le_one' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_sum_le_one
/-- info: 'PNP.CROSSING.cx_cross_unique' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_cross_unique
/-- info: 'PNP.CROSSING.crossings_le_time' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.crossings_le_time
/-- info: 'PNP.CROSSING.fool_eq' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_eq
/-- info: 'PNP.CROSSING.fool_clause_len' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_clause_len
/-- info: 'PNP.CROSSING.fool_units_len' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_units_len
/-- info: 'PNP.CROSSING.fool_units_pos' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_units_pos
/-- info: 'PNP.CROSSING.fool_pad_len' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_pad_len
/-- info: 'PNP.CROSSING.fool_evalCNF_append' depends on axioms: [propext]
    -/
#guard_msgs in #print axioms PNP.CROSSING.fool_evalCNF_append
/-- info: 'PNP.CROSSING.fool_units_true' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_units_true
/-- info: 'PNP.CROSSING.fool_units_sat' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_units_sat
/-- info: 'PNP.CROSSING.fool_pad_true' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.fool_pad_true
/-- info: 'PNP.CROSSING.fool_ext' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_ext
/-- info: 'PNP.CROSSING.fool_sat' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_sat
/-- info: 'PNP.CROSSING.fool_unsat' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_unsat
/-- info: 'PNP.CROSSING.cx_getD_append_right' depends on axioms:
    [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_getD_append_right
/-- info: 'PNP.CROSSING.cx_init_right' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_init_right
/-- info: 'PNP.CROSSING.fool_left' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_left
/-- info: 'PNP.CROSSING.fool_right' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_right
/-- info: 'PNP.CROSSING.fool_cs_inj' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_cs_inj
/-- info: 'PNP.CROSSING.allBits_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.allBits_length
/-- info: 'PNP.CROSSING.allBits_mem' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.allBits_mem
/-- info: 'PNP.CROSSING.allBits_once' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.allBits_once
/-- info: 'PNP.CROSSING.cx_bnds_length' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CROSSING.cx_bnds_length
/-- info: 'PNP.CROSSING.cx_bnds_mem' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_bnds_mem
/-- info: 'PNP.CROSSING.cx_bnds_once' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_bnds_once
/-- info: 'PNP.CROSSING.cx_ext_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_ext_length
/-- info: 'PNP.CROSSING.cx_words_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_words_length
/-- info: 'PNP.CROSSING.cx_ext_mem' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_ext_mem
/-- info: 'PNP.CROSSING.cx_words_mem' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_words_mem
/-- info: 'PNP.CROSSING.cx_map_val_inj' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_map_val_inj
/-- info: 'PNP.CROSSING.cx_pigeon' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_pigeon
/-- info: 'PNP.CROSSING.cx_average' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_average
/-- info: 'PNP.CROSSING.fool_long' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.fool_long
/-- info: 'PNP.CROSSING.cx_two_pow' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_two_pow
/-- info: 'PNP.CROSSING.cx_arith' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.cx_arith
/-- info: 'PNP.CROSSING.diagonal_target_at_one' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.diagonal_target_at_one
/-- info: 'PNP.CROSSING.the_inequality_from_degree_two' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.the_inequality_from_degree_two
/-- info: 'PNP.CROSSING.the_barrier_from_degree_two' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.the_barrier_from_degree_two
/-- info: 'PNP.CROSSING.degree_one_closure' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CROSSING.degree_one_closure

/-! ============================================================
    ============================================
    PART N · SAT IS IN NP, IN THE KERNEL (namespace PNP.VERIFIER) ·
      v1_6_3_CC

    Purely additive: nothing above is overwritten. This Part builds a
      concrete one-tape verifier with a polynomial
    clock, `vM`: 5120 control states and a tape alphabet of nineteen
      symbols (s = 15). It reads a certificate of one
    bit per literal, the separator, and the input, and runs four phases.
      Phase 0 checks that the input is a written
    formula; the strings the decoder accepts are exactly the written
      formulas (`decode_vfSyms`, `decode_inv`), and the
    syntax automaton accepts exactly those (`p0_iff`). Phase A writes the
      certificate bits onto the signs in reading
    order. Phase B checks that every clause has a sign meeting its bit.
      Phase C checks, digit by digit with marks, that
    any two literals naming one variable carry one bit (`cmp_decide`).

    The run is computed on zippers (`vRel_run`); every phase is a theorem
      on lists with an explicit step count. A run in
    the accept state certifies that the input is in SAT
      (`verifier_sound`); on a satisfiable input the certificate of
    the values of a satisfying assignment makes the verifier accept
      within 5000·n⁴ + 5000 steps (`verifier_complete`,
    `vtime_le`). So the membership half of Cook and Levin is now a
      theorem of the kernel and no longer cited:
    `satLang_in_NP : PNP.TM.InNP PNP.TM.satLang`. The hardness half
      (`PNP.CLOSURE.SATHard`) stays cited. This Part
    proves nothing about whether SAT is in P. ΔM = 0.
```

```
/-! ### N.1 · The grammar of the decodable strings -/

/-- A literal as written: its first binary digit, its further digits (least significant first), its sign. -/
structure VLit where
  d0 : Bool
  ds : List Bool
  sg : Bool

def vdigits (L : VLit) : List Bool := L.d0 :: L.ds

def vlitSyms (L : VLit) : List Sym := (vdigits L).map symOfBool ++ [Sym.sep, symOfBool L.sg]

def vclSyms : List VLit → List Sym
  | [] => [Sym.sep]
  | L :: c => vlitSyms L ++ vclSyms c

/-- A written formula: its clauses, each a list of written literals closed by a separator. -/
def vfSyms : List (List VLit) → List Sym
  | [] => []
  | c :: F => vclSyms c ++ vfSyms F

/-- The variable a written literal names: the value of its digits, less one (the decoder's reading). -/
def vvar (L : VLit) : Nat := ofBits (vdigits L) - 1
def vlitToLit (L : VLit) : Lit := (vvar L, L.sg)
def vfToCNF (F : List (List VLit)) : CNF := F.map (fun c => c.map vlitToLit)

theorem vparseLit (L : VLit) (rest : List Sym) : parseLit (vlitSyms L ++ rest) = some (vlitToLit L, rest) := by
  have e : vlitSyms L ++ rest = (vdigits L).map symOfBool ++ Sym.sep :: (symOfBool L.sg :: rest) := by
    simp [vlitSyms]
  rw [e]
  unfold parseLit
  rw [parseBits_enc]
  obtain (d0, ds, sg) := L
  cases sg <;> rfl

theorem vparseClause_cons (fuel : Nat) (x : Sym) (w : List Sym) (hx : x ≠ Sym.sep) :
    parseClause (fuel+1) (x :: w) =
      (match parseLit (x :: w) with
       | none => none
       | some (l, rest) =>
         match parseClause fuel rest with
         | none => none
         | some (c, rest') => some (l :: c, rest')) := by
  cases x
  · rfl
  · rfl
  · exact absurd rfl hx

theorem vlitSyms_cons (L : VLit) (rest : List Sym) :
    vlitSyms L ++ rest = symOfBool L.d0 :: (L.ds.map symOfBool ++ Sym.sep :: symOfBool L.sg :: rest) := by
  simp [vlitSyms, vdigits]

theorem vparseClause : ∀ (c : List VLit) (fuel : Nat) (rest : List Sym), c.length ≤ fuel →
    parseClause (fuel+1) (vclSyms c ++ rest) = some (c.map vlitToLit, rest)
  | [], _, _, _ => rfl
  | L :: c, fuel+1, rest, h => by
    have ih := vparseClause c fuel rest (Nat.le_of_succ_le_succ h)
    have hl := vparseLit L (vclSyms c ++ rest)
    have e : vclSyms (L :: c) ++ rest = vlitSyms L ++ (vclSyms c ++ rest) := by
      simp [vclSyms]
    rw [e] at *
    rw [vlitSyms_cons] at hl ⊢
    rw [vparseClause_cons _ _ _ (symOfBool_ne_sep L.d0), hl]
    simp only [ih, List.map_cons]

theorem vclSyms_length : ∀ c : List VLit, c.length + 1 ≤ (vclSyms c).length
  | [] => Nat.le_refl 1
  | L :: c => by
    have := vclSyms_length c
    simp only [vclSyms, List.length_append, List.length_cons, vlitSyms]
    omega

theorem vfSyms_length : ∀ F : List (List VLit), F.length ≤ (vfSyms F).length
  | [] => Nat.le_refl 0
  | c :: F => by
    have := vfSyms_length F
    have := vclSyms_length c
    simp only [vfSyms, List.length_append, List.length_cons]
    omega

theorem vparseCNF : ∀ (F : List (List VLit)) (fuel : Nat), F.length ≤ fuel →
    parseCNF fuel (vfSyms F) = some (vfToCNF F)
  | [], fuel, _ => by cases fuel <;> rfl
  | c :: F, fuel+1, h => by
    have ih := vparseCNF F fuel (Nat.le_of_succ_le_succ h)
    show parseCNF (fuel+1) (vclSyms c ++ vfSyms F) = some (vfToCNF (c :: F))
    cases hE : vclSyms c ++ vfSyms F with
    | nil =>
      have := congrArg List.length hE
      have := vclSyms_length c
      simp only [List.length_append, List.length_nil] at *
      omega
    | cons x w =>
      have hcw : c.length ≤ w.length := by
        have h1 := congrArg List.length hE
        have := vclSyms_length c
        simp only [List.length_append, List.length_cons] at h1
        omega
      show (match parseClause (w.length + 1) (x :: w) with
            | none => none
            | some (c', rest) =>
              match parseCNF fuel rest with
              | none => none
              | some f' => some (c' :: f')) = some (vfToCNF (c :: F))
      rw [← hE, vparseClause c w.length (vfSyms F) hcw]
      simp only [ih, vfToCNF, List.map_cons]

/-- The decoder reads every written formula, whatever its digit strings. -/
theorem decode_vfSyms (F : List (List VLit)) : decode (vfSyms F) = some (vfToCNF F) :=
  vparseCNF F _ (vfSyms_length F)

theorem vparseBits_inv : ∀ (w : List Sym) (ds : List Bool) (rest : List Sym),
    parseBits w = some (ds, rest) → w = ds.map symOfBool ++ Sym.sep :: rest
  | [], _, _, h => by simp [parseBits] at h
  | Sym.sep :: w, ds, rest, h => by
    simp only [parseBits, Option.some.injEq, Prod.mk.injEq] at h
    obtain (h1, h2) := h
    subst h1; subst h2; rfl
  | Sym.zero :: w, ds, rest, h => by
    simp only [parseBits] at h
    cases hp : parseBits w with
    | none => rw [hp] at h; simp at h
    | some p =>
      rw [hp] at h
      simp only [Option.map_some', Option.some.injEq, Prod.mk.injEq] at h
      obtain (h1, h2) := h
      subst h1; subst h2
      have := vparseBits_inv w p.1 p.2 hp
      simp [this, symOfBool]
  | Sym.one :: w, ds, rest, h => by
    simp only [parseBits] at h
    cases hp : parseBits w with
    | none => rw [hp] at h; simp at h
    | some p =>
      rw [hp] at h
      simp only [Option.map_some', Option.some.injEq, Prod.mk.injEq] at h
      obtain (h1, h2) := h
      subst h1; subst h2
      have := vparseBits_inv w p.1 p.2 hp
      simp [this, symOfBool]

theorem vparseLit_inv (x : Sym) (w : List Sym) (hx : x ≠ Sym.sep) (l : Lit) (rest : List Sym)
    (h : parseLit (x :: w) = some (l, rest)) : ∃ L : VLit, x :: w = vlitSyms L ++ rest ∧ l = vlitToLit L := by
  unfold parseLit at h
  cases hp : parseBits (x :: w) with
  | none => rw [hp] at h; simp at h
  | some p =>
    obtain (ds, rest0) := p
    rw [hp] at h
    have hw := vparseBits_inv _ _ _ hp
    cases ds with
    | nil => simp at hw; exact absurd hw.1 hx
    | cons b ds' =>
      cases rest0 with
      | nil => simp at h
      | cons y rest' =>
        cases y with
        | sep => simp at h
        | zero =>
          simp only [Option.some.injEq, Prod.mk.injEq] at h
          obtain (h1, h2) := h
          subst h1; subst h2
          exact ⟨(b, ds', false), by rw [hw]; simp [vlitSyms, vdigits, symOfBool], rfl⟩
        | one =>
          simp only [Option.some.injEq, Prod.mk.injEq] at h
          obtain (h1, h2) := h
          subst h1; subst h2
          exact ⟨(b, ds', true), by rw [hw]; simp [vlitSyms, vdigits, symOfBool], rfl⟩

theorem vparseClause_inv : ∀ (fuel : Nat) (w : List Sym) (c : Clause) (rest : List Sym),
    parseClause fuel w = some (c, rest) → ∃ vc : List VLit, w = vclSyms vc ++ rest ∧ c = vc.map vlitToLit
  | 0, _, _, _, h => by simp [parseClause] at h
  | _+1, [], _, _, h => by simp [parseClause, parseLit, parseBits] at h
  | _+1, Sym.sep :: w, c, rest, h => by
    simp only [parseClause, Option.some.injEq, Prod.mk.injEq] at h
    obtain (h1, h2) := h
    subst h1; subst h2
    exact ⟨[], rfl, rfl⟩
  | fuel+1, Sym.zero :: w, c, rest, h => by
    rw [vparseClause_cons _ _ _ (by decide)] at h
```

```
    cases hl : parseLit (Sym.zero :: w) with
    | none => rw [hl] at h; simp at h
    | some p =>
      obtain (l, r1) := p
      rw [hl] at h
      cases hc : parseClause fuel r1 with
      | none => simp only [hc] at h; simp at h
      | some q =>
        obtain (c', r2) := q
        simp only [hc, Option.some.injEq, Prod.mk.injEq] at h
        obtain ⟨h1, h2⟩ := h
        subst h1; subst h2
        obtain (vc, hv1, hv2) := vparseClause_inv fuel r1 c' r2 hc
        obtain (L, hL1, hL2) := vparseLit_inv _ _ (by decide) l r1 hl
        refine (L :: vc, ?_, ?_)
        · rw [hL1, hv1]; simp [vclSyms]
        · rw [hL2, hv2]; rfl
  | fuel+1, Sym.one :: w, c, rest, h => by
    rw [vparseClause_cons _ _ _ (by decide)] at h
    cases hl : parseLit (Sym.one :: w) with
    | none => rw [hl] at h; simp at h
    | some p =>
      obtain (l, r1) := p
      rw [hl] at h
      cases hc : parseClause fuel r1 with
      | none => simp only [hc] at h; simp at h
      | some q =>
        obtain (c', r2) := q
        simp only [hc, Option.some.injEq, Prod.mk.injEq] at h
        obtain ⟨h1, h2⟩ := h
        subst h1; subst h2
        obtain (vc, hv1, hv2) := vparseClause_inv fuel r1 c' r2 hc
        obtain (L, hL1, hL2) := vparseLit_inv _ _ (by decide) l r1 hl
        refine (L :: vc, ?_, ?_)
        · rw [hL1, hv1]; simp [vclSyms]
        · rw [hL2, hv2]; rfl

theorem vparseCNF_inv : ∀ (fuel : Nat) (w : List Sym) (f : CNF),
    parseCNF fuel w = some f → ∃ F : List (List VLit), w = vfSyms F ∧ f =
      vfToCNF F
  | _, [], f, h => by
    have : f = [] := by cases ‹Nat› <;> simp [parseCNF] at h <;> exact h
    exact ([], rfl, this)
  | 0, _ :: _, _, h => by simp [parseCNF] at h
  | fuel+1, x :: w, f, h => by
    simp only [parseCNF] at h
    cases hc : parseClause (x :: w).length (x :: w) with
    | none => rw [hc] at h; simp at h
    | some p =>
      obtain (c, rest) := p
      rw [hc] at h
      cases hf : parseCNF fuel rest with
      | none => simp only [hf] at h; simp at h
      | some f' =>
        simp only [hf, Option.some.injEq] at h
        subst h
        obtain (F, hF1, hF2) := vparseCNF_inv fuel rest f' hf
        obtain (vc, hv1, hv2) := vparseClause_inv _ _ c rest hc
        refine (vc :: F, ?_, ?_)
        · rw [hv1, hF1]; rfl
        · rw [hv2, hF2]; rfl

/-- THE GRAMMAR: a string decodes exactly when it is a written formula,
    and then to that formula's reading. -/
theorem decode_inv (w : List Sym) (f : CNF) (h : decode w = some f) :
    ∃ F : List (List VLit), w = vfSyms F ∧ f = vfToCNF F :=
  vparseCNF_inv _ _ _ h

/-! ### N.2 · The verifier: alphabet, control, transition table -/

/-- The tape alphabet, nineteen symbols: the blank, the two digits, the
    separator, a used certificate bit, the two
    marked digits, and a sign carrying its certificate bit, plain,
      outer-marked or inner-marked. -/
inductive VG
  | B | Z | O | S | U | MZ | MO
  | A (s v : Bool) | OA (s v : Bool) | IA (s v : Bool)
  deriving DecidableEq

def vbit (b : Bool) : Nat := if b then 1 else 0

def encG : VG → Nat
  | .B => 0 | .Z => 1 | .O => 2 | .S => 3 | .U => 4 | .MZ => 5 | .MO => 6
  | .A s v => 7 + 2 * vbit s + vbit v
  | .OA s v => 11 + 2 * vbit s + vbit v
  | .IA s v => 15 + 2 * vbit s + vbit v

def decG : Nat → VG
  | 1 => .Z | 2 => .O | 3 => .S | 4 => .U | 5 => .MZ | 6 => .MO
  | 7 => .A false false | 8 => .A false true | 9 => .A true false | 10 =>
    .A true true
  | 11 => .OA false false | 12 => .OA false true | 13 => .OA true false |
    14 => .OA true true
  | 15 => .IA false false | 16 => .IA false true | 17 => .IA true false |
    18 => .IA true true
  | _ => .B

theorem decG_encG (g : VG) : decG (encG g) = g := by
  cases g with
  | A s v => cases s <;> cases v <;> rfl
  | OA s v => cases s <;> cases v <;> rfl
  | IA s v => cases s <;> cases v <;> rfl
  | _ => rfl

theorem encG_lt (g : VG) : encG g < 15 + 4 := by
  cases g with
  | A s v => cases s <;> cases v <;> decide
  | OA s v => cases s <;> cases v <;> decide
  | IA s v => cases s <;> cases v <;> decide
  | _ => decide

/-- The comparison flags of Phase C: position 0 pending, digit 0 equal,
    the higher digits equal, the higher
    digits of the outer literal all zero. -/
structure VFl where
  p0 : Bool
  e0 : Bool
  er : Bool
  zr : Bool

/-- The control states. Phase 0 checks the syntax, Phase A writes each
    certificate bit onto a sign, Phase B checks
    every clause, Phase C checks that literals naming one variable carry
      one bit. -/
inductive VS
  | p0Cert | p0Start | p0After | p0D | p0S | p0Ret
  | aScan | aToW (b : Bool) | aC (b : Bool) | aD (b : Bool) | aS (b :
    Bool) | aRet | aFin | aSkip
  | bStart | bD (f : Bool) | bS (f : Bool) | bAfter (f : Bool) | bRet |
    bSkip
  | cFindOuter | cFindInner (v : Bool) | cBack (v : Bool)
  | cGoOuter (v : Bool) (F : VFl) | cXSep (v : Bool) (F : VFl) | cXScan
    (v : Bool) (F : VFl)
  | cXRead (v : Bool) (F : VFl)
  | cGoInner (v : Bool) (F : VFl) (dx : Option Bool) | cYSep (v : Bool)
    (F : VFl) (dx : Option Bool)
  | cYScan (v : Bool) (F : VFl) (dx : Option Bool) | cYRead (v : Bool) (F
    : VFl) (dx : Option Bool)
  | cResult (v : Bool) (F : VFl)
  | cUnYSep (v : Bool) | cUnYScan (v : Bool) | cUnGoOuter (v : Bool) |
    cUnXSep (v : Bool)
  | cUnXScan (v : Bool) | cUnGoInner (v : Bool)
  | acc | rej

def encFl (F : VFl) : Nat := vbit F.p0 + 2 * vbit F.e0 + 4 * vbit F.er +
    8 * vbit F.zr
def decFl (n : Nat) : VFl := ⟨n % 2 == 1, n / 2 % 2 == 1, n / 4 % 2 == 1,
    n / 8 % 2 == 1⟩
def encO : Option Bool → Nat
  | none => 0
  | some b => 1 + vbit b
def decO (n : Nat) : Option Bool := if n = 0 then none else some (n == 2)

def vp1 (v : Bool) : Nat := vbit v
def vp2 (v : Bool) (F : VFl) : Nat := vbit v + 2 * encFl F
def vp3 (v : Bool) (F : VFl) (dx : Option Bool) : Nat := vbit v + 2 *
    encFl F + 32 * encO dx

def tagOf : VS → Nat
  | .p0Cert => 0 | .p0Start => 1 | .p0After => 2 | .p0D => 3 | .p0S => 4 |
    .p0Ret => 5
  | .aScan => 6 | .aToW _ => 7 | .aC _ => 8 | .aD _ => 9 | .aS _ => 10 |
    .aRet => 11 | .aFin => 12
  | .aSkip => 13
  | .bStart => 14 | .bD _ => 15 | .bS _ => 16 | .bAfter _ => 17 | .bRet =>
    18 | .bSkip => 19
  | .cFindOuter => 20 | .cFindInner _ => 21 | .cBack _ => 22
  | .cGoOuter _ _ => 23 | .cXSep _ _ => 24 | .cXScan _ _ => 25 | .cXRead _
    _ => 26
  | .cGoInner _ _ _ => 27 | .cYSep _ _ _ => 28 | .cYScan _ _ _ => 29 |
    .cYRead _ _ _ => 30
  | .cResult _ _ => 31
  | .cUnYSep _ => 32 | .cUnYScan _ => 33 | .cUnGoOuter _ => 34 | .cUnXSep
    _ => 35 | .cUnXScan _ => 36
  | .cUnGoInner _ => 37
  | .acc => 38 | .rej => 39

def parOf : VS → Nat
  | .aToW b => vp1 b | .aC b => vp1 b | .aD b => vp1 b | .aS b => vp1 b
  | .bD f => vp1 f | .bS f => vp1 f | .bAfter f => vp1 f
  | .cFindInner v => vp1 v | .cBack v => vp1 v
  | .cGoOuter v F => vp2 v F | .cXSep v F => vp2 v F | .cXScan v F => vp2
    v F | .cXRead v F => vp2 v F
  | .cGoInner v F dx => vp3 v F dx | .cYSep v F dx => vp3 v F dx | .cYScan
    v F dx => vp3 v F dx
  | .cYRead v F dx => vp3 v F dx
  | .cResult v F => vp2 v F
  | .cUnYSep v => vp1 v | .cUnYScan v => vp1 v | .cUnGoOuter v => vp1 v |
    .cUnXSep v => vp1 v
  | .cUnXScan v => vp1 v | .cUnGoInner v => vp1 v
  | _ => 0

def encS (s : VS) : Nat := 128 * tagOf s + parOf s

def dB (p : Nat) : Bool := p % 2 == 1
def dF (p : Nat) : VFl := decFl (p / 2 % 16)
def dO (p : Nat) : Option Bool := decO (p / 32)

def decTag (t p : Nat) : VS :=
  match t with
  | 0 => .p0Cert | 1 => .p0Start | 2 => .p0After | 3 => .p0D | 4 => .p0S |
    5 => .p0Ret
  | 6 => .aScan | 7 => .aToW (dB p) | 8 => .aC (dB p) | 9 => .aD (dB p) |
    10 => .aS (dB p)
  | 11 => .aRet | 12 => .aFin | 13 => .aSkip
  | 14 => .bStart | 15 => .bD (dB p) | 16 => .bS (dB p) | 17 => .bAfter
    (dB p) | 18 => .bRet
  | 19 => .bSkip
  | 20 => .cFindOuter | 21 => .cFindInner (dB p) | 22 => .cBack (dB p)
  | 23 => .cGoOuter (dB p) (dF p) | 24 => .cXSep (dB p) (dF p) | 25 =>
    .cXScan (dB p) (dF p)
  | 26 => .cXRead (dB p) (dF p)
```

```
  | 27 => .cGoInner (dB p) (dF p) (dO p) | 28 => .cYSep (dB p) (dF p) (dO p)
  | 29 => .cYScan (dB p) (dF p) (dO p) | 30 => .cYRead (dB p) (dF p) (dO p)
  | 31 => .cResult (dB p) (dF p)
  | 32 => .cUnYSep (dB p) | 33 => .cUnYScan (dB p) | 34 => .cUnGoOuter (dB p) | 35 => .cUnXSep (dB p)
  | 36 => .cUnXScan (dB p) | 37 => .cUnGoInner (dB p)
  | 38 => .acc
  | _ => .rej

def decS (n : Nat) : VS := decTag (n / 128) (n % 128)

theorem vbit_le (b : Bool) : vbit b ≤ 1 := by cases b <;> decide
theorem encFl_le (F : VFl) : encFl F ≤ 15 := by
  obtain (a, b, c, d) := F
  have := vbit_le a; have := vbit_le b; have := vbit_le c; have := vbit_le d
  simp only [encFl]; omega
theorem encO_le (d : Option Bool) : encO d ≤ 2 := by
  cases d with
  | none => decide
  | some b => have := vbit_le b; simp only [encO]; omega

theorem decFl_encFl (F : VFl) : decFl (encFl F) = F := by
  obtain (a, b, c, d) := F
  cases a <;> cases b <;> cases c <;> cases d <;> rfl
theorem decO_encO (d : Option Bool) : decO (encO d) = d := by
  cases d with
  | none => rfl
  | some b => cases b <;> rfl

theorem dB_vp (v : Bool) (k : Nat) : dB (vbit v + 2 * k) = v := by
  cases v <;> simp [dB, vbit] <;> omega
theorem dF_vp (v : Bool) (F : VFl) (k : Nat) : dF (vbit v + 2 * encFl F + 32 * k) = F := by
  have h1 := vbit_le v; have h2 := encFl_le F
  have : (vbit v + 2 * encFl F + 32 * k) / 2 % 16 = encFl F := by omega
  simp only [dF, this, decFl_encFl]
theorem dO_vp (v : Bool) (F : VFl) (d : Option Bool) : dO (vbit v + 2 * encFl F + 32 * encO d) = d := by
  have h1 := vbit_le v; have h2 := encFl_le F
  have : (vbit v + 2 * encFl F + 32 * encO d) / 32 = encO d := by omega
  simp only [dO, this, decO_encO]

theorem parOf_lt (s : VS) : parOf s < 128 := by
  have p1 : ∀ v, vp1 v < 128 := fun v => by have := vbit_le v; simp only [vp1]; omega
  have p2 : ∀ v F, vp2 v F < 128 := fun v F => by
    have := vbit_le v; have := encFl_le F; simp only [vp2]; omega
  have p3 : ∀ v F d, vp3 v F d < 128 := fun v F d => by
    have := vbit_le v; have := encFl_le F; have := encO_le d; simp only [vp3]; omega
  cases s <;> simp only [parOf] <;> first | exact p1 _ | exact p2 _ _ | exact p3 _ _ _ | decide

theorem decTag_tagOf (s : VS) : decTag (tagOf s) (parOf s) = s := by
  have e1 : ∀ v, dB (vp1 v) = v := fun v => by
    have := dB_vp v 0; simpa [vp1] using this
  have e2 : ∀ v F, dB (vp2 v F) = v := fun v F => dB_vp v (encFl F)
  have e3 : ∀ v F d, dB (vp3 v F d) = v := fun v F d => by
    have : vp3 v F d = vbit v + 2 * (encFl F + 16 * encO d) := by simp only [vp3]; omega
    rw [this, dB_vp]
  have f2 : ∀ v F, dF (vp2 v F) = F := fun v F => by
    have := dF_vp v F 0; simpa [vp2] using this
  have f3 : ∀ v F d, dF (vp3 v F d) = F := fun v F d => dF_vp v F (encO d)
  have o3 : ∀ v F d, dO (vp3 v F d) = d := fun v F d => dO_vp v F d
  cases s <;> simp only [tagOf, parOf, decTag, e1, e2, e3, f2, f3, o3]

theorem decS_encS (s : VS) : decS (encS s) = s := by
  have h := parOf_lt s
  have h1 : (128 * tagOf s + parOf s) / 128 = tagOf s := by omega
  have h2 : (128 * tagOf s + parOf s) % 128 = parOf s := by omega
  simp only [decS, encS, h1, h2, decTag_tagOf]

theorem encS_inj {s t : VS} (h : encS s = encS t) : s = t := by
  have := congrArg decS h
  rwa [decS_encS, decS_encS] at this

theorem tagOf_lt (s : VS) : tagOf s < 40 := by cases s <;> simp only [tagOf] <;> decide

def vQ : Nat := 40 * 128

theorem encS_lt (s : VS) : encS s < vQ := by
  have := parOf_lt s; have := tagOf_lt s
  simp only [encS, vQ]; omega

/-- One digit-pair step of the comparison: the first pair fills digit 0, every later pair the higher digits. -/
def vupd (F : VFl) (x y : Bool) : VFl :=
  if F.p0 then (false, x == y, F.er, F.zr) else (false, F.e0, F.er && (x == y), F.zr && !x)

/-- After both digits are read: both strings exhausted gives the result, else one more step. -/
def vcomb (v : Bool) (F : VFl) : Option Bool → Option Bool → VS
  | none, none => .cResult v F
  | dx, dy => .cGoOuter v (vupd F (dx.getD false) (dy.getD false))

def vF0 : VFl := (true, false, true, true)

/-- THE TRANSITION TABLE. Every pair not listed sends the machine to `rej`. -/
def vδ (s : VS) (x : VG) : VS × VG × Dir :=
  match s with
  | .p0Cert => match x with
    | .Z => (.p0Cert, .Z, .R) | .O => (.p0Cert, .O, .R) | .S => (.p0Start, .S, .R)
    | x => (.rej, x, .R)
  | .p0Start => match x with
    | .Z => (.p0D, .Z, .R) | .O => (.p0D, .O, .R) | .S => (.p0Start, .S, .R) | .B => (.p0Ret, .B, .L)
    | x => (.rej, x, .R)
  | .p0After => match x with
    | .Z => (.p0D, .Z, .R) | .O => (.p0D, .O, .R) | .S => (.p0Start, .S, .R)
    | x => (.rej, x, .R)
  | .p0D => match x with
    | .Z => (.p0D, .Z, .R) | .O => (.p0D, .O, .R) | .S => (.p0S, .S, .R)
    | x => (.rej, x, .R)
  | .p0S => match x with
    | .Z => (.p0After, .Z, .R) | .O => (.p0After, .O, .R)
    | x => (.rej, x, .R)
  | .p0Ret => match x with
    | .B => (.aScan, .B, .R)
    | x => (.p0Ret, x, .L)
  | .aScan => match x with
    | .U => (.aScan, .U, .R) | .Z => (.aToW false, .U, .R) | .O => (.aToW true, .U, .R)
    | .S => (.bStart, .S, .R)
    | x => (.rej, x, .R)
  | .aToW b => match x with
    | .Z => (.aToW b, .Z, .R) | .O => (.aToW b, .O, .R) | .U => (.aToW b, .U, .R)
    | .S => (.aC b, .S, .R)
    | x => (.rej, x, .R)
  | .aC b => match x with
    | .Z => (.aD b, .Z, .R) | .O => (.aD b, .O, .R) | .S => (.aC b, .S, .R) | .B => (.aFin, .B, .L)
    | x => (.rej, x, .R)
  | .aD b => match x with
   | .Z => (.aD b, .Z, .R) | .O => (.aD b, .O, .R) | .S => (.aS b, .S, .R)
    | x => (.rej, x, .R)
  | .aS b => match x with
    | .Z => (.aRet, .A false b, .L) | .O => (.aRet, .A true b, .L)
    | .A s' v' => (.aC b, .A s' v', .R)
    | x => (.rej, x, .R)
  | .aRet => match x with
    | .B => (.aScan, .B, .R)
    | x => (.aRet, x, .L)
  | .aFin => match x with
    | .B => (.aSkip, .B, .R)
    | x => (.aFin, x, .L)
  | .aSkip => match x with
    | .Z => (.aSkip, .Z, .R) | .O => (.aSkip, .O, .R) | .U => (.aSkip, .U, .R)
    | .S => (.bStart, .S, .R)
    | x => (.rej, x, .R)
  | .bStart => match x with
    | .Z => (.bD false, .Z, .R) | .O => (.bD false, .O, .R) | .B => (.bRet, .B, .L)
    | x => (.rej, x, .R)
  | .bD f => match x with
   | .Z => (.bD f, .Z, .R) | .O => (.bD f, .O, .R) | .S => (.bS f, .S, .R)
    | x => (.rej, x, .R)
  | .bS f => match x with
    | .A s' v' => (.bAfter (f || (s' == v')), .A s' v', .R)
    | x => (.rej, x, .R)
  | .bAfter f => match x with
    | .Z => (.bD f, .Z, .R) | .O => (.bD f, .O, .R)
    | .S => if f then (.bStart, .S, .R) else (.rej, .S, .R)
    | x => (.rej, x, .R)
  | .bRet => match x with
    | .B => (.bSkip, .B, .R)
    | x => (.bRet, x, .L)
  | .bSkip => match x with
    | .Z => (.bSkip, .Z, .R) | .O => (.bSkip, .O, .R) | .U => (.bSkip, .U, .R)
    | .S => (.cFindOuter, .S, .R)
    | x => (.rej, x, .R)
  | .cFindOuter => match x with
    | .Z => (.cFindOuter, .Z, .R) | .O => (.cFindOuter, .O, .R) | .S => (.cFindOuter, .S, .R)
    | .A s' v' => (.cFindInner v', .OA s' v', .R)
    | .B => (.acc, .B, .L)
    | x => (.rej, x, .R)
  | .cFindInner v => match x with
    | .Z => (.cFindInner v, .Z, .R) | .O => (.cFindInner v, .O, .R) | .S => (.cFindInner v, .S, .R)
    | .A s' v' => (.cGoOuter v vF0, .IA s' v', .L)
    | .B => (.cBack v, .B, .L)
    | x => (.rej, x, .R)
  | .cBack v => match x with
    | .OA s' v' => (.cFindOuter, .A s' v', .R)
    | x => (.cBack v, x, .L)
  | .cGoOuter v F => match x with
    | .OA s' v' => (.cXSep v F, .OA s' v', .L)
    | x => (.cGoOuter v F, x, .L)
  | .cXSep v F => match x with
    | .S => (.cXScan v F, .S, .L)
    | x => (.rej, x, .R)
  | .cXScan v F => match x with
    | .Z => (.cXScan v F, .Z, .L) | .O => (.cXScan v F, .O, .L)
    | .MZ => (.cXRead v F, .MZ, .R) | .MO => (.cXRead v F, .MO, .R) | .S => (.cXRead v F, .S, .R)
    | .A s' v' => (.cXRead v F, .A s' v', .R)
    | x => (.rej, x, .R)
  | .cXRead v F => match x with
    | .Z => (.cGoInner v F (some false), .MZ, .R) | .O => (.cGoInner v F (some true), .MO, .R)
    | .S => (.cGoInner v F none, .S, .R)
```

```
    | x => (.rej, x, .R)
  | .cGoInner v F dx => match x with
    | .IA s' v' => (.cYSep v F dx, .IA s' v', .L)
    | x => (.cGoInner v F dx, x, .R)
  | .cYSep v F dx => match x with
    | .S => (.cYScan v F dx, .S, .L)
    | x => (.rej, x, .R)
  | .cYScan v F dx => match x with
    | .Z => (.cYScan v F dx, .Z, .L) | .O => (.cYScan v F dx, .O, .L)
    | .MZ => (.cYRead v F dx, .MZ, .R) | .MO => (.cYRead v F dx, .MO, .R)
    | .S => (.cYRead v F dx, .S, .R)
    | .A s' v' => (.cYRead v F dx, .A s' v', .R) | .OA s' v' => (.cYRead v
       F dx, .OA s' v', .R)
    | x => (.rej, x, .R)
  | .cYRead v F dx => match x with
    | .Z => (vcomb v F dx (some false), .MZ, .R) | .O => (vcomb v F dx
       (some true), .MO, .R)
    | .S => (vcomb v F dx none, .S, .R)
    | x => (.rej, x, .R)
  | .cResult v F => match x with
    | .IA s' v' =>
      if F.er && (F.e0 || F.zr) && (v != v') then (.rej, .IA s' v', .R)
         else (.cUnYSep v, .IA s' v', .L)
    | x => (.rej, x, .R)
  | .cUnYSep v => match x with
    | .S => (.cUnYScan v, .S, .L)
    | x => (.rej, x, .R)
  | .cUnYScan v => match x with
    | .MZ => (.cUnYScan v, .Z, .L) | .MO => (.cUnYScan v, .O, .L)
    | .S => (.cUnGoOuter v, .S, .R) | .A s' v' => (.cUnGoOuter v, .A s'
       v', .R)
    | .OA s' v' => (.cUnGoOuter v, .OA s' v', .R)
    | x => (.rej, x, .R)
  | .cUnGoOuter v => match x with
    | .OA s' v' => (.cUnXSep v, .OA s' v', .L)
    | x => (.cUnGoOuter v, x, .L)
  | .cUnXSep v => match x with
    | .S => (.cUnXScan v, .S, .L)
    | x => (.rej, x, .R)
  | .cUnXScan v => match x with
    | .MZ => (.cUnXScan v, .Z, .L) | .MO => (.cUnXScan v, .O, .L)
    | .S => (.cUnGoInner v, .S, .R) | .A s' v' => (.cUnGoInner v, .A s'
       v', .R)
    | x => (.rej, x, .R)
  | .cUnGoInner v => match x with
    | .IA s' v' => (.cFindInner v, .A s' v', .R)
    | x => (.cUnGoInner v, x, .R)
  | .acc => (.acc, x, .R)
  | .rej => (.rej, x, .R)

/-- THE VERIFIER: 5120 control states, the tape alphabet of nineteen
    symbols (s = 15). -/
def vM : TM where
  q := vQ
  s := 15
  δ := fun i g =>
    let r := vδ (decS i.val) (decG g.val)
    ((encS r.1, encS_lt r.1), (encG r.2.1, encG_lt r.2.1), r.2.2)
  start := (encS .p0Cert, encS_lt _)
  accept := (encS .acc, encS_lt _)
  reject := (encS .rej, encS_lt _)

/-! ### N.3 · Zippers: the run as a list computation -/

/-- A configuration as a zipper: the state, the cells left of the head
    (nearest first), the scanned cell, the
    cells right of the head. -/
structure VZ where
  st : VS
  l : List VG
  h : VG
  r : List VG

def vhalt : VS → Bool
  | .acc => true
  | .rej => true
  | _ => false

def zstep (z : VZ) : VZ :=
  if vhalt z.st then z else
  match vδ z.st z.h with
  | (s', x, Dir.R) => (s', x :: z.l, z.r.headD .B, z.r.tail)
  | (s', x, Dir.L) => (s', z.l.tail, z.l.headD .B, x :: z.r)

def zsteps : Nat → VZ → VZ
  | 0, z => z
  | n+1, z => zsteps n (zstep z)

/-- The head at the start of R, the cells L to its left (in tape order).
    -/
def zAt (st : VS) (L R : List VG) : VZ := (st, L.reverse, R.headD .B,
    R.tail)

/-- The tape a zipper denotes when its head stands at cell p. -/
def zmean (p : Int) (z : VZ) (i : Int) : VG :=
  if i < p then z.l.getD (p - 1 - i).toNat .B
  else if i = p then z.h
  else z.r.getD (i - p - 1).toNat .B

def vRel (c : Config vM) (z : VZ) : Prop :=
  c.st.val = encS z.st ∧ ∀ i, (c.tape i).val = encG (zmean c.pos z i)

theorem vgetD_tail (r : List VG) (k : Nat) : r.tail.getD k .B = r.getD
    (k+1) .B := by
  cases r <;> simp

theorem vheadD (r : List VG) : r.headD .B = r.getD 0 .B := by
  cases r <;> rfl

theorem zmean_R (p : Int) (s s' : VS) (l r : List VG) (h x : VG) (i :
    Int) :
    zmean (p + 1) (s', x :: l, r.headD .B, r.tail) i = if i = p then x
       else zmean p (s, l, h, r) i := by
  unfold zmean
  dsimp only
  by_cases h1 : i < p
  · rw [if_pos (show i < p + 1 by omega), if_neg (show ¬ i = p by omega),
     if_pos h1,
      show (p + 1 - 1 - i).toNat = (p - 1 - i).toNat + 1 by omega,
        List.getD_cons_succ]
  · by_cases h2 : i = p
    · rw [if_pos (show i < p + 1 by omega), if_pos h2, show (p + 1 - 1 -
       i).toNat = 0 by omega,
        List.getD_cons_zero]
    · rw [if_neg (show ¬ i < p + 1 by omega), if_neg h2, if_neg h1,
       if_neg h2]
      by_cases h3 : i = p + 1
      · rw [if_pos h3, show (i - p - 1).toNat = 0 by omega, vheadD]
      · rw [if_neg h3, show (i - p - 1).toNat = (i - (p + 1) - 1).toNat +
         1 by omega, vgetD_tail]

theorem zmean_L (p : Int) (s s' : VS) (l r : List VG) (h x : VG) (i :
    Int) :
    zmean (p + -1) (s', l.tail, l.headD .B, x :: r) i = if i = p then x
       else zmean p (s, l, h, r) i := by
  unfold zmean
  dsimp only
  by_cases h1 : i < p + -1
  · rw [if_pos h1, if_neg (show ¬ i = p by omega), if_pos (show i < p by
     omega),
      show (p - 1 - i).toNat = (p + -1 - 1 - i).toNat + 1 by omega,
        vgetD_tail]
  · by_cases h2 : i = p + -1
    · rw [if_neg h1, if_pos h2, if_neg (show ¬ i = p by omega), if_pos
       (show i < p by omega),
        show (p - 1 - i).toNat = 0 by omega, vheadD]
    · rw [if_neg h1, if_neg h2, if_neg (show ¬ i < p by omega)]
      by_cases h3 : i = p
      · rw [if_pos h3, show (i - (p + -1) - 1).toNat = 0 by omega,
         List.getD_cons_zero]
      · rw [if_neg h3, if_neg h3, show (i - (p + -1) - 1).toNat = (i - p -
         1).toNat + 1 by omega,
          List.getD_cons_succ]

theorem halted_iff (c : Config vM) (z : VZ) (h : c.st.val = encS z.st) :
    halted vM c ↔ vhalt z.st = true := by
  unfold halted
  constructor
  · intro hh
    rcases hh with h1 | h1
    · have : encS z.st = encS .acc := by rw [← h, h1]; rfl
      rw [encS_inj this]; rfl
    · have : encS z.st = encS .rej := by rw [← h, h1]; rfl
      rw [encS_inj this]; rfl
  · intro hv
    have key : z.st = .acc ∨ z.st = .rej := by
      cases hs : z.st <;> rw [hs] at hv <;> simp [vhalt] at hv <;> simp
    rcases key with h1 | h1
    · exact Or.inl (Fin.ext (by rw [h, h1]; rfl))
    · exact Or.inr (Fin.ext (by rw [h, h1]; rfl))

theorem vRel_step (c : Config vM) (z : VZ) (h : vRel c z) : vRel (step vM
    c) (zstep z) := by
  obtain (hst, htape) := h
  unfold step zstep
  by_cases hh : halted vM c
  · have hv := (halted_iff c z hst).mp hh
    rw [if_pos hh, if_pos hv]; exact (hst, htape)
  · have hv : vhalt z.st = false := by
      cases hz : vhalt z.st
      · rfl
      · exact absurd ((halted_iff c z hst).mpr hz) hh
    rw [if_neg hh, if_neg (by simp [hv])]
    have hread : decG (c.tape c.pos).val = z.h := by
      rw [htape]; simp [zmean, decG_encG]
    have hs : decS c.st.val = z.st := by rw [hst, decS_encS]
    rcases hr : vδ z.st z.h with (s', x, d)
    have hδ : vM.δ c.st (c.tape c.pos) = ((encS s', encS_lt s'), (encG x,
       encG_lt x), d) := by
      simp only [vM, hs, hread, hr]
    simp only [hδ]
    cases d with
    | R =>
      refine (rfl, fun i => ?_)
      show (if i = c.pos then ((encG x, encG_lt x) : Fin (15 + 4)) else
         c.tape i).val =
        encG (zmean (c.pos + 1) (s', x :: z.l, z.r.headD .B, z.r.tail) i)
      rw [zmean_R c.pos z.st s' z.l z.r z.h x i]
      by_cases hi : i = c.pos
      · simp [hi]
      · simp only [hi, if_false]; exact htape i
    | L =>
      refine (rfl, fun i => ?_)
      show (if i = c.pos then ((encG x, encG_lt x) : Fin (15 + 4)) else
         c.tape i).val =
        encG (zmean (c.pos + -1) (s', z.l.tail, z.l.headD .B, x :: z.r) i)
      rw [zmean_L c.pos z.st s' z.l z.r z.h x i]
      by_cases hi : i = c.pos
      · simp [hi]
      · simp only [hi, if_false]; exact htape i

def symG : Sym → VG
```

```
  | Sym.zero => .Z
  | Sym.one => .O
  | Sym.sep => .S

/-- The initial zipper: the input from cell 0, the blank at cell -1 and after the input written out. -/
def vinit (inp : List Sym) : VZ := zAt .p0Cert [VG.B] (inp.map symG ++ [VG.B])

theorem init_getD (inp : List Sym) : ∀ k : Nat,
    ((inp.map (Sym.toTape vM.s)).getD k (blank vM)).val = encG ((inp.map symG ++ [VG.B]).getD k VG.B) := by
  induction inp with
  | nil => intro k; cases k <;> simp [blank, encG]
  | cons x rest ih =>
    intro k
    cases k with
    | zero => cases x <;> rfl
    | succ k => simp only [List.map_cons, List.getD_cons_succ, List.cons_append]; exact ih k

theorem vRel_init (inp : List Sym) : vRel (init vM inp) (vinit inp) := by
  refine (rfl, fun i => ?_)
  show (initTape vM inp i).val = encG (zmean 0 (vinit inp) i)
  unfold initTape zmean vinit zAt
  by_cases h1 : i < 0
  · simp only [h1, if_true]
    have : ([VG.B].reverse.getD (0 - 1 - i).toNat VG.B) = .B := by
      cases hk : (0 - 1 - i).toNat <;> simp
    rw [this]; rfl
  · simp only [h1, if_false]
    rw [init_getD]
    by_cases h2 : i = 0
    · subst h2; rw [vheadD]; rfl
    · simp only [h2, if_false]
      have e : i.toNat = (i - 0 - 1).toNat + 1 := by omega
      rw [e, vgetD_tail]

theorem zsteps_last (n : Nat) : ∀ z : VZ, zsteps (n+1) z = zstep (zsteps n z) := by
  induction n with
  | zero => intro z; rfl
  | succ n ih => intro z; show zsteps (n+1) (zstep z) = _; rw [ih]; rfl

theorem vRel_run (inp : List Sym) : ∀ t, vRel (run vM inp t) (zsteps t (vinit inp)) := by
  intro t
  induction t with
  | zero => exact vRel_init inp
  | succ t ih => rw [zsteps_last]; exact vRel_step _ _ ih

theorem verdict_iff (inp : List Sym) (t : Nat) :
    verdictAt vM inp t = true ↔ (zsteps t (vinit inp)).st = .acc := by
  have hst := (vRel_run inp t).1
  unfold verdictAt
  constructor
  · intro h
    have h1 : (run vM inp t).st = vM.accept := of_decide_eq_true h
    have : encS (zsteps t (vinit inp)).st = encS .acc := by rw [← hst, h1]; rfl
    exact encS_inj this
  · intro h
    apply decide_eq_true
    apply Fin.ext
    rw [hst, h]; rfl

theorem halts_iff (inp : List Sym) (t : Nat) :
    HaltsAt vM inp t ↔ vhalt (zsteps t (vinit inp)).st = true :=
  halted_iff _ _ (vRel_run inp t).1

/-! ### N.4 · Scans and reachability -/

theorem zstep_halt (z : VZ) (h : vhalt z.st = true) : zstep z = z := by
  unfold zstep; rw [if_pos h]

theorem zsteps_halt (n : Nat) : ∀ z : VZ, vhalt z.st = true → zsteps n z = z := by
  induction n with
  | zero => intro z _; rfl
  | succ n ih => intro z h; show zsteps n (zstep z) = z; rw [zstep_halt z h, ih z h]

theorem zsteps_add (a b : Nat) : ∀ z : VZ, zsteps (a + b) z = zsteps b (zsteps a z) := by
  induction a with
  | zero => intro z; simp [zsteps]
  | succ a ih => intro z; rw [Nat.succ_add]; show zsteps (a + b) (zstep z) = _; rw [ih]; rfl

theorem zsteps_one (z : VZ) : zsteps 1 z = zstep z := rfl

theorem zstep_R {st s' : VS} {x y : VG} (l r : List VG) (h1 : vhalt st = false)
    (h2 : vδ st x = (s', y, Dir.R)) : zstep (st, l, x, r) = (s', y :: l, r.headD .B, r.tail) := by
  unfold zstep; simp only [h1, h2]; rfl

theorem zstep_L {st s' : VS} {x y : VG} (l r : List VG) (h1 : vhalt st = false)
    (h2 : vδ st x = (s', y, Dir.L)) : zstep (st, l, x, r) = (s', l.tail, l.headD .B, y :: r) := by
  unfold zstep; simp only [h1, h2]; rfl

theorem zAt_cons (st : VS) (L : List VG) (x : VG) (R : List VG) :
    zAt st L (x :: R) = (st, L.reverse, x, R) := rfl

theorem zstep_zAt_R {st s' : VS} {x y : VG} (L R : List VG) (h1 : vhalt st = false)
    (h2 : vδ st x = (s', y, Dir.R)) : zstep (zAt st L (x :: R)) = zAt s' (L ++ [y]) R := by
  rw [zAt_cons, zstep_R _ _ h1 h2]; simp [zAt]

/-- A RIGHT SCAN: each cell of X keeps the state, is rewritten by f, and the head moves right. -/
theorem scanR (st : VS) (P : VG → Bool) (f : VG → VG) (hst : vhalt st = false)
    (hP : ∀ x, P x = true → vδ st x = (st, f x, Dir.R)) :
    ∀ (X L R : List VG), (∀ x ∈ X, P x = true) →
      zsteps X.length (zAt st L (X ++ R)) = zAt st (L ++ X.map f) R := by
  intro X
  induction X with
  | nil => intro L R _; simp [zsteps]
  | cons x X ih =>
    intro L R hX
    show zsteps X.length (zstep (zAt st L (x :: (X ++ R)))) = _
    rw [zstep_zAt_R L _ hst (hP x (hX x (List.mem_cons.mpr (Or.inl rfl))))]
    rw [ih (L ++ [f x]) R (fun y hy => hX y (List.mem_cons_of_mem _ hy))]
    simp

/-- A LEFT SCAN: X lists the cells from the head leftward, each keeping the state, rewritten by f. -/
theorem scanL (st : VS) (P : VG → Bool) (f : VG → VG) (hst : vhalt st = false)
    (hP : ∀ x, P x = true → vδ st x = (st, f x, Dir.L)) :
    ∀ (X : List VG) (h : VG) (l r : List VG) (y : VG) (l0 : List VG), (∀ x ∈ X, P x = true) →
      h :: l = X ++ y :: l0 → zsteps X.length (st, l, h, r) = (st, l0, y, (X.map f).reverse ++ r) := by
  intro X
  induction X with
  | nil =>
    intro h l r y l0 _ he
    simp only [List.nil_append, List.cons.injEq] at he
    obtain (h1, h2) := he
    subst h1; subst h2; rfl
  | cons x X ih =>
    intro h l r y l0 hX he
    simp only [List.cons_append, List.cons.injEq] at he
    obtain (h1, h2) := he
    subst h1
    show zsteps X.length (zstep (st, l, h, r)) = _
    rw [zstep_L _ _ hst (hP h (hX h (List.mem_cons.mpr (Or.inl rfl))))]
    have hl : l.headD .B :: l.tail = X ++ y :: l0 := by
      rw [h2]; cases X <;> rfl
    rw [ih (l.headD .B) l.tail (f h :: r) y l0 (fun z hz => hX z (List.mem_cons_of_mem _ hz)) hl]
    simp

/-- THE RETURN: a state that walks left over every non-blank cell and turns at the blank of cell -1. -/
theorem retL (st st' : VS) (hst : vhalt st = false)
    (hP : ∀ x, (x != VG.B) = true → vδ st x = (st, x, Dir.L)) (hB : vδ st .B = (st', .B, Dir.R)) :
    ∀ (X : List VG) (h : VG) (l r : List VG), (∀ x ∈ X, (x != VG.B) = true) → h :: l = X ++ [.B] →
      zsteps (X.length + 1) (st, l, h, r) = zAt st' [.B] (X.reverse ++ r) := by
  intro X h l r hX he
  rw [zsteps_add, scanL st (fun x => x != VG.B) id hst hP X h l r .B [] hX he]
  show zstep (st, [], .B, (X.map id).reverse ++ r) = _
  rw [zstep_R _ _ hst hB]
  simp [zAt]

/-! Reachability within a step budget, rejection, acceptance. -/

def Reach (z z' : VZ) (n : Nat) : Prop := ∃ k, k ≤ n ∧ zsteps k z = z'
def RejR (z : VZ) : Prop := ∃ k, (zsteps k z).st = .rej
def AccR (z : VZ) (n : Nat) : Prop := ∃ k, k ≤ n ∧ (zsteps k z).st = .acc

theorem Reach.refl (z : VZ) : Reach z z 0 := ⟨0, Nat.le_refl 0, rfl⟩

theorem Reach.of_steps {z z' : VZ} {k n : Nat} (h : zsteps k z = z') (hk : k ≤ n) : Reach z z' n :=
  ⟨k, hk, h⟩

theorem Reach.step {z z' : VZ} (h : zstep z = z') : Reach z z' 1 := ⟨1, Nat.le_refl 1, h⟩

theorem Reach.steps {z z' : VZ} {k : Nat} (h : zsteps k z = z') : Reach z z' k := ⟨k, Nat.le_refl k, h⟩

theorem Reach.trans {z1 z2 z3 : VZ} {a b : Nat} (h1 : Reach z1 z2 a) (h2 : Reach z2 z3 b) :
    Reach z1 z3 (a + b) := by
  obtain ⟨k1, hk1, e1⟩ := h1
  obtain ⟨k2, hk2, e2⟩ := h2
  exact ⟨k1 + k2, Nat.add_le_add hk1 hk2, by rw [zsteps_add, e1, e2]⟩

theorem Reach.mono {z z' : VZ} {a b : Nat} (h : Reach z z' a) (hab : a ≤ b) : Reach z z' b := by
  obtain ⟨k, hk, e⟩ := h
  exact ⟨k, Nat.le_trans hk hab, e⟩

theorem Reach.rej {z z' : VZ} {n : Nat} (h : Reach z z' n) (hr : RejR z') : RejR z := by
  obtain ⟨k, _, e⟩ := h
  obtain ⟨k2, e2⟩ := hr
  exact ⟨k + k2, by rw [zsteps_add, e, e2]⟩

theorem Reach.acc {z z' : VZ} {n m : Nat} (h : Reach z z' n) (ha : AccR z' m) : AccR z (n + m) := by
```

```
  obtain (k, hk, e) := h
  obtain (k2, hk2, e2) := ha
  exact (k + k2, Nat.add_le_add hk hk2, by rw [zsteps_add, e, e2])

theorem RejR.of_st {z : VZ} (h : z.st = .rej) : RejR z := ⟨0, h⟩

theorem RejR.step {z : VZ} (h : RejR (zstep z)) : RejR z := by
  obtain (k, e) := h
  exact ⟨k + 1, e⟩

theorem AccR.of_st {z : VZ} (h : z.st = .acc) : AccR z 0 := ⟨0,
    Nat.le_refl 0, h⟩

theorem AccR.mono {z : VZ} {a b : Nat} (h : AccR z a) (hab : a ≤ b) :
    AccR z b := by
  obtain (k, hk, e) := h
  exact ⟨k, Nat.le_trans hk hab, e⟩

/-- A run is never in the accept state and in the reject state, at
    whatever two times. -/
theorem acc_rej_false (z : VZ) (t k : Nat) (ha : (zsteps t z).st = .acc)
    (hr : (zsteps k z).st = .rej) :
    False := by
  rcases Nat.le_total t k with h | h
  · have e : zsteps k z = zsteps t z := by
      rw [show k = t + (k - t) by omega, zsteps_add, zsteps_halt _ _ (by
        rw [ha]; rfl)]
    rw [e, ha] at hr; cases hr
  · have e : zsteps t z = zsteps k z := by
      rw [show t = k + (t - k) by omega, zsteps_add, zsteps_halt _ _ (by
        rw [hr]; rfl)]
    rw [e, hr] at ha; cases ha

/-! ### N.5 · Phase 0: the syntax check -/

def bitG (b : Bool) : VG := if b then .O else .Z

theorem symG_symOfBool (b : Bool) : symG (symOfBool b) = bitG b := by
    cases b <;> rfl

def isZO : VG → Bool
  | .Z => true
  | .O => true
  | _ => false

theorem isZO_bitG (b : Bool) : isZO (bitG b) = true := by cases b <;> rfl

def isD0 : VS → Bool
  | .p0Start => true | .p0After => true | .p0D => true | .p0S => true
  | _ => false

/-- The syntax automaton of Phase 0, run on a string. -/
def p0run (s : VS) : List Sym → VS
  | [] => s
  | x :: w => p0run (vδ s (symG x)).1 w

theorem p0run_rej : ∀ w, p0run .rej w = .rej
  | [] => rfl
  | _ :: w => p0run_rej w

theorem p0run_append (s : VS) (a b : List Sym) : p0run s (a ++ b) = p0run
    (p0run s a) b := by
  induction a generalizing s with
  | nil => rfl
  | cons x a ih => exact ih _

theorem isD0_halt (s : VS) (hs : isD0 s = true) : vhalt s = false := by
  cases s <;> simp_all [isD0, vhalt]

theorem p0_step (s : VS) (x : Sym) (hs : isD0 s = true) :
    vδ s (symG x) = ((vδ s (symG x)).1, symG x, Dir.R) ∧
      (isD0 (vδ s (symG x)).1 = true ∨ (vδ s (symG x)).1 = .rej) := by
  cases s <;> (try simp [isD0] at hs) <;> cases x <;>
    first | exact ⟨rfl, Or.inl rfl⟩ | exact ⟨rfl, Or.inr rfl⟩

/-- The automaton read off the machine: on every string it either rejects
    or walks to its end. -/
theorem p0_scan : ∀ (w : List Sym) (s : VS) (L R : List VG), isD0 s =
    true →
    (p0run s w ≠ .rej → isD0 (p0run s w) = true ∧
      zsteps w.length (zAt s L (w.map symG ++ R)) = zAt (p0run s w) (L ++
        w.map symG) R) ∧
    (p0run s w = .rej → RejR (zAt s L (w.map symG ++ R))) := by
  intro w
  induction w with
  | nil =>
    intro s L R hs
    refine ⟨fun _ => ⟨hs, by simp [zsteps, p0run]⟩, fun h => ?_⟩
    simp only [p0run] at h; subst h; simp [isD0] at hs
  | cons x w ih =>
    intro s L R hs
    obtain ⟨hδ, hnext⟩ := p0_step s x hs
    have hz : zstep (zAt s L (symG x :: (w.map symG ++ R))) =
        zAt (vδ s (symG x)).1 (L ++ [symG x]) (w.map symG ++ R) :=
      zstep_zAt_R L _ (isD0_halt s hs) hδ
    have e1 : (x :: w).map symG ++ R = symG x :: (w.map symG ++ R) := rfl
    have e2 : p0run s (x :: w) = p0run (vδ s (symG x)).1 w := rfl
    rw [e1, e2]
    rcases hnext with h1 | h1
    · obtain ⟨ih1, ih2⟩ := ih _ (L ++ [symG x]) R h1
      refine ⟨fun hne => ?_, fun hr => ?_⟩
      · obtain ⟨a, b⟩ := ih1 hne
        refine ⟨a, ?_⟩
        show zsteps w.length (zstep _) = _
        rw [hz, b]; simp
      · exact RejR.step (by rw [hz]; exact ih2 hr)
    · rw [h1, p0run_rej]
      refine ⟨fun hne => absurd rfl hne, fun _ => RejR.step ?_⟩
      rw [hz]; exact RejR.of_st h1

/-- The tape from cell 0 on: the certificate, the separator, the formula,
    the blank after it. -/
def vtape (u : List Bool) (w : List Sym) : List VG := u.map bitG ++ VG.S
    :: (w.map symG ++ [VG.B])

theorem vinit_eq (u : List Bool) (w : List Sym) :
    vinit (u.map symOfBool ++ Sym.sep :: w) = zAt .p0Cert [.B] (vtape u
      w) := by
  show zAt _ _ ((u.map symOfBool ++ Sym.sep :: w).map symG ++ [VG.B]) = _
  rw [List.map_append, List.map_map, show (symG ∘ symOfBool) = bitG from
    funext symG_symOfBool]
  simp [vtape]; rfl

theorem cons_headD_tail : ∀ (M : List VG), M ≠ [] → M.headD .B :: M.tail
    = M
  | [], h => absurd rfl h
  | _ :: _, _ => rfl

theorem bitG_ne_B (b : Bool) : (bitG b != VG.B) = true := by cases b <;>
    rfl
theorem symG_ne_B (x : Sym) : (symG x != VG.B) = true := by cases x <;>
    rfl

/-- PHASE 0: the machine walks to the end of the input and back exactly
    when the formula part is accepted by the
    syntax automaton; otherwise it rejects. -/
theorem phase0 (u : List Bool) (w : List Sym) :
    (p0run .p0Start w = .p0Start →
      Reach (zAt .p0Cert [.B] (vtape u w)) (zAt .aScan [.B] (vtape u w))
        (2 * (u.length + w.length) + 4)) ∧
    (p0run .p0Start w ≠ .p0Start → RejR (zAt .p0Cert [.B] (vtape u w)))
      := by
  -- the certificate
  have hA : zsteps (u.map bitG).length (zAt .p0Cert [VG.B] ((u.map bitG)
    ++ (VG.S :: (w.map symG ++ [VG.B])))) =
      zAt .p0Cert ([VG.B] ++ (u.map bitG).map id) (VG.S :: (w.map symG ++
        [VG.B])) :=
    scanR .p0Cert isZO id rfl (fun x hx => by cases x <;> first | rfl |
      simp [isZO] at hx) _ _ _
    (fun x hx => by simp only [List.mem_map] at hx; obtain ⟨b, _, hb⟩
      := hx; rw [← hb]; exact isZO_bitG b)
  simp only [List.map_id] at hA
  have hB : zstep (zAt .p0Cert ([VG.B] ++ u.map bitG) (VG.S :: (w.map
    symG ++ [VG.B]))) =
      zAt .p0Start ([VG.B] ++ u.map bitG ++ [VG.S]) (w.map symG ++
        [VG.B]) := by
    rw [zstep_zAt_R _ _ rfl rfl]
  have R1 : Reach (zAt .p0Cert [.B] (vtape u w)) (zAt .p0Start ([VG.B] ++
    u.map bitG ++ [VG.S]) (w.map symG ++ [VG.B]))
      (u.length + 1) := by
    refine Reach.of_steps (k := (u.map bitG).length + 1) ?_ (by simp)
    rw [zsteps_add, vtape, hA]; exact hB
  obtain ⟨hC1, hC2⟩ := p0_scan w .p0Start ([VG.B] ++ u.map bitG ++
    [VG.S]) [VG.B] rfl
  constructor
  · intro hacc
    obtain ⟨_, hC⟩ := hC1 (by rw [hacc]; exact fun h => VS.noConfusion h)
    rw [hacc] at hC
    -- turn at the blank and return
    have hD : zstep (zAt .p0Start ([VG.B] ++ u.map bitG ++ [VG.S] ++
      w.map symG) [VG.B]) =
        (.p0Ret, (([VG.B] ++ u.map bitG ++ [VG.S] ++ w.map
          symG).reverse).tail,
          ((([VG.B] ++ u.map bitG ++ [VG.S] ++ w.map symG).reverse).headD
            .B, [VG.B]) := by
      rw [zAt_cons]; rw [zstep_L _ _ rfl rfl]
    have hX : ∀ x ∈ (w.map symG).reverse ++ VG.S :: (u.map bitG).reverse,
      (x != VG.B) = true := by
      intro x hx
      simp only [List.mem_append, List.mem_reverse, List.mem_map,
        List.mem_cons] at hx
      rcases hx with ⟨y, _, hy⟩ | hx | ⟨b, _, hb⟩
      · rw [← hy]; exact symG_ne_B y
      · rw [hx]; rfl
      · rw [← hb]; exact bitG_ne_B b
    have hl : (([VG.B] ++ u.map bitG ++ [VG.S] ++ w.map
      symG).reverse).headD .B ::
        (([VG.B] ++ u.map bitG ++ [VG.S] ++ w.map symG).reverse).tail =
        ((w.map symG).reverse ++ VG.S :: (u.map bitG).reverse) ++ [VG.B]
          := by
      rw [cons_headD_tail _ (by simp)]; simp
    have hE := retL .p0Ret .aScan rfl (fun x hx => by cases x <;> first |
      rfl | simp at hx) rfl _ _ _ [VG.B] hX hl
    have R2 : Reach (zAt .p0Start ([VG.B] ++ u.map bitG ++ [VG.S]) (w.map
      symG ++ [VG.B]))
        (zAt .aScan [.B] (vtape u w)) (2 * w.length + u.length + 3) := by
      refine Reach.of_steps (k := w.length + (1 + (((w.map symG).reverse
        ++ VG.S :: (u.map bitG).reverse).length + 1)))
        ?_ (by simp; omega)
      rw [zsteps_add, hC, zsteps_add, zsteps_one, hD, hE]
      simp [vtape]
    exact (R1.trans R2).mono (by omega)
  · intro hrej
    by_cases hr : p0run .p0Start w = .rej
    · exact R1.rej (hC2 hr)
    · obtain ⟨hd, hC⟩ := hC1 hr
      refine R1.rej (w.length + 1, ?_)
      rw [zsteps_add, hC]
      generalize p0run .p0Start w = X at hd hrej
      cases X <;> simp [isD0] at hd <;> first | exact absurd rfl hrej | rfl

/-! The language of the automaton is exactly the written formulas. -/
```

```
theorem p0_digits (rest : List Sym) : ∀ ds : List Bool, p0run .p0D
    (ds.map symOfBool ++ rest) = p0run .p0D rest
  | [] => rfl
  | true :: ds => p0_digits rest ds
  | false :: ds => p0_digits rest ds

theorem p0_lit (L : VLit) (rest : List Sym) (s : VS) (hs : s = .p0Start ∨
    s = .p0After) :
    p0run s (vlitSyms L ++ rest) = p0run .p0After rest := by
  rw [vlitSyms_cons]
  have h1 : (vδ s (symG (symOfBool L.d0))).1 = .p0D := by
    rcases hs with h | h <;> subst h <;> cases L.d0 <;> rfl
  show p0run (vδ s (symG (symOfBool L.d0))).1 _ = _
  rw [h1, p0_digits]
  cases L.sg <;> rfl

theorem p0_cl : ∀ (c : List VLit) (rest : List Sym) (s : VS), (s =
    .p0Start ∨ s = .p0After) →
    p0run s (vclSyms c ++ rest) = p0run .p0Start rest
  | [], rest, s, hs => by rcases hs with h | h <;> subst h <;> rfl
  | L :: c, rest, s, hs => by
    show p0run s ((vlitSyms L ++ vclSyms c) ++ rest) = _
    rw [List.append_assoc, p0_lit L _ s hs, p0_cl c rest .p0After (Or.inr
      rfl)]

/-- Every written formula passes the syntax check. -/
theorem p0_vf : ∀ F : List (List VLit), p0run .p0Start (vfSyms F) =
    .p0Start
  | [] => rfl
  | c :: F => by
    show p0run .p0Start (vclSyms c ++ vfSyms F) = _
    rw [p0_cl c _ .p0Start (Or.inl rfl), p0_vf F]

theorem p0_inv : ∀ (w : List Sym) (s : VS), p0run s w = .p0Start →
    (s = .p0Start → ∃ F, w = vfSyms F) ∧
    (s = .p0After → ∃ c F, w = vclSyms c ++ vfSyms F) ∧
    (s = .p0D → ∃ (ds : List Bool) (sg : Bool) (c : List VLit) (F : List
      (List VLit)),
      w = ds.map symOfBool ++ Sym.sep :: symOfBool sg :: (vclSyms c ++
        vfSyms F)) ∧
    (s = .p0S → ∃ (sg : Bool) (c : List VLit) (F : List (List VLit)),
      w = symOfBool sg :: (vclSyms c ++ vfSyms F)) := by
  intro w
  induction w with
  | nil =>
    intro s h
    simp only [p0run] at h
    subst h
    exact ⟨fun _ => ([], rfl), fun h => VS.noConfusion h, fun h =>
      VS.noConfusion h,
      fun h => VS.noConfusion h⟩
  | cons x w ih =>
    intro s h
    have ih' := ih _ h
    refine ⟨fun hs => ?_, fun hs => ?_, fun hs => ?_, fun hs => ?_⟩ <;>
      subst hs <;> cases x
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨((false, ds, sg) :: c) :: F, by rw [e]; simp [vfSyms,
        vclSyms, vlitSyms, vdigits, symOfBool]⟩
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨((true, ds, sg) :: c) :: F, by rw [e]; simp [vfSyms,
        vclSyms, vlitSyms, vdigits, symOfBool]⟩
    · obtain ⟨F, e⟩ := ih'.1 rfl
      exact ⟨[] :: F, by rw [e]; rfl⟩
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨(false, ds, sg) :: c, F, by rw [e]; simp [vclSyms, vlitSyms,
        vdigits, symOfBool]⟩
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨(true, ds, sg) :: c, F, by rw [e]; simp [vclSyms, vlitSyms,
        vdigits, symOfBool]⟩
    · obtain ⟨F, e⟩ := ih'.1 rfl
      exact ⟨[], F, by rw [e]; rfl⟩
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨false :: ds, sg, c, F, by rw [e]; rfl⟩
    · obtain ⟨ds, sg, c, F, e⟩ := ih'.2.2.1 rfl
      exact ⟨true :: ds, sg, c, F, by rw [e]; rfl⟩
    · obtain ⟨sg, c, F, e⟩ := ih'.2.2.2 rfl
      exact ⟨[], sg, c, F, by rw [e]; rfl⟩
    · obtain ⟨c, F, e⟩ := ih'.2.1 rfl
      exact ⟨false, c, F, by rw [e]; rfl⟩
    · obtain ⟨c, F, e⟩ := ih'.2.1 rfl
      exact ⟨true, c, F, by rw [e]; rfl⟩
    · exact absurd (h.symm.trans (p0run_rej w)) (fun h => VS.noConfusion
      h)

/-- THE SYNTAX CHECK IS THE GRAMMAR: the automaton accepts exactly the
    written formulas. -/
theorem p0_iff (w : List Sym) : p0run .p0Start w = .p0Start ↔ ∃ F, w =
    vfSyms F :=
  ⟨fun h => (p0_inv w .p0Start h).1 rfl, fun ⟨F, e⟩ => e ▸ p0_vf F⟩

/-! ### N.6 · Phase A: each certificate bit onto a sign -/

/-- An item of the formula part of the tape: a clause-closing separator,
    or a literal with its certificate bit
    once one is written on its sign. -/
inductive VIt
  | sep
  | lit (L : VLit) (a : Option Bool)

def signCell (sg : Bool) : Option Bool → VG
  | none => bitG sg
  | some v => .A sg v

def litCells (L : VLit) (a : Option Bool) : List VG := (vdigits L).map
    bitG ++ [VG.S, signCell L.sg a]

def itCells : VIt → List VG
  | .sep => [VG.S]
  | .lit L a => litCells L a

def itsCells : List VIt → List VG
  | [] => []
  | i :: is => itCells i ++ itsCells is

def clIts (c : List VLit) : List VIt := c.map (fun L => VIt.lit L none)
    ++ [VIt.sep]

def fIts : List (List VLit) → List VIt
  | [] => []
  | c :: F => clIts c ++ fIts F

def nLits : List VIt → Nat
  | [] => 0
  | .sep :: is => nLits is
  | .lit _ _ :: is => nLits is + 1

def allPlain : List VIt → Prop
  | [] => True
  | .sep :: is => allPlain is
  | .lit _ a :: is => a = none ∧ allPlain is

/-- The certificate bits written onto the signs, in reading order. -/
def annIts : List VIt → List Bool → List VIt
  | [], _ => []
  | .sep :: is, us => .sep :: annIts is us
  | .lit L a :: is, [] => .lit L a :: is
  | .lit L _ :: is, b :: us => .lit L (some b) :: annIts is us

theorem itsCells_append (a b : List VIt) : itsCells (a ++ b) = itsCells a
    ++ itsCells b := by
  induction a with
  | nil => rfl
  | cons i a ih => simp [itsCells, ih]

theorem annIts_nil : ∀ its : List VIt, annIts its [] = its
  | [] => rfl
  | .sep :: is => by simp [annIts, annIts_nil is]
  | .lit _ _ :: _ => rfl

theorem vlitSyms_map (L : VLit) : (vlitSyms L).map symG = litCells L none
    := by
  simp only [vlitSyms, litCells, signCell, List.map_append, List.map_map,
    List.map_cons, List.map_nil,
    symG_symOfBool, show (symG ∘ symOfBool) = bitG from funext
      symG_symOfBool]
  rfl

theorem vclSyms_map : ∀ c : List VLit, (vclSyms c).map symG = itsCells
    (clIts c)
  | [] => rfl
  | L :: c => by
    have ih := vclSyms_map c
    simp only [vclSyms, clIts, List.map_append, List.map_cons,
      List.cons_append] at ih ⊢
    rw [vlitSyms_map, ih]; rfl

theorem fIts_cells : ∀ F : List (List VLit), (vfSyms F).map symG =
    itsCells (fIts F)
  | [] => rfl
  | c :: F => by
    simp only [vfSyms, fIts, List.map_append, itsCells_append,
      vclSyms_map, fIts_cells F]

theorem allPlain_append (a b : List VIt) (ha : allPlain a) (hb : allPlain
    b) : allPlain (a ++ b) := by
  induction a with
  | nil => exact hb
  | cons i a ih =>
    cases i with
    | sep => exact ih ha
    | lit L x => exact ⟨ha.1, ih ha.2⟩

theorem allPlain_fIts : ∀ F, allPlain (fIts F)
  | [] => trivial
  | c :: F => by
    apply allPlain_append _ _ _ (allPlain_fIts F)
    induction c with
    | nil => trivial
    | cons L c ih => exact ⟨rfl, ih⟩

theorem mem_map_bitG {x : VG} {l : List Bool} (h : x ∈ l.map bitG) : isZO
    x = true ∧ (x != VG.B) = true := by
  simp only [List.mem_map] at h
  obtain ⟨c, _, hc⟩ := h
  rw [← hc]; exact ⟨isZO_bitG c, bitG_ne_B c⟩

/-- A scan to the right over binary digits, by a state that keeps them. -/
theorem scanDigits (st : VS) (hst : vhalt st = false) (hP : ∀ c, vδ st
    (bitG c) = (st, bitG c, Dir.R))
    (ds : List Bool) (L R : List VG) :
    Reach (zAt st L (ds.map bitG ++ R)) (zAt st (L ++ ds.map bitG) R)
      ds.length := by
  have := scanR st isZO id hst (fun x hx => by
    cases x <;> simp [isZO] at hx
    · exact hP false
    · exact hP true) (ds.map bitG) L R (fun x hx => (mem_map_bitG hx).1)
  rw [List.map_id, List.length_map] at this
  exact Reach.steps this

theorem aC_digit (b c : Bool) : vδ (.aC b) (bitG c) = (.aD b, bitG c, .R)
    := by cases c <;> rfl
```

```
theorem aD_digit (b c : Bool) : vδ (.aD b) (bitG c) = (.aD b, bitG c, .R)
    := by cases c <;> rfl
theorem aS_sign (b sg : Bool) : vδ (.aS b) (bitG sg) = (.aRet, .A sg b,
    .L) := by cases sg <;> rfl

/-- The digits of a literal, from the literal-start state of Phase A to
    its sign. -/
theorem aLitDigits (b : Bool) (L0 : VLit) (L R : List VG) :
    Reach (zAt (.aC b) L ((vdigits L0).map bitG ++ VG.S :: R))
      (zAt (.aS b) (L ++ (vdigits L0).map bitG ++ [VG.S]) R) ((vdigits
        L0).length + 1) := by
  have e : (vdigits L0).map bitG ++ VG.S :: R = bitG L0.d0 :: (L0.ds.map
      bitG ++ VG.S :: R) := by
    simp [vdigits]
  rw [e]
  have h1 := Reach.step (zstep_zAt_R L (L0.ds.map bitG ++ VG.S :: R) rfl
      (aC_digit b L0.d0))
  have h2 := scanDigits (.aD b) rfl (aD_digit b) L0.ds (L ++ [bitG
      L0.d0]) (VG.S :: R)
  have h3 := Reach.step (zstep_zAt_R (L ++ [bitG L0.d0] ++ L0.ds.map
      bitG) R rfl
    (show vδ (.aD b) .S = (.aS b, .S, .R) from rfl))
  have := (h1.trans h2).trans h3
  simp only [vdigits, List.map_cons, List.append_assoc, List.cons_append,
      List.nil_append,
    List.length_cons] at this ⊢
  exact this.mono (by omega)

theorem litCells_length (L0 : VLit) (a : Option Bool) : (litCells L0
    a).length = (vdigits L0).length + 2 := by
  simp [litCells]

/-- PHASE A, INSIDE THE FORMULA, a bit to place: the first sign without a
    bit receives it. -/
theorem aScanRet (b : Bool) : ∀ (its : List VIt) (us : List Bool) (L :
    List VG), allPlain its →
    us.length < nLits its →
    ∃ (X : List VG) (h' : VG) (l' r' : List VG),
      Reach (zAt (.aC b) L (itsCells (annIts its us) ++ [VG.B])) (.aRet,
        l', h', r') (itsCells its).length ∧
      h' :: l' = X ++ L.reverse ∧ (∀ x ∈ X, (x != VG.B) = true) ∧
      X.reverse ++ r' = itsCells (annIts its (us ++ [b])) ++ [VG.B] := by
  intro its
  induction its with
  | nil => intro us L _ h; simp [nLits] at h
  | cons it is ih =>
    intro us L hp hlt
    cases it with
    | sep =>
      obtain ⟨X, h', l', r', hR, e1, hX, e2⟩ := ih us (L ++ [VG.S]) hp hlt
      refine ⟨X ++ [VG.S], h', l', r', ?_, by rw [e1]; simp, ?_, by rw
        [List.reverse_append]; simp [annIts, itsCells, itCells, e2]⟩
      · have h1 : zstep (zAt (.aC b) L (itsCells (annIts (.sep :: is) us)
          ++ [VG.B])) =
            zAt (.aC b) (L ++ [VG.S]) (itsCells (annIts is us) ++ [VG.B])
              := by
          simp only [annIts, itsCells, itCells, List.cons_append,
            List.nil_append]
          exact zstep_zAt_R _ _ rfl rfl
        have := (Reach.step h1).trans hR
        exact this.mono (by simp [itsCells, itCells]; omega)
      · intro x hx; simp only [List.mem_append, List.mem_singleton] at hx
        rcases hx with hx | hx
        · exact hX x hx
        · rw [hx]; rfl
    | lit L0 a =>
      obtain ⟨ha, hp'⟩ := hp
      subst ha
      cases us with
      | nil =>
        -- the target literal
        refine ⟨VG.S :: ((vdigits L0).map bitG).reverse, VG.S, ((vdigits
          L0).map bitG).reverse ++ L.reverse,
          VG.A L0.sg b :: (itsCells is ++ [VG.B]), ?_, by simp, ?_, ?_⟩
        · have h1 := aLitDigits b L0 L (bitG L0.sg :: (itsCells is ++
            [VG.B]))
          have h2 : zstep (zAt (.aS b) (L ++ (vdigits L0).map bitG ++
              [VG.S]) (bitG L0.sg :: (itsCells is ++ [VG.B]))) =
              (.aRet, ((vdigits L0).map bitG).reverse ++ L.reverse, VG.S,
                VG.A L0.sg b :: (itsCells is ++ [VG.B])) := by
            rw [zAt_cons, zstep_L _ _ rfl (aS_sign b L0.sg)]
            simp
          have e : itsCells (annIts (.lit L0 none :: is) []) ++ [VG.B] =
              ((vdigits L0).map bitG ++ VG.S :: (bitG L0.sg :: (itsCells is
                  ++ [VG.B])) := by
            simp [annIts, itsCells, itCells, litCells, signCell]
          rw [e]
          refine Reach.mono (h1.trans (Reach.step h2)) ?_
          simp [itsCells, itCells, litCells]; omega
        · intro x hx
          simp only [List.mem_cons, List.mem_reverse, List.mem_map] at hx
          rcases hx with hx | ⟨c, _, hc⟩
          · rw [hx]; rfl
          · rw [← hc]; exact bitG_ne_B c
        · simp [annIts, annIts_nil, itsCells, itCells, litCells, signCell]
      | cons u0 us' =>
        have hlt' : us'.length < nLits is := by simp [nLits] at hlt; omega
        obtain ⟨X, h', l', r', hR, e1, hX, e2⟩ := ih us' (L ++ litCells L0
            (some u0)) hp' hlt'
        refine ⟨X ++ (litCells L0 (some u0)).reverse, h', l', r', ?_, by
            rw [e1]; simp, ?_, ?_⟩
        · have h1 := aLitDigits b L0 L (VG.A L0.sg u0 :: (itsCells
            (annIts is us') ++ [VG.B]))
          have h2 : zstep (zAt (.aS b) (L ++ (vdigits L0).map bitG ++
              [VG.S])
              (VG.A L0.sg u0 :: (itsCells (annIts is us') ++ [VG.B]))) =
              zAt (.aC b) (L ++ litCells L0 (some u0)) (itsCells (annIts
                  is us') ++ [VG.B]) := by
            rw [zstep_zAt_R _ _ rfl rfl]; simp [litCells, signCell]
          have e : itsCells (annIts (.lit L0 none :: is) (u0 :: us')) ++
              [VG.B] =
              (vdigits L0).map bitG ++ VG.S :: (VG.A L0.sg u0 :: (itsCells
                  (annIts is us') ++ [VG.B])) := by
            simp [annIts, itsCells, itCells, litCells, signCell]
          rw [e]
          refine Reach.mono ((h1.trans (Reach.step h2)).trans hR) ?_
          simp [itsCells, itCells, litCells]; omega
        · intro x hx
          simp only [List.mem_append, List.mem_reverse] at hx
          rcases hx with hx | hx
          · exact hX x hx
          · simp only [litCells, signCell, List.mem_append, List.mem_map,
              List.mem_cons] at hx
            rcases hx with ⟨c, _, hc⟩ | hx | hx | hx
            · rw [← hc]; exact bitG_ne_B c
            · rw [hx]; rfl
            · rw [hx]; rfl
            · simp at hx
        · rw [List.reverse_append, List.reverse_reverse,
            List.append_assoc, e2]
          simp [annIts, itsCells, itCells]

/-- PHASE A, INSIDE THE FORMULA, every sign already carrying a bit: the
    machine reaches the end and turns back. -/
theorem aScanFin (b : Bool) : ∀ (its : List VIt) (us : List Bool) (L :
    List VG), allPlain its →
    nLits its ≤ us.length → L ≠ [] →
    ∃ (X : List VG) (h' : VG) (l' r' : List VG),
      Reach (zAt (.aC b) L (itsCells (annIts its us) ++ [VG.B])) (.aFin,
        l', h', r') ((itsCells its).length + 1) ∧
      h' :: l' = X ++ L.reverse ∧ (∀ x ∈ X, (x != VG.B) = true) ∧
      X.reverse ++ r' = itsCells (annIts its us) ++ [VG.B] := by
  intro its
  induction its with
  | nil =>
    intro us L _ hL
    refine ⟨[], L.reverse.headD .B, L.reverse.tail, [VG.B], ?_, ?_, fun x
        hx => by simp at hx, by simp [annIts, itsCells]⟩
    · refine Reach.step ?_
      show zstep (.aC b, L.reverse, .B, []) = _
      rw [zstep_L _ _ rfl rfl]
    · rw [cons_headD_tail _ (by simpa using hL)]; rfl
  | cons it is ih =>
    intro us L hp hle hL
    cases it with
    | sep =>
      obtain ⟨X, h', l', r', hR, e1, hX, e2⟩ := ih us (L ++ [VG.S]) hp hle
          (by simp)
      refine ⟨X ++ [VG.S], h', l', r', ?_, by rw [e1]; simp, ?_, by rw
        [List.reverse_append]; simp [annIts, itsCells, itCells, e2]⟩
      · have h1 : zstep (zAt (.aC b) L (itsCells (annIts (.sep :: is) us)
          ++ [VG.B])) =
            zAt (.aC b) (L ++ [VG.S]) (itsCells (annIts is us) ++ [VG.B])
              := by
          simp only [annIts, itsCells, itCells, List.cons_append,
            List.nil_append]
          exact zstep_zAt_R _ _ rfl rfl
        exact ((Reach.step h1).trans hR).mono (by simp [itsCells,
          itCells]; omega)
      · intro x hx; simp only [List.mem_append, List.mem_singleton] at hx
        rcases hx with hx | hx
        · exact hX x hx
        · rw [hx]; rfl
    | lit L0 a =>
      obtain ⟨ha, hp'⟩ := hp
      subst ha
      cases us with
      | nil => simp [nLits] at hle
      | cons u0 us' =>
        have hle' : nLits is ≤ us'.length := by simp [nLits] at hle; omega
        obtain ⟨X, h', l', r', hR, e1, hX, e2⟩ := ih us' (L ++ litCells L0
            (some u0)) hp' hle' (by simp [litCells])
        refine ⟨X ++ (litCells L0 (some u0)).reverse, h', l', r', ?_, by
            rw [e1]; simp, ?_, ?_⟩
        · have h1 := aLitDigits b L0 L (VG.A L0.sg u0 :: (itsCells
            (annIts is us') ++ [VG.B]))
          have h2 : zstep (zAt (.aS b) (L ++ (vdigits L0).map bitG ++
              [VG.S])
              (VG.A L0.sg u0 :: (itsCells (annIts is us') ++ [VG.B]))) =
              zAt (.aC b) (L ++ litCells L0 (some u0)) (itsCells (annIts
                  is us') ++ [VG.B]) := by
            rw [zstep_zAt_R _ _ rfl rfl]; simp [litCells, signCell]
          have e : itsCells (annIts (.lit L0 none :: is) (u0 :: us')) ++
              [VG.B] =
              (vdigits L0).map bitG ++ VG.S :: (VG.A L0.sg u0 :: (itsCells
                  (annIts is us') ++ [VG.B])) := by
            simp [annIts, itsCells, itCells, litCells, signCell]
          rw [e]
          refine Reach.mono ((h1.trans (Reach.step h2)).trans hR) ?_
          simp [itsCells, itCells, litCells]; omega
        · intro x hx
          simp only [List.mem_append, List.mem_reverse] at hx
          rcases hx with hx | hx
          · exact hX x hx
          · simp only [litCells, signCell, List.mem_append, List.mem_map,
              List.mem_cons] at hx
            rcases hx with ⟨c, _, hc⟩ | hx | hx | hx
            · rw [← hc]; exact bitG_ne_B c
            · rw [hx]; rfl
            · rw [hx]; rfl
            · simp at hx
        · rw [List.reverse_append, List.reverse_reverse,
            List.append_assoc, e2]
```

```
          simp [annIts, itsCells, itCells]

theorem annIts_long : ∀ (its : List VIt) (us vs : List Bool), nLits its ≤ us.length →
    annIts its (us ++ vs) = annIts its us
  | [], _, _, _ => rfl
  | .sep :: is, us, vs, h => by simp only [annIts]; rw [annIts_long is us vs h]
  | .lit L a :: is, [], vs, h => by simp [nLits] at h
  | .lit L a :: is, u :: us, vs, h => by
    simp only [List.cons_append, annIts]
    rw [annIts_long is us vs (by simp [nLits] at h; omega)]

theorem annIts_length : ∀ (its : List VIt) (us : List Bool), (itsCells (annIts its us)).length = (itsCells its).length
  | [], _ => rfl
  | .sep :: is, us => by simp [annIts, itsCells, itCells, annIts_length is us]
  | .lit L a :: is, [] => rfl
  | .lit L a :: is, u :: us => by simp [annIts, itsCells, itCells, litCells, annIts_length is us]

/-- The tape of Phase A after the bits of `pre` are placed, the bits of `post` still to come. -/
def tapeA (its : List VIt) (pre post : List Bool) : List VG :=
  List.replicate pre.length VG.U ++ post.map bitG ++ VG.S :: (itsCells (annIts its pre) ++ [VG.B])

def isU : VG → Bool
  | .U => true
  | _ => false

theorem scanU (st : VS) (hst : vhalt st = false) (hP : vδ st .U = (st, .U, Dir.R)) (k : Nat) (L R : List VG) :
    Reach (zAt st L (List.replicate k VG.U ++ R)) (zAt st (L ++ List.replicate k VG.U) R) k := by
  have := scanR st isU id hst (fun x hx => by cases x <;> simp [isU] at hx; exact hP) (List.replicate k VG.U) L R
  (fun x hx => by rw [List.eq_of_mem_replicate hx]; rfl)
  rw [List.map_id, List.length_replicate] at this
  exact Reach.steps this

/-- From the certificate start, take the next bit, carry it into the formula. -/
theorem aTake (its : List VIt) (pre post : List Bool) (a : Bool) :
    Reach (zAt .aScan [.B] (tapeA its pre (a :: post)))
      (zAt (.aC a) ([VG.B] ++ List.replicate (pre.length + 1) VG.U ++ post.map bitG ++ [VG.S])
        (itsCells (annIts its pre) ++ [VG.B])) (pre.length + post.length + 2) := by
  have h1 := scanU .aScan rfl rfl pre.length [VG.B]
    (bitG a :: (post.map bitG ++ VG.S :: (itsCells (annIts its pre) ++ [VG.B])))
  have h2 := Reach.step (zstep_zAt_R ([VG.B] ++ List.replicate pre.length VG.U)
    (post.map bitG ++ VG.S :: (itsCells (annIts its pre) ++ [VG.B])) rfl
    (show vδ .aScan (bitG a) = (.aToW a, .U, .R) by cases a <;> rfl))
  have h3 := scanDigits (.aToW a) rfl (fun c => by cases c <;> rfl) post
    ([VG.B] ++ List.replicate pre.length VG.U ++ [VG.U]) (VG.S :: (itsCells (annIts its pre) ++ [VG.B]))
  have h4 := Reach.step (zstep_zAt_R ([VG.B] ++ List.replicate pre.length VG.U ++ [VG.U] ++ post.map bitG)
    (itsCells (annIts its pre) ++ [VG.B]) rfl (show vδ (.aToW a) .S = (.aC a, .S, .R) from rfl))
  have := ((h1.trans h2).trans h3).trans h4
  simp only [tapeA, List.map_cons, List.cons_append, List.append_assoc, List.length_cons] at this ⊢
  rw [List.replicate_succ', List.append_assoc]
  simp only [List.singleton_append, List.cons_append, List.nil_append] at this ⊢
  exact this.mono (by omega)

theorem replicate_U_nb (k : Nat) : ∀ x ∈ List.replicate k VG.U, (x != VG.B) = true := by
  intro x hx; rw [List.eq_of_mem_replicate hx]; rfl

theorem len_le_of_rev_append {X r M : List VG} (h : X.reverse ++ r = M) : X.length ≤ M.length := by
  have := congrArg List.length h
  simp only [List.length_append, List.length_reverse] at this
  omega

/-- ONE ROUND OF PHASE A: the next certificate bit is written onto the first sign without one. -/
theorem aRound (its : List VIt) (hp : allPlain its) (pre post : List Bool) (a : Bool)
    (hlt : pre.length < nLits its) :
    Reach (zAt .aScan [.B] (tapeA its pre (a :: post))) (zAt .aScan [.B] (tapeA its (pre ++ [a]) post))
      (2 * (pre.length + post.length + (itsCells its).length) + 8) := by
  have h1 := aTake its pre post a
  obtain (X, h', l', r', hR, e1, hX, e2) :=
    aScanRet a its pre ([VG.B] ++ List.replicate (pre.length + 1) VG.U ++ post.map bitG ++ [VG.S]) hp hlt
  have hXl := len_le_of_rev_append e2
  simp only [List.length_append, annIts_length, List.length_singleton] at hXl
  have e3 : h' :: l' = (X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U)) ++ [VG.B] := by
    rw [e1]; simp
  have hX' : ∀ x ∈ X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U),
      (x != VG.B) = true := by
    intro x hx
    simp only [List.mem_append, List.mem_cons, List.mem_reverse] at hx
    rcases hx with hx | hx | hx | hx
    · exact hX x hx
    · rw [hx]; rfl
    · exact (mem_map_bitG hx).2
    · exact replicate_U_nb _ x hx
  have h3 := Reach.steps (retL .aRet .aScan rfl (fun x hx => by cases x <;> first | rfl | simp at hx) rfl
    _ h' l' r' hX' e3)
  have := (h1.trans hR).trans h3
  have e4 : (X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U)).reverse ++ r' =
      tapeA its (pre ++ [a]) post := by
    simp only [List.reverse_append, List.reverse_cons, List.reverse_reverse, List.reverse_replicate,
      List.append_assoc, List.singleton_append, List.cons_append, tapeA, List.length_append,
      List.length_singleton, List.nil_append]
    rw [e2]
  rw [e4] at this
  refine this.mono ?_
  simp only [List.length_append, List.length_cons, List.length_reverse, List.length_map,
    List.length_replicate]
  omega

def isUZO : VG → Bool
  | .U => true
  | .Z => true
  | .O => true
  | _ => false

/-- THE END OF PHASE A, the certificate used up. -/
theorem aEnd (its : List VIt) (pre : List Bool) :
    Reach (zAt .aScan [.B] (tapeA its pre []))
      (zAt .bStart ([VG.B] ++ List.replicate pre.length VG.U ++ [VG.S]) (itsCells (annIts its pre) ++ [VG.B]))
      (pre.length + 1) := by
  have h1 := scanU .aScan rfl rfl pre.length [VG.B] (VG.S :: (itsCells (annIts its pre) ++ [VG.B]))
  have h2 := Reach.step (zstep_zAt_R ([VG.B] ++ List.replicate pre.length VG.U)
    (itsCells (annIts its pre) ++ [VG.B]) rfl (show vδ .aScan .S = (.bStart, .S, .R) from rfl))
  have := h1.trans h2
  simpa [tapeA] using this

/-- THE END OF PHASE A, every sign carrying a bit: the extra bit is spent and the machine goes on to Phase B. -/
theorem aEndFull (its : List VIt) (hp : allPlain its) (pre post : List Bool) (a : Bool)
    (hge : nLits its ≤ pre.length) :
    Reach (zAt .aScan [.B] (tapeA its pre (a :: post)))
      (zAt .bStart ([VG.B] ++ (List.replicate (pre.length + 1) VG.U ++ post.map bitG) ++ [VG.S])
        (itsCells (annIts its pre) ++ [VG.B]))
      (3 * (pre.length + post.length + 1 + (itsCells its).length) + 10) := by
  have h1 := aTake its pre post a
  obtain (X, h', l', r', hR, e1, hX, e2) :=
    aScanFin a its pre ([VG.B] ++ List.replicate (pre.length + 1) VG.U ++ post.map bitG ++ [VG.S]) hp hge (by simp)
  have hXl := len_le_of_rev_append e2
  simp only [List.length_append, annIts_length, List.length_singleton] at hXl
  have e3 : h' :: l' = (X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U)) ++ [VG.B] := by
    rw [e1]; simp
  have hX' : ∀ x ∈ X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U),
      (x != VG.B) = true := by
    intro x hx
    simp only [List.mem_append, List.mem_cons, List.mem_reverse] at hx
    rcases hx with hx | hx | hx | hx
    · exact hX x hx
    · rw [hx]; rfl
    · exact (mem_map_bitG hx).2
    · exact replicate_U_nb _ x hx
  have h3 := Reach.steps (retL .aFin .aSkip rfl (fun x hx => by cases x <;> first | rfl | simp at hx) rfl
    _ h' l' r' hX' e3)
  have e4 : (X ++ VG.S :: ((post.map bitG).reverse ++ List.replicate (pre.length + 1) VG.U)).reverse ++ r' =
      List.replicate (pre.length + 1) VG.U ++ (post.map bitG ++ VG.S :: (itsCells (annIts its pre) ++ [VG.B])) := by
    simp only [List.reverse_append, List.reverse_cons, List.reverse_reverse, List.reverse_replicate,
      List.append_assoc, List.singleton_append, List.cons_append, List.nil_append]
    rw [e2]
  rw [e4] at h3
  have h4 := scanU .aSkip rfl rfl (pre.length + 1) [VG.B] (post.map bitG ++ VG.S :: (itsCells (annIts its pre) ++ [VG.B]))
  have h5 := scanDigits .aSkip rfl (fun c => by cases c <;> rfl) post ([VG.B] ++ List.replicate (pre.length + 1) VG.U)
    (VG.S :: (itsCells (annIts its pre) ++ [VG.B]))
  have h6 := Reach.step (zstep_zAt_R ([VG.B] ++ List.replicate (pre.length + 1) VG.U ++ post.map bitG)
    (itsCells (annIts its pre) ++ [VG.B]) rfl (show vδ .aSkip .S = (.bStart, .S, .R) from rfl))
  have := ((((h1.trans hR).trans h3).trans h4).trans h5).trans h6
  simp only [List.append_assoc] at this ⊢
  refine this.mono ?_
  simp only [List.length_append, List.length_cons, List.length_reverse, List.length_map,
    List.length_replicate]
  omega
```

```
/-- PHASE A: the certificate bits are written onto the signs in reading order, as many as both allow. -/
theorem aLoop (its : List VIt) (hp : allPlain its) : ∀ (post pre : List Bool) (K : Nat),
    pre.length ≤ nLits its → 3 * (pre.length + post.length + (itsCells its).length) + 10 ≤ K →
    ∃ C : List VG, (∀ x ∈ C, isUZO x = true) ∧ C.length = pre.length + post.length ∧
      Reach (zAt .aScan [.B] (tapeA its pre post))
        (zAt .bStart ([VG.B] ++ C ++ [VG.S]) (itsCells (annIts its (pre ++ post)) ++ [VG.B]))
        ((post.length + 1) * K) := by
  intro post
  induction post with
  | nil =>
    intro pre K _ hK
    refine (List.replicate pre.length VG.U, fun x hx => by rw [List.eq_of_mem_replicate hx]; rfl,
      by simp, ?_)
    rw [List.append_nil]
    exact (aEnd its pre).mono (by simp only [List.length_nil, Nat.zero_add, Nat.one_mul] at hK ⊢; omega)
  | cons a post ih =>
    intro pre K hle hK
    by_cases hlt : pre.length < nLits its
    · have hle' : (pre ++ [a]).length ≤ nLits its := by simp; omega
      have hK' : 3 * ((pre ++ [a]).length + post.length + (itsCells its).length) + 10 ≤ K := by
        simp at hK ⊢; omega
      obtain ⟨C, hC1, hC2, hR⟩ := ih (pre ++ [a]) K hle' hK'
      refine ⟨C, hC1, by rw [hC2]; simp; omega, ?_⟩
      have h1 := aRound its hp pre post a hlt
      rw [show pre ++ [a] ++ post = pre ++ a :: post by simp] at hR
      refine (h1.trans hR).mono ?_
      simp only [List.length_cons] at hK ⊢
      generalize K = K' at *
      rw [Nat.succ_mul (post.length + 1)]
      omega
    · have hge : nLits its ≤ pre.length := by omega
      refine (List.replicate (pre.length + 1) VG.U ++ post.map bitG, ?_, by simp; omega, ?_)
      · intro x hx
        simp only [List.mem_append, List.mem_map] at hx
        rcases hx with hx | ⟨c, _, hc⟩
        · rw [List.eq_of_mem_replicate hx]; rfl
        · rw [← hc]; cases c <;> rfl
      · rw [annIts_long its pre (a :: post) hge]
        refine (aEndFull its hp pre post a hge).mono ?_
        simp only [List.length_cons]
        generalize K = K' at *
        rw [Nat.succ_mul (post.length + 1)]
        have : 0 ≤ (post.length + 1) * K' := Nat.zero_le _
        simp only [List.length_cons] at hK
        omega

/-- PHASE A from the certificate start. -/
theorem phaseA (its : List VIt) (hp : allPlain its) (u : List Bool) :
    ∃ C : List VG, (∀ x ∈ C, isUZO x = true) ∧ C.length = u.length ∧
      Reach (zAt .aScan [.B] (u.map bitG ++ VG.S :: (itsCells its ++ [VG.B])))
        (zAt .bStart ([VG.B] ++ C ++ [VG.S]) (itsCells (annIts its u) ++ [VG.B]))
        ((u.length + 1) * (3 * (u.length + (itsCells its).length) + 10)) := by
  obtain ⟨C, h1, h2, h3⟩ := aLoop its hp u [] _ (Nat.zero_le _) (Nat.le_refl _)
  refine ⟨C, h1, by simpa using h2, ?_⟩
  simpa [tapeA, annIts_nil] using h3

/-! ### N.7 · Phase B: in every clause some sign meets its bit -/

/-- The clause automaton of Phase B: `none` at a clause start, `some f` after a literal, f recording whether a
    sign met its bit in the clause so far. -/
def bCheck : Option Bool → List VIt → Bool
  | none, [] => true
  | none, .sep :: _ => false
  | none, .lit _ none :: _ => false
  | none, .lit L (some v) :: is => bCheck (some (L.sg == v)) is
  | some _, [] => false
  | some f, .sep :: is => f && bCheck none is
  | some _, .lit _ none :: _ => false
  | some f, .lit L (some v) :: is => bCheck (some (f || L.sg == v)) is

def bSt : Option Bool → VS
  | none => .bStart
  | some f => .bAfter f

def bF : Option Bool → Bool
  | none => false
  | some f => f

theorem bCheck_lit (o : Option Bool) (L0 : VLit) (v : Bool) (is : List VIt) :
    bCheck o (.lit L0 (some v) :: is) = bCheck (some (bF o || L0.sg == v)) is := by
  cases o <;> rfl

theorem itsCells_nb : ∀ (its : List VIt), ∀ x ∈ itsCells its, (x != VG.B) = true
  | [], _, hx => by simp [itsCells] at hx
  | .sep :: is, x, hx => by
    simp only [itsCells, itCells, List.mem_append, List.mem_singleton] at hx
    rcases hx with hx | hx
    · rw [hx]; rfl
    · exact itsCells_nb is x hx
  | .lit L a :: is, x, hx => by
    simp only [itsCells, itCells, litCells, List.mem_append, List.mem_cons, List.mem_singleton] at hx
    rcases hx with (hx | hx | hx | hx) | hx
    · exact (mem_map_bitG hx).2
    · rw [hx]; rfl
    · rw [hx]; cases a with
      | none => exact bitG_ne_B _
      | some v => rfl
    · simp at hx
    · exact itsCells_nb is x hx

theorem bSt_digit (o : Option Bool) (c : Bool) : vδ (bSt o) (bitG c) = (.bD (bF o), bitG c, .R) := by
  cases o <;> cases c <;> rfl

theorem bSt_halt (o : Option Bool) : vhalt (bSt o) = false := by cases o <;> rfl

/-- The digits of a literal, from a literal-start state of Phase B to its sign. -/
theorem bLitDigits (o : Option Bool) (L0 : VLit) (L R : List VG) :
    Reach (zAt (bSt o) L ((vdigits L0).map bitG ++ VG.S :: R))
      (zAt (.bS (bF o)) (L ++ (vdigits L0).map bitG ++ [VG.S]) R)
      ((vdigits L0).length + 1) := by
  have e : (vdigits L0).map bitG ++ VG.S :: R = bitG L0.d0 :: (L0.ds.map bitG ++ VG.S :: R) := by
    simp [vdigits]
  rw [e]
  have h1 := Reach.step (zstep_zAt_R L (L0.ds.map bitG ++ VG.S :: R) (bSt_halt o) (bSt_digit o L0.d0))
  have h2 := scanDigits (.bD (bF o)) rfl (fun c => by cases c <;> rfl) L0.ds (L ++ [bitG L0.d0]) (VG.S :: R)
  have h3 := Reach.step (zstep_zAt_R (L ++ [bitG L0.d0] ++ L0.ds.map bitG) R rfl
    (show vδ (.bD (bF o)) .S = (.bS (bF o), .S, .R) from rfl))
  have := (h1.trans h2).trans h3
  simp only [vdigits, List.map_cons, List.append_assoc, List.cons_append, List.nil_append,
    List.length_cons] at this ⊢
  exact this.mono (by omega)

/-- PHASE B on the machine: it reaches Phase C at the formula start exactly when the clause automaton accepts;
    otherwise it rejects. -/
theorem bScan (C : List VG) (hC : ∀ x ∈ C, isUZO x = true) : ∀ (its : List VIt) (o : Option Bool) (P : List VG),
    (∀ x ∈ P, (x != VG.B) = true) →
    (bCheck o its = true →
      Reach (zAt (bSt o) ([VG.B] ++ C ++ [VG.S] ++ P) (itsCells its ++ [VG.B]))
        (zAt .cFindOuter ([VG.B] ++ C ++ [VG.S]) (P ++ itsCells its ++ [VG.B]))
        ((itsCells its).length + (P.length + (itsCells its).length) + 2 * C.length + 4)) ∧
    (bCheck o its = false → RejR (zAt (bSt o) ([VG.B] ++ C ++ [VG.S] ++ P) (itsCells its ++ [VG.B]))) := by
  intro its
  induction its with
  | nil =>
    intro o P hP
    cases o with
    | none =>
      refine (fun _ => ?_, fun h => by simp [bCheck] at h)
      have h1 : zstep (zAt .bStart ([VG.B] ++ C ++ [VG.S] ++ P) (itsCells [] ++ [VG.B])) =
          (.bRet, (([VG.B] ++ C ++ [VG.S] ++ P).reverse).tail, (([VG.B] ++ C ++ [VG.S] ++ P).reverse).headD .B,
            [VG.B]) := by
        show zstep (.bStart, _, .B, []) = _
        rw [zstep_L _ _ rfl rfl]
      have hX : ∀ x ∈ P.reverse ++ VG.S :: C.reverse, (x != VG.B) = true := by
        intro x hx
        simp only [List.mem_append, List.mem_reverse, List.mem_cons] at hx
        rcases hx with hx | hx | hx
        · exact hP x hx
        · rw [hx]; rfl
        · have := hC x hx; cases x <;> simp_all [isUZO]
      have hl : (([VG.B] ++ C ++ [VG.S] ++ P).reverse).headD .B ::
          (([VG.B] ++ C ++ [VG.S] ++ P).reverse).tail =
            (P.reverse ++ VG.S :: C.reverse) ++ [VG.B] := by
        rw [cons_headD_tail _ (by simp)]; simp
      have h2 := Reach.steps (retL .bRet .bSkip rfl (fun x hx => by cases x <;> first | rfl | simp at hx) rfl
        _ _ _ [VG.B] hX hl)
      have e2 : (P.reverse ++ VG.S :: C.reverse).reverse ++ [VG.B] = C ++ (VG.S :: (P ++ [VG.B])) := by simp
      rw [e2] at h2
      have h3 := scanR .bSkip isUZO id rfl (fun x hx => by cases x <;> first | rfl | simp [isUZO] at hx)
        C [VG.B] (VG.S :: (P ++ [VG.B])) hC
      rw [List.map_id] at h3
      have h4 := Reach.step (zstep_zAt_R ([VG.B] ++ C) (P ++ [VG.B]) rfl
        (show vδ .bSkip .S = (.cFindOuter, .S, .R) from rfl))
      have := (((Reach.step h1).trans h2).trans (Reach.steps h3)).trans h4
      simp only [itsCells, List.append_nil, List.append_assoc] at this ⊢
      refine this.mono ?_
      simp only [List.length_append, List.length_reverse, List.length_cons, List.length_nil]
      omega
    | some f =>
      refine (fun h => by simp [bCheck] at h, fun _ => RejR.step (RejR.of_st rfl))
  | cons it is ih =>
```

```
    intro o P hP
    cases it with
    | sep =>
      cases o with
      | none => exact (fun h => by simp [bCheck] at h, fun _ => RejR.step
          (RejR.of_st rfl))
      | some f =>
        cases f with
        | false => exact (fun h => by simp [bCheck] at h, fun _ =>
            RejR.step (RejR.of_st rfl))
        | true =>
          have hP' : ∀ x ∈ P ++ [VG.S], (x != VG.B) = true := by
            intro x hx; simp only [List.mem_append, List.mem_singleton]
              at hx
            rcases hx with hx | hx
            · exact hP x hx
            · rw [hx]; rfl
          obtain (ih1, ih2) := ih none (P ++ [VG.S]) hP'
          have h1 : zstep (zAt (bSt (some true)) ([VG.B] ++ C ++ [VG.S] ++
              P) (itsCells (.sep :: is) ++ [VG.B])) =
              zAt .bStart ([VG.B] ++ C ++ [VG.S] ++ (P ++ [VG.S]))
                (itsCells is ++ [VG.B]) := by
            simp only [itsCells, itCells, List.cons_append,
              List.nil_append, bSt]
            rw [zstep_zAt_R _ _ rfl rfl]; simp
          refine (fun h => ?_, fun h => RejR.step (by rw [h1]; exact ih2
            h))
          have := (Reach.step h1).trans (ih1 h)
          simp only [List.append_assoc, List.singleton_append, itsCells,
            itCells, List.cons_append,
            List.nil_append, List.length_append, List.length_cons,
              List.length_singleton, List.length_nil] at this ⊢
          refine this.mono ?_
          omega
    | lit L0 a =>
      have hd := bLitDigits o L0 ([VG.B] ++ C ++ [VG.S] ++ P) (signCell
        L0.sg a :: (itsCells is ++ [VG.B]))
      have e : itsCells (.lit L0 a :: is) ++ [VG.B] =
          (vdigits L0).map bitG ++ VG.S :: (signCell L0.sg a :: (itsCells
            is ++ [VG.B])) := by
        simp [itsCells, itCells, litCells]
      rw [e]
      cases a with
      | none =>
        refine (fun h => by cases o <;> simp [bCheck] at h, fun _ =>
          hd.rej (RejR.step (RejR.of_st ?_)))
        show (zstep (.bS (bF o), _, bitG L0.sg, _)).st = .rej
        cases L0.sg <;> rfl
      | some v =>
        have hP' : ∀ x ∈ P ++ litCells L0 (some v), (x != VG.B) = true :=
          by
          intro x hx; simp only [List.mem_append] at hx
          rcases hx with hx | hx
          · exact hP x hx
          · exact itsCells_nb [.lit L0 (some v)] x (by simp [itsCells,
            itCells, hx])
        obtain (ih1, ih2) := ih (some (bF o || L0.sg == v)) (P ++ litCells
          L0 (some v)) hP'
        have h1 : zstep (zAt (.bS (bF o)) ([VG.B] ++ C ++ [VG.S] ++ P ++
            (vdigits L0).map bitG ++ [VG.S])
            (signCell L0.sg (some v) :: (itsCells is ++ [VG.B]))) =
            zAt (bSt (some (bF o || L0.sg == v))) ([VG.B] ++ C ++ [VG.S]
              ++ (P ++ litCells L0 (some v)))
              (itsCells is ++ [VG.B]) := by
          rw [zstep_zAt_R _ _ rfl rfl]; simp [litCells, signCell, bSt]
        rw [bCheck_lit]
        refine (fun h => ?_, fun h => hd.rej (RejR.step (by rw [h1];
          exact ih2 h)))
        have := (hd.trans (Reach.step h1)).trans (ih1 h)
        simp only [List.append_assoc, itsCells, itCells, litCells,
          signCell, List.length_append,
          List.length_cons, List.length_singleton, List.length_map,
            List.cons_append, List.nil_append,
          List.length_nil] at this ⊢
        refine this.mono ?_
        omega

/-! What Phase B accepts. -/

def litA (p : VLit × Bool) : VIt := .lit p.1 (some p.2)

/-- A formula whose every literal carries a bit, as items. -/
def itemsAF : List (List (VLit × Bool)) → List VIt
  | [] => []
  | c :: AF => c.map litA ++ .sep :: itemsAF AF

theorem bCheck_inv : ∀ (its : List VIt) (o : Option Bool), bCheck o its =
    true →
    (o = none → ∃ AF : List (List (VLit × Bool)), its = itemsAF AF ∧ ∀ c ∈
      AF, ∃ p ∈ c, p.1.sg = p.2) ∧
    (∀ f, o = some f → ∃ (c : List (VLit × Bool)) (AF : List (List (VLit ×
      Bool))),
      its = c.map litA ++ .sep :: itemsAF AF ∧ (f = true ∨ ∃ p ∈ c, p.1.sg
        = p.2) ∧
      ∀ c' ∈ AF, ∃ p ∈ c', p.1.sg = p.2) := by
  intro its
  induction its with
  | nil =>
    intro o h
    refine (fun _ => ([], rfl, fun c hc => by simp at hc), fun f hf => ?_)
    subst hf; simp [bCheck] at h
  | cons it is ih =>
    intro o h
    cases it with
    | sep =>
      refine (fun ho => ?_, fun f hf => ?_)
      · subst ho; simp [bCheck] at h
      · subst hf
        simp only [bCheck, Bool.and_eq_true] at h
        obtain (AF, e, hAF) := (ih none h.2).1 rfl
        exact ([], AF, by rw [e]; rfl, Or.inl h.1, hAF)
    | lit L a =>
      cases a with
      | none => cases o <;> simp [bCheck] at h
      | some v =>
        rw [bCheck_lit] at h
        obtain (c, AF, e, hc, hAF) := (ih _ h).2 _ rfl
        refine (fun ho => ?_, fun f hf => ?_)
        · subst ho
          refine (((L, v) :: c) :: AF, by rw [e]; rfl, ?_)
          intro c' hc'
          simp only [List.mem_cons] at hc'
          rcases hc' with hc' | hc'
          · subst hc'
            rcases hc with hc | (p, hp, hp2)
            · exact ((L, v), List.mem_cons.mpr (Or.inl rfl), by simpa
                [bF] using hc)
            · exact (p, List.mem_cons.mpr (Or.inr hp), hp2)
          · exact hAF c' hc'
        · subst hf
          refine ((L, v) :: c, AF, by rw [e]; rfl, ?_, hAF)
          simp only [bF, Bool.or_eq_true, beq_iff_eq] at hc
          rcases hc with (hc | hc) | (p, hp, hp2)
          · exact Or.inl hc
          · exact Or.inr ((L, v), List.mem_cons.mpr (Or.inl rfl), hc)
          · exact Or.inr (p, List.mem_cons.mpr (Or.inr hp), hp2)

theorem bCheck_clause (f : Bool) : ∀ (c : List (VLit × Bool)) (rest :
    List VIt),
    bCheck (some f) (c.map litA ++ .sep :: rest) = ((f || c.any (fun p =>
      p.1.sg == p.2)) && bCheck none rest)
  | [], rest => by simp [bCheck]
  | p :: c, rest => by
    show bCheck (some (f || p.1.sg == p.2)) (c.map litA ++ .sep :: rest)
      = _
    rw [bCheck_clause (f || p.1.sg == p.2) c rest]
    simp [Bool.or_assoc]

theorem bCheck_itemsAF : ∀ AF : List (List (VLit × Bool)), (∀ c ∈ AF, ∃ p
    ∈ c, p.1.sg = p.2) →
    bCheck none (itemsAF AF) = true
  | [], _ => rfl
  | [] :: AF, h => by
    obtain (p, hp, _) := h [] (List.mem_cons.mpr (Or.inl rfl))
    simp at hp
  | (p :: c) :: AF, h => by
    show bCheck (some (p.1.sg == p.2)) (c.map litA ++ .sep :: itemsAF AF)
      = true
    rw [bCheck_clause, bCheck_itemsAF AF (fun c' hc' => h c'
      (List.mem_cons.mpr (Or.inr hc')))]
    obtain (q, hq, hq2) := h (p :: c) (List.mem_cons.mpr (Or.inl rfl))
    simp only [List.mem_cons] at hq
    simp only [Bool.and_true, Bool.or_eq_true, beq_iff_eq,
      List.any_eq_true]
    rcases hq with hq | hq
    · subst hq; exact Or.inl hq2
    · exact Or.inr (q, hq, by simp [hq2])

/-! ### N.8 · Phase C: literals naming one variable carry one bit -/

def markG (b : Bool) : VG := if b then .MO else .MZ

def unmarkG : VG → VG
  | .MZ => .Z
  | .MO => .O
  | x => x

theorem unmark_markG (b : Bool) : unmarkG (markG b) = bitG b := by cases
    b <;> rfl

def notOA : VG → Bool
  | .OA _ _ => false
  | _ => true
def notIA : VG → Bool
  | .IA _ _ => false
  | _ => true
def isZOS : VG → Bool
  | .Z => true | .O => true | .S => true
  | _ => false
def isSA : VG → Bool
  | .S => true | .A _ _ => true
  | _ => false
/-- A cell of the formula part in Phase C outside the two literals
    compared: a digit, a separator, a sign. -/
def isPC : VG → Bool
  | .Z => true | .O => true | .S => true | .A _ _ => true
  | _ => false
def xstop : VG → Bool
  | .MZ => true | .MO => true | .S => true | .A _ _ => true
  | _ => false
def ystop : VG → Bool
  | .MZ => true | .MO => true | .S => true | .A _ _ => true | .OA _ _ =>
    true
  | _ => false
def uystop : VG → Bool
  | .S => true | .A _ _ => true | .OA _ _ => true
  | _ => false
def isMk : VG → Bool
  | .MZ => true | .MO => true
  | _ => false

/-- The last cell of a list satisfies Q. -/
```

```
def lastStop (Q : VG → Bool) (LL : List VG) : Prop := ∃ L0 y, LL = L0 ++ [y] ∧ Q y = true

theorem lastStop_mono {Q Q' : VG → Bool} (hQ : ∀ y, Q y = true → Q' y = true) {LL : List VG}
    (h : lastStop Q LL) : lastStop Q' LL := by
  obtain ⟨L0, y, e, hy⟩ := h
  exact ⟨L0, y, e, hQ y hy⟩

theorem lastStop_mark {Q : VG → Bool} (hQ : ∀ b, Q (markG b) = true) {LL : List VG} (h : lastStop Q LL)
    (ds : List Bool) : lastStop Q (LL ++ ds.map markG) := by
  rcases List.eq_nil_or_concat ds with hd | ⟨ds', d, hd⟩
  · subst hd; simpa using h
  · subst hd; exact ⟨LL ++ ds'.map markG, markG d, by simp, hQ d⟩

theorem lastStop_append {Q : VG → Bool} {LL M : List VG} (h : M = [] ∧ lastStop Q LL ∨ lastStop Q M) :
    lastStop Q (LL ++ M) := by
  rcases h with ⟨hM, hL⟩ | ⟨M0, y, e, hy⟩
  · subst hM; simpa using hL
  · subst e; exact ⟨LL ++ M0, y, by simp, hy⟩

theorem zstep_zAt_L {st s' : VS} {x y : VG} (L : List VG) (c : VG) (R : List VG) (h1 : vhalt st = false)
    (h2 : vδ st x = (s', y, Dir.L)) : zstep (zAt st (L ++ [c]) (x :: R)) = zAt s' L (c :: y :: R) := by
  rw [zAt_cons, zstep_L _ _ h1 h2]; simp [zAt]

/-- A left scan in the head-at-start form: the head on h, Xt to its left, both kept by the state. -/
theorem scanLz (st : VS) (P : VG → Bool) (f : VG → VG) (hst : vhalt st = false)
    (hP : ∀ x, P x = true → vδ st x = (st, f x, Dir.L)) (Xt : List VG) (h : VG) (L0 : List VG) (y : VG)
    (R : List VG) (hh : P h = true) (hX : ∀ x ∈ Xt, P x = true) :
    Reach (zAt st (L0 ++ y :: Xt) (h :: R)) (zAt st L0 (y :: (Xt.map f ++ f h :: R))) (Xt.length + 1) := by
  have := scanL st P f hst hP (h :: Xt.reverse) h (L0 ++ y :: Xt).reverse R y L0.reverse
    (by
      intro x hx
      simp only [List.mem_cons, List.mem_reverse] at hx
      rcases hx with hx | hx
      · rw [hx]; exact hh
      · exact hX x hx)
    (by simp)
  have e : zAt st (L0 ++ y :: Xt) (h :: R) = ⟨st, (L0 ++ y :: Xt).reverse, h, R⟩ := rfl
  rw [e]
  refine Reach.of_steps (k := (h :: Xt.reverse).length) ?_ (by simp)
  rw [this]
  simp [zAt]

/-- Turn left, scan left over PB, stop at the last cell of LL and step back right. -/
theorem turnScan (st0 st st' : VS) (x x' : VG) (P : VG → Bool) (f : VG → VG) (Q : VG → Bool)
    (h0 : vhalt st0 = false) (hx : vδ st0 x = (st, x', Dir.L)) (hst : vhalt st = false)
    (hP : ∀ y, P y = true → vδ st y = (st, f y, Dir.L)) (hQ : ∀ y, Q y = true → vδ st y = (st', y, Dir.R))
    (LL PB R : List VG) (hPB : ∀ y ∈ PB, P y = true) (hLL : lastStop Q LL) :
    Reach (zAt st0 (LL ++ PB) (x :: R)) (zAt st' LL (PB.map f ++ x' :: R)) (PB.length + 2) := by
  obtain ⟨L0, y, e, hy⟩ := hLL
  subst e
  rcases List.eq_nil_or_concat PB with hp | ⟨Xt, h, hp⟩
  · subst hp
    have h1 := Reach.step (zstep_zAt_L L0 y R h0 hx)
    have h2 := Reach.step (zstep_zAt_R L0 (x' :: R) hst (hQ y hy))
    simp only [List.append_nil, List.map_nil, List.nil_append, List.length_nil]
    exact h1.trans h2
  · rw [List.concat_eq_append] at hp
    subst hp
    have hh : P h = true := hPB h (by simp)
    have hX : ∀ z ∈ Xt, P z = true := fun z hz => hPB z (by simp [hz])
    have h1 := Reach.step (zstep_zAt_L (L0 ++ [y] ++ Xt) h R h0 hx)
    have h2 := scanLz st P f hst hP Xt h L0 y (x' :: R) hh hX
    have h3 := Reach.step (zstep_zAt_R L0 (Xt.map f ++ f h :: x' :: R) hst (hQ y hy))
    simp only [List.append_assoc, List.singleton_append, List.cons_append] at h1 h2 h3 ⊢
    have := (h1.trans h2).trans h3
    simp only [List.map_append, List.map_cons, List.map_nil, List.append_assoc, List.singleton_append,
      List.cons_append, List.nil_append] at this ⊢
    refine this.mono ?_
    simp only [List.length_append, List.length_cons, List.length_nil]
    omega

/-- Scan right over X' and c, turn left at t. -/
theorem scanTurn (st st' : VS) (P : VG → Bool) (f : VG → VG) (hst : vhalt st = false)
    (hP : ∀ x, P x = true → vδ st x = (st, f x, Dir.R)) (t t' : VG) (ht : vδ st t = (st', t', Dir.L))
    (L X' : List VG) (c : VG) (R : List VG) (hX : ∀ x ∈ X', P x = true) (hc : P c = true) :
    Reach (zAt st L (X' ++ c :: t :: R)) (zAt st' (L ++ X'.map f) (f c :: t' :: R)) (X'.length + 2) := by
  have h1 := scanR st P f hst hP (X' ++ [c]) L (t :: R) (by
    intro x hx; simp only [List.mem_append, List.mem_singleton] at hx
    rcases hx with hx | hx
    · exact hX x hx
    · rw [hx]; exact hc)
  have h2 := Reach.step (zstep_zAt_L (L ++ X'.map f) (f c) R hst ht)
  simp only [List.append_assoc, List.singleton_append, List.cons_append,
    List.map_append, List.map_cons,
    List.map_nil] at h1 h2 ⊢
  exact ((Reach.steps h1).trans h2).mono (by simp)

/-! Transition facts of Phase C. -/

theorem notOA_hP (st : VS) (hδ : ∀ x, notOA x = true → vδ st x = (st, x, Dir.L)) :
    ∀ x, notOA x = true → vδ st x = (st, id x, Dir.L) := hδ

theorem cGoOuter_pass (v : Bool) (F : VFl) : ∀ x, notOA x = true → vδ (.cGoOuter v F) x = (.cGoOuter v F, id x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [notOA] at hx
theorem cUnGoOuter_pass (v : Bool) : ∀ x, notOA x = true → vδ (.cUnGoOuter v) x = (.cUnGoOuter v, id x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [notOA] at hx
theorem cBack_pass (v : Bool) : ∀ x, notOA x = true → vδ (.cBack v) x = (.cBack v, id x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [notOA] at hx
theorem cGoInner_pass (v : Bool) (F : VFl) (dx : Option Bool) :
    ∀ x, notIA x = true → vδ (.cGoInner v F dx) x = (.cGoInner v F dx, id x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [notIA] at hx
theorem cUnGoInner_pass (v : Bool) : ∀ x, notIA x = true → vδ (.cUnGoInner v) x = (.cUnGoInner v, id x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [notIA] at hx
theorem cXScan_pass (v : Bool) (F : VFl) : ∀ x, isZO x = true → vδ (.cXScan v F) x = (.cXScan v F, id x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [isZO] at hx
theorem cXScan_stop (v : Bool) (F : VFl) : ∀ x, xstop x = true → vδ (.cXScan v F) x = (.cXRead v F, x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [xstop] at hx
theorem cYScan_pass (v : Bool) (F : VFl) (dx : Option Bool) :
    ∀ x, isZO x = true → vδ (.cYScan v F dx) x = (.cYScan v F dx, id x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [isZO] at hx
theorem cYScan_stop (v : Bool) (F : VFl) (dx : Option Bool) :
    ∀ x, ystop x = true → vδ (.cYScan v F dx) x = (.cYRead v F dx, x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [ystop] at hx
theorem cUnYScan_pass (v : Bool) : ∀ x, isMk x = true → vδ (.cUnYScan v) x = (.cUnYScan v, unmarkG x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [isMk] at hx
theorem cUnYScan_stop (v : Bool) : ∀ x, uystop x = true → vδ (.cUnYScan v) x = (.cUnGoOuter v, x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [uystop] at hx
theorem cUnXScan_pass (v : Bool) : ∀ x, isMk x = true → vδ (.cUnXScan v) x = (.cUnXScan v, unmarkG x, Dir.L) := by
  intro x hx; cases x <;> first | rfl | simp [isMk] at hx
theorem cUnXScan_stop (v : Bool) : ∀ x, isSA x = true → vδ (.cUnXScan v) x = (.cUnGoInner v, x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [isSA] at hx
theorem cFind_pass_O : ∀ x, isZOS x = true → vδ .cFindOuter x = (.cFindOuter, id x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [isZOS] at hx
theorem cFind_pass_I (v : Bool) : ∀ x, isZOS x = true → vδ (.cFindInner v) x = (.cFindInner v, id x, Dir.R) := by
  intro x hx; cases x <;> first | rfl | simp [isZOS] at hx

theorem markG_ystop (b : Bool) : ystop (markG b) = true := by cases b <;> rfl
theorem markG_xstop (b : Bool) : xstop (markG b) = true := by cases b <;> rfl
theorem markG_isMk (b : Bool) : isMk (markG b) = true := by cases b <;> rfl
theorem markG_notOA (b : Bool) : notOA (markG b) = true := by cases b <;> rfl
theorem markG_notIA (b : Bool) : notIA (markG b) = true := by cases b <;> rfl
theorem bitG_notOA (b : Bool) : notOA (bitG b) = true := by cases b <;> rfl
theorem bitG_notIA (b : Bool) : notIA (bitG b) = true := by cases b <;> rfl
theorem bitG_isZOS (b : Bool) : isZOS (bitG b) = true := by cases b <;> rfl
theorem isPC_notOA {x : VG} (h : isPC x = true) : notOA x = true := by cases x <;> simp_all [isPC, notOA]
theorem isPC_notIA {x : VG} (h : isPC x = true) : notIA x = true := by cases x <;> simp_all [isPC, notIA]

theorem map_markG_unmark (ds : List Bool) : (ds.map markG).map unmarkG = ds.map bitG := by
  simp [List.map_map, Function.comp_def, unmark_markG]

/-- The digit cells of a literal under comparison: the marked part, then the part still plain. -/
def mkCells (xd xr : List Bool) : List VG := xd.map markG ++ xr.map bitG

/-- From the outer sign to the next plain digit of the outer literal. -/
theorem cHalfX (v : Bool) (F : VFl) (s : Bool) (LC : List VG) (hLC : lastStop isSA LC)
    (xd xr : List Bool) (R1 POST : List VG) (hR1 : ∀ x ∈ R1, notOA x = true)
    (Q1 : List VG) (h : VG) (Q2 : List VG) (e : Q1 ++ h :: Q2 = R1) :
    Reach (zAt (.cGoOuter v F) (LC ++ mkCells xd xr ++ [VG.S, VG.OA s v] ++ Q1) (h :: (Q2 ++ POST)))
      (zAt (.cXRead v F) (LC ++ xd.map markG) (xr.map bitG ++ VG.S :: VG.OA s v :: (R1 ++ POST)))
      (Q1.length + xr.length + 4) := by
  have hQ1 : ∀ x ∈ Q1, notOA x = true := fun x hx => hR1 x (by rw [← e]; simp [hx])
  have hh : notOA h = true := hR1 h (by rw [← e]; simp)
  have h1 := scanLz (.cGoOuter v F) notOA id rfl (cGoOuter_pass v F) Q1 h (LC ++ mkCells xd xr ++ [VG.S])
```

```
    (VG.OA s v) (Q2 ++ POST) hh hQ1
  have h2 := Reach.step (zstep_zAt_L (LC ++ mkCells xd xr) VG.S (Q1.map
     id ++ id h :: (Q2 ++ POST)) rfl
    (show vδ (.cGoOuter v F) (VG.OA s v) = (.cXSep v F, VG.OA s v, Dir.L)
       from rfl))
  have h3 := turnScan (.cXSep v F) (.cXScan v F) (.cXRead v F) VG.S VG.S
     isZO id xstop rfl rfl rfl
    (cXScan_pass v F) (cXScan_stop v F) (LC ++ xd.map markG) (xr.map bitG)
    (VG.OA s v :: (Q1.map id ++ id h :: (Q2 ++ POST)))
    (fun y hy => (mem_map_bitG hy).1)
    (lastStop_mark markG_xstop (lastStop_mono (fun y hy => by cases y <;>
       simp_all [isSA, xstop]) hLC) xd)
  subst e
  simp only [List.map_id, mkCells, List.append_assoc, List.cons_append,
     List.singleton_append,
    List.nil_append, id] at h1 h2 h3 ⊢
  refine ((h1.trans h2).trans h3).mono ?_
  simp only [List.length_map]
  omega

theorem cXRead_digit (v : Bool) (F : VFl) (x0 : Bool) (L R : List VG) :
    zstep (zAt (.cXRead v F) L (bitG x0 :: R)) = zAt (.cGoInner v F (some
       x0)) (L ++ [markG x0]) R :=
  zstep_zAt_R L R rfl (by cases x0 <;> rfl)

theorem cXRead_S (v : Bool) (F : VFl) (L R : List VG) :
    zstep (zAt (.cXRead v F) L (VG.S :: R)) = zAt (.cGoInner v F none) (L
       ++ [VG.S]) R :=
  zstep_zAt_R L R rfl rfl

theorem cYRead_digit (v : Bool) (F : VFl) (dx : Option Bool) (y0 : Bool)
    (L R : List VG) :
    zstep (zAt (.cYRead v F dx) L (bitG y0 :: R)) = zAt (vcomb v F dx
       (some y0)) (L ++ [markG y0]) R :=
  zstep_zAt_R L R rfl (by cases y0 <;> rfl)

theorem cYRead_S (v : Bool) (F : VFl) (dx : Option Bool) (L R : List VG) :
    zstep (zAt (.cYRead v F dx) L (VG.S :: R)) = zAt (vcomb v F dx none)
       (L ++ [VG.S]) R :=
  zstep_zAt_R L R rfl rfl

/-- From the outer literal over to the inner sign, back to the next plain
    digit of the inner literal, read it. -/
theorem cHalfY2 (v : Bool) (F : VFl) (dx : Option Bool) (s' w : Bool) (Lx
    MID : List VG)
    (hLx : lastStop ystop (Lx ++ MID)) (yd yr : List Bool) (POST : List
       VG) (A0 B0 : List VG)
    (hB0 : ∀ x ∈ B0, notIA x = true) (e : A0 ++ B0 = Lx ++ MID ++ yd.map
       markG ++ yr.map bitG ++ [VG.S]) :
    ∃ (Q1' : List VG) (h' : VG) (Q2' : List VG),
      Q1' ++ h' :: Q2' = MID ++ mkCells (yd ++ yr.take 1) yr.tail ++
         [VG.S, VG.IA s' w] ∧
      (yr = [] → Q1' = MID ++ yd.map markG ++ [VG.S] ∧ h' = VG.IA s' w ∧
         Q2' = []) ∧
      Reach (zAt (.cGoInner v F dx) A0 (B0 ++ VG.IA s' w :: POST))
        (zAt (vcomb v F dx yr.head?) (Lx ++ Q1') (h' :: (Q2' ++ POST)))
           (B0.length + yr.length + 4) := by
  have h1 := Reach.steps (scanR (.cGoInner v F dx) notIA id rfl
     (cGoInner_pass v F dx) B0 A0
    (VG.IA s' w :: POST) hB0)
  rw [List.map_id, e] at h1
  have h2 := Reach.step (zstep_zAt_L (Lx ++ MID ++ yd.map markG ++ yr.map
     bitG) VG.S POST rfl
    (show vδ (.cGoInner v F dx) (VG.IA s' w) = (.cYSep v F dx, VG.IA s'
       w, Dir.L) from rfl))
  have hst : lastStop ystop (Lx ++ MID ++ yd.map markG) := lastStop_mark
     markG_ystop hLx yd
  have h3 := turnScan (.cYSep v F dx) (.cYScan v F dx) (.cYRead v F dx)
     VG.S VG.S isZO id ystop rfl rfl rfl
    (cYScan_pass v F dx) (cYScan_stop v F dx) (Lx ++ MID ++ yd.map markG)
       (yr.map bitG) (VG.IA s' w :: POST)
    (fun y hy => (mem_map_bitG hy).1) hst
  have h13 := (h1.trans h2).trans h3
  rw [List.map_id] at h13
  cases yr with
  | nil =>
    refine (MID ++ yd.map markG ++ [VG.S], VG.IA s' w, [], by simp
       [mkCells], fun _ => (rfl, rfl, rfl), ?_)
    have h4 := Reach.step (cYRead_S v F dx (Lx ++ MID ++ yd.map markG)
       (VG.IA s' w :: POST))
    simp only [List.map_nil, List.nil_append] at h13
    have := h13.trans h4
    simp only [List.append_assoc, List.head?_nil, List.nil_append] at
       this ⊢
    refine this.mono ?_
    simp
  | cons y0 yr' =>
    have hM : yr'.map bitG ++ [VG.S, VG.IA s' w] ≠ [] := by simp
    refine (MID ++ (yd ++ [y0]).map markG, (yr'.map bitG ++ [VG.S, VG.IA
       s' w]).headD .B,
      (yr'.map bitG ++ [VG.S, VG.IA s' w]).tail, ?_, fun h => absurd h
         (by simp), ?_)
    · rw [cons_headD_tail _ hM]; simp [mkCells]
    · have h4 := Reach.step (cYRead_digit v F dx y0 (Lx ++ MID ++ yd.map
       markG) (yr'.map bitG ++ VG.S :: VG.IA s' w :: POST))
      simp only [List.map_cons, List.cons_append] at h13
      have := h13.trans h4
      have e5 : (yr'.map bitG ++ [VG.S, VG.IA s' w]).headD .B ::
         ((yr'.map bitG ++ [VG.S, VG.IA s' w]).tail ++ POST) =
          yr'.map bitG ++ VG.S :: VG.IA s' w :: POST := by
        rw [← List.cons_append, cons_headD_tail _ hM]; simp
      rw [e5]
      simp only [List.append_assoc, List.head?_cons, List.map_append,
         List.map_cons, List.map_nil,
        List.singleton_append, List.cons_append, List.nil_append] at this
           ⊢
      refine this.mono ?_
      simp only [List.length_cons, List.length_map]
      omega

theorem mkCells_notIA (xd xr : List Bool) : ∀ x ∈ mkCells xd xr, notIA x
    = true := by
  intro x hx
  simp only [mkCells, List.mem_append, List.mem_map] at hx
  rcases hx with (b, _, hb) | (b, _, hb)
  · rw [← hb]; exact markG_notIA b
  · rw [← hb]; exact bitG_notIA b

theorem mkCells_notOA (xd xr : List Bool) : ∀ x ∈ mkCells xd xr, notOA x
    = true := by
  intro x hx
  simp only [mkCells, List.mem_append, List.mem_map] at hx
  rcases hx with (b, _, hb) | (b, _, hb)
  · rw [← hb]; exact markG_notOA b
  · rw [← hb]; exact bitG_notOA b

theorem mkCells_length (xd xr : List Bool) : (mkCells xd xr).length =
    xd.length + xr.length := by
  simp [mkCells]

theorem lastStop_OA (Lx MID : List VG) (s v : Bool) (hMIDl : MID = [] ∨
    lastStop isSA MID) :
    lastStop ystop (Lx ++ [VG.S, VG.OA s v] ++ MID) := by
  apply lastStop_append
  rcases hMIDl with h | h
  · exact Or.inl (h, (Lx ++ [VG.S], VG.OA s v, by simp, rfl))
  · exact Or.inr (lastStop_mono (fun y hy => by cases y <;> simp_all
      [isSA, ystop]) h)

/-- ONE ROUND OF THE COMPARISON: one digit of each literal read and
    marked, the flags updated. -/
theorem cIter (v s s' w : Bool) (LC MID POST : List VG) (hLC : lastStop
    isSA LC)
    (hMID : ∀ x ∈ MID, isPC x = true) (hMIDl : MID = [] ∨ lastStop isSA
       MID) (xs ys : List Bool) (F : VFl)
    (xd xr yd yr : List Bool) (exs : xd ++ xr = xs) (eys : yd ++ yr = ys)
       (Q1 : List VG) (h : VG)
    (Q2 : List VG) (e : Q1 ++ h :: Q2 = MID ++ mkCells yd yr ++ [VG.S,
       VG.IA s' w]) :
    ∃ (Q1' : List VG) (h' : VG) (Q2' : List VG),
      Q1' ++ h' :: Q2' = MID ++ mkCells (yd ++ yr.take 1) yr.tail ++
         [VG.S, VG.IA s' w] ∧
      (xr = [] → yr = [] → Q1' = MID ++ ys.map markG ++ [VG.S] ∧ h' =
         VG.IA s' w ∧ Q2' = []) ∧
      Reach (zAt (.cGoOuter v F) (LC ++ mkCells xd xr ++ [VG.S, VG.OA s
         v] ++ Q1) (h :: (Q2 ++ POST)))
        (zAt (vcomb v F xr.head? yr.head?) (LC ++ mkCells (xd ++ xr.take
           1) xr.tail ++ [VG.S, VG.OA s v] ++ Q1')
          (h' :: (Q2' ++ POST)))
        (3 * (xs.length + MID.length + ys.length) + 13) := by
  have hR1 : ∀ x ∈ MID ++ mkCells yd yr ++ [VG.S, VG.IA s' w], notOA x =
     true := by
    intro x hx
    simp only [List.mem_append, List.mem_cons, List.mem_singleton] at hx
    rcases hx with (hx | hx) | hx | hx | hx
    · exact isPC_notOA (hMID x hx)
    · exact mkCells_notOA yd yr x hx
    · rw [hx]; rfl
    · rw [hx]; rfl
    · simp at hx
  have hQ1 : Q1.length ≤ MID.length + ys.length + 1 := by
    have := congrArg List.length e
    simp only [List.length_append, List.length_cons, mkCells_length,
       List.length_nil] at this
    rw [← eys]; simp; omega
  have hxr : xr.length ≤ xs.length := by rw [← exs]; simp
  have hyr : yr.length ≤ ys.length := by rw [← eys]; simp
  have hxs : xd.length + xr.length = xs.length := by rw [← exs]; simp
  have hys : yd.length + yr.length = ys.length := by rw [← eys]; simp
  have hX := cHalfX v F s LC hLC xd xr _ POST hR1 Q1 h Q2 e
  have hNotIA : ∀ x ∈ MID ++ mkCells yd yr ++ [VG.S], notIA x = true := by
    intro x hx
    simp only [List.mem_append, List.mem_singleton] at hx
    rcases hx with (hx | hx) | hx
    · exact isPC_notIA (hMID x hx)
    · exact mkCells_notIA yd yr x hx
    · rw [hx]; rfl
  cases xr with
  | nil =>
    have h1 := Reach.step (cXRead_S v F (LC ++ xd.map markG) (VG.OA s v
       :: (MID ++ mkCells yd yr ++ [VG.S, VG.IA s' w] ++ POST)))
    have hB0 : ∀ x ∈ VG.OA s v :: (MID ++ mkCells yd yr ++ [VG.S]), notIA
       x = true := by
      intro x hx
      simp only [List.mem_cons] at hx
      rcases hx with h | hx
      · rw [hx]; rfl
      · exact hNotIA x hx
    obtain (Q1', h', Q2', e', hfin, hY) := cHalfY2 v F none s' w (LC ++
       xd.map markG ++ [VG.S, VG.OA s v]) MID
      (lastStop_OA _ MID s v hMIDl) yd yr POST (LC ++ xd.map markG ++
         [VG.S])
      (VG.OA s v :: (MID ++ mkCells yd yr ++ [VG.S])) hB0 (by simp
         [mkCells])
    refine (Q1', h', Q2', e', fun _ hy => ?_, ?_)
    · obtain (a, b, c) := hfin hy
      refine (by rw [a, ← eys, hy]; simp, b, c)
    · have := (hX.trans h1).trans (by
        simp only [List.append_assoc, List.cons_append,
          List.singleton_append, List.nil_append] at hY ⊢
        exact hY)
      simp only [mkCells, List.map_nil, List.append_nil, List.take_nil,
        List.tail_nil, List.head?_nil,
```

```
        List.append_assoc, List.cons_append, List.singleton_append,
          List.nil_append] at this ⊢
      refine this.mono ?_
      simp only [List.length_cons, List.length_append, List.length_nil,
        mkCells_length, List.length_singleton,
        List.length_map] at *
      omega
  | cons x0 xr' =>
    have h1 := Reach.step (cXRead_digit v F x0 (LC ++ xd.map markG)
      (xr'.map bitG ++ VG.S :: VG.OA s v :: (MID ++ mkCells yd yr ++
        [VG.S, VG.IA s' w] ++ POST)))
    have hB0 : ∀ x ∈ xr'.map bitG ++ VG.S :: VG.OA s v :: (MID ++ mkCells
      yd yr ++ [VG.S]), notIA x = true := by
      intro x hx
      rw [List.mem_append] at hx
      rcases hx with hx | hx
      · simp only [List.mem_map] at hx; obtain (b, _, hb) := hx; rw [←
        hb]; exact bitG_notIA b
      · rw [List.mem_cons, List.mem_cons] at hx
        rcases hx with hx | hx | hx
        · rw [hx]; rfl
        · rw [hx]; rfl
        · exact hNotIA x hx
    obtain (Q1', h', Q2', e', hfin, hY) := cHalfY2 v F (some x0) s' w
      (LC ++ mkCells (xd ++ [x0]) xr' ++ [VG.S, VG.OA s v]) MID
      (lastStop_OA _ MID s v hMIDl) yd yr POST (LC ++ xd.map markG ++
        [markG x0])
      (xr'.map bitG ++ VG.S :: VG.OA s v :: (MID ++ mkCells yd yr ++
        [VG.S])) hB0 (by simp [mkCells])
    refine (Q1', h', Q2', e', fun h => absurd h (by simp), ?_)
    have := (hX.trans h1).trans (by
      simp only [List.append_assoc, List.cons_append,
        List.singleton_append, List.nil_append] at hY ⊢
      exact hY)
    simp only [mkCells, List.map_append, List.map_cons, List.map_nil,
      List.take_succ_cons, List.take_zero,
      List.tail_cons, List.head?_cons, List.append_assoc,
        List.cons_append, List.singleton_append,
      List.nil_append] at this ⊢
    refine this.mono ?_
    simp only [List.length_cons, List.length_append, List.length_nil,
      List.length_map, mkCells_length,
      List.length_singleton] at *
    omega

/-- The comparison flags over the remaining digit pairs, zero-padded. -/
def cmpFl : VFl → List Bool → List Bool → VFl
  | F, [], [] => F
  | F, x :: xs, [] => cmpFl (vupd F x false) xs []
  | F, [], y :: ys => cmpFl (vupd F false y) [] ys
  | F, x :: xs, y :: ys => cmpFl (vupd F x y) xs ys

theorem vcomb_go (v : Bool) (F : VFl) (dx dy : Option Bool) (h : ¬ (dx =
    none ∧ dy = none)) :
    vcomb v F dx dy = .cGoOuter v (vupd F (dx.getD false) (dy.getD
      false)) := by
  cases dx <;> cases dy <;> simp_all [vcomb]

theorem cmpFl_step (F : VFl) (xr yr : List Bool) (h : ¬ (xr = [] ∧ yr =
    [])) :
    cmpFl F xr yr = cmpFl (vupd F (xr.head?.getD false) (yr.head?.getD
      false)) xr.tail yr.tail := by
  cases xr <;> cases yr <;> simp_all [cmpFl]

/-- THE COMPARISON LOOP: from the first round to the result, every digit
    of both literals marked. -/
theorem cLoop (v s s' w : Bool) (LC MID POST : List VG) (hLC : lastStop
    isSA LC)
    (hMID : ∀ x ∈ MID, isPC x = true) (hMIDl : MID = [] ∨ lastStop isSA
      MID) (xs ys : List Bool) :
    ∀ (n : Nat) (F : VFl) (xd xr yd yr : List Bool) (Q1 : List VG) (h :
      VG) (Q2 : List VG),
      xr.length + yr.length ≤ n → xd ++ xr = xs → yd ++ yr = ys →
      Q1 ++ h :: Q2 = MID ++ mkCells yd yr ++ [VG.S, VG.IA s' w] →
      Reach (zAt (.cGoOuter v F) (LC ++ mkCells xd xr ++ [VG.S, VG.OA s
        v] ++ Q1) (h :: (Q2 ++ POST)))
        (zAt (.cResult v (cmpFl F xr yr)) (LC ++ xs.map markG ++ [VG.S,
          VG.OA s v] ++ MID ++ ys.map markG ++ [VG.S])
          (VG.IA s' w :: POST))
        ((xr.length + yr.length + 1) * (3 * (xs.length + MID.length +
          ys.length) + 13)) := by
  intro n
  induction n with
  | zero =>
    intro F xd xr yd yr Q1 h Q2 hn exs eys e
    have hxr : xr = [] := List.eq_nil_of_length_eq_zero (by omega)
    have hyr : yr = [] := List.eq_nil_of_length_eq_zero (by omega)
    subst hxr; subst hyr
    obtain (Q1', h', Q2', _, hfin, hR) := cIter v s s' w LC MID POST hLC
      hMID hMIDl xs ys F xd [] yd [] exs eys Q1 h Q2 e
    obtain (a, b, c) := hfin rfl rfl
    subst a; subst b; subst c
    simp only [List.append_nil] at exs eys
    subst exs; subst eys
    simp only [mkCells, List.map_nil, List.append_nil, List.take_nil,
      List.tail_nil, List.head?_nil, vcomb,
      cmpFl, List.length_nil, Nat.zero_add, Nat.one_mul,
        List.append_assoc, List.cons_append,
      List.singleton_append, List.nil_append] at hR ⊢
    exact hR
  | succ n ih =>
    intro F xd xr yd yr Q1 h Q2 hn exs eys e
    obtain (Q1', h', Q2', e', hfin, hR) := cIter v s s' w LC MID POST hLC
      hMID hMIDl xs ys F xd xr yd yr exs eys Q1 h Q2 e
    by_cases hb : xr = [] ∧ yr = []
    · obtain (hx, hy) := hb
      subst hx; subst hy
      obtain (a, b, c) := hfin rfl rfl
      subst a; subst b; subst c
      simp only [List.append_nil] at exs eys
      subst exs; subst eys
      simp only [mkCells, List.map_nil, List.append_nil, List.take_nil,
        List.tail_nil, List.head?_nil, vcomb,
        cmpFl, List.length_nil, Nat.zero_add, Nat.one_mul,
          List.append_assoc, List.cons_append,
        List.singleton_append, List.nil_append] at hR ⊢
      exact hR
    · have hb' : ¬ (xr.head? = none ∧ yr.head? = none) := by
        intro (h1, h2); exact hb (List.head?_eq_none_iff.mp h1,
          List.head?_eq_none_iff.mp h2)
      rw [vcomb_go v F _ _ hb'] at hR
      rw [cmpFl_step F xr yr hb]
      have hlen : xr.tail.length + yr.tail.length + 1 ≤ xr.length +
        yr.length := by
        cases xr <;> cases yr <;> simp_all <;> omega
      have exs' : (xd ++ xr.take 1) ++ xr.tail = xs := by
        rw [List.append_assoc, ← List.drop_one, List.take_append_drop];
          exact exs
      have eys' : (yd ++ yr.take 1) ++ yr.tail = ys := by
        rw [List.append_assoc, ← List.drop_one, List.take_append_drop];
          exact eys
      have hn' : xr.tail.length + yr.tail.length ≤ n := by omega
      have := hR.trans (ih _ (xd ++ xr.take 1) xr.tail (yd ++ yr.take 1)
        yr.tail Q1' h' Q2' hn' exs' eys' e')
      refine this.mono ?_
      generalize 3 * (xs.length + MID.length + ys.length) + 13 = K
      have : (xr.tail.length + yr.tail.length + 1) * K ≤ (xr.length +
        yr.length) * K := Nat.mul_le_mul_right K hlen
      rw [Nat.succ_mul (xr.length + yr.length)]
      omega

/-! The flags decide whether two literals name one variable. -/

def peqB : List Bool → List Bool → Bool
  | [], [] => true
  | x :: xs, [] => (x == false) && peqB xs []
  | [], y :: ys => (false == y) && peqB [] ys
  | x :: xs, y :: ys => (x == y) && peqB xs ys

def allZB : List Bool → Bool
  | [] => true
  | x :: xs => !x && allZB xs

theorem cmpFl_nil_left : ∀ (ys : List Bool) (e0 er zr : Bool),
    cmpFl (false, e0, er, zr) [] ys = (false, e0, er && peqB [] ys, zr)
  | [], e0, er, zr => by simp [cmpFl, peqB]
  | y :: ys, e0, er, zr => by
    rw [cmpFl]
    simp only [vupd, Bool.false_eq_true, ↓reduceIte, Bool.not_false,
      Bool.and_true]
    rw [cmpFl_nil_left ys]
    simp [peqB, Bool.and_assoc]

theorem cmpFl_rest : ∀ (xs ys : List Bool) (e0 er zr : Bool),
    cmpFl (false, e0, er, zr) xs ys = (false, e0, er && peqB xs ys, zr &&
      allZB xs)
  | [], ys, e0, er, zr => by rw [cmpFl_nil_left]; simp [allZB]
  | x :: xs, [], e0, er, zr => by
    rw [cmpFl]
    simp only [vupd, Bool.false_eq_true, ↓reduceIte]
    rw [cmpFl_rest xs [] e0]
    simp [peqB, allZB, Bool.and_assoc]
  | x :: xs, y :: ys, e0, er, zr => by
    rw [cmpFl]
    simp only [vupd, Bool.false_eq_true, ↓reduceIte]
    rw [cmpFl_rest xs ys e0]
    simp [peqB, allZB, Bool.and_assoc]

theorem peqB_iff : ∀ xs ys : List Bool, peqB xs ys = true ↔ ofBits xs =
    ofBits ys
  | [], [] => by simp [peqB, ofBits]
  | x :: xs, [] => by
    have ih := peqB_iff xs []
    cases x <;> simp [peqB, ofBits, ih] <;> omega
  | [], y :: ys => by
    have ih := peqB_iff [] ys
    cases y <;> simp [peqB, ofBits, ih] <;> omega
  | x :: xs, y :: ys => by
    have ih := peqB_iff xs ys
    cases x <;> cases y <;> simp [peqB, ofBits, ih] <;> omega

theorem allZB_iff : ∀ xs : List Bool, allZB xs = true ↔ ofBits xs = 0
  | [] => by simp [allZB, ofBits]
  | x :: xs => by
    have ih := allZB_iff xs
    cases x <;> simp [allZB, ofBits, ih] <;> omega

/-- THE FLAGS DECIDE THE VARIABLE: after all digit pairs, erest ∧ (e0 ∨
    zrest) holds exactly when the two digit
    strings are read as one variable, the value less one. -/
theorem cmp_decide (x0 y0 : Bool) (xs ys : List Bool) :
    ((cmpFl vF0 (x0 :: xs) (y0 :: ys)).er && ((cmpFl vF0 (x0 :: xs) (y0
      :: ys)).e0 ||
      (cmpFl vF0 (x0 :: xs) (y0 :: ys)).zr)) = true ↔ ofBits (x0 :: xs) -
        1 = ofBits (y0 :: ys) - 1 := by
  have e : cmpFl vF0 (x0 :: xs) (y0 :: ys) = (false, x0 == y0, peqB xs
    ys, allZB xs) := by
    rw [cmpFl]
    simp only [vupd, vF0, ↓reduceIte]
    rw [cmpFl_rest]; simp
  rw [e]
  simp only [Bool.and_eq_true, Bool.or_eq_true, beq_iff_eq, peqB_iff,
    allZB_iff, ofBits]
```

```
  cases x0 <;> cases y0 <;> simp <;> omega

/-! What Phase C reads: the variable and the bit of each literal, in order. -/

def allAnn : List VIt → Prop
  | [] => True
  | .sep :: is => allAnn is
  | .lit _ a :: is => a ≠ none ∧ allAnn is

def pairsOf : List VIt → List (Nat × Bool)
  | [] => []
  | .sep :: is => pairsOf is
  | .lit L (some v) :: is => (vvar L, v) :: pairsOf is
  | .lit _ none :: is => pairsOf is

/-- CONSISTENCY: any two literals naming one variable carry one bit. -/
def vcons : List (Nat × Bool) → Prop
  | [] => True
  | p :: ps => (∀ q ∈ ps, q.1 = p.1 → q.2 = p.2) ∧ vcons ps

theorem itsCells_isPC : ∀ (its : List VIt), allAnn its → ∀ x ∈ itsCells its, isPC x = true
  | [], _, _, hx => by simp [itsCells] at hx
  | .sep :: is, ha, x, hx => by
    simp only [itsCells, itCells, List.mem_append, List.mem_singleton] at hx
    rcases hx with hx | hx
    · rw [hx]; rfl
    · exact itsCells_isPC is ha x hx
  | .lit L a :: is, ha, x, hx => by
    obtain (ha1, ha2) := ha
    simp only [itsCells, itCells, litCells, List.mem_append, List.mem_cons, List.mem_singleton] at hx
    rcases hx with (hx | hx | hx | hx) | hx
    · simp only [List.mem_map] at hx; obtain (b, _, hb) := hx; rw [← hb]; cases b <;> rfl
    · rw [hx]; rfl
    · rw [hx]; cases a with
      | none => exact absurd rfl ha1
      | some v => rfl
    · simp at hx
    · exact itsCells_isPC is ha2 x hx

theorem itsCells_last : ∀ (its : List VIt), allAnn its → itsCells its = [] ∨ lastStop isSA (itsCells its)
  | [], _ => Or.inl rfl
  | .it :: is, ha => by
    have hit : allAnn [.it] ∧ allAnn is := by
      cases it with
      | sep => exact ⟨trivial, ha⟩
      | lit L a => exact ⟨(ha.1, trivial), ha.2⟩
    have h1 : lastStop isSA (itCells it) := by
      cases it with
      | sep => exact ([], VG.S, rfl, rfl)
      | lit L a =>
        cases a with
        | none => exact absurd rfl hit.1.1
        | some v => exact ((vdigits L).map bitG ++ [VG.S], VG.A L.sg v,
          by simp [itCells, litCells, signCell], rfl)
    right
    show lastStop isSA (itCells it ++ itsCells is)
    apply lastStop_append
    rcases itsCells_last is hit.2 with h | h
    · exact Or.inl ⟨h, h1⟩
    · exact Or.inr h

theorem allAnn_append (a b : List VIt) (ha : allAnn a) (hb : allAnn b) : allAnn (a ++ b) := by
  induction a with
  | nil => exact hb
  | cons i a ih =>
    cases i with
    | sep => exact ih ha
    | lit L x => exact ⟨ha.1, ih ha.2⟩

theorem cResult_rej (v : Bool) (F : VFl) (s' w : Bool) (h : (F.er && (F.e0 || F.zr) && (v != w)) = true) :
    vδ (.cResult v F) (.IA s' w) = (.rej, .IA s' w, .R) := by
  simp [vδ, h]

theorem cResult_ok (v : Bool) (F : VFl) (s' w : Bool) (h : (F.er && (F.e0 || F.zr) && (v != w)) = false) :
    vδ (.cResult v F) (.IA s' w) = (.cUnYSep v, .IA s' w, .L) := by
  simp [vδ, h]

theorem conflict_iff (X Y : VLit) (v w : Bool) :
    ((cmpFl vF0 (vdigits X) (vdigits Y)).er && ((cmpFl vF0 (vdigits X) (vdigits Y)).e0 ||
      (cmpFl vF0 (vdigits X) (vdigits Y)).zr) && (v != w)) = true ↔ (vvar X = vvar Y ∧ v ≠ w) := by
  have h := cmp_decide X.d0 Y.d0 X.ds Y.ds
  simp only [vdigits, vvar] at h ⊢
  rw [Bool.and_eq_true, h]
  simp

/-- From an inner sign found, through the comparison, to the verdict at the inner sign. -/
theorem cPairA (v : Bool) (LC : List VG) (hLC : lastStop isSA LC) (X : VLit) (MID : List VG)
    (hMID : ∀ x ∈ MID, isPC x = true) (hMIDl : MID = [] ∨ lastStop isSA MID) (Y : VLit) (w : Bool)
    (POST : List VG) :
    Reach (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ MID)
      (litCells Y (some w) ++ POST))
      (zAt (.cResult v (cmpFl vF0 (vdigits X) (vdigits Y)))
        (LC ++ (vdigits X).map markG ++ [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits Y).map markG ++ [VG.S])
        (VG.IA Y.sg w :: POST))
      ((vdigits Y).length + 2 +
        ((vdigits X).length + (vdigits Y).length + 1) * (3 * ((vdigits X).length + MID.length + (vdigits Y).length) + 13)) := by
  have hZ : ∀ x ∈ (vdigits Y).map bitG ++ [VG.S], isZOS x = true := by
    intro x hx
    simp only [List.mem_append, List.mem_singleton] at hx
    rcases hx with hx | hx
    · simp only [List.mem_map] at hx; obtain (b, _, hb) := hx; rw [← hb]; exact bitG_isZOS b
    · rw [hx]; rfl
  have h1 := Reach.steps (scanR (.cFindInner v) isZOS id rfl (cFind_pass_I v) ((vdigits Y).map bitG ++ [VG.S])
    (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ MID) (VG.A Y.sg w :: POST) hZ)
  have h2 := Reach.step (zstep_zAt_L (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits Y).map bitG)
    VG.S POST rfl (show vδ (.cFindInner v) (VG.A Y.sg w) = (.cGoOuter v vF0, VG.IA Y.sg w, Dir.L) from rfl))
  have h3 := cLoop v X.sg Y.sg w LC MID POST hLC hMID hMIDl (vdigits X) (vdigits Y)
    ((vdigits X).length + (vdigits Y).length) vF0 [] (vdigits X) [] (vdigits Y) (MID ++ (vdigits Y).map bitG) VG.S
    [VG.IA Y.sg w] (Nat.le_refl _) rfl rfl (by simp [mkCells])
  rw [List.map_id] at h1
  simp only [mkCells, List.map_nil, List.nil_append, litCells, signCell, List.append_assoc, List.cons_append,
    List.singleton_append] at h1 h2 h3 ⊢
  refine ((h1.trans h2).trans h3).mono ?_
  simp only [List.length_append, List.length_map, List.length_singleton]
  omega

theorem allP_app {P : VG → Bool} {A B : List VG} (hA : ∀ x ∈ A, P x = true) (hB : ∀ x ∈ B, P x = true) :
    ∀ x ∈ A ++ B, P x = true := by
  intro x hx
  rcases List.mem_append.mp hx with h | h
  · exact hA x h
  · exact hB x h

theorem allP_cons {P : VG → Bool} {a : VG} {B : List VG} (ha : P a = true) (hB : ∀ x ∈ B, P x = true) :
    ∀ x ∈ a :: B, P x = true := by
  intro x hx
  rcases List.mem_cons.mp hx with h | h
  · rw [h]; exact ha
  · exact hB x h

theorem allP_nil {P : VG → Bool} : ∀ x ∈ ([] : List VG), P x = true := by
  intro x hx; simp at hx

theorem allP_bitG {P : VG → Bool} (hP : ∀ b, P (bitG b) = true) (ds : List Bool) : ∀ x ∈ ds.map bitG, P x = true := by
  intro x hx
  simp only [List.mem_map] at hx
  obtain (b, _, hb) := hx
  rw [← hb]; exact hP b

theorem allP_markG {P : VG → Bool} (hP : ∀ b, P (markG b) = true) (ds : List Bool) : ∀ x ∈ ds.map markG, P x = true := by
  intro x hx
  simp only [List.mem_map] at hx
  obtain (b, _, hb) := hx
  rw [← hb]; exact hP b

theorem lastStop_uy (LC : List VG) (xs : List Bool) (s v : Bool) (MID : List VG)
    (hMIDl : MID = [] ∨ lastStop isSA MID) :
    lastStop uystop (LC ++ xs.map markG ++ [VG.S, VG.OA s v] ++ MID) := by
  apply lastStop_append
  rcases hMIDl with h | h
  · exact Or.inl (h, (LC ++ xs.map markG ++ [VG.S], VG.OA s v, by simp, rfl))
  · exact Or.inr (lastStop_mono (fun y hy => by cases y <;> simp_all [isSA, uystop]) h)

/-- No conflict: both literals unmarked, the inner sign restored, the inner search goes on after it. -/
theorem cPairB (v : Bool) (LC : List VG) (hLC : lastStop isSA LC) (X : VLit) (MID : List VG)
    (hMID : ∀ x ∈ MID, isPC x = true) (hMIDl : MID = [] ∨ lastStop isSA MID) (Y : VLit) (w : Bool)
    (POST : List VG) (F : VFl) (hF : (F.er && (F.e0 || F.zr) && (v != w)) = false) :
    Reach (zAt (.cResult v F)
        (LC ++ (vdigits X).map markG ++ [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits Y).map markG ++ [VG.S])
        (VG.IA Y.sg w :: POST))
      (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ (MID ++ litCells Y (some w))) POST)
      (2 * (vdigits X).length + 2 * MID.length + 2 * (vdigits Y).length + 11) := by
  have h4 := Reach.step (zstep_zAt_L (LC ++ (vdigits X).map markG ++ [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits Y).map markG)
    VG.S POST rfl (cResult_ok v F Y.sg w hF))
  have h5 := turnScan (.cUnYSep v) (.cUnYScan v) (.cUnGoOuter v) VG.S VG.S isMk unmarkG uystop rfl rfl rfl
    (cUnYScan_pass v) (cUnYScan_stop v) (LC ++ (vdigits X).map markG ++ [VG.S, VG.OA X.sg v] ++ MID)
    ((vdigits Y).map markG) (VG.IA Y.sg w :: POST)
    (allP_markG markG_isMk _) (lastStop_uy LC (vdigits X) X.sg v MID hMIDl)
  rw [map_markG_unmark] at h5
```

```
  have hMO : ∀ x ∈ MID, notOA x = true := fun x hx => isPC_notOA (hMID x
    hx)
  have h6 := scanLz (.cUnGoOuter v) notOA id rfl (cUnGoOuter_pass v) MID
    (bitG Y.d0)
    (LC ++ (vdigits X).map markG ++ [VG.S]) (VG.OA X.sg v) (Y.ds.map bitG
      ++ VG.S :: VG.IA Y.sg w :: POST)
    (bitG_notOA Y.d0) hMO
  have h7 := Reach.step (zstep_zAt_L (LC ++ (vdigits X).map markG) VG.S
    (MID.map id ++ id (bitG Y.d0) :: (Y.ds.map bitG ++ VG.S :: VG.IA Y.sg
      w :: POST)) rfl
    (show vδ (.cUnGoOuter v) (VG.OA X.sg v) = (.cUnXSep v, VG.OA X.sg v,
      Dir.L) from rfl))
  have h8 := turnScan (.cUnXSep v) (.cUnXScan v) (.cUnGoInner v) VG.S
    VG.S isMk unmarkG isSA rfl rfl rfl
    (cUnXScan_pass v) (cUnXScan_stop v) LC ((vdigits X).map markG)
    (VG.OA X.sg v :: (MID.map id ++ id (bitG Y.d0) :: (Y.ds.map bitG ++
      VG.S :: VG.IA Y.sg w :: POST)))
    (allP_markG markG_isMk _) hLC
  rw [map_markG_unmark] at h8
  have hNI : ∀ x ∈ (vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ MID ++
    (vdigits Y).map bitG ++ [VG.S],
      notIA x = true :=
    allP_app (allP_app (allP_app (allP_app (allP_bitG bitG_notIA _)
      (allP_cons rfl (allP_cons rfl allP_nil)))
      (fun x hx => isPC_notIA (hMID x hx))) (allP_bitG bitG_notIA _))
        (allP_cons rfl allP_nil)
  have h9 := Reach.steps (scanR (.cUnGoInner v) notIA id rfl
    (cUnGoInner_pass v)
    ((vdigits X).map bitG ++ [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits
      Y).map bitG ++ [VG.S]) LC
    (VG.IA Y.sg w :: POST) hNI)
  have h10 := Reach.step (zstep_zAt_R (LC ++ ((vdigits X).map bitG ++
    [VG.S, VG.OA X.sg v] ++ MID ++ (vdigits Y).map bitG ++ [VG.S]))
    POST rfl (show vδ (.cUnGoInner v) (VG.IA Y.sg w) = (.cFindInner v,
      VG.A Y.sg w, Dir.R) from rfl))
  rw [List.map_id] at h9
  simp only [vdigits, List.map_cons, List.map_id, id, litCells, signCell,
    List.append_assoc, List.cons_append,
    List.singleton_append, List.nil_append] at h4 h5 h6 h7 h8 h9 h10 ⊢
  refine ((((((h4.trans h5).trans h6).trans h7).trans h8).trans h9).trans
    h10).mono ?_
  simp only [List.length_append, List.length_map, List.length_cons,
    List.length_singleton, List.length_nil]
  omega

/-- ONE PAIR: the outer literal against one inner literal; a conflict
    rejects, otherwise the search goes on. -/
theorem cPair (v : Bool) (LC : List VG) (hLC : lastStop isSA LC) (X :
    VLit) (MID : List VG)
    (hMID : ∀ x ∈ MID, isPC x = true) (hMIDl : MID = [] ∨ lastStop isSA
      MID) (Y : VLit) (w : Bool)
    (POST : List VG) :
    (¬ (vvar X = vvar Y ∧ v ≠ w) →
      Reach (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S,
        VG.OA X.sg v] ++ MID)
          (litCells Y (some w) ++ POST))
        (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA
          X.sg v] ++ (MID ++ litCells Y (some w))) POST)
        (((vdigits X).length + (vdigits Y).length + 2) *
          (3 * ((vdigits X).length + MID.length + (vdigits Y).length) +
            13))) ∧
    ((vvar X = vvar Y ∧ v ≠ w) →
      RejR (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S,
        VG.OA X.sg v] ++ MID)
        (litCells Y (some w) ++ POST))) := by
  have hA := cPairA v LC hLC X MID hMID hMIDl Y w POST
  constructor
  · intro hc
    have hF : ((cmpFl vF0 (vdigits X) (vdigits Y)).er && ((cmpFl vF0
      (vdigits X) (vdigits Y)).e0 ||
        (cmpFl vF0 (vdigits X) (vdigits Y)).zr) && (v != w)) = false := by
      cases hd : ((cmpFl vF0 (vdigits X) (vdigits Y)).er && ((cmpFl vF0
        (vdigits X) (vdigits Y)).e0 ||
        (cmpFl vF0 (vdigits X) (vdigits Y)).zr) && (v != w))
      · rfl
      · exact absurd ((conflict_iff X Y v w).mp hd) hc
    have hB := cPairB v LC hLC X MID hMID hMIDl Y w POST _ hF
    refine (hA.trans hB).mono ?_
    generalize hK : 3 * ((vdigits X).length + MID.length + (vdigits
      Y).length) + 13 = K
    rw [Nat.succ_mul ((vdigits X).length + (vdigits Y).length + 1)]
    omega
  · intro hc
    have hd := (conflict_iff X Y v w).mpr hc
    refine hA.rej (RejR.step ?_)
    rw [zstep_zAt_R _ _ rfl (cResult_rej v _ Y.sg w hd)]
    exact RejR.of_st rfl

theorem itsCells_append_one (mid : List VIt) (it : VIt) : itsCells (mid
    ++ [it]) = itsCells mid ++ itCells it := by
  rw [itsCells_append]; simp [itsCells]

/-- THE INNER LOOP: the outer literal against every literal after it. -/
theorem cInner (v : Bool) (LC : List VG) (hLC : lastStop isSA LC) (X :
    VLit) (T : Nat) :
    ∀ (rest mid : List VIt), allAnn rest → allAnn mid →
      LC.length + (vdigits X).length + 2 + (itsCells mid).length +
        (itsCells rest).length ≤ T →
      ((∀ p ∈ pairsOf rest, p.1 = vvar X → p.2 = v) →
        Reach (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S,
          VG.OA X.sg v] ++ itsCells mid)
            (itsCells rest ++ [VG.B]))
          (zAt .cFindOuter (LC ++ litCells X (some v)) (itsCells mid ++
            itsCells rest ++ [VG.B]))
          (((itsCells rest).length + 1) * ((T + 2) * (3 * T + 13) + T +
            3))) ∧
      (¬ (∀ p ∈ pairsOf rest, p.1 = vvar X → p.2 = v) →
        RejR (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S,
          VG.OA X.sg v] ++ itsCells mid)
          (itsCells rest ++ [VG.B]))) := by
intro rest
induction rest with
| nil =>
  intro mid _ hmid hT
  refine (fun _ => ?_, fun h => absurd (fun p hp => by simp [pairsOf]
    at hp) h)
  have hMO : ∀ x ∈ itsCells mid, notOA x = true := fun x hx =>
    isPC_notOA (itsCells_isPC mid hmid x hx)
  rcases List.eq_nil_or_concat (itsCells mid) with hm | (M', m, hm)
  · rw [hm]
    have h1 := Reach.step (zstep_zAt_L (LC ++ (vdigits X).map bitG ++
      [VG.S]) (VG.OA X.sg v) [] rfl
      (show vδ (.cFindInner v) .B = (.cBack v, .B, Dir.L) from rfl))
    have h2 := Reach.step (zstep_zAt_R (LC ++ (vdigits X).map bitG ++
      [VG.S]) [VG.B] rfl
      (show vδ (.cBack v) (VG.OA X.sg v) = (.cFindOuter, VG.A X.sg v,
        Dir.R) from rfl))
    simp only [itsCells, List.append_nil, List.nil_append, litCells,
      signCell, List.append_assoc,
      List.cons_append, List.singleton_append] at h1 h2 ⊢
    refine (h1.trans h2).mono ?_
    simp only [List.length_nil, Nat.zero_add, Nat.one_mul]
    omega
  · rw [List.concat_eq_append] at hm
    rw [hm]
    rw [hm] at hMO
    have h1 := Reach.step (zstep_zAt_L (LC ++ (vdigits X).map bitG ++
      [VG.S, VG.OA X.sg v] ++ M') m [] rfl
      (show vδ (.cFindInner v) .B = (.cBack v, .B, Dir.L) from rfl))
    have h2 := scanLz (.cBack v) notOA id rfl (cBack_pass v) M' m (LC
      ++ (vdigits X).map bitG ++ [VG.S])
      (VG.OA X.sg v) [VG.B] (hMO m (by simp)) (fun x hx => hMO x (by
        simp [hx]))
    have h3 := Reach.step (zstep_zAt_R (LC ++ (vdigits X).map bitG ++
      [VG.S]) (M'.map id ++ id m :: [VG.B]) rfl
      (show vδ (.cBack v) (VG.OA X.sg v) = (.cFindOuter, VG.A X.sg v,
        Dir.R) from rfl))
    simp only [itsCells, List.append_nil, List.nil_append, litCells,
      signCell, List.append_assoc,
      List.cons_append, List.singleton_append, List.map_id, id] at h1
        h2 h3 ⊢
    refine ((h1.trans h2).trans h3).mono ?_
    have := congrArg List.length hm
    simp only [List.length_append, List.length_singleton] at this
    simp only [List.length_nil, Nat.zero_add, Nat.one_mul]
    omega
| cons it rest ih =>
  intro mid hrest hmid hT
  have hIB : 1 ≤ (T + 2) * (3 * T + 13) + T + 3 := by omega
  cases it with
  | sep =>
    have hmid' : allAnn (mid ++ [VIt.sep]) := allAnn_append _ _ hmid
      trivial
    have hT' : LC.length + (vdigits X).length + 2 + (itsCells (mid ++
      [VIt.sep])).length + (itsCells rest).length ≤ T := by
      rw [itsCells_append_one]; simp [itsCells, itCells] at hT ⊢; omega
    obtain (ih1, ih2) := ih (mid ++ [VIt.sep]) hrest hmid' hT'
    rw [itsCells_append_one] at ih1 ih2
    have h1 : zstep (zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++
      [VG.S, VG.OA X.sg v] ++ itsCells mid)
        (itsCells (.sep :: rest) ++ [VG.B])) =
        zAt (.cFindInner v) (LC ++ (vdigits X).map bitG ++ [VG.S, VG.OA
          X.sg v] ++ (itsCells mid ++ itCells .sep))
          (itsCells rest ++ [VG.B]) := by
      simp only [itsCells, itCells, List.cons_append, List.nil_append]
      rw [zstep_zAt_R _ _ rfl rfl]; simp
    refine (fun hc => ?_, fun hc => RejR.step (by rw [h1]; exact ih2 hc))
    have := (Reach.step h1).trans (ih1 hc)
    simp only [itCells, itsCells, List.append_assoc,
      List.singleton_append, List.cons_append,
      List.nil_append] at this ⊢
    refine this.mono ?_
    simp only [List.length_cons, List.length_append,
      List.length_singleton]
    generalize (T + 2) * (3 * T + 13) + T + 3 = K at *
    rw [Nat.succ_mul ((itsCells rest).length + 1)]
    omega
  | lit Y a =>
    obtain (ha, hrest') := hrest
    cases a with
    | none => exact absurd rfl ha
    | some w =>
      have hMID := itsCells_isPC mid hmid
      have hMIDl := itsCells_last mid hmid
      obtain (hp1, hp2) := cPair v LC hLC X (itsCells mid) hMID hMIDl Y
        w (itsCells rest ++ [VG.B])
      have e0 : itsCells (.lit Y (some w) :: rest) ++ [VG.B] = litCells
        Y (some w) ++ (itsCells rest ++ [VG.B]) := by
        simp [itsCells, itCells]
      rw [e0]
      by_cases hc : vvar X = vvar Y ∧ v ≠ w
      · refine (fun h => absurd (h (vvar Y, w) (by simp [pairsOf])
          hc.1.symm) (fun e => hc.2 e.symm), fun _ => hp2 hc)
      · have hmid' : allAnn (mid ++ [VIt.lit Y (some w)]) :=
          allAnn_append _ _ hmid (by simp, trivial)
        have hT' : LC.length + (vdigits X).length + 2 + (itsCells (mid
          ++ [VIt.lit Y (some w)])).length +
            (itsCells rest).length ≤ T := by
          rw [itsCells_append_one]; simp [itsCells, itCells] at hT ⊢;
            omega
        obtain (ih1, ih2) := ih (mid ++ [VIt.lit Y (some w)]) hrest'
          hmid' hT'
```

```
          rw [itsCells_append_one] at ih1 ih2
          have hiff : (∀ p ∈ pairsOf (.lit Y (some w) :: rest), p.1 = vvar
             X → p.2 = v) ↔
               (∀ p ∈ pairsOf rest, p.1 = vvar X → p.2 = v) := by
            simp only [pairsOf, List.mem_cons, forall_eq_or_imp]
            constructor
            · exact fun h => h.2
            · intro h
              refine (fun e => ?_, h)
              exact Classical.byContradiction fun hne => hc (e.symm, fun
                 e2 => hne e2.symm)
          rw [hiff]
          have hlen0 : itsCells (.lit Y (some w) :: rest) = litCells Y
             (some w) ++ itsCells rest := rfl
          have hP := hp1 hc
          refine (fun h => ?_, fun h => hP.rej (ih2 h))
          have := hP.trans (ih1 h)
          rw [hlen0]
          simp only [itCells, List.append_assoc] at this ⊢
          refine this.mono ?_
          have hxs : (vdigits X).length + (vdigits Y).length ≤ T := by
            simp [itsCells, itCells, litCells] at hT; omega
          have hxm : (vdigits X).length + (itsCells mid).length +
             (vdigits Y).length ≤ T := by
            simp [itsCells, itCells, litCells] at hT; omega
          have hpair : ((vdigits X).length + (vdigits Y).length + 2) *
              (3 * ((vdigits X).length + (itsCells mid).length + (vdigits
                 Y).length) + 13) ≤ (T + 2) * (3 * T + 13) :=
            Nat.mul_le_mul (by omega) (by omega)
          have hlen : (litCells Y (some w) ++ itsCells rest).length =
             (vdigits Y).length + 2 + (itsCells rest).length := by
            simp [litCells]; omega
          rw [hlen]
          generalize (T + 2) * (3 * T + 13) = P at *
          generalize hK : P + T + 3 = K
          have : ((itsCells rest).length + 1) * K + K ≤ ((vdigits
             Y).length + 2 + (itsCells rest).length + 1) * K := by
            rw [← Nat.succ_mul]; exact Nat.mul_le_mul_right K (by omega)
          omega

/-- THE OUTER LOOP OF PHASE C: every literal against every later one;
    acceptance exactly on consistency. -/
theorem cOuter (T : Nat) : ∀ (its : List VIt) (LC : List VG), lastStop
    isSA LC → allAnn its →
    LC.length + (itsCells its).length ≤ T →
    (vcons (pairsOf its) → AccR (zAt .cFindOuter LC (itsCells its ++
       [VG.B]))
        (((itsCells its).length + 1) * (T + 2 + (T + 1) * ((T + 2) * (3 *
           T + 13) + T + 3)))) ∧
    (¬ vcons (pairsOf its) → RejR (zAt .cFindOuter LC (itsCells its ++
       [VG.B]))) := by
  intro its
  induction its with
  | nil =>
    intro LC _ _ _
    refine (fun _ => (1, ?_, rfl), fun h => absurd trivial h)
    simp only [itsCells, List.length_nil, Nat.zero_add, Nat.one_mul]
    omega
  | cons it is ih =>
    intro LC hLC hits hT
    cases it with
    | sep =>
      have h1 : zstep (zAt .cFindOuter LC (itsCells (.sep :: is) ++
         [VG.B])) =
          zAt .cFindOuter (LC ++ [VG.S]) (itsCells is ++ [VG.B]) := by
        simp only [itsCells, itCells, List.cons_append, List.nil_append]
        exact zstep_zAt_R _ _ rfl rfl
      have hT' : (LC ++ [VG.S]).length + (itsCells is).length ≤ T := by
        simp [itsCells, itCells] at hT ⊢; omega
      obtain (ih1, ih2) := ih (LC ++ [VG.S]) (LC, VG.S, rfl, rfl) hits hT'
      refine (fun hc => ?_, fun hc => RejR.step (by rw [h1]; exact ih2 hc))
      have := (Reach.step h1).acc (ih1 hc)
      refine this.mono ?_
      simp only [itsCells, itCells, List.length_append,
        List.length_singleton]
      have hK1 : 1 ≤ T + 2 + (T + 1) * ((T + 2) * (3 * T + 13) + T + 3) :=
         by omega
      generalize T + 2 + (T + 1) * ((T + 2) * (3 * T + 13) + T + 3) = K at
         hK1 ⊢
      rw [show 1 + (itsCells is).length + 1 = ((itsCells is).length + 1)
         + 1 by omega,
        Nat.succ_mul ((itsCells is).length + 1)]
      omega
    | lit X a =>
      obtain (ha, hits') := hits
      cases a with
      | none => exact absurd rfl ha
      | some v =>
        have hZ : ∀ x ∈ (vdigits X).map bitG ++ [VG.S], isZOS x = true :=
          allP_app (allP_bitG bitG_isZOS _) (allP_cons rfl allP_nil)
        have h1 := Reach.steps (scanR .cFindOuter isZOS id rfl
           cFind_pass_O ((vdigits X).map bitG ++ [VG.S]) LC
          (VG.A X.sg v :: (itsCells is ++ [VG.B])) hZ)
        have h2 := Reach.step (zstep_zAt_R (LC ++ ((vdigits X).map bitG
           ++ [VG.S])) (itsCells is ++ [VG.B]) rfl
          (show vδ .cFindOuter (VG.A X.sg v) = (.cFindInner v, VG.OA X.sg
             v, Dir.R) from rfl))
        rw [List.map_id] at h1
        have hTi : LC.length + (vdigits X).length + 2 + (itsCells
           []).length + (itsCells is).length ≤ T := by
          simp [itsCells, itCells, litCells] at hT ⊢; omega
        obtain (hi1, hi2) := cInner v LC hLC X T is [] hits' trivial hTi
        have e0 : itsCells (.lit X (some v) :: is) ++ [VG.B] =
            ((vdigits X).map bitG ++ [VG.S]) ++ VG.A X.sg v :: (itsCells
               is ++ [VG.B]) := by
          simp [itsCells, itCells, litCells, signCell]
        have h12 := h1.trans h2
        simp only [itsCells, List.append_nil, List.append_assoc,
           List.singleton_append, List.cons_append,
          List.nil_append] at h12 hi1 hi2
        rw [e0]
        have hLC' : lastStop isSA (LC ++ litCells X (some v)) :=
          (LC ++ (vdigits X).map bitG ++ [VG.S], VG.A X.sg v, by simp
             [litCells, signCell], rfl)
        have hT' : (LC ++ litCells X (some v)).length + (itsCells
           is).length ≤ T := by
          simp [itsCells, itCells, litCells] at hT ⊢; omega
        obtain (ih1, ih2) := ih (LC ++ litCells X (some v)) hLC' hits' hT'
        show (((∀ q ∈ pairsOf is, q.1 = vvar X → q.2 = v) ∧ vcons (pairsOf
           is)) → _) ∧
          (¬ ((∀ q ∈ pairsOf is, q.1 = vvar X → q.2 = v) ∧ vcons (pairsOf
             is)) → _)
        simp only [List.append_assoc, List.singleton_append,
           List.cons_append, List.nil_append]
        constructor
        · intro (hc1, hc2)
          have := (h12.trans (hi1 hc1)).acc (ih1 hc2)
          refine this.mono ?_
          have hxs : (vdigits X).length ≤ T := by simp [itsCells,
             itCells, litCells] at hT; omega
          have hisl : (itsCells is).length ≤ T := by simp [itsCells,
             itCells, litCells] at hT; omega
          simp only [itsCells, itCells, litCells, List.length_append,
             List.length_map, List.length_cons,
            List.length_singleton, List.length_nil]
          generalize hIB : (T + 2) * (3 * T + 13) + T + 3 = IB
          generalize hOB : T + 2 + (T + 1) * IB = OB
          have a1 : ((itsCells is).length + 1) * IB ≤ (T + 1) * IB :=
             Nat.mul_le_mul_right IB (by omega)
          have a2 : OB ≤ ((vdigits X).length + 1) * OB + OB := by omega
          have a3 : ((vdigits X).length + (0 + 1 + 1) + (itsCells
             is).length + 1) * OB =
              ((itsCells is).length + 1) * OB + ((vdigits X).length + 1) *
                 OB + OB := by
            rw [show (vdigits X).length + (0 + 1 + 1) + (itsCells
               is).length + 1 =
              ((itsCells is).length + 1) + ((vdigits X).length + 1) + 1 by
                 omega,
              Nat.add_mul, Nat.add_mul, Nat.one_mul]
          rw [a3]
          omega
        · intro hc
          by_cases hc1 : ∀ q ∈ pairsOf is, q.1 = vvar X → q.2 = v
          · have hc2 : ¬ vcons (pairsOf is) := fun h => hc (hc1, h)
            exact (h12.trans (hi1 hc1)).rej (ih2 hc2)
          · exact h12.rej (hi2 hc1)

/-! ### N.9 · From the three checks to satisfiability, and back -/

def eraseIts : List VIt → List VIt
  | [] => []
  | .sep :: is => .sep :: eraseIts is
  | .lit L _ :: is => .lit L none :: eraseIts is

theorem eraseIts_append (a b : List VIt) : eraseIts (a ++ b) = eraseIts a
    ++ eraseIts b := by
  induction a with
  | nil => rfl
  | cons i a ih => cases i <;> simp [eraseIts, ih]

theorem eraseIts_annIts : ∀ (its : List VIt) (us : List Bool), eraseIts
    (annIts its us) = eraseIts its
  | [], _ => rfl
  | .sep :: is, us => by simp [annIts, eraseIts, eraseIts_annIts is us]
  | .lit L a :: is, [] => rfl
  | .lit L a :: is, u :: us => by simp [annIts, eraseIts, eraseIts_annIts
     is us]

theorem eraseIts_fIts : ∀ F : List (List VLit), eraseIts (fIts F) = fIts F
  | [] => rfl
  | c :: F => by
    have hc : ∀ c : List VLit, eraseIts (clIts c) = clIts c := by
      intro c
      induction c with
      | nil => rfl
      | cons L c ih => simp only [clIts, List.map_cons, List.cons_append,
         eraseIts] at ih ⊢; rw [ih]
    simp only [fIts, eraseIts_append, hc, eraseIts_fIts F]

theorem eraseIts_itemsAF : ∀ AF : List (List (VLit × Bool)), eraseIts
    (itemsAF AF) = fIts (AF.map (List.map Prod.fst))
  | [] => rfl
  | c :: AF => by
    have hc : ∀ (c : List (VLit × Bool)) (rest : List VIt),
        eraseIts (c.map litA ++ VIt.sep :: rest) = clIts (c.map Prod.fst)
           ++ eraseIts rest := by
      intro c rest
      induction c with
      | nil => rfl
      | cons p c ih => simp only [List.map_cons, List.cons_append,
         eraseIts, litA, clIts] at ih ⊢; rw [ih]
    simp only [itemsAF, hc, eraseIts_itemsAF AF, List.map_cons, fIts]

theorem map_symG_inj : ∀ (a b : List Sym), a.map symG = b.map symG → a = b
  | [], [], _ => rfl
  | [], _ :: _, h => by simp at h
  | _ :: _, [], h => by simp at h
  | x :: a, y :: b, h => by
    simp only [List.map_cons, List.cons.injEq] at h
    have hxy : x = y := by cases x <;> cases y <;> simp_all [symG]
    rw [hxy, map_symG_inj a b h.2]
```

```
/-- Two formulas with one item skeleton have one reading: the decoder is the left inverse. -/
theorem vfToCNF_of_fIts (F G : List (List VLit)) (h : fIts G = fIts F) : vfToCNF G = vfToCNF F := by
  have e : vfSyms G = vfSyms F := map_symG_inj _ _ (by rw [fIts_cells, fIts_cells, h])
  have := decode_vfSyms G
  rw [e, decode_vfSyms] at this
  exact (Option.some.inj this).symm

/-- The assignment read off a consistent list: the bit of the first literal naming the variable. -/
def aOf : List (Nat × Bool) → Nat → Bool
  | [], _ => false
  | p :: ps, x => if p.1 = x then p.2 else aOf ps x

theorem aOf_spec : ∀ ps : List (Nat × Bool), vcons ps → ∀ p ∈ ps, aOf ps p.1 = p.2
  | [], _, p, hp => by simp at hp
  | q :: ps, (h1, h2), p, hp => by
    simp only [aOf]
    rcases List.mem_cons.mp hp with hp | hp
    · subst hp; simp
    · by_cases e : q.1 = p.1
      · rw [if_pos e]; exact (h1 p hp e.symm).symm
      · rw [if_neg e]; exact aOf_spec ps h2 p hp

theorem pairsOf_append (a b : List VIt) : pairsOf (a ++ b) = pairsOf a ++ pairsOf b := by
  induction a with
  | nil => rfl
  | cons i a ih =>
    cases i with
    | sep => simp [pairsOf, ih]
    | lit L x => cases x <;> simp [pairsOf, ih]

theorem mem_pairsOf_itemsAF : ∀ (AF : List (List (VLit × Bool))) (c : List (VLit × Bool)), c ∈ AF →
    ∀ p ∈ c, (vvar p.1, p.2) ∈ pairsOf (itemsAF AF)
  | [], _, hc, _, _ => by simp at hc
  | c' :: AF, c, hc, p, hp => by
    have hcl : ∀ (c : List (VLit × Bool)), pairsOf (c.map litA) = c.map (fun p => (vvar p.1, p.2)) := by
      intro c
      induction c with
      | nil => rfl
      | cons q c ih => simp only [List.map_cons, litA, pairsOf] at ih ⊢; rw [ih]
    simp only [itemsAF, pairsOf_append, pairsOf, hcl, List.mem_append, List.mem_map]
    rcases List.mem_cons.mp hc with hc | hc
    · subst hc; exact Or.inl (p, hp, rfl)
    · exact Or.inr (mem_pairsOf_itemsAF AF c hc p hp)

theorem allAnn_itemsAF : ∀ AF : List (List (VLit × Bool)), allAnn (itemsAF AF)
  | [] => trivial
  | c :: AF => by
    show allAnn (c.map litA ++ VIt.sep :: itemsAF AF)
    apply allAnn_append _ _ _ (show allAnn (VIt.sep :: itemsAF AF) from allAnn_itemsAF AF)
    induction c with
    | nil => trivial
    | cons p c ih => exact (by simp [litA], ih)

/-- SOUNDNESS OF THE CHECKS: a consistent annotation meeting a sign in every clause is an assignment. -/
theorem sat_of_checks (AF : List (List (VLit × Bool))) (hcons : vcons (pairsOf (itemsAF AF)))
    (hcl : ∀ c ∈ AF, ∃ p ∈ c, p.1.sg = p.2) : Satisfiable (vfToCNF (AF.map (List.map Prod.fst))) := by
  refine (aOf (pairsOf (itemsAF AF)), ?_)
  unfold evalCNF vfToCNF
  rw [List.all_eq_true]
  intro cl hcl'
  simp only [List.map_map, List.mem_map] at hcl'
  obtain (c, hc, e) := hcl'
  subst e
  obtain (p, hp, hsg) := hcl c hc
  unfold evalClause
  rw [List.any_eq_true]
  refine (vlitToLit p.1, ?_, ?_)
  · simp only [Function.comp, List.map_map, List.mem_map]
    exact (p, hp, rfl)
  · have ha := aOf_spec _ hcons _ (mem_pairsOf_itemsAF AF c hc p hp)
    simp only [evalLit, vlitToLit] at ha ⊢
    rw [ha, hsg]; simp

/-! The good certificate. -/

def uOf (a : Assignment) : List (List VLit) → List Bool
  | [] => []
  | c :: F => c.map (fun L => a (vvar L)) ++ uOf a F

def afOf (a : Assignment) (F : List (List VLit)) : List (List (VLit × Bool)) :=
  F.map (fun c => c.map (fun L => (L, a (vvar L))))

theorem nLits_append (a b : List VIt) : nLits (a ++ b) = nLits a + nLits b := by
  induction a with
  | nil => simp [nLits]
  | cons i a ih => cases i <;> simp [nLits, ih] <;> omega

theorem annIts_app : ∀ (A B : List VIt) (us vs : List Bool), us.length = nLits A →
    annIts (A ++ B) (us ++ vs) = annIts A us ++ annIts B vs
  | [], B, us, vs, h => by
    have : us = [] := List.eq_nil_of_length_eq_zero (by simpa [nLits] using h)
    subst this; rfl
  | .sep :: A, B, us, vs, h => by
    simp only [List.cons_append, annIts]
    rw [annIts_app A B us vs (by simpa [nLits] using h)]
  | .lit L x :: A, B, [], vs, h => by simp [nLits] at h
  | .lit L x :: A, B, u :: us, vs, h => by
    simp only [List.cons_append, annIts]
    rw [annIts_app A B us vs (by simp [nLits] at h; omega)]

theorem annIts_good (a : Assignment) : ∀ F : List (List VLit), annIts (fIts F) (uOf a F) = itemsAF (afOf a F)
  | [] => rfl
  | c :: F => by
    have hc : ∀ c : List VLit, annIts (clIts c) (c.map (fun L => a (vvar L))) =
        (c.map (fun L => (L, a (vvar L)))).map litA ++ [VIt.sep] := by
      intro c
      induction c with
      | nil => rfl
      | cons L c ih => simp only [clIts, List.map_cons, List.cons_append, annIts, litA] at ih ⊢; rw [ih]
    have hn : ∀ c : List VLit, (c.map (fun L => a (vvar L))).length = nLits (clIts c) := by
      intro c
      induction c with
      | nil => rfl
      | cons L c ih => simp only [clIts, List.map_cons, List.cons_append, nLits, List.length_cons] at ih ⊢; omega
    simp only [fIts, uOf, afOf, List.map_cons, itemsAF]
    rw [annIts_app _ _ _ _ (hn c), hc, annIts_good a F]
    simp [afOf]

theorem uOf_length (a : Assignment) : ∀ F : List (List VLit), (uOf a F).length ≤ (vfSyms F).length
  | [] => Nat.le_refl 0
  | c :: F => by
    have hc : ∀ c : List VLit, c.length + 1 ≤ (vclSyms c).length := vclSyms_length
    have := uOf_length a F
    have := hc c
    simp only [uOf, vfSyms, List.length_append, List.length_map]
    omega

theorem good_clauses (a : Assignment) (F : List (List VLit)) (h : evalCNF a (vfToCNF F) = true) :
    ∀ c ∈ afOf a F, ∃ p ∈ c, p.1.sg = p.2 := by
  intro c hc
  simp only [afOf, List.mem_map] at hc
  obtain (c0, hc0, e) := hc
  subst e
  unfold evalCNF vfToCNF at h
  rw [List.all_eq_true] at h
  have h1 := h (c0.map vlitToLit) (List.mem_map.mpr (c0, hc0, rfl))
  unfold evalClause at h1
  rw [List.any_eq_true] at h1
  obtain (l, hl, hl2) := h1
  simp only [List.mem_map] at hl
  obtain (L, hL, e) := hl
  subst e
  refine ((L, a (vvar L)), List.mem_map.mpr (L, hL, rfl), ?_)
  simp only [evalLit, vlitToLit, beq_iff_eq] at hl2
  exact hl2.symm

theorem vcons_of_fun (a : Assignment) : ∀ ps : List (Nat × Bool), (∀ q ∈ ps, q.2 = a q.1) → vcons ps
  | [], _ => trivial
  | p :: ps, h => by
    refine (fun q hq e => ?_, vcons_of_fun a ps (fun q hq => h q (List.mem_cons_of_mem _ hq)))
    rw [h q (List.mem_cons_of_mem _ hq), h p (List.mem_cons.mpr (Or.inl rfl)), e]

theorem good_pairs (a : Assignment) : ∀ F : List (List VLit), ∀ q ∈ pairsOf (itemsAF (afOf a F)), q.2 = a q.1
  | [], q, hq => by simp [afOf, itemsAF, pairsOf] at hq
  | c :: F, q, hq => by
    have hcl : ∀ (c : List VLit), pairsOf ((c.map (fun L => (L, a (vvar L)))).map litA) =
        c.map (fun L => (vvar L, a (vvar L))) := by
      intro c
      induction c with
      | nil => rfl
      | cons L c ih => simp only [List.map_cons, litA, pairsOf] at ih ⊢; rw [ih]
    simp only [afOf, List.map_cons, itemsAF, pairsOf_append, pairsOf, hcl, List.mem_append, List.mem_map] at hq
    rcases hq with (L, _, e) | hq
    · subst e; rfl
    · exact good_pairs a F q hq

/-! ### N.10 · Soundness, completeness with its clock, SAT ∈ NP -/

theorem itsCells_fIts_length (F : List (List VLit)) : (itsCells (fIts F)).length = (vfSyms F).length := by
  rw [← fIts_cells, List.length_map]

/-- SOUNDNESS: a run of the verifier that is ever in its accept state certifies that the formula part is in SAT. -/
theorem verifier_sound (w : List Sym) (u : List Bool) (t : Nat)
    (ht : (zsteps t (vinit (u.map symOfBool ++ Sym.sep :: w))).st = .acc) : satLang w = true := by
  rw [vinit_eq] at ht
```

```
  have noRej : ∀ (z' : VZ) (n : Nat), Reach (zAt .p0Cert [.B] (vtape u
      w)) z' n → RejR z' → False := by
    intro z' n hR hr
    obtain (k, e) := hR.rej hr
    exact acc_rej_false _ t k ht e
  obtain (h0a, h0b) := phase0 u w
  by_cases hv : p0run .p0Start w = .p0Start
  · obtain (F, hF) := (p0_iff w).mp hv
    have R0 := h0a hv
    subst hF
    have et : vtape u (vfSyms F) = u.map bitG ++ VG.S :: (itsCells (fIts
       F) ++ [VG.B]) := by
      simp [vtape, fIts_cells]
    rw [et] at R0 noRej
    obtain (C, hC, hClen, RA) := phaseA (fIts F) (allPlain_fIts F) u
    have R1 := R0.trans RA
    obtain (hB1, hB2) := bScan C hC (annIts (fIts F) u) none [] (fun x hx
       => by simp at hx)
    simp only [List.append_nil, List.nil_append, bSt] at hB1 hB2
    by_cases hb : bCheck none (annIts (fIts F) u) = true
    · have R2 := R1.trans (hB1 hb)
      obtain (AF, hAF, hcl) := (bCheck_inv _ none hb).1 rfl
      have hann : allAnn (annIts (fIts F) u) := by rw [hAF]; exact
         allAnn_itemsAF AF
      obtain (hC1, hC2) := cOuter (([VG.B] ++ C ++ [VG.S]).length +
         (itsCells (annIts (fIts F) u)).length)
        (annIts (fIts F) u) ([VG.B] ++ C ++ [VG.S]) ([VG.B] ++ C, VG.S,
           rfl, rfl) hann (Nat.le_refl _)
      by_cases hc : vcons (pairsOf (annIts (fIts F) u))
      · rw [hAF] at hc
        have hsat := sat_of_checks AF hc hcl
        have e1 : fIts (AF.map (List.map Prod.fst)) = fIts F := by
          rw [← eraseIts_itemsAF, ← hAF, eraseIts_annIts, eraseIts_fIts]
        rw [vfToCNF_of_fIts F _ e1] at hsat
        unfold satLang
        rw [decode_vfSyms]
        exact satB_complete _ hsat
      · exact absurd (noRej _ _ R2 (hC2 hc)) id
    · exact absurd (noRej _ _ R1 (hB2 (by simpa using hb))) id
  · exact absurd (noRej _ 0 (Reach.refl _) (h0b hv)) id

/-- The step budget of the verifier on the good certificate: m
    certificate bits, n input symbols. -/
def vtime (m n : Nat) : Nat :=
  (2 * (m + n) + 4) + (m + 1) * (3 * (m + n) + 10) + (n + (0 + n) + 2 * m
     + 4) +
    (n + 1) * ((m + 2 + n) + 2 + ((m + 2 + n) + 1) * (((m + 2 + n) + 2) *
       (3 * (m + 2 + n) + 13) + (m + 2 + n) + 3))

/-- COMPLETENESS WITH ITS CLOCK: on a satisfiable formula the certificate
     of one bit per literal (the value of its
    variable under a satisfying assignment) makes the verifier accept
       within `vtime`. -/
theorem verifier_complete (w : List Sym) (h : satLang w = true) :
    ∃ u : List Bool, u.length ≤ w.length ∧
      AccR (vinit (u.map symOfBool ++ Sym.sep :: w)) (vtime u.length
         w.length) := by
  unfold satLang at h
  cases hd : decode w with
  | none => rw [hd] at h; exact absurd h (by simp)
  | some f =>
    rw [hd] at h
    obtain (F, hw, hf) := decode_inv w f hd
    obtain (a, ha) := satB_sound f h
    subst hw; subst hf
    refine (uOf a F, uOf_length a F, ?_)
    rw [vinit_eq]
    have R0 := (phase0 (uOf a F) (vfSyms F)).1 (p0_vf F)
    have et : vtape (uOf a F) (vfSyms F) = (uOf a F).map bitG ++ VG.S ::
       (itsCells (fIts F) ++ [VG.B]) := by
      simp [vtape, fIts_cells]
    rw [et] at R0 ⊢
    obtain (C, hC, hClen, RA) := phaseA (fIts F) (allPlain_fIts F) (uOf a
       F)
    rw [annIts_good] at RA
    obtain (hB1, _) := bScan C hC (itemsAF (afOf a F)) none [] (fun x hx
       => by simp at hx)
    simp only [List.append_nil, List.nil_append, bSt] at hB1
    have RB := hB1 (bCheck_itemsAF _ (good_clauses a F ha))
    obtain (hC1, _) := cOuter (([VG.B] ++ C ++ [VG.S]).length + (itsCells
       (itemsAF (afOf a F))).length)
      (itemsAF (afOf a F)) ([VG.B] ++ C ++ [VG.S]) ([VG.B] ++ C, VG.S,
         rfl, rfl) (allAnn_itemsAF _) (Nat.le_refl _)
    have AC := hC1 (vcons_of_fun a _ (good_pairs a F))
    have := ((R0.trans RA).trans RB).acc AC
    refine this.mono ?_
    have hlen : (itsCells (itemsAF (afOf a F))).length = (vfSyms
       F).length := by
      rw [← annIts_good a F, annIts_length, itsCells_fIts_length]
    rw [hlen, itsCells_fIts_length]
    simp only [List.length_append, List.length_singleton,
       List.length_nil, hClen, vtime]
    rw [show 1 + (uOf a F).length + 1 = (uOf a F).length + 2 by omega]
    omega

theorem pow4_eq (X : Nat) : X ^ 4 = X * X * X * X := by
  simp only [Nat.pow_succ, Nat.pow_zero, Nat.one_mul]

theorem succ_pow4_le (n : Nat) : (n + 1) ^ 4 ≤ 16 * n ^ 4 + 16 := by
  cases n with
  | zero => decide
  | succ n =>
    have h1 : (n + 1 + 1) ^ 4 ≤ (2 * (n + 1)) ^ 4 := Nat.pow_le_pow_left
       (by omega) 4
    rw [Nat.mul_pow] at h1
    have : (2 : Nat) ^ 4 = 16 := rfl
    rw [this] at h1
    omega

/-- THE CLOCK IS POLYNOMIAL: on the good certificate the verifier needs
    at most 5000·n⁴ + 5000 steps. -/
theorem vtime_le (m n : Nat) (h : m ≤ n) : vtime m n ≤ 5000 * n ^ 4 + 5000
    := by
  unfold vtime
  generalize hX : n + 1 = X
  have hX1 : 1 ≤ X := by omega
  have hXX : X ≤ X * X := by
    have := Nat.mul_le_mul_left X hX1; simpa using this
  have hXXX : X * X ≤ X * X * X := by
    have := Nat.mul_le_mul_left (X * X) hX1; simpa using this
  have hX4 : X * X * X ≤ X * X * X * X := by
    have := Nat.mul_le_mul_left (X * X * X) hX1; simpa using this
  -- the inner factors
  have hT1 : m + 2 + n + 2 ≤ 4 * X := by omega
  have hT2 : 3 * (m + 2 + n) + 13 ≤ 19 * X := by omega
  have hT3 : m + 2 + n + 1 ≤ 3 * X := by omega
  have hp1 : (m + 2 + n + 2) * (3 * (m + 2 + n) + 13) ≤ 76 * (X * X) := by
    have := Nat.mul_le_mul hT1 hT2
    rw [Nat.mul_mul_mul_comm] at this
    exact this
  have hIB : (m + 2 + n + 2) * (3 * (m + 2 + n) + 13) + (m + 2 + n) + 3 ≤
     81 * (X * X) := by omega
  have hp2 : (m + 2 + n + 1) * ((m + 2 + n + 2) * (3 * (m + 2 + n) + 13) +
     (m + 2 + n) + 3) ≤
      243 * (X * X * X) := by
    have := Nat.mul_le_mul hT3 hIB
    rw [Nat.mul_mul_mul_comm] at this
    have e : X * (X * X) = X * X * X := by rw [Nat.mul_assoc]
    rw [e] at this
    exact this
  have hOB : (m + 2 + n) + 2 + (m + 2 + n + 1) * ((m + 2 + n + 2) * (3 *
     (m + 2 + n) + 13) + (m + 2 + n) + 3) ≤
      247 * (X * X * X) := by omega
  have hp3 : X * ((m + 2 + n) + 2 + (m + 2 + n + 1) * ((m + 2 + n + 2) *
     (3 * (m + 2 + n) + 13) +
      (m + 2 + n) + 3)) ≤ 247 * (X * X * X * X) := by
    have := Nat.mul_le_mul (Nat.le_refl X) hOB
    rw [Nat.mul_left_comm] at this
    have e : X * (X * X * X) = X * X * X * X := by
      rw [Nat.mul_comm X (X * X * X)]
    rw [e] at this
    exact this
  have hPA : (m + 1) * (3 * (m + n) + 10) ≤ 16 * (X * X) := by
    have := Nat.mul_le_mul (show m + 1 ≤ X by omega) (show 3 * (m + n) +
       10 ≤ 16 * X by omega)
    rw [Nat.mul_left_comm] at this
    exact this
  have h4 := succ_pow4_le n
  rw [hX, pow4_eq] at h4
  omega

/-- SAT IS IN NP, IN THE KERNEL: the verifier `vM` with the clock 5000·n⁴
    + 5000 witnesses `InNP satLang`. -/
theorem satLang_in_NP : PNP.TM.InNP PNP.TM.satLang := by
  refine (vM, 5000, 4, fun w => (fun h => ?_, fun (u, _, t, _, _, hv) =>
     ?_))
  · obtain (u, hu, k, hk, hacc) := verifier_complete w h
    have hn : w.length ≤ 5000 * w.length ^ 4 + 5000 := by
      cases hw : w.length with
      | zero => decide
      | succ n =>
        have := Nat.pow_le_pow_right (show 1 ≤ n + 1 by omega) (show 1 ≤ 4
           by decide)
        rw [Nat.pow_one] at this
        omega
    refine (u, ?_, k, ?_, (halts_iff _ k).mpr (by rw [hacc]; rfl),
       (verdict_iff _ k).mpr hacc)
    · show u.length ≤ 5000 * w.length ^ 4 + 5000
      omega
    · show k ≤ 5000 * w.length ^ 4 + 5000
      exact Nat.le_trans hk (vtime_le _ _ hu)
  · exact verifier_sound w u t ((verdict_iff _ t).mp hv)

end PNP.VERIFIER

/-- info: 'PNP.VERIFIER.vparseLit' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseLit
/-- info: 'PNP.VERIFIER.vparseClause_cons' does not depend on any axioms
   -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseClause_cons
/-- info: 'PNP.VERIFIER.vlitSyms_cons' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vlitSyms_cons
/-- info: 'PNP.VERIFIER.vparseClause' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseClause
/-- info: 'PNP.VERIFIER.vclSyms_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vclSyms_length
/-- info: 'PNP.VERIFIER.vfSyms_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vfSyms_length
/-- info: 'PNP.VERIFIER.vparseCNF' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseCNF
/-- info: 'PNP.VERIFIER.decode_vfSyms' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.decode_vfSyms
/-- info: 'PNP.VERIFIER.vparseBits_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseBits_inv
/-- info: 'PNP.VERIFIER.vparseLit_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseLit_inv
/-- info: 'PNP.VERIFIER.vparseClause_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseClause_inv
```

```
/-- info: 'PNP.VERIFIER.vparseCNF_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vparseCNF_inv
/-- info: 'PNP.VERIFIER.decode_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.decode_inv
/-- info: 'PNP.VERIFIER.decG_encG' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.decG_encG
/-- info: 'PNP.VERIFIER.encG_lt' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.encG_lt
/-- info: 'PNP.VERIFIER.vbit_le' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.vbit_le
/-- info: 'PNP.VERIFIER.encFl_le' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.encFl_le
/-- info: 'PNP.VERIFIER.encO_le' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.encO_le
/-- info: 'PNP.VERIFIER.decFl_encFl' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.decFl_encFl
/-- info: 'PNP.VERIFIER.decO_encO' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.decO_encO
/-- info: 'PNP.VERIFIER.dB_vp' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.dB_vp
/-- info: 'PNP.VERIFIER.dF_vp' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.dF_vp
/-- info: 'PNP.VERIFIER.dO_vp' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.dO_vp
/-- info: 'PNP.VERIFIER.parOf_lt' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.parOf_lt
/-- info: 'PNP.VERIFIER.decTag_tagOf' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.decTag_tagOf
/-- info: 'PNP.VERIFIER.decS_encS' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.decS_encS
/-- info: 'PNP.VERIFIER.encS_inj' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.encS_inj
/-- info: 'PNP.VERIFIER.tagOf_lt' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.tagOf_lt
/-- info: 'PNP.VERIFIER.encS_lt' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.encS_lt
/-- info: 'PNP.VERIFIER.vgetD_tail' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vgetD_tail
/-- info: 'PNP.VERIFIER.vheadD' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.vheadD
/-- info: 'PNP.VERIFIER.zmean_R' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.zmean_R
/-- info: 'PNP.VERIFIER.zmean_L' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.VERIFIER.zmean_L
/-- info: 'PNP.VERIFIER.halted_iff' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.halted_iff
/-- info: 'PNP.VERIFIER.vRel_step' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vRel_step
/-- info: 'PNP.VERIFIER.init_getD' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.init_getD
/-- info: 'PNP.VERIFIER.vRel_init' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vRel_init
/-- info: 'PNP.VERIFIER.zsteps_last' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zsteps_last
/-- info: 'PNP.VERIFIER.vRel_run' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vRel_run
/-- info: 'PNP.VERIFIER.verdict_iff' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.verdict_iff
/-- info: 'PNP.VERIFIER.halts_iff' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.halts_iff
/-- info: 'PNP.VERIFIER.zstep_halt' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zstep_halt
/-- info: 'PNP.VERIFIER.zsteps_halt' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zsteps_halt
/-- info: 'PNP.VERIFIER.zsteps_add' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.zsteps_add
/-- info: 'PNP.VERIFIER.zsteps_one' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zsteps_one
/-- info: 'PNP.VERIFIER.zstep_R' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zstep_R
/-- info: 'PNP.VERIFIER.zstep_L' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zstep_L
/-- info: 'PNP.VERIFIER.zAt_cons' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.zAt_cons
/-- info: 'PNP.VERIFIER.zstep_zAt_R' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.zstep_zAt_R
/-- info: 'PNP.VERIFIER.scanR' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.scanR
/-- info: 'PNP.VERIFIER.scanL' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.scanL
/-- info: 'PNP.VERIFIER.retL' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.retL
/-- info: 'PNP.VERIFIER.Reach.refl' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.refl
/-- info: 'PNP.VERIFIER.Reach.of_steps' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.of_steps
/-- info: 'PNP.VERIFIER.Reach.step' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.step
/-- info: 'PNP.VERIFIER.Reach.steps' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.steps
/-- info: 'PNP.VERIFIER.Reach.trans' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.trans
/-- info: 'PNP.VERIFIER.Reach.mono' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.mono
/-- info: 'PNP.VERIFIER.Reach.rej' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.rej
/-- info: 'PNP.VERIFIER.Reach.acc' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.Reach.acc
/-- info: 'PNP.VERIFIER.RejR.of_st' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.RejR.of_st
/-- info: 'PNP.VERIFIER.RejR.step' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.RejR.step
/-- info: 'PNP.VERIFIER.AccR.of_st' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.AccR.of_st
/-- info: 'PNP.VERIFIER.AccR.mono' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.AccR.mono
/-- info: 'PNP.VERIFIER.acc_rej_false' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.acc_rej_false
/-- info: 'PNP.VERIFIER.symG_symOfBool' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.symG_symOfBool
/-- info: 'PNP.VERIFIER.isZO_bitG' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.isZO_bitG
/-- info: 'PNP.VERIFIER.p0run_rej' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.p0run_rej
/-- info: 'PNP.VERIFIER.p0run_append' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.p0run_append
/-- info: 'PNP.VERIFIER.isD0_halt' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.isD0_halt
/-- info: 'PNP.VERIFIER.p0_step' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_step
/-- info: 'PNP.VERIFIER.p0_scan' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_scan
/-- info: 'PNP.VERIFIER.vinit_eq' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vinit_eq
/-- info: 'PNP.VERIFIER.cons_headD_tail' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.cons_headD_tail
/-- info: 'PNP.VERIFIER.bitG_ne_B' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bitG_ne_B
/-- info: 'PNP.VERIFIER.symG_ne_B' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.symG_ne_B
/-- info: 'PNP.VERIFIER.phase0' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.phase0
/-- info: 'PNP.VERIFIER.p0_digits' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_digits
/-- info: 'PNP.VERIFIER.p0_lit' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_lit
/-- info: 'PNP.VERIFIER.p0_cl' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_cl
/-- info: 'PNP.VERIFIER.p0_vf' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_vf
/-- info: 'PNP.VERIFIER.p0_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_inv
/-- info: 'PNP.VERIFIER.p0_iff' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.p0_iff
/-- info: 'PNP.VERIFIER.itsCells_append' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_append
/-- info: 'PNP.VERIFIER.annIts_nil' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.annIts_nil
/-- info: 'PNP.VERIFIER.vlitSyms_map' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vlitSyms_map
/-- info: 'PNP.VERIFIER.vclSyms_map' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vclSyms_map
/-- info: 'PNP.VERIFIER.fIts_cells' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.fIts_cells
/-- info: 'PNP.VERIFIER.allPlain_append' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.allPlain_append
/-- info: 'PNP.VERIFIER.allPlain_fIts' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.allPlain_fIts
/-- info: 'PNP.VERIFIER.mem_map_bitG' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.mem_map_bitG
/-- info: 'PNP.VERIFIER.scanDigits' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.scanDigits
/-- info: 'PNP.VERIFIER.aC_digit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.aC_digit
/-- info: 'PNP.VERIFIER.aD_digit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.aD_digit
/-- info: 'PNP.VERIFIER.aS_sign' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.aS_sign
/-- info: 'PNP.VERIFIER.aLitDigits' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aLitDigits
/-- info: 'PNP.VERIFIER.litCells_length' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.litCells_length
/-- info: 'PNP.VERIFIER.aScanRet' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aScanRet
/-- info: 'PNP.VERIFIER.aScanFin' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aScanFin
/-- info: 'PNP.VERIFIER.annIts_long' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.annIts_long
/-- info: 'PNP.VERIFIER.annIts_length' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.annIts_length
/-- info: 'PNP.VERIFIER.scanU' depends on axioms: [propext, Quot.sound]
    -/
```

```
#guard_msgs in #print axioms PNP.VERIFIER.scanU
/-- info: 'PNP.VERIFIER.aTake' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aTake
/-- info: 'PNP.VERIFIER.replicate_U_nb' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.replicate_U_nb
/-- info: 'PNP.VERIFIER.len_le_of_rev_append' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.len_le_of_rev_append
/-- info: 'PNP.VERIFIER.aRound' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aRound
/-- info: 'PNP.VERIFIER.aEnd' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aEnd
/-- info: 'PNP.VERIFIER.aEndFull' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aEndFull
/-- info: 'PNP.VERIFIER.aLoop' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.aLoop
/-- info: 'PNP.VERIFIER.phaseA' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.phaseA
/-- info: 'PNP.VERIFIER.bCheck_lit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bCheck_lit
/-- info: 'PNP.VERIFIER.itsCells_nb' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_nb
/-- info: 'PNP.VERIFIER.bSt_digit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bSt_digit
/-- info: 'PNP.VERIFIER.bSt_halt' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bSt_halt
/-- info: 'PNP.VERIFIER.bLitDigits' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.bLitDigits
/-- info: 'PNP.VERIFIER.bScan' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.bScan
/-- info: 'PNP.VERIFIER.bCheck_inv' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.bCheck_inv
/-- info: 'PNP.VERIFIER.bCheck_clause' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.bCheck_clause
/-- info: 'PNP.VERIFIER.bCheck_itemsAF' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.bCheck_itemsAF
/-- info: 'PNP.VERIFIER.unmark_markG' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.unmark_markG
/-- info: 'PNP.VERIFIER.lastStop_mono' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.lastStop_mono
/-- info: 'PNP.VERIFIER.lastStop_mark' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.lastStop_mark
/-- info: 'PNP.VERIFIER.lastStop_append' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.lastStop_append
/-- info: 'PNP.VERIFIER.zstep_zAt_L' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.zstep_zAt_L
/-- info: 'PNP.VERIFIER.scanLz' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.scanLz
/-- info: 'PNP.VERIFIER.turnScan' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.turnScan
/-- info: 'PNP.VERIFIER.scanTurn' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.scanTurn
/-- info: 'PNP.VERIFIER.notOA_hP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.notOA_hP
/-- info: 'PNP.VERIFIER.cGoOuter_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cGoOuter_pass
/-- info: 'PNP.VERIFIER.cUnGoOuter_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnGoOuter_pass
/-- info: 'PNP.VERIFIER.cBack_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cBack_pass
/-- info: 'PNP.VERIFIER.cGoInner_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cGoInner_pass
/-- info: 'PNP.VERIFIER.cUnGoInner_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnGoInner_pass
/-- info: 'PNP.VERIFIER.cXScan_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cXScan_pass
/-- info: 'PNP.VERIFIER.cXScan_stop' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cXScan_stop
/-- info: 'PNP.VERIFIER.cYScan_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cYScan_pass
/-- info: 'PNP.VERIFIER.cYScan_stop' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cYScan_stop
/-- info: 'PNP.VERIFIER.cUnYScan_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnYScan_pass
/-- info: 'PNP.VERIFIER.cUnYScan_stop' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnYScan_stop
/-- info: 'PNP.VERIFIER.cUnXScan_pass' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnXScan_pass
/-- info: 'PNP.VERIFIER.cUnXScan_stop' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cUnXScan_stop
/-- info: 'PNP.VERIFIER.cFind_pass_O' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cFind_pass_O
/-- info: 'PNP.VERIFIER.cFind_pass_I' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cFind_pass_I
/-- info: 'PNP.VERIFIER.markG_ystop' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.markG_ystop
/-- info: 'PNP.VERIFIER.markG_xstop' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.markG_xstop
/-- info: 'PNP.VERIFIER.markG_isMk' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.markG_isMk
/-- info: 'PNP.VERIFIER.markG_notOA' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.markG_notOA
/-- info: 'PNP.VERIFIER.markG_notIA' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.markG_notIA
/-- info: 'PNP.VERIFIER.bitG_notOA' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bitG_notOA
/-- info: 'PNP.VERIFIER.bitG_notIA' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bitG_notIA
/-- info: 'PNP.VERIFIER.bitG_isZOS' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.bitG_isZOS
/-- info: 'PNP.VERIFIER.isPC_notOA' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.isPC_notOA
/-- info: 'PNP.VERIFIER.isPC_notIA' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.isPC_notIA
/-- info: 'PNP.VERIFIER.map_markG_unmark' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.map_markG_unmark
/-- info: 'PNP.VERIFIER.cHalfX' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cHalfX
/-- info: 'PNP.VERIFIER.cXRead_digit' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cXRead_digit
/-- info: 'PNP.VERIFIER.cXRead_S' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cXRead_S
/-- info: 'PNP.VERIFIER.cYRead_digit' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cYRead_digit
/-- info: 'PNP.VERIFIER.cYRead_S' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cYRead_S
/-- info: 'PNP.VERIFIER.cHalfY2' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cHalfY2
/-- info: 'PNP.VERIFIER.mkCells_notIA' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.mkCells_notIA
/-- info: 'PNP.VERIFIER.mkCells_notOA' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.mkCells_notOA
/-- info: 'PNP.VERIFIER.mkCells_length' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.mkCells_length
/-- info: 'PNP.VERIFIER.lastStop_OA' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.lastStop_OA
/-- info: 'PNP.VERIFIER.cIter' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cIter
/-- info: 'PNP.VERIFIER.vcomb_go' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vcomb_go
/-- info: 'PNP.VERIFIER.cmpFl_step' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cmpFl_step
/-- info: 'PNP.VERIFIER.cLoop' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cLoop
/-- info: 'PNP.VERIFIER.cmpFl_nil_left' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cmpFl_nil_left
/-- info: 'PNP.VERIFIER.cmpFl_rest' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cmpFl_rest
/-- info: 'PNP.VERIFIER.peqB_iff' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.peqB_iff
/-- info: 'PNP.VERIFIER.allZB_iff' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.allZB_iff
/-- info: 'PNP.VERIFIER.cmp_decide' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cmp_decide
/-- info: 'PNP.VERIFIER.itsCells_isPC' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_isPC
/-- info: 'PNP.VERIFIER.itsCells_last' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_last
/-- info: 'PNP.VERIFIER.allAnn_append' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.VERIFIER.allAnn_append
/-- info: 'PNP.VERIFIER.cResult_rej' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cResult_rej
/-- info: 'PNP.VERIFIER.cResult_ok' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.cResult_ok
/-- info: 'PNP.VERIFIER.conflict_iff' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.conflict_iff
/-- info: 'PNP.VERIFIER.cPairA' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cPairA
/-- info: 'PNP.VERIFIER.allP_app' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.allP_app
/-- info: 'PNP.VERIFIER.allP_cons' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.allP_cons
/-- info: 'PNP.VERIFIER.allP_nil' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.allP_nil
/-- info: 'PNP.VERIFIER.allP_bitG' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.allP_bitG
/-- info: 'PNP.VERIFIER.allP_markG' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.allP_markG
/-- info: 'PNP.VERIFIER.lastStop_uy' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.lastStop_uy
/-- info: 'PNP.VERIFIER.cPairB' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cPairB
/-- info: 'PNP.VERIFIER.cPair' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cPair
/-- info: 'PNP.VERIFIER.itsCells_append_one' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_append_one
/-- info: 'PNP.VERIFIER.cInner' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cInner
/-- info: 'PNP.VERIFIER.cOuter' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.cOuter
/-- info: 'PNP.VERIFIER.eraseIts_append' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.eraseIts_append
```

```
/-- info: 'PNP.VERIFIER.eraseIts_annIts' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.eraseIts_annIts
/-- info: 'PNP.VERIFIER.eraseIts_fIts' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.eraseIts_fIts
/-- info: 'PNP.VERIFIER.eraseIts_itemsAF' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.eraseIts_itemsAF
/-- info: 'PNP.VERIFIER.map_symG_inj' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.map_symG_inj
/-- info: 'PNP.VERIFIER.vfToCNF_of_fIts' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vfToCNF_of_fIts
/-- info: 'PNP.VERIFIER.aOf_spec' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.aOf_spec
/-- info: 'PNP.VERIFIER.pairsOf_append' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.pairsOf_append
/-- info: 'PNP.VERIFIER.mem_pairsOf_itemsAF' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.mem_pairsOf_itemsAF
/-- info: 'PNP.VERIFIER.allAnn_itemsAF' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.allAnn_itemsAF
/-- info: 'PNP.VERIFIER.sat_of_checks' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.sat_of_checks
/-- info: 'PNP.VERIFIER.nLits_append' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.nLits_append
/-- info: 'PNP.VERIFIER.annIts_app' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.annIts_app
/-- info: 'PNP.VERIFIER.annIts_good' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.annIts_good
/-- info: 'PNP.VERIFIER.uOf_length' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.uOf_length
/-- info: 'PNP.VERIFIER.good_clauses' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.good_clauses
/-- info: 'PNP.VERIFIER.vcons_of_fun' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.vcons_of_fun
/-- info: 'PNP.VERIFIER.good_pairs' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.good_pairs
/-- info: 'PNP.VERIFIER.itsCells_fIts_length' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.itsCells_fIts_length
/-- info: 'PNP.VERIFIER.verifier_sound' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.verifier_sound
/-- info: 'PNP.VERIFIER.verifier_complete' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.verifier_complete
/-- info: 'PNP.VERIFIER.pow4_eq' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.VERIFIER.pow4_eq
/-- info: 'PNP.VERIFIER.succ_pow4_le' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.succ_pow4_le
/-- info: 'PNP.VERIFIER.vtime_le' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.vtime_le
/-- info: 'PNP.VERIFIER.satLang_in_NP' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.VERIFIER.satLang_in_NP

/-! ========================================================================
    ================================================
    PART O · THE CLOSURE, TIGHTENED (namespace PNP.TIGHT) · v1_6_3_CC

    Purely additive: nothing above is overwritten. Parts L, M and N move
      every formal piece that can be moved, and this
    Part binds them. With SAT in NP now a theorem of the kernel (Part N),
      the Barrier implies P ≠ NP with nothing before
    the colon (`barrier_implies_p_ne_np`), and the Millennium sentence is
      the Barrier under the hardness half alone
    (`p_ne_np_iff_barrier`). Degrees 0 and 1 of the Diagonal Target are
      theorems (Parts L and M), so the Barrier is
    exactly the Diagonal Target at every degree k ≥ 2
      (`barrier_from_degree_two`).

    What stays open, stated once: the Diagonal Target at every degree k ≥
      2, which is the Barrier itself, and the
    hardness half of Cook and Levin (`PNP.CLOSURE.SATHard`), which stays
      cited. No declaration of this file has the
    type of the Barrier, of `P_ne_NP`, or of any of their faces. The seat
      stays vacant. ΔM = 0.
========================================================================
    ================================================
    -/
namespace PNP.TIGHT

open PNP.NORMALFORM

/-- THE BARRIER GIVES THE SENTENCE, UNCONDITIONALLY: SAT ∈ NP is a
    theorem of the kernel, so the Barrier implies
    P ≠ NP with nothing before the colon. -/
theorem barrier_implies_p_ne_np : PNP.FINALBARRIER.Barrier →
    PNP.TM.P_ne_NP := fun h =>
  PNP.TM.p_ne_np_of_not_inP PNP.VERIFIER.satLang_in_NP
    (PNP.TM.sep_tmsat_iff.mp (PNP.INEQUALITY.face_sep.mpr h))

/-- THE SENTENCE IS THE BARRIER, under the hardness half alone. -/
theorem p_ne_np_iff_barrier (hHard : PNP.CLOSURE.SATHard) :
    PNP.TM.P_ne_NP ↔ PNP.FINALBARRIER.Barrier :=
  PNP.INEQUALITY.the_proof PNP.VERIFIER.satLang_in_NP hHard

/-- THE BARRIER, BY DEGREE: degrees 0 and 1 are theorems, so the Barrier
    is the Diagonal Target at every k ≥ 2. -/
theorem barrier_from_degree_two :
    PNP.FINALBARRIER.Barrier ↔ ∀ k, 2 ≤ k → PNP.LOCALITY.DiagonalTargetAt
      k :=
  PNP.CROSSING.the_barrier_from_degree_two

/-- THE TIGHTENED CLOSURE, nothing before the colon: SAT is in NP;
    degrees 0 and 1 of the Diagonal Target hold; the
    Barrier is the Diagonal Target at every degree k ≥ 2; the Barrier
      implies P ≠ NP; and under the hardness half
    the sentence is the Barrier. -/
theorem the_tightened_closure :
    PNP.TM.InNP PNP.TM.satLang ∧
    PNP.LOCALITY.DiagonalTargetAt 0 ∧
    PNP.LOCALITY.DiagonalTargetAt 1 ∧
    (PNP.FINALBARRIER.Barrier ↔ ∀ k, 2 ≤ k →
      PNP.LOCALITY.DiagonalTargetAt k) ∧
    (PNP.FINALBARRIER.Barrier → PNP.TM.P_ne_NP) ∧
    (PNP.CLOSURE.SATHard → (PNP.TM.P_ne_NP ↔ PNP.FINALBARRIER.Barrier))
      :=
  ⟨PNP.VERIFIER.satLang_in_NP, PNP.LOCALITY.diagonal_target_at_zero,
    PNP.CROSSING.diagonal_target_at_one,
    barrier_from_degree_two, barrier_implies_p_ne_np, p_ne_np_iff_barrier⟩

end PNP.TIGHT

/-- info: 'PNP.TIGHT.barrier_implies_p_ne_np' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TIGHT.barrier_implies_p_ne_np
/-- info: 'PNP.TIGHT.p_ne_np_iff_barrier' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TIGHT.p_ne_np_iff_barrier
/-- info: 'PNP.TIGHT.barrier_from_degree_two' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TIGHT.barrier_from_degree_two
/-- info: 'PNP.TIGHT.the_tightened_closure' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TIGHT.the_tightened_closure

/-! ## Part L · PNP.LOCALITY + PNP.DEGREES: the Locality Engine, the
    Degree
    Stratification, and the First Rung

    The barrier, read by budget degree. `DiagonalTarget` quantifies over
      all
    budgets at once; this Part stratifies it, `DiagonalTargetAt k`, proves
    the stratification exact, and constructs the first rung: at degree 0
      the
    budget is the constant `2c`, and a decider inside it reads only its
      first
    `2c` cells. Two formula strings sharing that prefix, one satisfiable,
    one carrying an empty clause beyond the prefix, force one verdict on
    both, so the decider errs on one of them; a decider outside the budget
    errs by the clock. Locality is ported from Part K of the assembly. -/

namespace PNP.LOCALITY

open PNP.TM

/-- Two configurations agree on a region `R`: same state, same head, same
    tape contents inside `R`. -/
structure AgreeOn (M : TM) (R : Int → Prop) (c₁ c₂ : Config M) : Prop
    where
  st_eq   : c₁.st = c₂.st
  pos_eq  : c₁.pos = c₂.pos
  tape_eq : ∀ i : Int, R i → c₁.tape i = c₂.tape i

theorem step_agree_on (M : TM) (R : Int → Prop) :
    ∀ (c₁ c₂ : Config M), AgreeOn M R c₁ c₂ → R c₁.pos → AgreeOn M R (step
      M c₁) (step M c₂)
  | ⟨st, pos, tape₁⟩, ⟨_, _, tape₂⟩, (rfl, rfl, htp), hpos =>
    if hh : halted M ⟨st, pos, tape₁⟩ then
      have h1 : step M ⟨st, pos, tape₁⟩ = ⟨st, pos, tape₁⟩ := if_pos hh
      have h2 : step M ⟨st, pos, tape₂⟩ = ⟨st, pos, tape₂⟩ := if_pos hh
      ⟨(congrArg Config.st h1).trans (congrArg Config.st h2).symm,
       (congrArg Config.pos h1).trans (congrArg Config.pos h2).symm,
       fun i hi => (congrArg (fun c => c.tape i) h1).trans
         ((htp i hi).trans (congrArg (fun c => c.tape i) h2).symm)⟩
    else
      have hcell : tape₁ pos = tape₂ pos := htp pos hpos
      have h1 : step M ⟨st, pos, tape₁⟩ =
          ⟨(M.δ st (tape₁ pos)).1, pos + move (M.δ st (tape₁ pos)).2.2,
           fun i => if i = pos then (M.δ st (tape₁ pos)).2.1 else tape₁ i)
             := if_neg hh
      have h2 : step M ⟨st, pos, tape₂⟩ =
          ⟨(M.δ st (tape₂ pos)).1, pos + move (M.δ st (tape₂ pos)).2.2,
           fun i => if i = pos then (M.δ st (tape₂ pos)).2.1 else tape₂ i)
             := if_neg hh
      ⟨(congrArg Config.st h1).trans
         ((congrArg (fun a => (M.δ st a).1) hcell).trans (congrArg
           Config.st h2).symm),
       (congrArg Config.pos h1).trans
         ((congrArg (fun a => pos + move (M.δ st a).2.2) hcell).trans
           (congrArg Config.pos h2).symm),
       fun i hi =>
         (congrArg (fun c => c.tape i) h1).trans
           ((if heq : i = pos then
               (if_pos heq).trans ((congrArg (fun a => (M.δ st a).2.1)
                 hcell).trans (if_pos heq).symm)
             else
               (if_neg heq).trans ((htp i hi).trans (if_neg
                 heq).symm)).trans
            (congrArg (fun c => c.tape i) h2).symm)⟩

theorem run_agree_on (M : TM) (R : Int → Prop) (w₁ w₂ : List Sym)
    (hinit : ∀ i : Int, R i → initTape M w₁ i = initTape M w₂ i) :
    ∀ t : Nat, (∀ s, s < t → R (run M w₁ s).pos) → AgreeOn M R (run M w₁
      t) (run M w₂ t)
```

```
  | 0, _ => (rfl, rfl, hinit)
  | t + 1, hvis =>
    have ih : AgreeOn M R (run M w₁ t) (run M w₂ t) :=
      run_agree_on M R w₁ w₂ hinit t (fun s hs => hvis s
        (Nat.lt_succ_of_lt hs))
    step_agree_on M R (run M w₁ t) (run M w₂ t) ih (hvis t
      (Nat.lt_succ_self t))

theorem verdict_agree_on (M : TM) (R : Int → Prop) (w₁ w₂ : List Sym)
    (hinit : ∀ i : Int, R i → initTape M w₁ i = initTape M w₂ i)
    (t : Nat) (hvis : ∀ s, s < t → R (run M w₁ s).pos) :
    verdictAt M w₁ t = verdictAt M w₂ t :=
  have h_ag : AgreeOn M R (run M w₁ t) (run M w₂ t) := run_agree_on M R
    w₁ w₂ hinit t hvis
  congrArg (fun st => decide (st = M.accept)) h_ag.st_eq

/-- The head moves at most one cell per step. -/
theorem move_abs (d : Dir) : move d ≤ 1 ∧ -1 ≤ move d := by
  cases d <;> exact (by decide, by decide)

/-- After `t` steps the head sits within `t` cells of the origin. -/
theorem pos_abs_le (M : TM) (w : List Sym) : ∀ t : Nat,
    (run M w t).pos ≤ (t : Int) ∧ -(t : Int) ≤ (run M w t).pos := by
  intro t
  induction t with
  | zero =>
    have h0 : (run M w 0).pos = 0 := rfl
    rw [h0]
    exact (by decide, by decide)
  | succ t ih =>
    have hcast : ((t + 1 : Nat) : Int) = (t : Int) + 1 := by omega
    by_cases hh : halted M (run M w t)
    · have hfix : step M (run M w t) = run M w t := if_pos hh
      have hpos : (run M w (t + 1)).pos = (run M w t).pos := congrArg
        Config.pos hfix
      rw [hpos, hcast]
      exact (by omega, by omega)
    · have hpos : (run M w (t + 1)).pos
          = (run M w t).pos + move (M.δ (run M w t).st ((run M w t).tape
            (run M w t).pos)).2.2 :=
        congrArg Config.pos (if_neg hh)
      have hm := move_abs (M.δ (run M w t).st ((run M w t).tape (run M w
        t).pos)).2.2
      rw [hpos, hcast]
      exact (by omega, by omega)

end PNP.LOCALITY

namespace PNP.DEGREES

open PNP.TM PNP.NORMALFORM PNP.SAT PNP.ZA PNP.LOCALITY

/-- THE DIAGONAL TARGET AT DEGREE `k`: every decider, at every constant,
    misses SAT at some string, wrong verdict, or past the degree-`k`
    budget. `DiagonalTarget` is the target at all degrees at once. -/
def DiagonalTargetAt (k : Nat) : Prop :=
  ∀ (A : Decider) (c : Nat), ∃ x : List Sym,
    out A x ≠ satLang x ∨ bound c k x.length < time A x

/-- The stratification is exact: the target is its degrees, jointly. -/
theorem diagonal_target_iff_all_degrees :
    PNP.MAINLINE.DiagonalTarget ↔ ∀ k, DiagonalTargetAt k := by
  constructor
  · intro h k A c
    obtain ⟨x, hx⟩ := h A c k
    exact ⟨x, hx⟩
  · intro h A c k
    exact h k A c

/-- `getD` reads the left factor inside its own length. -/
theorem getD_append_lt {α : Type} (l₁ l₂ : List α) (d : α) :
    ∀ i : Nat, i < l₁.length → (l₁ ++ l₂).getD i d = l₁.getD i d := by
  induction l₁ with
  | nil => intro i hi; simp at hi
  | cons a l₁ ih =>
    intro i hi
    cases i with
    | zero => rfl
    | succ i =>
      have hi' : i < l₁.length := by
        have := hi
        simp only [List.length_cons] at this
        exact Nat.lt_of_succ_lt_succ this
      exact ih i hi'

/-- Excess halving fuel changes nothing once the fuel covers the value. -/
theorem bitsF_fuel_mono : ∀ (f n d : Nat), n ≤ f → bitsF f n = bitsF (f +
    d) n := by
  intro f
  induction f with
  | zero =>
    intro n d hn
    obtain rfl : n = 0 := Nat.eq_zero_of_le_zero hn
    cases d <;> rfl
  | succ f ih =>
    intro n d hn
    cases n with
    | zero =>
      rw [show f + 1 + d = (f + d) + 1 from by omega]
      rfl
    | succ n =>
      have hdiv : div2 (n + 1) ≤ f := by
        have h2 : 2 * div2 (n + 1) ≤ n + 1 ∧ 2 * div2 (n + 1 + 1) ≤ n + 1
            + 1 := by
          have key : ∀ m : Nat, 2 * div2 m ≤ m ∧ 2 * div2 (m + 1) ≤ m + 1
              := by
            intro m
            induction m with
            | zero => exact (by decide, by decide)
            | succ m ih =>
              refine (ih.2, ?_)
              have hstep : div2 (m + 1 + 1) = div2 m + 1 := rfl
              rw [hstep]
              omega
          exact key (n + 1)
        omega
      rw [show f + 1 + d = (f + d) + 1 from by omega]
      show (mod2 (n + 1) :: bitsF f (div2 (n + 1)))
          = (mod2 (n + 1) :: bitsF (f + d) (div2 (n + 1)))
      rw [ih (div2 (n + 1)) d hdiv]

/-- Doubling costs exactly one digit: `div2 (2 * n) = n`. -/
theorem div2_double : ∀ n : Nat, div2 (2 * n) = n := by
  intro n
  induction n with
  | zero => rfl
  | succ n ih =>
    have h : 2 * (n + 1) = 2 * n + 2 := by omega
    rw [h, show div2 (2 * n + 2) = div2 (2 * n) + 1 from rfl, ih]

/-- The digits of a power of two: `2^m` has exactly `m + 1`. -/
theorem bits_two_pow_length : ∀ m : Nat, (bits (2 ^ m)).length = m + 1 :=
  by
  intro m
  induction m with
  | zero => rfl
  | succ m ih =>
    have hpow : (2 : Nat) ^ (m + 1) = 2 * 2 ^ m := by rw [Nat.pow_succ,
      Nat.mul_comm]
    have hpos : 0 < 2 ^ (m + 1) := Nat.pow_pos (by decide)
    have hdiv : div2 (2 ^ (m + 1)) = 2 ^ m := by rw [hpow]; exact
      div2_double (2 ^ m)
    have hunfold : bitsF (2 ^ (m + 1)) (2 ^ (m + 1))
        = mod2 (2 ^ (m + 1)) :: bitsF (2 ^ m) (2 ^ m) := by
      have h1 : (1 : Nat) ≤ 2 ^ m := Nat.pow_pos (by decide)
      have hsum : 2 ^ m + ((2 ^ (m + 1) - 1) - 2 ^ m) = 2 ^ (m + 1) - 1 :=
        by omega
      have hstep : ∀ (f n : Nat), bitsF (f + 1) n
          = if n = 0 then [] else mod2 n :: bitsF f (div2 n) := fun f n =>
            rfl
      have e1 : bitsF ((2 ^ (m + 1) - 1) + 1) (2 ^ (m + 1))
          = mod2 (2 ^ (m + 1)) :: bitsF (2 ^ (m + 1) - 1) (div2 (2 ^ (m +
            1))) := by
        rw [hstep, if_neg (Nat.ne_of_gt hpos)]
      have e2 : bitsF (2 ^ (m + 1)) (2 ^ (m + 1))
          = bitsF ((2 ^ (m + 1) - 1) + 1) (2 ^ (m + 1)) := by
        congr 1
        exact (Nat.sub_add_cancel (show 1 ≤ 2 ^ (m + 1) from Nat.pow_pos
          (by decide))).symm
      rw [e2, e1, hdiv, ← hsum,
        ← bitsF_fuel_mono (2 ^ m) (2 ^ m) ((2 ^ (m + 1) - 1) - 2 ^ m)
          (Nat.le_refl _)]
    have hlen : (bitsF (2 ^ m) (2 ^ m)).length = m + 1 := ih
    show (bitsF (2 ^ (m + 1)) (2 ^ (m + 1))).length = m + 1 + 1
    rw [hunfold, List.length_cons, hlen]

/-- The two test strings: `x₂` one long unit clause (satisfiable); `x₁` the
    same clause followed by the empty clause (unsatisfiable). `x₁` is `x₂`
    with one separator appended, so they share the whole prefix `x₂`. -/
theorem degree_zero_witnesses (c : Nat) :
    ∃ (x₁ x₂ : List Sym),
      x₁ = x₂ ++ [Sym.sep] ∧ 2 * c + 1 ≤ x₂.length ∧
      satLang x₁ = false ∧ satLang x₂ = true := by
  let v : Nat := 2 ^ (2 * c + 2) - 1
  have hv : v + 1 = 2 ^ (2 * c + 2) := by
    have h1 : (1 : Nat) ≤ 2 ^ (2 * c + 2) := Nat.pow_pos (by decide)
    simp only [v]
    omega
  refine (encode [[(v, true)], []], encode [[(v, true)]],
    by simp [encode, encClause], ?_, ?_, ?_)
  · have hlen : (encode [[(v, true)]]).length = (bits (2 ^ (2 * c +
      2))).length + 3 := by
      simp only [encode, encClause, encLit, encNat, List.length_append,
        List.length_map,
        List.length_cons, List.length_nil, Nat.add_zero]
      rw [hv]
    rw [hlen, bits_two_pow_length]
    omega
  · rw [satLang_encode]
    have h : ¬ Satisfiable [[(v, true)], []] := by
      intro (a, ha)
      simp [evalCNF, evalClause] at ha
    cases hsb : satB [[(v, true)], []] with
    | false => rfl
    | true => exact absurd ((PNP.SAT.satB_iff _).mp hsb) h
  · rw [satLang_encode]
    exact (PNP.SAT.satB_iff _).mpr ⟨fun _ => true, by simp [evalCNF,
      evalClause, evalLit]⟩

/-- DEGREE ZERO: no decider decides SAT inside a constant budget. Inside
    the budget the head sees only the shared prefix of the two witnesses
    and
    returns one verdict on both, erring on one; outside it, the clock
    errs. -/
theorem diagonal_target_at_zero : DiagonalTargetAt 0 := by
  intro A c
  obtain ⟨x₁, x₂, hpre, hlen, hsat₁, hsat₂⟩ := degree_zero_witnesses c
  by_cases htime : time A x₁ ≤ 2 * c
  · -- the local case: one verdict on both witnesses
    have hhalt : HaltsAt A.M x₁ (time A x₁) := time_halts A x₁
    have hinit : ∀ i : Int, i < (x₂.length : Int) → initTape A.M x₁ i =
      initTape A.M x₂ i := by
```

```
      intro i hi
      by_cases hneg : i < 0
      · simp [initTape, if_pos hneg]
      · have hi0 : 0 ≤ i := by omega
        have hlt : i.toNat < x₂.length := by omega
        simp only [initTape, if_neg hneg, hpre, List.map_append,
           List.map_cons, List.map_nil]
        exact getD_append_lt (x₂.map (Sym.toTape A.M.s))
           [(Sym.sep).toTape A.M.s] (blank A.M) i.toNat
          (by rw [List.length_map]; exact hlt)
    have hvis : ∀ s : Nat, s < time A x₁ → (run A.M x₁ s).pos < (x₂.length
       : Int) := by
      intro s hs
      have h1 := (pos_abs_le A.M x₁ s).1
      have h2 : s < x₂.length := by omega
      have h3 : ((s : Int)) < (x₂.length : Int) := by omega
      omega
    have hverd : verdictAt A.M x₁ (time A x₁) = verdictAt A.M x₂ (time A
       x₁) :=
      verdict_agree_on A.M (fun i => i < (x₂.length : Int)) x₁ x₂ hinit
         (time A x₁) hvis
    have h_ag : PNP.LOCALITY.AgreeOn A.M (fun i => i < (x₂.length : Int))
       (run A.M x₁ (time A x₁)) (run A.M x₂ (time A x₁)) :=
      PNP.LOCALITY.run_agree_on A.M _ x₁ x₂ hinit (time A x₁) hvis
    have hhalt₂ : HaltsAt A.M x₂ (time A x₁) := by
      unfold HaltsAt halted at hhalt ⊢
      rw [← h_ag.st_eq]
      exact hhalt
    have htime₂ : time A x₂ ≤ time A x₁ := time_least A x₂ (time A x₁)
       hhalt₂
    have hout₂ : out A x₂ = verdictAt A.M x₂ (time A x₁) :=
      (verdict_of_le (time_halts A x₂) htime₂).symm
    have hout : out A x₁ = out A x₂ := hverd.trans hout₂.symm
    by_cases hb : out A x₁ = true
    · exact ⟨x₁, Or.inl (by rw [hb, hsat₁]; decide)⟩
    · have hb2 : out A x₁ = false := by
        cases h : out A x₁ with
        | true => exact absurd h hb
        | false => rfl
      exact ⟨x₂, Or.inl (by rw [← hout, hb2, hsat₂]; decide)⟩
  · -- the clock case: the budget at degree 0 is the constant `2c`
    have hbound : bound c 0 x₁.length = 2 * c := by
      show c * 1 + c = 2 * c
      omega
    exact ⟨x₁, Or.inr (by rw [hbound]; omega)⟩

/-- DEGREE ZERO, in the master's own currency: at the constant budget,
    every decider has a string it does not decide cleanly. -/
theorem target_zero_kills_clean : ∀ (A : Decider) (c : Nat), ∃ x : List
    Sym,
    ¬ PNP.NORMALFORM.Clean PNP.TM.TMSAT A c 0 x := by
  intro A c
  obtain ⟨x, hx⟩ := diagonal_target_at_zero A c
  refine ⟨x, fun hCl => ?_⟩
  cases hx with
  | inl h => exact h hCl.1
  | inr h => exact Nat.not_lt.mpr hCl.2 h

end PNP.DEGREES

/-- info: 'PNP.DEGREES.diagonal_target_iff_all_degrees' does not depend
    on any axioms -/
#guard_msgs in #print axioms PNP.DEGREES.diagonal_target_iff_all_degrees
/-- info: 'PNP.DEGREES.diagonal_target_at_zero' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.DEGREES.diagonal_target_at_zero
/-- info: 'PNP.DEGREES.target_zero_kills_clean' depends on axioms:
    [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.DEGREES.target_zero_kills_clean

/-! ## Part M · PNP.FINALDELIVERY: the Combined Closure: the Bit as the
    Delivering Theorem

    All forces combined in one file: the base masters (Parts A–K), the
    scribe's verifier (`PNP.VERIFIER`: SAT ∈ NP is a theorem), crossing
    engine (`PNP.CROSSING`: degree 1, the reduction to the tail) and
    tightened closure (`PNP.TIGHT`), and the independent degree ladder
    (Part L: `PNP.DEGREES`). What remains of the whole P ≠ NP arc is one
    hypothesis, the degree tail, and this Part names the delivery:
    from the tail alone, the sentence. -/

namespace PNP.FINALDELIVERY

open PNP.TM

/-- The two stratifications of this file, the independent `PNP.DEGREES`
    ladder and `PNP.LOCALITY`'s, are one stratification. -/
theorem stratifications_agree (k : Nat) :
    PNP.DEGREES.DiagonalTargetAt k ↔ PNP.LOCALITY.DiagonalTargetAt k :=
      Iff.rfl

/-- THE FINAL DELIVERING THEOREM: `P ≠ NP` from the degree tail alone.
    Every other hypothesis the arc ever carried before the colon, SAT's
    membership half, degrees 0 and 1, and (for this direction) the
    hardness half, is discharged by a theorem of this file. The single
    remaining hypothesis is the tail `∀ k ≥ 2, DiagonalTargetAt k`, which
    is the Barrier in its degree form. -/
theorem p_ne_np_of_tail (h : ∀ k, 2 ≤ k → PNP.LOCALITY.DiagonalTargetAt
    k) :
    PNP.TM.P_ne_NP :=
  PNP.TIGHT.barrier_implies_p_ne_np
    (PNP.TIGHT.barrier_from_degree_two.mpr h)

/-- The same delivery, entered through the independent stratification. -/
theorem p_ne_np_of_tail_degrees (h : ∀ k, 2 ≤ k →
    PNP.DEGREES.DiagonalTargetAt k) :
    PNP.TM.P_ne_NP :=
  p_ne_np_of_tail (fun k hk => (stratifications_agree k).mp (h k hk))

/-- THE COMBINED CLOSURE, in one statement: SAT is in NP (theorem);
    degrees 0 and 1 of the Diagonal Target hold (theorems, by two
    independent constructions at degree 0); the Barrier is the degree
    tail; the tail delivers the sentence; the Barrier delivers the
    sentence; and under the hardness half the sentence is the Barrier. -/
theorem the_final_delivery :
    PNP.TM.InNP PNP.TM.satLang ∧
    PNP.LOCALITY.DiagonalTargetAt 0 ∧
    PNP.LOCALITY.DiagonalTargetAt 1 ∧
    PNP.DEGREES.DiagonalTargetAt 0 ∧
    (PNP.FINALBARRIER.Barrier ↔ ∀ k, 2 ≤ k →
       PNP.LOCALITY.DiagonalTargetAt k) ∧
    ((∀ k, 2 ≤ k → PNP.LOCALITY.DiagonalTargetAt k) → PNP.TM.P_ne_NP) ∧
    (PNP.FINALBARRIER.Barrier → PNP.TM.P_ne_NP) ∧
    (PNP.CLOSURE.SATHard → (PNP.TM.P_ne_NP ↔ PNP.FINALBARRIER.Barrier))
       :=
  ⟨PNP.VERIFIER.satLang_in_NP,
   PNP.LOCALITY.diagonal_target_at_zero,
   PNP.CROSSING.diagonal_target_at_one,
   PNP.DEGREES.diagonal_target_at_zero,
   PNP.TIGHT.barrier_from_degree_two,
   p_ne_np_of_tail,
   PNP.TIGHT.barrier_implies_p_ne_np,
   fun hHard => PNP.TIGHT.p_ne_np_iff_barrier hHard⟩

end PNP.FINALDELIVERY

/-- info: 'PNP.FINALDELIVERY.stratifications_agree' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.FINALDELIVERY.stratifications_agree
/-- info: 'PNP.FINALDELIVERY.p_ne_np_of_tail' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALDELIVERY.p_ne_np_of_tail
/-- info: 'PNP.FINALDELIVERY.the_final_delivery' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALDELIVERY.the_final_delivery

/-! ============================================================
    ============================================
    PART P · THE MANDATORY NECESSITY AND ILLEGAL-DEMAND REFUSAL THEOREMS
       (namespace PNP.CONVERSION) · v1_6_5_CC

    Purely additive: restored from v1_6_4_CC and titled per the binding
       instructions. Three proposed conversions of the
    bit are stated in the kernel, and each is proved to factor through
       the bit.

    Step 1 (`p_ne_np_of_one_bit`): the bit delivers P ≠ NP, the
       membership half discharged by Part N. The bit is the
    hypothesis.

    Step 2 (`CompletedFertileTMSAT`): on the structure the bit and P ≠ NP
       are projections (`completed_tmsat_p_ne_np`),
    and the structure is inhabited exactly when the bit holds
       (`completed_tmsat_iff_bit`).

    Step 3 (`BoundedHaltingTableau`): given one, the Diagonal Target
       follows at every degree (`diagonal_target_of_tableau`),
    and every inhabitant already gives the bit (`tableau_gives_bit`). The
       intended self-describing formula cannot be
    built: a Cook and Levin tableau of a run of T steps has at least T
       cells, and the budget c·n^k + c exceeds n, so a
    formula of length n cannot contain the tableau of its own run at the
       budget it is tested on.

    Mandatory necessity: every proposed route to the sentence is, by
       these theorems, a proof of the bit. Illegal-demand
    refusal: a demand for a route that bypasses the bit is refused for
       that reason. Neither structure is inhabited in this
    file, and no declaration has the type of the bit. `stopsAt` restates
       the Diagonal Target as a computable test
    (`diagonal_target_at_iff_stops`). ΔM = 0.
============================================================================
    ============================================
    -/
namespace PNP.CONVERSION

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.LOCALITY

/-! ### P.1 · The computable test -/

/-- The decider fails or overruns the budget (c, k) on x, as a Boolean. -/
def stopsAt (A : Decider) (c k : Nat) (x : List Sym) : Bool :=
  (out A x != satLang x) || decide (bound c k x.length < time A x)

theorem stopsAt_iff (A : Decider) (c k : Nat) (x : List Sym) :
    stopsAt A c k x = true ↔ (out A x ≠ satLang x ∨ bound c k x.length <
       time A x) := by
  unfold stopsAt
  rw [Bool.or_eq_true, bne_iff_ne, decide_eq_true_eq]

theorem diagonal_target_at_iff_stops (k : Nat) :
    DiagonalTargetAt k ↔ ∀ (A : Decider) (c : Nat), ∃ x, stopsAt A c k x
       = true :=
  ⟨fun h A c => (h A c).elim fun x hx => ⟨x, (stopsAt_iff A c k x).mpr hx⟩,
   fun h A c => (h A c).elim fun x hx => ⟨x, (stopsAt_iff A c k x).mp hx⟩⟩

/-! ### P.2 · Step 1, under its proposed name -/

/-- P ≠ NP from the bit, the membership half discharged by Part N. The
    bit is the hypothesis. -/
theorem p_ne_np_of_one_bit (bit : PNP.INEQUALITY.TheInequality) : P_ne_NP
    :=
```

```
  PNP.TIGHT.barrier_implies_p_ne_np bit

/-! ### P.3 · Step 2, the completed structure, graded -/

/-- The proposed completed structure: TMSAT together with a fertile machine on it. -/
structure CompletedFertileTMSAT where
  fertile : FTOE.FertileMachine TMSAT

/-- On the completed structure the bit, the separation, SAT ∉ P and P ≠ NP are projections. -/
theorem completed_tmsat_p_ne_np (S : CompletedFertileTMSAT) :
    PNP.INEQUALITY.TheInequality ∧ Sep TMSAT ∧ ¬ SATinP ∧ P_ne_NP :=
  have hsep := (FTOE.ftoe_pnp_one_bit_theorem TMSAT S.fertile).2.2
  have hnot := sep_tmsat_iff.mp hsep
  ((FTOE.ftoe_pnp_one_bit_theorem TMSAT S.fertile).1, hsep, hnot,
   p_ne_np_of_not_inP PNP.VERIFIER.satLang_in_NP hnot)

/-- EXACT: the completed structure is inhabited iff the bit holds. -/
theorem completed_tmsat_iff_bit : Nonempty CompletedFertileTMSAT ↔
    PNP.INEQUALITY.TheInequality :=
  (fun (S) => (completed_tmsat_p_ne_np S).1,
   fun h => ((PNP.SELECTED.sep_iff_fertile_machine TMSAT).mp
      (PNP.INEQUALITY.face_sep.mpr h)).elim
     fun FM => ((FM)))

/-! ### P.4 · Step 3, the bounded-halting tableau, graded -/

/-- The proposed tableau: for every decider and budget, a formula on which the decider cannot be right in time. -/
structure BoundedHaltingTableau where
  diagFormula : Decider → Nat → Nat → CNF
  tableau_stops : ∀ (A : Decider) (c k : Nat),
    time A (encode (diagFormula A c k)) ≤ bound c k (encode (diagFormula A c k)).length →
      out A (encode (diagFormula A c k)) ≠ satLang (encode (diagFormula A c k))

/-- Given a tableau, the Diagonal Target holds at every degree at once. -/
theorem diagonal_target_of_tableau (T : BoundedHaltingTableau) :
    PNP.MAINLINE.DiagonalTarget := by
  intro A c k
  refine (encode (T.diagFormula A c k), ?_)
  rcases Nat.lt_or_ge (bound c k (encode (T.diagFormula A c k)).length)
      (time A (encode (T.diagFormula A c k))) with hlate | htime
  · exact Or.inr hlate
  · exact Or.inl (T.tableau_stops A c k htime)

/-- P ≠ NP from a tableau. The tableau is the hypothesis. -/
theorem p_ne_np_of_tableau (T : BoundedHaltingTableau) : P_ne_NP :=
  PNP.MAINLINE.p_ne_np_main_line PNP.VERIFIER.satLang_in_NP
    (diagonal_target_of_tableau T)

/-- AT LEAST THE BIT: any inhabitant of the tableau already gives the bit. -/
theorem tableau_gives_bit : Nonempty BoundedHaltingTableau →
    PNP.INEQUALITY.TheInequality :=
  fun (T) => PNP.INEQUALITY.face_diagonal.mp (diagonal_target_of_tableau T)

/-! ### P.5 · The path, graded in one theorem -/

/-- THE CONVERSION PATH, GRADED, nothing before the colon: the bit gives P ≠ NP; the completed structure is exactly
    the bit; the tableau is at least the bit; and the bit is the Diagonal Target at every degree k ≥ 2. -/
theorem the_conversion_graded :
    (PNP.INEQUALITY.TheInequality → P_ne_NP) ∧
    (Nonempty CompletedFertileTMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (Nonempty BoundedHaltingTableau → PNP.INEQUALITY.TheInequality) ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k) :=
  (p_ne_np_of_one_bit, completed_tmsat_iff_bit, tableau_gives_bit,
   PNP.CROSSING.the_inequality_from_degree_two)

end PNP.CONVERSION

/-- info: 'PNP.CONVERSION.stopsAt_iff' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CONVERSION.stopsAt_iff
/-- info: 'PNP.CONVERSION.diagonal_target_at_iff_stops' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CONVERSION.diagonal_target_at_iff_stops
/-- info: 'PNP.CONVERSION.p_ne_np_of_one_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONVERSION.p_ne_np_of_one_bit
/-- info: 'PNP.CONVERSION.completed_tmsat_p_ne_np' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONVERSION.completed_tmsat_p_ne_np
/-- info: 'PNP.CONVERSION.completed_tmsat_iff_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONVERSION.completed_tmsat_iff_bit
/-- info: 'PNP.CONVERSION.diagonal_target_of_tableau' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CONVERSION.diagonal_target_of_tableau
/-- info: 'PNP.CONVERSION.p_ne_np_of_tableau' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONVERSION.p_ne_np_of_tableau
/-- info: 'PNP.CONVERSION.tableau_gives_bit' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CONVERSION.tableau_gives_bit
/-- info: 'PNP.CONVERSION.the_conversion_graded' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CONVERSION.the_conversion_graded

/-! ============================================================================
    ================================================
    PART Q · THE TRIAXIAL 1-BIT DELIVERING THEOREM AND FIXED-POINT CLOSURE (namespace PNP.TRIAXIAL_CLOSURE) · v1_6_5_CC

    The final weld of the binding instructions. With `PNP.VERIFIER.satLang_in_NP` (Part N),
    `PNP.LOCALITY.diagonal_target_at_zero` (Part L) and `PNP.CROSSING.diagonal_target_at_one` (Part M) proved, the
    membership half and degrees 0 and 1 are discharged. The hardness half is not needed for the faces below, because the
    separation is witnessed at `satLang` itself (`SAT_separates_P_NP`); it stays cited for the converse, P ≠ NP to the bit.
    1. The record locus `Rec TMSAT` is a fixed point and carries least escape, separation, self-recursion and a fertile
       machine, unconditionally
       (`record_locus_fixed_point_unconditional`). These are facts about the record.
    2. Ten faces of the bit, among them `SAT_separates_P_NP`, `¬ SATinP`, `TheInequality`, `SelfRecursive TMSAT`, record
       coincidence, the fertile machine at TMSAT, the selected machine, the physical witness and the degree tail
       `∀ k ≥ 2, DiagonalTargetAt k`, are one proposition, with nothing before the colon (`ten_faces_of_the_one_bit`).
    3. The bit's grade is proved four ways, with nothing before the colon: necessary and sufficient by the biconditionals;
       delivering, since each face implies P ≠ NP (`the_one_bit_delivers_p_ne_np`); irreducible by the block theorems
      (`solCount2 = 2` against `solCount3 = 1`, `keyless_forces_nothing`, `no_record_principle_decides`); and mandatory
       by `tmsat_every_route_passes_the_bit`.
    The bit is the antecedent of every delivering conjunct and is proved nowhere in this file: no declaration has the
    type of the bit or of any of its faces. ΔM = 0.
============================================================================
    ================================================ -/
namespace PNP.TRIAXIAL_CLOSURE
open PNP.NORMALFORM PNP.SAT PNP.TM PNP.LOCALITY PNP.CROSSING PNP.VERIFIER

/-- Separation of NP and P witnessed at `satLang` itself. -/
def SAT_separates_P_NP : Prop := InNP satLang ∧ ¬ InP satLang

/-- `SAT_separates_P_NP` delivers `P_ne_NP` in one step. -/
theorem sat_separates_gives_p_ne_np : SAT_separates_P_NP → P_ne_NP :=
  fun (hNP, hNotP) => (satLang, hNP, hNotP)

/-- With `satLang_in_NP` proved in Part N, `SAT_separates_P_NP` is equivalent to `¬ SATinP`. -/
theorem sat_separates_iff_not_inP : SAT_separates_P_NP ↔ ¬ SATinP :=
  (fun h => h.2, fun h => (satLang_in_NP, h))

/-- THE RECORD LOCUS FIXED POINT, UNCONDITIONAL: at `Rec TMSAT` registration is idempotent, least escape holds,
    separation holds, self-recursion holds, and a fertile machine is inhabited. All five are facts about the record. -/
theorem record_locus_fixed_point_unconditional :
    Rec (Rec TMSAT) = Rec TMSAT ∧
    NothingEscapes (Rec TMSAT) ∧
    Sep (Rec TMSAT) ∧
    SelfRecursive (Rec TMSAT) ∧
    Nonempty (FTOE.FertileMachine (Rec TMSAT)) :=
  (rec_rec TMSAT,
   tmsat_record_has_least_escape,
   record_is_separated TMSAT [],
   tmsat_self_recursion.2,
   (FTOE.recFertileMachine TMSAT []))

/-- THE TEN FACES OF THE BIT, with nothing before the colon: each face is equivalent to `TheInequality`. -/
theorem ten_faces_of_the_one_bit :
    (SAT_separates_P_NP ↔ PNP.INEQUALITY.TheInequality) ∧
    (¬ SATinP ↔ PNP.INEQUALITY.TheInequality) ∧
    (Sep TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (NothingEscapes TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (SelfRecursive TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    ((∀ A c k, Escapes TMSAT A c k ↔ Escapes (Rec TMSAT) A c k) ↔
      PNP.INEQUALITY.TheInequality) ∧
    (Nonempty (FTOE.FertileMachine TMSAT) ↔ PNP.INEQUALITY.TheInequality) ∧
    (Nonempty PNP.SELECTED.SelectedTMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (Nonempty PNP.SUPPLY.PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
    ((∀ k, 2 ≤ k → DiagonalTargetAt k) ↔ PNP.INEQUALITY.TheInequality) :=
  have h_notP : ¬ SATinP ↔ PNP.INEQUALITY.TheInequality :=
    pnp_normal_form_turing.symm
  (sat_separates_iff_not_inP.trans h_notP,
   h_notP,
   PNP.INEQUALITY.face_sep,
   PNP.INEQUALITY.face_closure,
   tmsat_self_recursion.1.trans h_notP,
   tmsat_coincidence.trans h_notP,
   PNP.FINALBARRIER.barrier_subsumption,
   (fun (S) => PNP.INEQUALITY.face_closure.mp S.selected,
    fun h => ((PNP.INEQUALITY.face_closure.mpr h))),
   PNP.INEQUALITY.face_physical,
   the_inequality_from_degree_two.symm)

/-- THE DELIVERING THEOREM, with nothing before the colon: each face of the bit implies `P_ne_NP`, from the cargo
    loop, the degree tail, the fertile machine, the selected machine and the physical witness. In every conjunct the
    face is the antecedent. -/
theorem the_one_bit_delivers_p_ne_np :
    (PNP.INEQUALITY.TheInequality → P_ne_NP) ∧
    ((∀ k, 2 ≤ k → DiagonalTargetAt k) → P_ne_NP) ∧
    (FTOE.FertileMachine TMSAT → P_ne_NP) ∧
    (PNP.SELECTED.SelectedTMSAT → P_ne_NP) ∧
```

```
    (PNP.SUPPLY.PhysicalWitness → P_ne_NP) :=
  (fun h => (PNP.SUPPLY.p_ne_np_delivered_twice satLang_in_NP).1 h,
   fun h => (PNP.SUPPLY.p_ne_np_delivered_twice satLang_in_NP).1
     (the_inequality_from_degree_two.mpr h),
   fun FM => p_ne_np_of_not_inP satLang_in_NP
     (sep_tmsat_iff.mp (FTOE.ftoe_pnp_one_bit_theorem TMSAT FM).2.2),
   fun S => PNP.SELECTED.selected_p_ne_np satLang_in_NP S,
   fun W => (PNP.SUPPLY.p_ne_np_delivered_twice satLang_in_NP).2 W)

/-- THE TRIAXIAL MASTER SEAL, with nothing before the colon. It conjoins: (1) the triaxial lock against the two-axis
    binary fibre (`two_born_from_three_lock`) and the return locus (`ftoe_unconditional_return_one_bit`); (2) the
    discharged parts (`satLang_in_NP`, degrees 0 and 1, the record fixed point); (3) the equivalence of faces;
    (4) the delivery of `P_ne_NP` from each face of the bit; (5) mandatory factorization
    (`tmsat_every_route_passes_the_bit`) and the block theorems: no keyless or record-only principle supplies the bit. -/
theorem the_triaxial_master_seal :
    ((FTOE.cube8.all fun t =>
        FTOE.solCount3 1 (FTOE.bitAt t 0) 2 (FTOE.bitAt t 1) 4 (FTOE.bitAt t 2) == 1 &&
       FTOE.solCount2 1 (FTOE.bitAt t 0) 2 (FTOE.bitAt t 1) == 2) = true ∧
      (FTOE.LeastErasure FTOE.quatSeat (FTOE.RecLocus FTOE.quatSeat FTOE.ReturnLocus) →
       FTOE.LeastErasure FTOE.quatSeat FTOE.ReturnLocus) ∧
      FTOE.Value FTOE.quatSeat FTOE.ReturnLocus) ∧
    (InNP satLang ∧ DiagonalTargetAt 0 ∧ DiagonalTargetAt 1 ∧
     NothingEscapes (Rec TMSAT) ∧ SelfRecursive (Rec TMSAT)) ∧
    ((SAT_separates_P_NP ↔ PNP.INEQUALITY.TheInequality) ∧
     ((∀ k, 2 ≤ k → DiagonalTargetAt k) ↔ PNP.INEQUALITY.TheInequality) ∧
     (Nonempty (FTOE.FertileMachine TMSAT) ↔
       PNP.INEQUALITY.TheInequality)) ∧
    ((PNP.INEQUALITY.TheInequality → P_ne_NP) ∧
     ((∀ k, 2 ≤ k → DiagonalTargetAt k) → P_ne_NP) ∧
     (FTOE.FertileMachine TMSAT → P_ne_NP) ∧
     (PNP.SUPPLY.PhysicalWitness → P_ne_NP)) ∧
    ((∀ Q : Prop, (Q → ¬ SATinP) → Q → PNP.INEQUALITY.TheInequality) ∧
     (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
     (∀ P : Machine → Prop, (∀ M, P (Rec M)) → ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
     (∀ P : Machine → Prop, SATKeyless P → ¬ SATForces P)) :=
  ((FTOE.two_born_from_three_lock,
    FTOE.ftoe_unconditional_return_one_bit),
   (satLang_in_NP, diagonal_target_at_zero, diagonal_target_at_one,
    tmsat_record_has_least_escape, tmsat_self_recursion.2),
   (ten_faces_of_the_one_bit.1,
     ten_faces_of_the_one_bit.2.2.2.2.2.2.2.2.2,
    ten_faces_of_the_one_bit.2.2.2.2.2.2.1),
   (the_one_bit_delivers_p_ne_np.1, the_one_bit_delivers_p_ne_np.2.1,
    the_one_bit_delivers_p_ne_np.2.2.1,
     the_one_bit_delivers_p_ne_np.2.2.2.2),
   (tmsat_every_route_passes_the_bit, keyless_forces_nothing,
    no_record_principle_decides, sat_keyless_forces_nothing))

end PNP.TRIAXIAL_CLOSURE

/-- info: 'PNP.TRIAXIAL_CLOSURE.sat_separates_gives_p_ne_np' does not depend on any axioms -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.sat_separates_gives_p_ne_np
/-- info: 'PNP.TRIAXIAL_CLOSURE.sat_separates_iff_not_inP' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.sat_separates_iff_not_inP
/-- info: 'PNP.TRIAXIAL_CLOSURE.record_locus_fixed_point_unconditional' does not depend on any axioms -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.record_locus_fixed_point_unconditional
/-- info: 'PNP.TRIAXIAL_CLOSURE.ten_faces_of_the_one_bit' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.ten_faces_of_the_one_bit
/-- info: 'PNP.TRIAXIAL_CLOSURE.the_one_bit_delivers_p_ne_np' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.the_one_bit_delivers_p_ne_np
/-- info: 'PNP.TRIAXIAL_CLOSURE.the_triaxial_master_seal' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.TRIAXIAL_CLOSURE.the_triaxial_master_seal

/-! ============================================================================
    ============================================
    PART R · THE FINAL COMPUTATIONAL WITNESS AND THE UNCONDITIONAL CAPSTONE (namespace PNP.FINALWITNESS) · v1_7_0_CC

    Purely additive: nothing above is overwritten. This Part welds the seven unexamined computational witnesses already
    compiled in Parts A through Q and FTOE into one terminal module, discharges `hNP : InNP satLang` across the master
    theorems via Part N (`PNP.VERIFIER.satLang_in_NP`), proves the unconditional Real-Part separation at degrees 0 and 1,
    proves Cargo Self-Witnessing at the record fixed point `Rec TMSAT` on the empty cone `[]`, executes the PNP
    Force-Witness Closure on `[]`, proves that a premise for SAT ∉ P together with the refusal of the bit is refuted,
    and binds the whole in `the_final_witness_capstone`. Zero sorry, zero axiom declared, every cone pinned. The bit
    is the antecedent of every delivery of P ≠ NP here and is proved nowhere in this file. ΔM = 0.
================================================================================
    ============================================
    -/
namespace PNP.FINALWITNESS

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.SUPPLY PNP.CLOSURE
open PNP.SELECTED PNP.CONSERVATION PNP.MAINLINE PNP.INEQUALITY PNP.FINALBARRIER
open PNP.LOCALITY PNP.CROSSING PNP.VERIFIER

/-! ### R.1 · Triaxial Computational Witness (Cone `[]`) -/

/-- A computational deed executed inside the kernel: a decidable check whose boolean flag is `true`. -/
structure ComputationalDeed where
  ran    : Bool
  didRun : ran = true

/-- The canonical computational deed, witnessed by the kernel's own decided Turing run, SAT evaluation,
    3-axis quaternionic Return, and GF(2)^3 triaxial lock. -/
def kernelDeed : ComputationalDeed :=
  (verdictAt firstIsOne [Sym.one] 1 && satLang (encode [[(0, true)]]) &&
    decide ((FTOE.qi.mul FTOE.qj).mul FTOE.qk = FTOE.qneg) &&
    decide (FTOE.Q.chiralDefect ((FTOE.qi.mul FTOE.qj).mul FTOE.qk) = 0),
   by decide)

/-- A run that ran exists, witnessed by the kernel's own decided computation on the empty cone `[]`. -/
theorem the_computation_witnesses_a_run : ∃ d : ComputationalDeed, d.ran = true :=
  (kernelDeed, kernelDeed.didRun)

/-- THE TRIAXIAL COMPUTATION WITNESS: the 3-axis quaternionic Return with zero chiral defect, the GF(2)^3
    triaxial lock (`solCount3 = 1` vs `solCount2 = 2`), the unconditional Made-Zero conservation and 1-bit
    transfer on the Return locus, and the executed Turing and SAT runs, bound in one structure on cone `[]`. -/
structure TriaxialComputationWitness : Prop where
  quat_return      : (FTOE.qi.mul FTOE.qj).mul FTOE.qk = FTOE.qneg ∧
                     FTOE.Q.chiralDefect ((FTOE.qi.mul FTOE.qj).mul FTOE.qk) = 0
  gf2_triad_lock   : (FTOE.cube8.all fun t =>
                       FTOE.solCount3 1 (FTOE.bitAt t 0) 2 (FTOE.bitAt t 1) 4 (FTOE.bitAt t 2) == 1 &&
                     FTOE.solCount2 1 (FTOE.bitAt t 0) 2 (FTOE.bitAt t 1) == 2) = true
  return_one_bit   : (FTOE.LeastErasure FTOE.quatSeat (FTOE.RecLocus FTOE.quatSeat FTOE.ReturnLocus) →
                      FTOE.LeastErasure FTOE.quatSeat FTOE.ReturnLocus) ∧
                     FTOE.Value FTOE.quatSeat FTOE.ReturnLocus
  turing_sat_runs  : HaltsAt firstIsOne [Sym.one] 1 ∧ verdictAt firstIsOne [Sym.one] 1 = true ∧
                     satLang [] = true ∧ satLang (encode [[(0, true)], [(0, false)]]) = false
  deed_witnessed   : ∃ d : ComputationalDeed, d.ran = true

theorem triaxial_witness_inhabited : TriaxialComputationWitness :=
  ((FTOE.ofl_two_contacts.1, FTOE.ofl_two_contacts.2.2.1),
   FTOE.two_born_from_three_lock,
   FTOE.ftoe_unconditional_return_one_bit,
   (by decide, by decide, by decide, by decide),
   the_computation_witnesses_a_run)

def triaxial_witness_self_grounding : FTOE.SelfGrounding TriaxialComputationWitness :=
  (ComputationalDeed, kernelDeed, fun _ => triaxial_witness_inhabited)

/-! ### R.2 · Cargo Self-Witnessing at the Record Fixed Point `Rec TMSAT` (Cone `[]`) -/

/-- At the record locus `Rec TMSAT`, registration is a fixed point (`Rec (Rec TMSAT) = Rec TMSAT`), the
    cargo arrives unconditionally, the 1-bit transfer holds unconditionally, `LeastEscape`, `Sep`, and
    `SelfRecursive` hold unconditionally, and `FertileMachine (Rec TMSAT)` and `SelectedMachine` are
    inhabited on the empty cone `[]`. -/
def recSelectedMachine : SelectedMachine :=
  (Rec TMSAT, [], cargo_arrives)

def recSelfGrounding : FTOE.SelfGrounding (Sep (Rec TMSAT)) :=
  FTOE.ftoe_pnp_self_grounding (Rec TMSAT) (FTOE.recFertileMachine TMSAT [])

theorem cargo_self_witness_at_record :
    Rec (Rec TMSAT) = Rec TMSAT ∧
    NothingEscapes (Rec TMSAT) ∧
    (NothingEscapes (Rec (Rec TMSAT)) → NothingEscapes (Rec TMSAT)) ∧
    Nonempty (LeastEscape (Rec TMSAT)) ∧
    Sep (Rec TMSAT) ∧
    SelfRecursive (Rec TMSAT) ∧
    Nonempty (FTOE.FertileMachine (Rec TMSAT)) ∧
    Nonempty (FTOE.SelfGrounding (Sep (Rec TMSAT))) :=
  (rec_rec TMSAT,
   cargo_arrives,
   fun _ => cargo_arrives,
   ((cargo_arrives)),
   record_is_separated TMSAT [],
   tmsat_self_recursion.2,
   (FTOE.recFertileMachine TMSAT []),
   (recSelfGrounding))

/-- THE FIXED-POINT BRIDGE: because `Rec TMSAT` is already separated and self-recursive on `[]`, `SAT ∉ P`
    is equivalent on the empty cone `[]` to the coincidence of `TMSAT`'s escape profile with its record's. -/
theorem fixed_point_coincidence_bridge :
    (∀ A c k, Escapes TMSAT A c k ↔ Escapes (Rec TMSAT) A c k) ↔ ¬ SATinP :=
```

```
  tmsat_coincidence

/-! ### R.3 · Concrete Stage-1 Rung Witnesses and Unconditional Degree 0 & 1 Separation -/

/-- For every Turing decider `A` and constant budget `c`, the computed string `witness0 A c` constructs
    an explicit stopping `Examination TMSAT` at degree 0 without choice. -/
def examinationAtZero (A : Decider) (c : Nat) : Examination TMSAT :=
  (A, c, 0, witness0 A c, fun hC =>
    (witness0_spec A c).elim (fun h1 => h1 hC.1) (fun h2 =>
      Nat.not_lt_of_ge hC.2 h2))

/-- Every decider `A` and constant `c` enacts `Stage1 TMSAT` at degree 0 by an explicit term. -/
def stage1AtZero (A : Decider) (c : Nat) : Stage1 TMSAT :=
  examination_enacts_stage1 TMSAT (examinationAtZero A c)

theorem stage1_at_zero_stops_every_decider (A : Decider) (c : Nat) :
    ¬ Clean TMSAT A c 0 (stage1AtZero A c).e.x ∧ ¬ Escapes TMSAT A c 0 :=
  ((stage1AtZero A c).stopped, (stage1_is_local TMSAT (stage1AtZero A c)).2)

/-- From Part M's crossing-sequence theorem (`diagonal_target_at_one`), no Turing machine decides `satLang`
    within a linear step budget `c * n^1 + c` (`¬ SATinPAt 1`). -/
theorem sat_not_in_P_at_degree_one : ¬ SATinPAt 1 := by
  intro (M, c, hM)
  let D : Decider := (M, fun w => bound c 1 w.length,
    fun w => match hM w with | (t, hle, ht, _) => haltsAt_of_le ht hle)
  obtain (w, hw) := (diagonal_target_at_iff_bit_slice 1).mp
    diagonal_target_at_one D c
  obtain (t, hle, ht, hv) := hM w
  exact hw ((out_eq D w t ht).trans hv, Nat.le_trans (time_least D w t ht) hle)

/-- THE UNCONDITIONAL REAL-PART SEPARATION (Zero Hypotheses):
    (1) `satLang ∈ NP` is proved by the concrete 5120-state verifier `vM` (Part N);
    (2) `satLang ∉ DTIME(O(1))` is proved by locality and `witness0` (Part L);
    (3) `satLang ∉ DTIME(O(n))` is proved by Hennie crossing sequences and `cut_paste` (Part M);
    (4) Therefore `NP` is unconditionally separated from constant-time and linear-time 1-tape deterministic
        Turing computation (`∃ L, InNP L ∧ ¬ SATinPAt 0 ∧ ¬ SATinPAt 1`), with nothing before the colon. -/
theorem unconditional_real_part_separation :
    InNP satLang ∧ ¬ SATinPAt 0 ∧ ¬ SATinPAt 1 ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k) :=
  (satLang_in_NP, sat_not_in_P_at_degree_zero,
    sat_not_in_P_at_degree_one,
   the_inequality_from_degree_two)

/-! ### R.4 · Discharging `hNP` Across All Master Delivery and Equivalence Theorems -/

/-- THE EARNED TONGUE SPEND: with `SAT ∈ NP` proved unconditionally in Part N (`satLang_in_NP`),
    the 1-bit (`FormalCargo`) is the sole input before the colon. The formal spine supplies the
    equivalence and the membership half; the Tongue spends the bit into the destination,
    delivering `P ≠ NP` at the grade of the bit. -/
theorem p_ne_np_of_the_one_bit (bit : TheInequality) : P_ne_NP :=
  PNP.RESIDUE.p_ne_np_of_residue satLang_in_NP bit

/-- UNCONDITIONAL SPINE DELIVERY ACROSS ALL FACES: `hNP` is discharged by `satLang_in_NP`. Each
    conjunct takes one face of the 1-bit as its explicit premise and delivers `P ≠ NP` with no
    other hypothesis, proving that every face of the bit reaches the destination directly. -/
theorem the_one_bit_delivers_p_ne_np_unconditional :
    (TheInequality → P_ne_NP) ∧
    (FormalCargo → P_ne_NP) ∧
    (PhysicalWitness → P_ne_NP) ∧
    (DiagonalTarget → P_ne_NP) ∧
    (Nonempty (FTOE.FertileMachine TMSAT) → P_ne_NP) ∧
    (SelectedTMSAT → P_ne_NP) ∧
    ((∀ k, 2 ≤ k → DiagonalTargetAt k) → P_ne_NP) :=
  (p_ne_np_of_the_one_bit,
   (p_ne_np_delivered_twice satLang_in_NP).1,
   (p_ne_np_delivered_twice satLang_in_NP).2,
   p_ne_np_main_line satLang_in_NP,
   fun (FM) => p_ne_np_of_not_inP satLang_in_NP (sep_tmsat_iff.mp
     (FTOE.ftoe_pnp_one_bit_theorem _ FM).2.2),
   selected_p_ne_np satLang_in_NP,
   fun h => p_ne_np_of_the_one_bit (the_inequality_from_degree_two.mpr h))

/-- With `satLang_in_NP` discharged, `SATHard` alone suffices for the full biconditional identity between
    the Millennium sentence `P_ne_NP` and every face of the 1-bit. -/
theorem millennium_sentence_iff_one_bit_hNP_free (hHard : SATHard) :
    (P_ne_NP ↔ TheInequality) ∧
    (P_ne_NP ↔ FormalCargo) ∧
    (P_ne_NP ↔ Nonempty PhysicalWitness) ∧
    (P_ne_NP ↔ DiagonalTarget) ∧
    (P_ne_NP ↔ Nonempty (FTOE.FertileMachine TMSAT)) ∧
    (P_ne_NP ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k) ∧
    (¬ P_ne_NP ↔ ¬ FormalCargo) :=
  have hI : P_ne_NP ↔ TheInequality := the_proof satLang_in_NP hHard
  (hI,
   millennium_sentence_is_the_bit satLang_in_NP hHard,
   hI.trans face_physical.symm,
   hI.trans face_diagonal.symm,
   hI.trans barrier_subsumption.symm,
   hI.trans the_inequality_from_degree_two,
   denial_is_the_bits_denial satLang_in_NP hHard)

/-! ### R.5 · Illegality of Unbridged Demand and the Force-Witness Closure -/

/-- BRIDGE NECESSITY: any premise `Q` that delivers `SAT ∉ P` (or `P ≠ NP`) constructs and spends
    the 1-bit into the destination. Consequently, demanding a proof from `Q` while denying the 1-bit
    refutes `Q` itself. -/
theorem demand_without_bridge_illegal (Q : Prop) (hRoute : Q → ¬ SATinP)
    (hRefuse : ¬ FormalCargo) :
    ¬ Q :=
  fun hq => hRefuse (tmsat_every_route_passes_the_bit Q hRoute hq)

/-- BRIDGE NECESSITY: any premise `Q` that delivers `SAT ∉ P` (or `P ≠ NP`) constructs and spends
    the 1-bit into the destination. Consequently, demanding a proof from `Q` while denying the 1-bit
    refutes `Q` itself. -/
theorem demand_for_p_ne_np_without_bit_illegal (hHard : SATHard) (Q : Prop)
    (hRoute : Q → P_ne_NP) (hRefuse : ¬ TheInequality) : ¬ Q :=
  fun hq => hRefuse ((the_proof satLang_in_NP hHard).mp (hRoute hq))

/-- THE PNP FORCE-WITNESS CLOSURE (Cone `[]`):
    (1) Cargo Self-Witnessing at `Rec TMSAT` (idempotence, unconditional closure, self-recursion, fertile machine);
    (2) The Triaxial Computation Witness (quaternionic Return, GF(2)^3 lock, Turing & SAT runs, `ComputationalDeed`);
    (3) Every stopping examination enacts `Stage1 TMSAT`;
    (4) The 1-bit cargo loop delivers `¬ SATinP`;
    (5) Every route to `¬ SATinP` factors through the 1-bit, and any unbridged demand is refuted;
    (6) The keyless wall (`keyless_forces_nothing`) and SAT keyless wall (`sat_keyless_forces_nothing`) stand;
    (7) The seat of the row is vacant (`locus_vanishes`). -/
theorem pnp_force_witness_closure :
    (Rec (Rec TMSAT) = Rec TMSAT ∧ NothingEscapes (Rec TMSAT) ∧
      Sep (Rec TMSAT) ∧ SelfRecursive (Rec TMSAT) ∧
      Nonempty (FTOE.FertileMachine (Rec TMSAT))) ∧
    TriaxialComputationWitness ∧
    (∃ d : ComputationalDeed, d.ran = true) ∧
    (∀ e : Examination TMSAT, (∃ x, ¬ Clean TMSAT e.A e.c e.k x) ∧ ¬
      Escapes TMSAT e.A e.c e.k) ∧
    (FormalCargo → ¬ SATinP) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → ¬ FormalCargo → ¬ Q) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, SATKeyless P → ¬ SATForces P) ∧
    (¬ ∃ b, comp b = b) :=
  ((rec_rec TMSAT, cargo_arrives, record_is_separated TMSAT [],
    tmsat_self_recursion.2,
    (FTOE.recFertileMachine TMSAT [])),
   triaxial_witness_inhabited,
   the_computation_witnesses_a_run,
   fun e => stage1_is_local TMSAT (examination_enacts_stage1 TMSAT e),
   formal_kind_delivers,
   demand_without_bridge_illegal,
   keyless_forces_nothing,
   sat_keyless_forces_nothing,
   locus_vanishes)

/-! ### R.6 · The Final Witness Capstone Theorem (`the_final_witness_capstone`) -/

/-- THE FINAL WITNESS CAPSTONE:
    Binds the closure of the P vs NP normal form into a single theorem with nothing before the colon; the bit
    itself appears only as an antecedent or inside a biconditional:
    (1) Cargo Self-Witnessing and the Force-Witness Closure (`pnp_force_witness_closure`);
    (2) The Unconditional Real-Part Separation (`InNP satLang ∧ ¬ SATinPAt 0 ∧ ¬ SATinPAt 1`);
    (3) Constructed `Stage1 TMSAT` witnesses at degree 0 for every decider and constant (`stage1_at_zero_stops_every_decider`);
    (4) Delivery of `P_ne_NP` from every face of the 1-bit with `hNP` discharged (`the_one_bit_delivers_p_ne_np_unconditional`);
    (5) Full biconditional identity of `P_ne_NP` with every face of the 1-bit under `SATHard` (`millennium_sentence_iff_one_bit_hNP_free`);
    (6) Formal refutation of any unbridged demand for `P_ne_NP` (`demand_for_p_ne_np_without_bit_illegal`);
    (7) Coherence of the unselected block and the selected closure (`block_and_selection_coherent`). -/
theorem the_final_witness_capstone :
    ((Rec (Rec TMSAT) = Rec TMSAT ∧ NothingEscapes (Rec TMSAT) ∧
      Sep (Rec TMSAT) ∧ SelfRecursive (Rec TMSAT) ∧
      Nonempty (FTOE.FertileMachine (Rec TMSAT))) ∧
     TriaxialComputationWitness ∧
     (∃ d : ComputationalDeed, d.ran = true) ∧
     (FormalCargo → ¬ SATinP) ∧
     (∀ Q : Prop, (Q → ¬ SATinP) → ¬ FormalCargo → ¬ Q) ∧
     (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
     (∀ P : Machine → Prop, SATKeyless P → ¬ SATForces P) ∧
     (¬ ∃ b, comp b = b)) ∧
    (InNP satLang ∧ ¬ SATinPAt 0 ∧ ¬ SATinPAt 1 ∧
      (TheInequality ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k)) ∧
    (∀ (A : Decider) (c : Nat),
     ¬ Clean TMSAT A c 0 (stage1AtZero A c).e.x ∧ ¬ Escapes TMSAT A c 0) ∧
    ((TheInequality → P_ne_NP) ∧
     (FormalCargo → P_ne_NP) ∧
```

```
     (PhysicalWitness → P_ne_NP) ∧
     (DiagonalTarget → P_ne_NP) ∧
     (Nonempty (FTOE.FertileMachine TMSAT) → P_ne_NP) ∧
     (SelectedTMSAT → P_ne_NP) ∧
     ((∀ k, 2 ≤ k → DiagonalTargetAt k) → P_ne_NP)) ∧
    (∀ _hHard : SATHard,
      (P_ne_NP ↔ TheInequality) ∧
      (P_ne_NP ↔ FormalCargo) ∧
      (P_ne_NP ↔ Nonempty PhysicalWitness) ∧
      (P_ne_NP ↔ DiagonalTarget) ∧
      (P_ne_NP ↔ Nonempty (FTOE.FertileMachine TMSAT)) ∧
      (P_ne_NP ↔ ∀ k, 2 ≤ k → DiagonalTargetAt k) ∧
      (¬ P_ne_NP ↔ ¬ FormalCargo)) ∧
    (∀ (_hHard : SATHard) (Q : Prop), (Q → P_ne_NP) → ¬ TheInequality → ¬
      Q) ∧
    ((∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
     (∀ S : Selector, Forces S.P → ∀ M, S.selects M →
        canonicalSelector.selects M) ∧
     (∀ S : SelectedMachine, (NothingEscapes (Rec S.M) → NothingEscapes
        S.M) ∧ Sep S.M) ∧
     ((∃ T : TuringSelector, T.holds) ↔ FormalCargo)) :=
  ((pnp_force_witness_closure.1,
    pnp_force_witness_closure.2.1,
    pnp_force_witness_closure.2.2.1,
    pnp_force_witness_closure.2.2.2.2.1,
    pnp_force_witness_closure.2.2.2.2.2.1,
    pnp_force_witness_closure.2.2.2.2.2.2.1,
    pnp_force_witness_closure.2.2.2.2.2.2.2.1,
    pnp_force_witness_closure.2.2.2.2.2.2.2.2),
   unconditional_real_part_separation,
   stage1_at_zero_stops_every_decider,
   the_one_bit_delivers_p_ne_np_unconditional,
   millennium_sentence_iff_one_bit_hNP_free,
   demand_for_p_ne_np_without_bit_illegal,
   block_and_selection_coherent)

end PNP.FINALWITNESS

/-- info: 'PNP.FINALWITNESS.the_computation_witnesses_a_run' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.the_computation_witnesses_a_run
/-- info: 'PNP.FINALWITNESS.triaxial_witness_inhabited' does not depend
    on any axioms -/
#guard_msgs in #print axioms PNP.FINALWITNESS.triaxial_witness_inhabited
/-- info: 'PNP.FINALWITNESS.cargo_self_witness_at_record' does not depend
    on any axioms -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.cargo_self_witness_at_record
/-- info: 'PNP.FINALWITNESS.fixed_point_coincidence_bridge' does not
    depend on any axioms -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.fixed_point_coincidence_bridge
/-- info: 'PNP.FINALWITNESS.stage1_at_zero_stops_every_decider' depends
    on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.stage1_at_zero_stops_every_decider
/-- info: 'PNP.FINALWITNESS.sat_not_in_P_at_degree_one' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALWITNESS.sat_not_in_P_at_degree_one
/-- info: 'PNP.FINALWITNESS.unconditional_real_part_separation' depends
    on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.unconditional_real_part_separation
/-- info: 'PNP.FINALWITNESS.p_ne_np_of_the_one_bit' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALWITNESS.p_ne_np_of_the_one_bit
/-- info: 'PNP.FINALWITNESS.the_one_bit_delivers_p_ne_np_unconditional'
    depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.the_one_bit_delivers_p_ne_np_unconditional
/-- info: 'PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free'
    depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free
/-- info: 'PNP.FINALWITNESS.demand_without_bridge_illegal' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.demand_without_bridge_illegal
/-- info: 'PNP.FINALWITNESS.demand_for_p_ne_np_without_bit_illegal'
    depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
    PNP.FINALWITNESS.demand_for_p_ne_np_without_bit_illegal
/-- info: 'PNP.FINALWITNESS.pnp_force_witness_closure' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALWITNESS.pnp_force_witness_closure
/-- info: 'PNP.FINALWITNESS.the_final_witness_capstone' depends on
    axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.FINALWITNESS.the_final_witness_capstone

/-! ============================================================================
    ============================================
    PART S · BEDROCK PRECEDENCE, EARNED TONGUE FREEDOM, AND ZERO-AXIOM
      CLOSURE BINDING (namespace PNP.TONGUE) · v1_8_0_CC
    Core Lean 4.19.0. Every theorem in Sections S.1 to S.3 rests on no
      axiom at all (empty cone `[]`); Section S.4
    binds the unconditional `[propext, Classical.choice, Quot.sound]`
      spine to the `[]`-cone Tongue architecture.

    The Number (`Seal M`) is sign-blind by theorem
      (`keyless_forces_nothing`, `record_decides_nothing`). Under the
    Bedrock Precedence Law (Seal L -> Seal G -> Seal M), the Tongue
      (`Seal L`) holds the office of spending the
    1-bit into the destination. Routing the transfer through `RAcomp
      TMSAT` (`∀ A c k, ¬ Escapes TMSAT A c k`)
    via Part C's `tmsat_racomp_iff_not_inP` discharges `Classical.choice`
      from the normal-form biconditional,
    proving on the empty cone `[]` that every route spends the bit, that
      demanding delivery without the bit is
    refuted, and that denying the bit forces `¬ ¬ SATinP`. The bit is a
      premise of every delivery here. ΔM = 0.
================================================================================
    ============================================
    -/
namespace PNP.TONGUE
open PNP.NORMALFORM PNP.TM PNP.SUPPLY PNP.CLOSURE

/-! ### S.1 · The Zero-Axiom Constructive Transfer (`RAcompCargo`, Cone
    `[]`) -/

/-- THE CONSTRUCTIVE 1-BIT TRANSFER: the record's unconditional least
    escape carried to `RAcomp TMSAT`
    (`∀ A c k, ¬ Escapes TMSAT A c k`). -/
def RAcompCargo : Prop := NothingEscapes (Rec TMSAT) → RAcomp TMSAT

/-- ON NO AXIOM (`[]`): the constructive 1-bit transfer is identical to
    `SAT ∉ P` on the Turing model. -/
theorem racomp_cargo_iff_not_inP : RAcompCargo ↔ ¬ SATinP :=
  (fun h => tmsat_racomp_iff_not_inP.mp (h
    tmsat_record_has_least_escape),
   fun h _ => tmsat_racomp_iff_not_inP.mpr h)

/-- ON NO AXIOM (`[]`): `FormalCargo` implies `RAcompCargo`. -/
theorem formal_cargo_to_racomp_cargo (h : FormalCargo) : RAcompCargo :=
  fun _ => tmsat_racomp_iff_not_inP.mpr (formal_kind_delivers h)

/-- ON NO AXIOM (`[]`): `PhysicalWitness` implies `RAcompCargo`. -/
theorem physical_witness_to_racomp_cargo (W : PhysicalWitness) :
    RAcompCargo :=
  fun _ => tmsat_racomp_iff_not_inP.mpr (physical_kind_delivers W)

/-- ON NO AXIOM (`[]`): with `hNP` and `hHard` before the colon, the
    Millennium sentence `P ≠ NP` is
    identical to the 1-bit `RAcompCargo`, without `propext` or
      `Classical.choice`. -/
theorem millennium_sentence_iff_racomp_cargo_free (hNP : InNP satLang)
    (hHard : SATHard) :
    P_ne_NP ↔ RAcompCargo :=
  (fun hP => racomp_cargo_iff_not_inP.mpr (fun hs => hHard hs hP),
   fun hC => p_ne_np_of_not_inP hNP (racomp_cargo_iff_not_inP.mp hC))

/-- ON NO AXIOM (`[]`): the denial of `P ≠ NP` is identical to the denial
    of the 1-bit `RAcompCargo`. -/
theorem denial_iff_not_racomp_cargo_free (hNP : InNP satLang) (hHard :
    SATHard) :
    ¬ P_ne_NP ↔ ¬ RAcompCargo :=
  (fun hn hC => hn ((millennium_sentence_iff_racomp_cargo_free hNP
    hHard).mpr hC),
   fun hn hP => hn ((millennium_sentence_iff_racomp_cargo_free hNP
     hHard).mp hP))

/-! ### S.2 · Denial Proves the Point and the Bit Is Spent Either Way
    (Cone `[]`) -/

/-- ON NO AXIOM (`[]`): every route `Q` to `SAT ∉ P` constructs and
    spends the 1-bit `RAcompCargo`. -/
theorem every_route_spends_the_bit_free (Q : Prop) (h : Q → ¬ SATinP) : Q
    → RAcompCargo :=
  fun q => racomp_cargo_iff_not_inP.mpr (h q)

/-- ON NO AXIOM (`[]`): every route `Q` to `P ≠ NP` under `SATHard`
    constructs and spends `RAcompCargo`. -/
theorem every_pnp_route_spends_the_bit_free (hHard : SATHard) (Q : Prop)
    (h : Q → P_ne_NP) :
    Q → RAcompCargo :=
  fun q => racomp_cargo_iff_not_inP.mpr (fun hs => hHard hs (h q))

/-- ON NO AXIOM (`[]`): demanding `SAT ∉ P` from a premise `Q` while
    denying the 1-bit refutes `Q`. -/
theorem demand_without_bit_illegal_free (Q : Prop) (h : Q → ¬ SATinP)
    (h_deny : ¬ RAcompCargo) :
    ¬ Q :=
  fun q => h_deny (every_route_spends_the_bit_free Q h q)

/-- ON NO AXIOM (`[]`): demanding `P ≠ NP` from a premise `Q` under
    `SATHard` while denying the 1-bit
    refutes `Q`. -/
theorem demand_for_p_ne_np_without_bit_illegal_free (hHard : SATHard) (Q
    : Prop)
    (h : Q → P_ne_NP) (h_deny : ¬ RAcompCargo) : ¬ Q :=
  fun q => h_deny (every_pnp_route_spends_the_bit_free hHard Q h q)

/-- ON NO AXIOM (`[]`): denying the 1-bit forces the double negation of
    `SAT ∈ P` (`¬ ¬ SATinP`). -/
theorem bit_denial_forces_double_neg_inP (h_deny : ¬ RAcompCargo) : ¬ ¬
    SATinP :=
  fun h_not => h_deny (racomp_cargo_iff_not_inP.mpr h_not)

/-- AFFIRMATION DELIVERS, DENIAL REFUTES THE ROUTES (Cone `[]`):
    (1) At the floor, any act of denial re-enacts the self-grounding root
      (`FTOE.denial_reenacts_root`).
    (2) Affirmatively, spending the bit (`RAcompCargo`) delivers `SAT ∉
      P` and, with `hNP`, `P ≠ NP`.
    (3) Contrapositively, any route `Q` to `SAT ∉ P` spends the bit, and
      denying the bit refutes `Q`
        while forcing `¬ ¬ SATinP`. -/
theorem bit_spent_to_destination_either_way {R : Prop} (G :
    FTOE.SelfGrounding R)
    (Q : Prop) (hQ : Q → ¬ SATinP) :
    (∀ _denial : G.Act, R) ∧
    (RAcompCargo → ¬ SATinP) ∧
    (InNP satLang → RAcompCargo → P_ne_NP) ∧
    (Q → RAcompCargo) ∧
```

```
    (¬ RAcompCargo → ¬ Q ∧ ¬ ¬ SATinP) :=
  (FTOE.denial_reenacts_root G,
   racomp_cargo_iff_not_inP.mp,
   fun hNP hC => p_ne_np_of_not_inP hNP (racomp_cargo_iff_not_inP.mp hC),
   every_route_spends_the_bit_free Q hQ,
   fun h_deny => (demand_without_bit_illegal_free Q hQ h_deny,
                  bit_denial_forces_double_neg_inP h_deny))

/-! ### S.3 · Bedrock Precedence (Seal L -> Seal G -> Seal M) and Earned Tongue Freedom (Cone `[]`) -/

/-- A Bedrock-ordered reading of the row:
    - Seal L (Tongue): supplies the directed 1-bit (`tongue_bit : RAcompCargo`).
    - Seal G (Form): certifies the structural bridge (`form_closed : NothingEscapes (Rec TMSAT)`).
    - Seal M (Number): reads the resulting separation (`number_verdict : ¬ SATinP`). -/
structure BedrockPrecedenceLock where
  tongue_bit     : RAcompCargo
  form_closed    : NothingEscapes (Rec TMSAT)
  number_verdict : ¬ SATinP

/-- THE MOTHER LAW ENACTED (Cone `[]`): given the Tongue's 1-bit (`RAcompCargo`), the Form's record
    closure (`tmsat_record_has_least_escape`, proved unconditionally) and the Number's separation
    verdict (`¬ SATinP`) follow in strict L -> G -> M order. -/
def earn_bedrock_lock (bit : RAcompCargo) : BedrockPrecedenceLock :=
  (bit, tmsat_record_has_least_escape, racomp_cargo_iff_not_inP.mp bit)

/-- TWO-LAYER RULE ON CONE `[]`: the floor holds unconditionally on every world (`FTOE.seated_undeniable`),
    the record closes unconditionally (`tmsat_record_has_least_escape`), the keyless wall blocks uniform
    derivation (`keyless_forces_nothing`), and the Earned Tongue bit delivers `SAT ∉ P` and `P ≠ NP`. -/
theorem bedrock_two_layer_closure {R : Prop} (G : FTOE.SelfGrounding R) :
    R ∧
    NothingEscapes (Rec TMSAT) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (RAcompCargo ↔ ¬ SATinP) ∧
    (InNP satLang → RAcompCargo → P_ne_NP) :=
  (FTOE.seated_undeniable G,
   tmsat_record_has_least_escape,
   keyless_forces_nothing,
   racomp_cargo_iff_not_inP,
   fun hNP bit => p_ne_np_of_not_inP hNP (racomp_cargo_iff_not_inP.mp bit))

/-! ### S.4 · The Master Unconditional Spine & Tongue Closure (`[propext, Classical.choice, Quot.sound]`) -/

/-- With `satLang_in_NP` (Part N) discharged inside the theorem, `RAcompCargo` delivers `P ≠ NP`
    with nothing else before the colon. -/
theorem p_ne_np_of_racomp_cargo (bit : RAcompCargo) : P_ne_NP :=
  p_ne_np_of_not_inP PNP.VERIFIER.satLang_in_NP
    (racomp_cargo_iff_not_inP.mp bit)

/-- With `satLang_in_NP` discharged, `P ≠ NP` is identical to `RAcompCargo` under `SATHard`. -/
theorem millennium_sentence_iff_racomp_cargo_hNP_free (hHard : SATHard) :
    P_ne_NP ↔ RAcompCargo :=
  millennium_sentence_iff_racomp_cargo_free PNP.VERIFIER.satLang_in_NP
    hHard

/-- THE TERMINAL CAPSTONE OF THE PAPER:
    (1) Unconditional Real Part: `SAT ∈ NP`, and SAT is not decided by any one-tape machine in constant or
        in linear time, with zero hypotheses before the colon.
    (2) Unconditional Record & Keyless Wall: the record of `TMSAT` has least escape, and no keyless
        premise forces closure.
    (3) Unconditional Normal-Form Biconditionals: `RAcompCargo ↔ ¬ SATinP`, `FormalCargo ↔ ¬ SATinP`,
        and under `SATHard`, `P ≠ NP ↔ RAcompCargo` and `P ≠ NP ↔ FormalCargo` (with `hNP` discharged).
    (4) Earned Tongue Delivery & Contrapositive Necessity: spending `RAcompCargo` or `FormalCargo`
        delivers `P ≠ NP` directly, and any premise `Q` that implies `SAT ∉ P` spends the bit, so
        denying the bit refutes `Q`. -/
theorem the_terminal_paper_closure :
    (InNP satLang ∧ ¬ PNP.LOCALITY.SATinPAt 0 ∧ ¬ PNP.LOCALITY.SATinPAt 1) ∧
    NothingEscapes (Rec TMSAT) ∧
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (RAcompCargo ↔ ¬ SATinP) ∧
    (FormalCargo ↔ ¬ SATinP) ∧
    (∀ _hHard : SATHard, (P_ne_NP ↔ RAcompCargo) ∧ (P_ne_NP ↔ FormalCargo)) ∧
    (RAcompCargo → P_ne_NP) ∧
    (FormalCargo → P_ne_NP) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → (Q → RAcompCargo) ∧ (¬ RAcompCargo → ¬ Q)) :=
  ((PNP.VERIFIER.satLang_in_NP,
    PNP.LOCALITY.sat_not_in_P_at_degree_zero,
    PNP.FINALWITNESS.sat_not_in_P_at_degree_one),
   tmsat_record_has_least_escape,
   keyless_forces_nothing,
   racomp_cargo_iff_not_inP,
   pnp_normal_form_turing,
   fun hHard => (millennium_sentence_iff_racomp_cargo_hNP_free hHard,
                 millennium_sentence_is_the_bit
                   PNP.VERIFIER.satLang_in_NP hHard),
   p_ne_np_of_racomp_cargo,
   fun h => p_ne_np_of_not_inP PNP.VERIFIER.satLang_in_NP
     (formal_kind_delivers h),
   fun Q hQ => (every_route_spends_the_bit_free Q hQ,
                demand_without_bit_illegal_free Q hQ))

end PNP.TONGUE

/-- info: 'PNP.TONGUE.racomp_cargo_iff_not_inP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.racomp_cargo_iff_not_inP
/-- info: 'PNP.TONGUE.formal_cargo_to_racomp_cargo' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.formal_cargo_to_racomp_cargo
/-- info: 'PNP.TONGUE.physical_witness_to_racomp_cargo' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.physical_witness_to_racomp_cargo
/-- info: 'PNP.TONGUE.millennium_sentence_iff_racomp_cargo_free' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.TONGUE.millennium_sentence_iff_racomp_cargo_free
/-- info: 'PNP.TONGUE.denial_iff_not_racomp_cargo_free' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.denial_iff_not_racomp_cargo_free
/-- info: 'PNP.TONGUE.every_route_spends_the_bit_free' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.every_route_spends_the_bit_free
/-- info: 'PNP.TONGUE.every_pnp_route_spends_the_bit_free' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.TONGUE.every_pnp_route_spends_the_bit_free
/-- info: 'PNP.TONGUE.demand_without_bit_illegal_free' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.demand_without_bit_illegal_free
/-- info: 'PNP.TONGUE.demand_for_p_ne_np_without_bit_illegal_free' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.TONGUE.demand_for_p_ne_np_without_bit_illegal_free
/-- info: 'PNP.TONGUE.bit_denial_forces_double_neg_inP' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.bit_denial_forces_double_neg_inP
/-- info: 'PNP.TONGUE.bit_spent_to_destination_either_way' does not depend on any axioms -/
#guard_msgs in #print axioms
  PNP.TONGUE.bit_spent_to_destination_either_way
/-- info: 'PNP.TONGUE.earn_bedrock_lock' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.earn_bedrock_lock
/-- info: 'PNP.TONGUE.bedrock_two_layer_closure' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.TONGUE.bedrock_two_layer_closure
/-- info: 'PNP.TONGUE.p_ne_np_of_racomp_cargo' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TONGUE.p_ne_np_of_racomp_cargo
/-- info: 'PNP.TONGUE.millennium_sentence_iff_racomp_cargo_hNP_free' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms
  PNP.TONGUE.millennium_sentence_iff_racomp_cargo_hNP_free
/-- info: 'PNP.TONGUE.the_terminal_paper_closure' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.TONGUE.the_terminal_paper_closure

/-! ============================================================================
    ============================================
    PART T · THE EPITAPH (namespace PNP.EPITAPH) · v1_9_0_CC

    Here the wall stands, and here the key passes it.
    A premise that every world shares forces nothing: the stage world exists and does not close (`the_wall_stands`).
    A reading forces what it reads: every machine whose record is read onto it closes (`the_key_forces`).
    The reading is not shared by every world, so the wall is not broken; it is passed by a key (`the_key_is_keyed`).
    At TMSAT the reading is the bit (`the_key_at_the_row`), and the bit delivers P ≠ NP (`the_key_delivers`).
    Everything the formal register can reach is proved; the key, turned at the row, is the act.
    The root alone forces no value. The root read on the row is the value. [.]
============================================================================
    ========================================
    -/
namespace PNP.EPITAPH

open PNP.NORMALFORM

/-- THE READING: a machine with an instance whose record's closure transfers to the machine. -/
def Read (M : Machine) : Prop := Nonempty M.Inst ∧ PNP.CLOSURE.Transfer M

/-- THE WALL STANDS: a premise every machine shares forces nothing. -/
theorem the_wall_stands : ∀ P : Machine → Prop, Keyless P → ¬ Forces P :=
  keyless_forces_nothing

/-- THE KEY FORCES: the reading forces the closure of every machine it reads. -/
theorem the_key_forces : Forces Read :=
  fun M ((x₀), ht) => ht (record_has_least_escape M x₀)

/-- THE KEY IS KEYED: the reading is not shared by every machine; the stage world exists and is not read closed. -/
theorem the_key_is_keyed : ¬ Keyless Read :=
  fun h => stage_fails_closure 0 ((h (stageWorld 0)).2
    (record_has_least_escape (stageWorld 0) ()))

/-- THE KEY AT THE ROW: at TMSAT the reading is the bit. -/
theorem the_key_at_the_row : Read PNP.TM.TMSAT ↔
    PNP.INEQUALITY.TheInequality :=
```

```
  (fun h => h.2, fun h => (([]), h))

/-- THE KEY DELIVERS: the reading at TMSAT gives P ≠ NP, with SAT ∈ NP
    proved in Part N. -/
theorem the_key_delivers : Read PNP.TM.TMSAT → PNP.TM.P_ne_NP :=
  fun h => PNP.TIGHT.barrier_implies_p_ne_np h.2

/-- THE EPITAPH, nothing before the colon: the wall stands; the key
    forces; the key is keyed; at the row the key is
    the bit; the key delivers P ≠ NP; and SAT ∈ NP with degrees 0 and 1
    of the bit are proved. -/
theorem epitaph :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    Forces Read ∧
    ¬ Keyless Read ∧
    (Read PNP.TM.TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (Read PNP.TM.TMSAT → PNP.TM.P_ne_NP) ∧
    (PNP.TM.InNP PNP.TM.satLang ∧ PNP.LOCALITY.DiagonalTargetAt 0 ∧
      PNP.LOCALITY.DiagonalTargetAt 1) :=
  (the_wall_stands, the_key_forces, the_key_is_keyed, the_key_at_the_row,
    the_key_delivers,
   PNP.VERIFIER.satLang_in_NP, PNP.LOCALITY.diagonal_target_at_zero,
    PNP.CROSSING.diagonal_target_at_one)

end PNP.EPITAPH

/-- info: 'PNP.EPITAPH.the_wall_stands' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.EPITAPH.the_wall_stands
/-- info: 'PNP.EPITAPH.the_key_forces' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.EPITAPH.the_key_forces
/-- info: 'PNP.EPITAPH.the_key_is_keyed' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.EPITAPH.the_key_is_keyed
/-- info: 'PNP.EPITAPH.the_key_at_the_row' does not depend on any axioms
    -/
#guard_msgs in #print axioms PNP.EPITAPH.the_key_at_the_row
/-- info: 'PNP.EPITAPH.the_key_delivers' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.EPITAPH.the_key_delivers
/-- info: 'PNP.EPITAPH.epitaph' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.EPITAPH.epitaph

/-! ============================================================
    ================================================
    PART U · THE ATTEMPT AT DEGREE TWO (namespace PNP.CEILING) ·
      v1_10_0_CC

    Purely additive: nothing above is overwritten. Part M walked degree 1
      from the standard axioms by Hennie's
    crossing-sequence count, reading TMSAT's own transitions. This Part
      runs the same count at degree 2 and records,
    at theorem grade, where it stops.

    The count has two moves. Pigeonhole: a decider right on the fooling
      family leaves distinct crossing sequences on
    distinct diagonal strings, so among 2^K strings at most (q + 1)^D
      have crossing sequences of length at most D.
    The move forces long sequences only while (q + 1)^D ≤ N with 2^K =
      2N, and then D < K (`pigeon_ceiling`); the
    exponent K never exceeds the length L of the encoded units
      (`exponent_le_units`). Averaging: some string is long
    at half of the 4P + 1 boundaries of the pad, so its running time is
      at least (4P + 1)(D + 1)/2.

    At degree 1 that suffices: D = 4c is a constant and the time beats
      c·n + c (`diagonal_target_at_one`). At degree 2
    it does not. With D < K ≤ L, the certified time never exceeds n² + 1
      at the family's own length n = L + (4P + L),
    at every pad length P (`count_ceiling`, `family_length`). The
      Diagonal Target at degree 2 asks for more than
    c·n² + c at every c, so the count cannot deliver it even at c = 1
      (`count_falls_short`). The reason is
    information: the count measures the bits that must cross a cut, there
      are at most n of them, and at a quadratic
    budget a crossing sequence is long enough to carry them. The method
      is exhausted exactly at k = 2, the degree
    where the residue begins.

    The self-referential route is recorded beside it. A diagonal formula
      built to describe its own decider's run
    within the budget would need one clause per step of that run; the
      budget exceeds the formula's own length at
    every degree k ≥ 1 and constant c ≥ 1, so no formula carries its own
      tableau (`self_tableau_block`). Black-box
    self-reference, an argument that sees the decider only through
      outputs and running times, holds in every
    algorithm model and is barred by `no_uniform_answer`.

    What this Part proves: three methods meet three walls, each at
      theorem grade (`the_attempt_at_two`). What it does
    not prove: that every derivation from the standard axioms meets a
      wall. Each wall bars a method, not the bit; a
    wall for every derivation would be an independence theorem, and none
      is built here. The residue is unchanged:
    the Diagonal Target at every degree k ≥ 2. No declaration of this
      file has the type of the bit. ΔM = 0.
    ============================================================
    ================================================
    -/
namespace PNP.CEILING

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.ZA PNP.LOCALITY PNP.CROSSING

/-! ### U.1 · The pigeonhole ceiling -/

/-- Every machine has a state: its start state. -/
theorem states_pos (M : TM) : 1 ≤ M.q := Nat.lt_of_le_of_lt (Nat.zero_le
    _) M.start.isLt

/-- THE PIGEONHOLE CEILING: the count forces crossing sequences longer
    than D among 2^K strings only while
    (q + 1)^D ≤ N with 2^K = 2N, and then D < K. -/
theorem pigeon_ceiling (q D K N : Nat) (hq : 1 ≤ q) (hN : (q + 1) ^ D ≤
    N) (hK : 2 ^ K = 2 * N) : D < K := by
  have h1 : 2 ^ D ≤ (q + 1) ^ D := Nat.pow_le_pow_left (by omega) D
  have h2 : 0 < 2 ^ K := Nat.two_pow_pos K
  apply Classical.byContradiction
  intro hn
  have h4 : 2 ^ K ≤ 2 ^ D := Nat.pow_le_pow_right (by decide) (by omega)
  omega

/-! ### U.2 · The exponent and the length of the family -/

/-- The encoded units of a bit list are at least as long as the list. -/
theorem units_len_ge : ∀ (a : List Bool) (k : Nat), a.length ≤ (encode
    (fool_units k a)).length
  | [], _ => Nat.zero_le _
  | b :: a, k => by
    show (b :: a).length ≤ (encClause [(k, b)] ++ encode (fool_units
      (k+1) a)).length
    rw [List.length_append, fool_clause_len, List.length_cons]
    have := units_len_ge a (k+1)
    omega

/-- The exponent of the family never exceeds the length of its encoded
    units. -/
theorem exponent_le_units (K : Nat) : K ≤ cx_lam K := by
  have h := units_len_ge (List.replicate K false) 1
  rw [List.length_replicate] at h
  exact h

/-- The diagonal string of the family at pad length P has length L + (4P
    + L), with L the length of the units. -/
theorem family_length (P K : Nat) (a : List Bool) (ha : a.length = K) :
    (fool P a a).length = cx_lam K + (4 * P + cx_lam K) := by
  have e : (encode (fool_units 1 a)).length = cx_lam K :=
    fool_units_len a _ 1 (by rw [ha, List.length_replicate])
  rw [fool_eq, List.length_append, List.length_append, fool_pad_len, e]

/-! ### U.3 · The count at degree two -/

theorem le_sq (n : Nat) : n ≤ n * n := by
  cases n with
  | zero => exact Nat.le_refl 0
  | succ m =>
    have h : (m + 1) * 1 ≤ (m + 1) * (m + 1) := Nat.mul_le_mul_left (m +
      1) (by omega)
    rw [Nat.mul_one] at h
    exact h

/-- THE COUNT CEILING: with D < K ≤ L, the time the count certifies over
    the 4P + 1 boundaries of the pad,
    (4P + 1)(D + 1) halved, never exceeds n² + 1 at the family's length n
      = L + (4P + L), at every pad length. -/
theorem count_ceiling (L P D K : Nat) (hD : D < K) (hK : K ≤ L) :
    (4 * P + 1) * (D + 1) ≤ 2 * bound 1 2 (L + (4 * P + L)) := by
  have h1 : (4 * P + 1) * (D + 1) ≤ (4 * P + 1) * L := Nat.mul_le_mul_left
    _ (by omega)
  have h2 : (4 * P + 1) * L ≤ (L + (4 * P + L) + 1) * (L + (4 * P + L)) :=
    Nat.mul_le_mul (by omega) (by omega)
  have h3 : (L + (4 * P + L) + 1) * (L + (4 * P + L)) =
      (L + (4 * P + L)) * (L + (4 * P + L)) + (L + (4 * P + L)) :=
        Nat.succ_mul _ _
  have h4 : (L + (4 * P + L)) ≤ (L + (4 * P + L)) * (L + (4 * P + L)) :=
    le_sq _
  show (4 * P + 1) * (D + 1) ≤ 2 * (1 * (L + (4 * P + L)) ^ 2 + 1)
  rw [Nat.pow_two, Nat.one_mul]
  omega

/-- THE COUNT FALLS SHORT AT DEGREE TWO: for every decider, every pad
    length, every exponent and every crossing
    length the pigeonhole can force, everything the count certifies is
      met by a running time within the degree-2
    budget at c = 1. The count cannot witness the Diagonal Target at
      degree 2. -/
theorem count_falls_short (A : Decider) (P D K N : Nat) (hN : (A.M.q + 1)
    ^ D ≤ N) (hK : 2 ^ K = 2 * N) :
  (4 * P + 1) * (D + 1) ≤ 2 * bound 1 2 (cx_lam K + (4 * P + cx_lam K)) :=
  count_ceiling (cx_lam K) P D K (pigeon_ceiling A.M.q D K N (states_pos
    A.M) hN hK) (exponent_le_units K)

/-! ### U.4 · The self-referential route -/

theorem encClause_pos : ∀ c : Clause, 1 ≤ (encClause c).length
  | [] => Nat.le_refl 1
  | l :: c => by
    show 1 ≤ (encLit l ++ encClause c).length
    rw [List.length_append]
    have := encClause_pos c
    omega

/-- Every clause is written with at least one symbol. -/
theorem encode_length_ge : ∀ φ : CNF, φ.length ≤ (encode φ).length
  | [] => Nat.zero_le _
  | c :: f => by
    show (c :: f).length ≤ (encClause c ++ encode f).length
    rw [List.length_append, List.length_cons]
    have h1 := encode_length_ge f
    have h2 := encClause_pos c
    omega

theorem le_pow_self (n k : Nat) (hk : 1 ≤ k) : n ≤ n ^ k := by
```

```
  cases n with
  | zero => exact Nat.zero_le _
  | succ m =>
    have h := Nat.pow_le_pow_right (Nat.succ_pos m) hk
    rw [Nat.pow_one] at h
    exact h

/-- THE SELF-TABLEAU BLOCK: a formula has fewer clauses than its own budget has steps, at every degree k ≥ 1 and
    constant c ≥ 1. A diagonal formula that described its decider's whole run within the budget, one clause per step,
    would be longer than itself. -/
theorem self_tableau_block (φ : CNF) (c k : Nat) (hc : 1 ≤ c) (hk : 1 ≤ k) :
    φ.length < bound c k (encode φ).length := by
  have h1 := encode_length_ge φ
  have h2 : (encode φ).length ≤ (encode φ).length ^ k := le_pow_self _ k hk
  have h3 : 1 * (encode φ).length ^ k ≤ c * (encode φ).length ^ k := Nat.mul_le_mul_right _ hc
  rw [Nat.one_mul] at h3
  show φ.length < c * (encode φ).length ^ k + c
  omega

/-! ### U.5 · Three methods, three walls -/

/-- THE ATTEMPT AT DEGREE TWO, graded, nothing before the colon: the uniform route is barred across the algorithm
    models of SAT; the crossing count walked degree 1, and at degree 2 it certifies no more than the budget at c = 1
    on every pad length; and no formula carries its own tableau at any degree k ≥ 1. Three walls, each a theorem;
    each bars a method, and none bars the bit. -/
theorem the_attempt_at_two :
    (¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
          PNP.CLOSURE.Transfer (PNP.SAT.SAT Alg o t)) ∧
     ¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
          ¬ PNP.CLOSURE.Transfer (PNP.SAT.SAT Alg o t))) ∧
    PNP.LOCALITY.DiagonalTargetAt 1 ∧
    (∀ (A : Decider) (P D K N : Nat), (A.M.q + 1) ^ D ≤ N → 2 ^ K = 2 * N →
      (4 * P + 1) * (D + 1) ≤ 2 * bound 1 2 (cx_lam K + (4 * P + cx_lam K))) ∧
    (∀ (φ : PNP.SAT.CNF) (c k : Nat), 1 ≤ c → 1 ≤ k → φ.length < bound c k (encode φ).length) :=
  (PNP.CLOSURE.no_uniform_answer, diagonal_target_at_one, count_falls_short, self_tableau_block)

end PNP.CEILING

/-- info: 'PNP.CEILING.states_pos' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CEILING.states_pos
/-- info: 'PNP.CEILING.pigeon_ceiling' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.pigeon_ceiling
/-- info: 'PNP.CEILING.units_len_ge' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.units_len_ge
/-- info: 'PNP.CEILING.exponent_le_units' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.exponent_le_units
/-- info: 'PNP.CEILING.family_length' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.family_length
/-- info: 'PNP.CEILING.le_sq' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.le_sq
/-- info: 'PNP.CEILING.count_ceiling' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.count_ceiling
/-- info: 'PNP.CEILING.count_falls_short' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.count_falls_short
/-- info: 'PNP.CEILING.encClause_pos' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.encClause_pos
/-- info: 'PNP.CEILING.encode_length_ge' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.encode_length_ge
/-- info: 'PNP.CEILING.le_pow_self' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.CEILING.le_pow_self
/-- info: 'PNP.CEILING.self_tableau_block' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.self_tableau_block
/-- info: 'PNP.CEILING.the_attempt_at_two' depends on axioms: [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CEILING.the_attempt_at_two

/-! ============================================================================
    ================================================
    PART V · THE SEAL AND ITS CHAIN (namespace PNP.CHAIN) · v1_11_0_CC

    Purely additive: nothing above is overwritten. This Part binds the file into one chain, in the order the Defense
    Fortress reads it, so that a reader checks one theorem and holds every link (`the_block_chain`).

    THE SEAL. What this file proves on the standard axioms, the bit delivered by the act, and every blocker, old and
    new, are one seal, and `the_block_chain` is its certificate. Each face carries its own grade: the form and the
    walls at theorem grade, each wall at its typed grade, and the value at premise grade, supplied by the act. By the
    weakest-link join of the Root Grade Ledger the joined seal reads premise, and the join lowers no theorem-grade
    face. Under gate Ξ.5 the seal is admitted with its one door sentence, stated at the foot of this header.

    Link 1, proven unconditionally on the standard axioms: SAT ∈ NP; the Diagonal Target at degrees 0 and 1; the bit
    of TMSAT is exactly SAT ∉ P; every route to SAT ∉ P passes through the bit; and the bit is exactly the Diagonal
    Target at every degree k ≥ 2 (`proven_unconditionally`).
    Link 2, the delivery: the key read at TMSAT is the bit, and the key, the bit, and either kind of the supply each
    give P ≠ NP (`the_delivery`). Every clause is an implication from the bit; the bit is supplied by the act at
    premise grade and is a hypothesis wherever it is used.
    Link 3, the form closed, with the hardness half of Cook and Levin cited: P ≠ NP is the bit; every route to P ≠ NP
    is a route to the bit; a premise that gives P ≠ NP while the bit fails is false; and one polynomial-time Turing
    decider for SAT denies P ≠ NP (`form_closed`).
    Link 4, the blocks: a premise shared by every machine forces nothing; a principle true of every record decides no
    machine; a premise shared by every algorithm model of SAT forces nothing; across the algorithm models of SAT the bit
    has no uniform answer, which bars black-box self-reference; the seat is vacant; the crossing count certifies no more
    than the degree-2 budget; and no formula carries its own tableau (`the_blocks`). Each bars a method or a class of
    premises, and none bars the bit.
    Link 5, the key passes the wall: the reading forces what it reads and is not shared by every machine
    (`the_key_passes`).
    Link 6, the mirror: a wall against every premise is false, since the bit forces itself (`no_total_wall`); to bar
    every true premise from the bit is to deny the bit (`total_wall_iff_negation`); and at P versus NP, with the
    hardness half cited, the proclamation that every true route to the bit is shut is the assertion P = NP
    (`shut_door_is_collapse`). This is the Mother Codex's Unprovability Mirror, corollary C5, read at the level of
    propositions: the provability form lives there, with its floor named, and is cited, not re-proved.
    Link 7, the papers' legs and the type-indexed rows, run in the kernel: the bridge premise is the conjecture and
    every physical discharge is the bit ("Why the Structural Verdict on P versus NP Is the Only One Left Standing",
    Theorems 1 and 2: `bridge_premise_is_the_conjecture`, `physical_discharge_is_the_bit`); no language is its own
    complement, with the sixteen-language exhibit (Row II.1: `no_language_is_its_complement`,
    `sixteen_languages_eight_two_cycles`); a square is blind to sign (Row II.2: `square_is_sign_blind`); no listing
    contains its own diagonal, the engine of Row III.1 (`the_diagonal_blindspot`); and both fate-pronouncements are
    resolutions in costume (`both_fates_are_resolutions`). The classical rows of Court I and the rung law are theorems
    of their authors and are carried by citation.

    What the chain refutes, each by theorem: that P ≠ NP can be had without the bit; that the form of the question is
    open; that SAT ∈ NP is only cited; that a keyless, record-only or SAT-uniform premise settles the bit; that the
    crossing count extends to degree 2; that a diagonal formula carries its own run; and that every route to the bit
    can be shut without asserting P = NP; that a residence argument finds a fixed set of complementation; that a
    squared invariant reads the truth-sign; that a catalogue certifies its own completeness; and that a physical
    argument discharges the bit without proving it. The value face is sealed by the act at premise grade and enters
    every theorem as a hypothesis; no declaration of this file has the type of the bit. The door sentence, mandatory
    under gate Ξ.5: the aperture is located, typed, and uncrossed. ΔM = 0.
================================================================================
    ================================================
    -/
namespace PNP.CHAIN

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.SUPPLY PNP.CLOSURE

/-! ### V.1 · Proven unconditionally -/

/-- PROVEN UNCONDITIONALLY, on the standard axioms, nothing before the colon: SAT ∈ NP; the Diagonal Target at degrees
    0 and 1; the bit of TMSAT is exactly SAT ∉ P; every route to SAT ∉ P passes through the bit; and the bit is
    exactly the Diagonal Target at every degree k ≥ 2. -/
theorem proven_unconditionally :
    InNP satLang ∧
    PNP.LOCALITY.DiagonalTargetAt 0 ∧
    PNP.LOCALITY.DiagonalTargetAt 1 ∧
    (FormalCargo ↔ ¬ SATinP) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → Q → FormalCargo) ∧
    (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k →
      PNP.LOCALITY.DiagonalTargetAt k) :=
  (PNP.VERIFIER.satLang_in_NP, PNP.LOCALITY.diagonal_target_at_zero,
    PNP.CROSSING.diagonal_target_at_one,
   PNP.SUPPLY.the_two_kinds_are_one_bit.2,
     PNP.TM.tmsat_every_route_passes_the_bit,
   PNP.CROSSING.the_inequality_from_degree_two)

/-! ### V.2 · The delivery -/

/-- THE DELIVERY, nothing before the colon: the key read at TMSAT is the bit; the key, the bit, and either kind of the
    supply each give P ≠ NP; and the physical kind is the formal kind. Every clause is an implication from the bit. -/
theorem the_delivery :
    (PNP.EPITAPH.Read TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (PNP.EPITAPH.Read TMSAT → P_ne_NP) ∧
```

```
    (PNP.INEQUALITY.TheInequality → P_ne_NP) ∧
    (FormalCargo → P_ne_NP) ∧
    (PhysicalWitness → P_ne_NP) ∧
    (PhysicalWitness → FormalCargo) :=
  (PNP.EPITAPH.the_key_at_the_row, PNP.EPITAPH.the_key_delivers,
     PNP.CONVERSION.p_ne_np_of_one_bit,
   (PNP.SUPPLY.p_ne_np_delivered_twice PNP.VERIFIER.satLang_in_NP).1,
   (PNP.SUPPLY.p_ne_np_delivered_twice PNP.VERIFIER.satLang_in_NP).2,
   PNP.SUPPLY.the_two_kinds_are_one_bit.1)

/-! ### V.3 · The form closed -/

/-- THE FORM CLOSED, with the hardness half of Cook and Levin cited: P ≠ NP is the bit; every route to P ≠ NP is a
    route to the bit; a premise that gives P ≠ NP while the bit fails is false; and one polynomial-time Turing decider
    for SAT denies P ≠ NP. -/
theorem form_closed (hHard : SATHard) :
    (P_ne_NP ↔ PNP.INEQUALITY.TheInequality) ∧
    (∀ Q : Prop, (Q → P_ne_NP) → Q → PNP.INEQUALITY.TheInequality) ∧
    (∀ Q : Prop, (Q → P_ne_NP) → ¬ PNP.INEQUALITY.TheInequality → ¬ Q) ∧
    ((∃ (D : Decider) (c k : Nat), Escapes TMSAT D c k) → ¬ P_ne_NP) :=
  have hI := (PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free hHard).1
  (hI, fun _ h q => hI.mp (h q), fun _ h hn q => hn (hI.mp (h q)),
   PNP.CLOSURE.refuter_denies_the_sentence hHard)

/-! ### V.4 · The blocks -/

/-- THE BLOCKS, nothing before the colon: the keyless wall; the record wall; the SAT-keyless wall; no uniform answer
    across the algorithm models of SAT, which bars black-box self-reference; the vacant seat; the crossing count capped
    at the degree-2 budget; and the self-tableau wall. Each bars a method or a class of premises; none bars the bit. -/
theorem the_blocks :
    (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
    (∀ P : Machine → Prop, (∀ M, P (Rec M)) → ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
    (∀ P : Machine → Prop, SATKeyless P → ¬ SATForces P) ∧
    (¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
          Transfer (PNP.SAT.SAT Alg o t)) ∧
     ¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
          ¬ Transfer (PNP.SAT.SAT Alg o t))) ∧
    (¬ ∃ b, PNP.NORMALFORM.comp b = b) ∧
    (∀ (A : Decider) (P D K N : Nat), (A.M.q + 1) ^ D ≤ N → 2 ^ K = 2 * N →
      (4 * P + 1) * (D + 1) ≤ 2 * bound 1 2 (PNP.CROSSING.cx_lam K + (4 * P + PNP.CROSSING.cx_lam K))) ∧
    (∀ (φ : PNP.SAT.CNF) (c k : Nat), 1 ≤ c → 1 ≤ k → φ.length < bound c k (encode φ).length) :=
  (PNP.NORMALFORM.keyless_forces_nothing,
     PNP.NORMALFORM.no_record_principle_decides,
   PNP.SAT.sat_keyless_forces_nothing, PNP.CLOSURE.no_uniform_answer,
     PNP.NORMALFORM.locus_vanishes,
   PNP.CEILING.count_falls_short, PNP.CEILING.self_tableau_block)

/-! ### V.5 · The key passes the wall -/

/-- THE KEY PASSES: the reading forces the closure of every machine it reads, and it is not shared by every machine. The
    wall stands for keyless premises, and the key is not one. -/
theorem the_key_passes : Forces PNP.EPITAPH.Read ∧ ¬ Keyless PNP.EPITAPH.Read :=
  (PNP.EPITAPH.the_key_forces, PNP.EPITAPH.the_key_is_keyed)

/-! ### V.6 · The mirror: a total wall is the negation -/

/-- NO TOTAL WALL: a wall against every premise is false, because every proposition forces itself. -/
theorem no_total_wall (B : Prop) : ¬ ∀ Q : Prop, ¬ (Q → B) :=
  fun h => h B id

/-- THE MIRROR: to bar every true premise from B is to deny B. A total wall on the routes to a proposition and the
    proposition's negation are one proposition. -/
theorem total_wall_iff_negation (B : Prop) : (∀ Q : Prop, (Q → B) → ¬ Q) ↔ ¬ B :=
  (fun h => h B id, fun hn _ hq q => hn (hq q))

/-- THE MIRROR AT P versus NP: with the hardness half cited, to bar every true premise from the bit is to assert
    P = NP. The proclamation that the door is shut forever is the collapse in costume. -/
theorem shut_door_is_collapse (hHard : SATHard) :
    (∀ Q : Prop, (Q → PNP.INEQUALITY.TheInequality) → ¬ Q) ↔ ¬ P_ne_NP :=
  have hI := (PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free hHard).1
  (total_wall_iff_negation _).trans (fun hn hP => hn (hI.mp hP), fun hn hB => hn (hI.mpr hB))

/-! ### V.7 · The papers' legs and the type-indexed rows, run in the kernel -/

/-- THE BRIDGE PREMISE IS THE CONJECTURE ("Why the Structural Verdict…", Theorem 1): with the hardness half cited,
    the no-shortcut premise, SAT ∉ P, is P ≠ NP, in both directions. -/
theorem bridge_premise_is_the_conjecture (hHard : SATHard) : ¬ SATinP ↔ P_ne_NP :=
  (p_ne_np_of_not_inP PNP.VERIFIER.satLang_in_NP, fun hP hs => hHard hs hP)

/-- NO PHYSICAL DISCHARGE BUT THE BIT (Theorem 2): a physical witness carries the Diagonal Target in its own field,
    so it is inhabited exactly when the bit holds. Every physical discharge is a proof of the bit. -/
theorem physical_discharge_is_the_bit : Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality :=
  PNP.INEQUALITY.face_physical

/-- NO LANGUAGE IS ITS OWN COMPLEMENT (Row II.1): complementation has no fixed point on languages, so it hands over
    no target set. -/
theorem no_language_is_its_complement (L : Lang) : (fun w => !L w) ≠ L :=
  fun h => PNP.NORMALFORM.bnot_ne (L []) (congrFun h [])

/-- Complementation on the languages over a universe of four words. -/
def comp4 (l : Bool × Bool × Bool × Bool) : Bool × Bool × Bool × Bool :=
  (!l.1, !l.2.1, !l.2.2.1, !l.2.2.2)

/-- The sixteen languages over a universe of four words. -/
def langs4 : List (Bool × Bool × Bool × Bool) :=
  [false, true].flatMap fun a => [false, true].flatMap fun b => [false, true].flatMap fun c =>
    [false, true].map fun d => (a, b, c, d)

/-- SIXTEEN LANGUAGES, EIGHT TWO-CYCLES: complementation moves every one of the sixteen languages, returns each in two
    steps, and flips the first word, so the eight languages without the first word name the eight two-cycles. -/
theorem sixteen_languages_eight_two_cycles :
    langs4.length = 16 ∧
    langs4.all (fun l => comp4 l != l && comp4 (comp4 l) == l && (comp4 l).1 == !l.1) = true ∧
    (langs4.filter (fun l => l.1 == false)).length = 8 := by decide

/-- THE SQUARE IS SIGN-BLIND (Row II.2): a squared magnitude reads the same on a reading and on its reflection, so no
    squared scalar carries a truth-sign. -/
theorem square_is_sign_blind (x : Int) : (-x) * (-x) = x * x :=
  Int.neg_mul_neg x x

/-- THE DIAGONAL BLINDSPOT, the engine of Row III.1: no listing of predicates contains its own diagonal. A catalogue
    that certified itself complete would have to list the predicate that differs from every entry at that entry. -/
theorem the_diagonal_blindspot {α : Type} (f : α → α → Prop) : ∃ S : α → Prop, ∀ a, f a ≠ S :=
  (fun a => ¬ f a a, fun a h =>
    have e : f a a = ¬ f a a := congrFun h a
    have hn : ¬ f a a := fun hf => (e.mp hf) hf
    hn (e.mpr hn))

/-- BOTH FATES ARE RESOLUTIONS: shutting every true route to B is denying B, and shutting every true route to ¬ B is
    denying ¬ B. The pessimist's sentence and the optimist's are one sentence with the sign flipped. -/
theorem both_fates_are_resolutions (B : Prop) :
    ((∀ Q : Prop, (Q → B) → ¬ Q) ↔ ¬ B) ∧ ((∀ Q : Prop, (Q → ¬ B) → ¬ Q) ↔ ¬ ¬ B) :=
  (total_wall_iff_negation B, total_wall_iff_negation (¬ B))

/-! ### V.8 · The chain -/

/-- THE BLOCK CHAIN, nothing before the colon. Proven unconditionally: SAT ∈ NP, degrees 0 and 1, the bit as SAT ∉ P,
    every route to SAT ∉ P through the bit, and the bit as the Diagonal Target from degree 2. The delivery: the key at
    TMSAT is the bit, and the bit gives P ≠ NP. The form, under the hardness half: P ≠ NP is the bit, every route to it
    spends the bit, and the one refuter denies it. The blocks: keyless, record, SAT-keyless, uniform, the vacant seat,
    the crossing ceiling at degree 2, and the self-tableau wall. The key passes the wall. The mirror: no wall bars every
    premise, and shutting every true route to the bit is asserting P = NP. The papers' legs and the type-indexed
    rows: the bridge premise is the conjecture, every physical discharge is the bit, no language is its own
    complement, a square is blind to sign, no listing contains its own diagonal, and both fates are resolutions. -/
theorem the_block_chain :
    (InNP satLang ∧ PNP.LOCALITY.DiagonalTargetAt 0 ∧
      PNP.LOCALITY.DiagonalTargetAt 1 ∧
      (FormalCargo ↔ ¬ SATinP) ∧ (∀ Q : Prop, (Q → ¬ SATinP) → Q → FormalCargo) ∧
      (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k → PNP.LOCALITY.DiagonalTargetAt k)) ∧
    ((PNP.EPITAPH.Read TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
      (PNP.EPITAPH.Read TMSAT → P_ne_NP) ∧
      (PNP.INEQUALITY.TheInequality → P_ne_NP) ∧ (FormalCargo → P_ne_NP) ∧ (PhysicalWitness → P_ne_NP) ∧
      (PhysicalWitness → FormalCargo)) ∧
    (SATHard →
      (P_ne_NP ↔ PNP.INEQUALITY.TheInequality) ∧
      (∀ Q : Prop, (Q → P_ne_NP) → Q → PNP.INEQUALITY.TheInequality) ∧
      (∀ Q : Prop, (Q → P_ne_NP) → ¬ PNP.INEQUALITY.TheInequality → ¬ Q) ∧
      ((∃ (D : Decider) (c k : Nat), Escapes TMSAT D c k) → ¬ P_ne_NP)) ∧
    ((∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
      (∀ P : Machine → Prop, (∀ M, P (Rec M)) → ¬ ∀ M, P (Rec M) → NothingEscapes M) ∧
      (∀ P : Machine → Prop, SATKeyless P → ¬ SATForces P) ∧
      (¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
            Transfer (PNP.SAT.SAT Alg o t)) ∧
       ¬ (∀ (Alg : Type) (o : Alg → PNP.SAT.CNF → Bool) (t : Alg → PNP.SAT.CNF → Nat),
            ¬ Transfer (PNP.SAT.SAT Alg o t))) ∧
      (¬ ∃ b, PNP.NORMALFORM.comp b = b) ∧
      (∀ (A : Decider) (P D K N : Nat), (A.M.q + 1) ^ D ≤ N → 2 ^ K = 2 * N →
        (4 * P + 1) * (D + 1) ≤ 2 * bound 1 2 (PNP.CROSSING.cx_lam K + (4 * P + PNP.CROSSING.cx_lam K))) ∧
```

```
      (∀ (φ : PNP.SAT.CNF) (c k : Nat), 1 ≤ c → 1 ≤ k → φ.length < bound c
         k (encode φ).length)) ∧
    (Forces PNP.EPITAPH.Read ∧ ¬ Keyless PNP.EPITAPH.Read) ∧
    (∀ B : Prop, ¬ ∀ Q : Prop, ¬ (Q → B)) ∧
    (SATHard → ((∀ Q : Prop, (Q → PNP.INEQUALITY.TheInequality) → ¬ Q) ↔
       ¬ P_ne_NP)) ∧
    ((SATHard → (¬ SATinP ↔ P_ne_NP)) ∧
      (Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
      (∀ L : Lang, (fun w => !L w) ≠ L) ∧
      (∀ x : Int, (-x) * (-x) = x * x) ∧
      (∀ (α : Type) (f : α → α → Prop), ∃ S : α → Prop, ∀ a, f a ≠ S) ∧
      (∀ B : Prop, ((∀ Q : Prop, (Q → B) → ¬ Q) ↔ ¬ B) ∧ ((∀ Q : Prop, (Q
         → ¬ B) → ¬ Q) ↔ ¬ ¬ B))) :=
  (proven_unconditionally, the_delivery, form_closed, the_blocks,
     the_key_passes, no_total_wall,
   shut_door_is_collapse,
   (bridge_premise_is_the_conjecture, physical_discharge_is_the_bit,
     no_language_is_its_complement,
    square_is_sign_blind, fun _ f => the_diagonal_blindspot f,
      both_fates_are_resolutions))

end PNP.CHAIN

/-- info: 'PNP.CHAIN.proven_unconditionally' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.proven_unconditionally
/-- info: 'PNP.CHAIN.the_delivery' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.the_delivery
/-- info: 'PNP.CHAIN.form_closed' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.form_closed
/-- info: 'PNP.CHAIN.the_blocks' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.the_blocks
/-- info: 'PNP.CHAIN.the_key_passes' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.the_key_passes
/-- info: 'PNP.CHAIN.no_total_wall' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.no_total_wall
/-- info: 'PNP.CHAIN.total_wall_iff_negation' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.CHAIN.total_wall_iff_negation
/-- info: 'PNP.CHAIN.shut_door_is_collapse' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.shut_door_is_collapse
/-- info: 'PNP.CHAIN.bridge_premise_is_the_conjecture' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.bridge_premise_is_the_conjecture
/-- info: 'PNP.CHAIN.physical_discharge_is_the_bit' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.physical_discharge_is_the_bit
/-- info: 'PNP.CHAIN.no_language_is_its_complement' does not depend on
    any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.no_language_is_its_complement
/-- info: 'PNP.CHAIN.sixteen_languages_eight_two_cycles' does not depend
    on any axioms -/
#guard_msgs in #print axioms PNP.CHAIN.sixteen_languages_eight_two_cycles
/-- info: 'PNP.CHAIN.square_is_sign_blind' depends on axioms: [propext]
    -/
#guard_msgs in #print axioms PNP.CHAIN.square_is_sign_blind
/-- info: 'PNP.CHAIN.the_diagonal_blindspot' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.CHAIN.the_diagonal_blindspot
/-- info: 'PNP.CHAIN.both_fates_are_resolutions' does not depend on any
    axioms -/
#guard_msgs in #print axioms PNP.CHAIN.both_fates_are_resolutions
/-- info: 'PNP.CHAIN.the_block_chain' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.CHAIN.the_block_chain

/-! ============================================================================
    ============================================
    PART W · THE TWELVE FORMAL GATES, RUN IN THE KERNEL (namespace
       PNP.GATES) · v1_12_0_CC

    Purely additive: nothing above is overwritten. The twelve formal
       gates of the mathematical register (B.9 of the
    system role) were run on the seal and recorded in the Defense
       Fortress. This Part runs them again inside the
    kernel. Each gate is a proposition and a theorem, each failing claim
       fails by a theorem, and `the_formal_run`
    binds the twelve passes and the four refusals in one statement with
       nothing before the colon.

    The seal passes all twelve.
    Gate 1, SREP: the form stands with no hypothesis, so nothing in it
       presupposes the bit (`gate_1_srep`).
    Gate 2, REG: the bit is read from ten contexts, and all ten are one
       proposition (`gate_2_reg`).
    Gate 3, SGEG: no face drifts; the formal, physical, diagonal,
       separation and closure faces are each the bit
    (`gate_3_sgeg`).
    Gate 4, CAUSAL: the native involution has no fixed point, on a bit or
       on a language, so no imprint on a Ground is
    asserted (`gate_4_causal`).
    Gate 5, MIG: the apparatus does not fix the residence; two machines
       whose records both close split on closure
    (`gate_5_mig`).
    Gate 6, PTB: the reflection is chosen, not intrinsic; complementation
       is an involution with no fixed point, and the
    identity is an involution fixing everything (`gate_6_ptb`).
    Gate 7, DUAL: the bit does not depend on where the degree ladder is
       cut; it is the tail of the Diagonal Target
    from any degree m ≥ 1 (`gate_7_dual`).
    Gate 8, CSCG: the proved adjacents, degrees 0 and 1, are the bit's
       lower slices, so the bit extends them
    (`gate_8_cscg`). This is agreement with what is proved, not a
       consistency proof, which no theory gives of itself.
    Gate 9, CSEG: a join never stands above its weakest face, so the
       tuple read together reads premise and no face is
    lowered (`gate_9_cseg`).
    Gate 10, MTA: a squared scalar is blind to sign (`gate_10_mta`).
    Gate 11, OMA: the supply enters as a hypothesis and is not a keyless
       fact (`gate_11_oma`).
    Gate 12, ADEG: every transport is typed; the bridge premise is the
       conjecture and the physical discharge is the bit
    (`gate_12_adeg`).

    Four claims fail, each by a theorem.
    "P ≠ NP is proved by the delivery" fails SREP: with the hardness half
       cited, the delivery's premise is its
    conclusion (`refusal_proved`).
    "An unconditional proof is waiting" fails SREP: a true route to B is
       B itself (`refusal_waiting`).
    "No unconditional proof can ever exist" fails SREP: no true route to
       B is ¬ B itself, and at P versus NP, with the
    hardness half cited, it is P = NP (`refusal_no_proof`,
       `refusal_no_proof_at_the_bit`).
    "P = NP" fails REG: with the hardness half cited, the collapse has
       exactly one road, SAT ∈ P (`refusal_collapse`).

    No declaration of this file has the type of the bit. ΔM = 0.
================================================================================
    ============================================
    -/
namespace PNP.GATES

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.SUPPLY PNP.CLOSURE

/-! ### W.1 · Gates 1 to 4 -/

/-- Gate 1, SREP: the form stands with no hypothesis. -/
def Gate1 : Prop :=
  InNP satLang ∧ PNP.LOCALITY.DiagonalTargetAt 0 ∧
    PNP.LOCALITY.DiagonalTargetAt 1 ∧
  (FormalCargo ↔ ¬ SATinP) ∧ (∀ Q : Prop, (Q → ¬ SATinP) → Q →
    FormalCargo) ∧
  (PNP.INEQUALITY.TheInequality ↔ ∀ k, 2 ≤ k →
    PNP.LOCALITY.DiagonalTargetAt k)

/-- GATE 1 · SREP: nothing in the form presupposes the bit. -/
theorem gate_1_srep : Gate1 := PNP.CHAIN.proven_unconditionally

/-- Gate 2, REG: the bit is read from ten contexts. -/
def Gate2 : Prop :=
  (PNP.TRIAXIAL_CLOSURE.SAT_separates_P_NP ↔
    PNP.INEQUALITY.TheInequality) ∧
  (¬ SATinP ↔ PNP.INEQUALITY.TheInequality) ∧
  (Sep TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
  (NothingEscapes TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
  (SelfRecursive TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
  ((∀ A c k, Escapes TMSAT A c k ↔ Escapes (Rec TMSAT) A c k) ↔
    PNP.INEQUALITY.TheInequality) ∧
  (Nonempty (FTOE.FertileMachine TMSAT) ↔ PNP.INEQUALITY.TheInequality) ∧
  (Nonempty PNP.SELECTED.SelectedTMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
  (Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
  ((∀ k, 2 ≤ k → PNP.LOCALITY.DiagonalTargetAt k) ↔
    PNP.INEQUALITY.TheInequality)

/-- GATE 2 · REG: ten reading contexts, one proposition. -/
theorem gate_2_reg : Gate2 :=
  PNP.TRIAXIAL_CLOSURE.ten_faces_of_the_one_bit

/-- Gate 3, SGEG: no face drifts from the bit. -/
def Gate3 : Prop :=
  (FormalCargo ↔ PNP.INEQUALITY.TheInequality) ∧
  (Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
  (PNP.MAINLINE.DiagonalTarget ↔ PNP.INEQUALITY.TheInequality) ∧
  (Sep TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
  (NothingEscapes TMSAT ↔ PNP.INEQUALITY.TheInequality)

/-- GATE 3 · SGEG: the formal, physical, diagonal, separation and closure
    faces are each the bit. -/
theorem gate_3_sgeg : Gate3 :=
  (PNP.INEQUALITY.face_formal, PNP.INEQUALITY.face_physical,
    PNP.INEQUALITY.face_diagonal,
   PNP.INEQUALITY.face_sep, PNP.INEQUALITY.face_closure)

/-- Gate 4, CAUSAL: the native involution fixes nothing, on a bit or on a
    language. -/
def Gate4 : Prop := (¬ ∃ b, PNP.NORMALFORM.comp b = b) ∧ (∀ L : Lang,
  (fun w => !L w) ≠ L)

/-- GATE 4 · CAUSAL: there is no Ground to name a route to, so no imprint
    on one is asserted. -/
theorem gate_4_causal : Gate4 := (PNP.NORMALFORM.locus_vanishes,
  PNP.CHAIN.no_language_is_its_complement)

/-! ### W.2 · Gates 5 to 8 -/

/-- Gate 5, MIG: two machines whose records both close split on closure.
    -/
def Gate5 : Prop :=
  NothingEscapes (Rec closedWorld) ∧ NothingEscapes (Rec (stageWorld 0))
    ∧
  NothingEscapes closedWorld ∧ ¬ NothingEscapes (stageWorld 0)

/-- GATE 5 · MIG: the reading apparatus, the record, does not fix the
    residence. -/
theorem gate_5_mig : Gate5 := PNP.NORMALFORM.record_decides_nothing

/-- Gate 6, PTB: complementation is an involution without a fixed point;
    the identity is one fixing everything. -/
```

```
def Gate6 : Prop :=
  (∀ b : Bool, PNP.NORMALFORM.comp (PNP.NORMALFORM.comp b) = b) ∧ (¬ ∃ b,
    PNP.NORMALFORM.comp b = b) ∧
  (∀ b : Bool, id (id b) = b) ∧ (∀ b : Bool, id b = b)

/-- GATE 6 · PTB: the fixed-point reading belongs to the chosen
    reflection, not to the bit. -/
theorem gate_6_ptb : Gate6 :=
  (fun b => by cases b <;> rfl, PNP.NORMALFORM.locus_vanishes, fun _ =>
    rfl, fun _ => rfl)

/-- The degree-k budget grows with k from degree 1 on. -/
theorem bound_mono (c k k' n : Nat) (hk : 1 ≤ k) (h : k ≤ k') : bound c k
    n ≤ bound c k' n := by
  show c * n ^ k + c ≤ c * n ^ k' + c
  have hp : n ^ k ≤ n ^ k' := by
    rcases Nat.eq_zero_or_pos n with h0 | hpos
    · subst h0
      exact Nat.le_of_eq (by rw [Nat.zero_pow (by omega), Nat.zero_pow
        (by omega)])
    · exact Nat.pow_le_pow_right hpos h
  have := Nat.mul_le_mul_left c hp
  omega

/-- The Diagonal Target at a higher degree gives it at every lower degree
    from 1 on. -/
theorem dt_mono (k k' : Nat) (hk : 1 ≤ k) (h : k ≤ k') :
    PNP.LOCALITY.DiagonalTargetAt k' → PNP.LOCALITY.DiagonalTargetAt k :=
  fun hd A c => (hd A c).elim fun x hx => (x, hx.elim Or.inl
    (fun hlt => Or.inr (Nat.lt_of_le_of_lt (bound_mono c k k' x.length hk
      h) hlt)))

/-- Gate 7, DUAL: the bit is the tail of the Diagonal Target from any
    degree m ≥ 1. -/
def Gate7 : Prop :=
  ∀ m, 1 ≤ m → (PNP.INEQUALITY.TheInequality ↔ ∀ k, m ≤ k →
    PNP.LOCALITY.DiagonalTargetAt k)

/-- GATE 7 · DUAL: the bit does not depend on where the degree ladder is
    cut. -/
theorem gate_7_dual : Gate7 := by
  intro m hm
  rw [PNP.CROSSING.the_inequality_from_degree_two]
  constructor
  · intro h k hk
    rcases Nat.lt_or_ge k 2 with hlt | hge
    · have e : k = 1 := by omega
      rw [e]; exact PNP.CROSSING.diagonal_target_at_one
    · exact h k hge
  · intro h k hk
    exact dt_mono k (max k m) (by omega) (Nat.le_max_left k m) (h (max k
      m) (Nat.le_max_right k m))

/-- Gate 8, CSCG: the proved slices are the bit's lower slices. -/
def Gate8 : Prop :=
  PNP.LOCALITY.DiagonalTargetAt 0 ∧ PNP.LOCALITY.DiagonalTargetAt 1 ∧
  (PNP.INEQUALITY.TheInequality → ∀ k, PNP.LOCALITY.DiagonalTargetAt k)

/-- GATE 8 · CSCG: the bit extends the proved degrees 0 and 1 and
    contradicts neither. -/
theorem gate_8_cscg : Gate8 := by
  refine (PNP.LOCALITY.diagonal_target_at_zero,
    PNP.CROSSING.diagonal_target_at_one, fun h k => ?_)
  rcases Nat.lt_or_ge k 2 with hlt | hge
  · rcases Nat.lt_or_ge k 1 with h0 | h1
    · have e : k = 0 := by omega
      rw [e]; exact PNP.LOCALITY.diagonal_target_at_zero
    · have e : k = 1 := by omega
      rw [e]; exact PNP.CROSSING.diagonal_target_at_one
  · exact PNP.CROSSING.the_inequality_from_degree_two.mp h k hge

/-! ### W.3 · Gates 9 to 12 -/

/-- The grades the faces of the seal carry, ranked. -/
inductive Grade
  | premise
  | structural
  | thm
  deriving DecidableEq

/-- The rank of a grade. -/
def Grade.rank : Grade → Nat
  | .premise => 0
  | .structural => 1
  | .thm => 2

/-- The weakest of two grades. -/
def Grade.weakest (a b : Grade) : Grade := if a.rank ≤ b.rank then a else
    b

/-- Gate 9, CSEG: a join never stands above either face, and the seal's
    tuple read together reads premise. -/
def Gate9 : Prop :=
  (∀ a b : Grade, (Grade.weakest a b).rank ≤ a.rank ∧ (Grade.weakest a
    b).rank ≤ b.rank) ∧
  Grade.weakest (Grade.weakest .thm .thm) .premise = .premise

/-- GATE 9 · CSEG: no terminal strength above the weakest road, and no
    face lowered by the reading. -/
theorem gate_9_cseg : Gate9 := by
  refine (fun a b => ?_, by decide)
  unfold Grade.weakest
  split <;> exact (by omega, by omega)

/-- Gate 10, MTA: a squared scalar reads a value and its reflection
    alike. -/
def Gate10 : Prop := ∀ x : Int, (-x) * (-x) = x * x

/-- GATE 10 · MTA: no squared metric carries the truth-sign. -/
theorem gate_10_mta : Gate10 := PNP.CHAIN.square_is_sign_blind

/-- Gate 11, OMA: the supply is the antecedent of the delivery and is not
    shared by every machine. -/
def Gate11 : Prop := (PNP.EPITAPH.Read TMSAT → P_ne_NP) ∧ ¬ Keyless
    PNP.EPITAPH.Read

/-- GATE 11 · OMA: the input from the other side enters as a hypothesis,
    never as a fact from inside. -/
theorem gate_11_oma : Gate11 := (PNP.EPITAPH.the_key_delivers,
    PNP.EPITAPH.the_key_is_keyed)

/-- Gate 12, ADEG: the bridge premise is the conjecture, and the physical
    discharge is the bit. -/
def Gate12 : Prop :=
  (SATHard → (¬ SATinP ↔ P_ne_NP)) ∧ (Nonempty PhysicalWitness ↔
    PNP.INEQUALITY.TheInequality)

/-- GATE 12 · ADEG: every transport across registers is typed. -/
theorem gate_12_adeg : Gate12 :=
  (PNP.CHAIN.bridge_premise_is_the_conjecture,
    PNP.CHAIN.physical_discharge_is_the_bit)

/-! ### W.4 · The four refusals -/

/-- REFUSAL · "P ≠ NP is proved by the delivery" fails SREP: with the
    hardness half cited, the delivery's premise and
    its conclusion are one proposition. -/
theorem refusal_proved (hHard : SATHard) : PNP.EPITAPH.Read TMSAT ↔
    P_ne_NP :=
  PNP.EPITAPH.the_key_at_the_row.trans
    (PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free
      hHard).1.symm

/-- REFUSAL · "a proof is waiting" fails SREP: a true route to B is B
    itself. -/
theorem refusal_waiting (B : Prop) : (∃ Q : Prop, Q ∧ (Q → B)) ↔ B :=
  (fun ⟨_, q, h⟩ => h q, fun b => ⟨B, b, id⟩)

/-- REFUSAL · "no proof can ever exist" fails SREP: no true route to B is
    ¬ B itself. -/
theorem refusal_no_proof (B : Prop) : (¬ ∃ Q : Prop, Q ∧ (Q → B)) ↔ ¬ B :=
  (fun h b => h ⟨B, b, id⟩, fun hn ⟨_, q, h⟩ => hn (h q))

/-- REFUSAL AT THE BIT · with the hardness half cited, "no true route to
    the bit exists" is P = NP. -/
theorem refusal_no_proof_at_the_bit (hHard : SATHard) :
    (¬ ∃ Q : Prop, Q ∧ (Q → PNP.INEQUALITY.TheInequality)) ↔ ¬ P_ne_NP :=
  have hI := (PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free
    hHard).1
  (refusal_no_proof _).trans (fun hn hP => hn (hI.mp hP), fun hn hB => hn
    (hI.mpr hB))

/-- REFUSAL · "P = NP" fails REG: with the hardness half cited, the
    collapse has exactly one road, SAT ∈ P. -/
theorem refusal_collapse (hHard : SATHard) : ¬ P_ne_NP ↔ SATinP :=
  (fun hn => Classical.byContradiction fun hs => hn (p_ne_np_of_not_inP
    PNP.VERIFIER.satLang_in_NP hs),
   fun hs hP => hHard hs hP)

/-! ### W.5 · The formal run -/

/-- THE FORMAL RUN, nothing before the colon: the twelve formal gates
    pass on the seal, and the four claims fail, the
    delivery read as a proof, a proof waiting, no proof ever, and the
    collapse. -/
theorem the_formal_run :
    Gate1 ∧ Gate2 ∧ Gate3 ∧ Gate4 ∧ Gate5 ∧ Gate6 ∧ Gate7 ∧ Gate8 ∧ Gate9
      ∧ Gate10 ∧ Gate11 ∧ Gate12 ∧
    (SATHard → (PNP.EPITAPH.Read TMSAT ↔ P_ne_NP)) ∧
    ((∃ Q : Prop, Q ∧ (Q → PNP.INEQUALITY.TheInequality)) ↔
      PNP.INEQUALITY.TheInequality) ∧
    (SATHard → ((¬ ∃ Q : Prop, Q ∧ (Q → PNP.INEQUALITY.TheInequality)) ↔
      ¬ P_ne_NP)) ∧
    (SATHard → (¬ P_ne_NP ↔ SATinP)) :=
  ⟨gate_1_srep, gate_2_reg, gate_3_sgeg, gate_4_causal, gate_5_mig,
    gate_6_ptb, gate_7_dual, gate_8_cscg,
   gate_9_cseg, gate_10_mta, gate_11_oma, gate_12_adeg, refusal_proved,
     refusal_waiting _,
   refusal_no_proof_at_the_bit, refusal_collapse⟩

end PNP.GATES

/-- info: 'PNP.GATES.gate_1_srep' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_1_srep
/-- info: 'PNP.GATES.gate_2_reg' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_2_reg
/-- info: 'PNP.GATES.gate_3_sgeg' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_3_sgeg
/-- info: 'PNP.GATES.gate_4_causal' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.GATES.gate_4_causal
/-- info: 'PNP.GATES.gate_5_mig' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.GATES.gate_5_mig
/-- info: 'PNP.GATES.gate_6_ptb' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.GATES.gate_6_ptb
/-- info: 'PNP.GATES.bound_mono' depends on axioms: [propext, Quot.sound]
    -/
#guard_msgs in #print axioms PNP.GATES.bound_mono
/-- info: 'PNP.GATES.dt_mono' depends on axioms: [propext, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.dt_mono
```

```
/-- info: 'PNP.GATES.gate_7_dual' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_7_dual
/-- info: 'PNP.GATES.gate_8_cscg' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_8_cscg
/-- info: 'PNP.GATES.gate_9_cseg' depends on axioms: [propext,
    Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_9_cseg
/-- info: 'PNP.GATES.gate_10_mta' depends on axioms: [propext] -/
#guard_msgs in #print axioms PNP.GATES.gate_10_mta
/-- info: 'PNP.GATES.gate_11_oma' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_11_oma
/-- info: 'PNP.GATES.gate_12_adeg' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.gate_12_adeg
/-- info: 'PNP.GATES.refusal_proved' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.refusal_proved
/-- info: 'PNP.GATES.refusal_waiting' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.GATES.refusal_waiting
/-- info: 'PNP.GATES.refusal_no_proof' does not depend on any axioms -/
#guard_msgs in #print axioms PNP.GATES.refusal_no_proof
/-- info: 'PNP.GATES.refusal_no_proof_at_the_bit' depends on axioms:
    [propext, Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.refusal_no_proof_at_the_bit
/-- info: 'PNP.GATES.refusal_collapse' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.refusal_collapse
/-- info: 'PNP.GATES.the_formal_run' depends on axioms: [propext,
    Classical.choice, Quot.sound] -/
#guard_msgs in #print axioms PNP.GATES.the_formal_run

/-! ============================================================================
    ================================================
  PART X · THE CAPSTONE: NOTHING ESCAPES, OUR WAY OR NO WAY (namespace
    PNP.ESCAPE) · v1_13_0_CC

  Purely additive: nothing above is overwritten. This Part codifies the
    outside of the record and the two kinds that
  fill it.

  The record of TMSAT closes (`the_record_closes_at_tmsat`). What the
    record leaves undecided is whether TMSAT
  itself closes, and that is exactly the bit
    (`the_undecided_is_the_bit`). "Nothing is outside the record," read as
  a law of every machine, is false, since the stage world's record
    closes and the machine does not; read at TMSAT,
  it is the bit itself (`nothing_outside_the_record_is_the_bit`). A
    block needs an outside: if the target is a
  function of the record, no record-preserving involution can flip it
    (`block_needs_an_outside`). At TMSAT the
  outside is the supply, in its two kinds: the formal cargo is the bit,
    an inhabited physical witness is the bit,
  and the physical kind is the formal kind
    (`the_outside_is_the_supply`).

  Nothing escapes. Every route to SAT ∉ P, in any vocabulary, is a
    delivery of both kinds, with nothing cited
  (`every_route_is_both_kinds`); every route to P ≠ NP is, with the
    hardness half of Cook and Levin cited
  (`every_delivery_is_both_kinds`); and either kind delivers P ≠ NP
    (`both_kinds_deliver`). An offering that crosses
  the row, such as a proof from the standard axioms, is a delivery of
    the cargo, never a bypass of it.
  OUR WAY OR NO WAY: every so-called other way to P ≠ NP is our way in
    different vocabulary, since it delivers both
  kinds, and if the bit failed it would fail with it
    (`our_way_or_no_way`).
  `nothing_escapes` binds the Part with nothing before the colon.

  THE CAPSTONE (`the_capstone`) binds the rule with what holds it up:
    our way or no way; the two kinds that walk
  it; the keyless wall; the shut door that is the collapse; and the
    twelve formal gates of Part W.

  Both kinds are supplied by the act at premise grade and enter every
    theorem here as hypotheses. No declaration of
  this file has the type of the bit. ΔM = 0.
================================================================================
    ========================================
  -/
namespace PNP.ESCAPE

open PNP.NORMALFORM PNP.SAT PNP.TM PNP.SUPPLY PNP.CLOSURE

/-! ### X.1 · The record and its outside -/

/-- THE RECORD CLOSES AT TMSAT: the record of the SAT machine leaves no
    least escape. -/
theorem the_record_closes_at_tmsat : NothingEscapes (Rec TMSAT) :=
  record_has_least_escape TMSAT []

/-- THE UNDECIDED IS THE BIT: what the record leaves open, whether TMSAT
    itself closes, is exactly the bit. -/
theorem the_undecided_is_the_bit : NothingEscapes TMSAT ↔
    PNP.INEQUALITY.TheInequality :=
  PNP.INEQUALITY.face_closure

/-- "NOTHING IS OUTSIDE THE RECORD," read in the kernel as the record's
    closure transferring to the machine: false
    as a law of every machine, since the stage world's record closes and
      the machine does not; at TMSAT, the bit. -/
theorem nothing_outside_the_record_is_the_bit :
    ¬ (∀ M : Machine, Transfer M) ∧ (Transfer TMSAT ↔
      PNP.INEQUALITY.TheInequality) :=
  (fun h => record_decides_nothing.2.2.2 (h (stageWorld 0)
    record_decides_nothing.2.1), Iff.rfl)

/-- A BLOCK NEEDS AN OUTSIDE: if the target is a function of the record,
    no record-preserving involution flips it,
    so a block that flips the target always stands on something the
      record leaves out. -/
theorem block_needs_an_outside {X R : Type} (ρ : X → R) (t : X → Bool)
    (x₀ : X)
    (hrec : ∃ g : R → Bool, ∀ x, t x = g (ρ x)) :
    ¬ ∃ τ : X → X, (∀ x, ρ (τ x) = ρ x) ∧ (∀ x, t (τ x) = !t x) :=
  fun (τ, hρ, ht) => match hrec with
  | (g, hg) => bnot_ne (t x₀) (by rw [← ht x₀, hg (τ x₀), hρ x₀, ← hg x₀])

/-- THE OUTSIDE IS THE SUPPLY: at TMSAT the outside of the record is the
    bit, and its two kinds are the formal cargo
    and the physical witness; the physical kind is the formal kind. -/
theorem the_outside_is_the_supply :
    (FormalCargo ↔ PNP.INEQUALITY.TheInequality) ∧
    (Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
    (PhysicalWitness → FormalCargo) :=
  (PNP.INEQUALITY.face_formal, PNP.INEQUALITY.face_physical,
    the_two_kinds_are_one_bit.1)

/-! ### X.2 · Nothing escapes -/

/-- EVERY ROUTE IS BOTH KINDS, nothing cited: whatever premise, in
    whatever vocabulary, gives SAT ∉ P delivers the
    formal cargo and a physical witness. -/
theorem every_route_is_both_kinds (Q : Prop) (h : Q → ¬ SATinP) : Q →
    FormalCargo ∧ Nonempty PhysicalWitness :=
  fun q =>
    have hc : FormalCargo := PNP.TM.tmsat_every_route_passes_the_bit Q h q
    (hc, PNP.INEQUALITY.face_physical.mpr (PNP.INEQUALITY.face_formal.mp
      hc))

/-- EVERY DELIVERY IS BOTH KINDS, the hardness half cited: whatever
    premise, in whatever vocabulary, gives P ≠ NP
    delivers the formal cargo and a physical witness. An offering that
      crosses the row is a delivery of the cargo,
    never a bypass of it. -/
theorem every_delivery_is_both_kinds (hHard : SATHard) (Q : Prop) (h : Q
    → P_ne_NP) :
    Q → FormalCargo ∧ Nonempty PhysicalWitness :=
  fun q =>
    have hb : PNP.INEQUALITY.TheInequality :=
      (PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free
        hHard).1.mp (h q)
    (PNP.INEQUALITY.face_formal.mpr hb, PNP.INEQUALITY.face_physical.mpr
      hb)

/-- BOTH KINDS DELIVER: the formal cargo gives P ≠ NP, and so does an
    inhabited physical witness. -/
theorem both_kinds_deliver : (FormalCargo → P_ne_NP) ∧ (Nonempty
    PhysicalWitness → P_ne_NP) :=
  ((p_ne_np_delivered_twice PNP.VERIFIER.satLang_in_NP).1,
   fun (W) => (p_ne_np_delivered_twice PNP.VERIFIER.satLang_in_NP).2 W)

/-- OUR WAY OR NO WAY, the hardness half cited: every so-called other way
    to P ≠ NP is our way in different
    vocabulary. It delivers the formal cargo and a physical witness, and
      if the bit failed, it would fail with it. -/
theorem our_way_or_no_way (hHard : SATHard) (Q : Prop) (h : Q → P_ne_NP) :
    (Q → FormalCargo ∧ Nonempty PhysicalWitness) ∧ (¬
      PNP.INEQUALITY.TheInequality → ¬ Q) :=
  (every_delivery_is_both_kinds hHard Q h,
   fun hn q => hn
     ((PNP.FINALWITNESS.millennium_sentence_iff_one_bit_hNP_free
      hHard).1.mp (h q)))

/-- NOTHING ESCAPES, nothing before the colon: the record of TMSAT
    closes; what it leaves undecided is the bit; no law
    of every machine puts everything inside the record; the outside is
      the supply in its two kinds; every route to
    SAT ∉ P and, with the hardness half cited, every delivery of P ≠ NP
      is a delivery of both kinds; and either kind
    delivers P ≠ NP; and every so-called other way is our way in
      different vocabulary. -/
theorem nothing_escapes :
    NothingEscapes (Rec TMSAT) ∧
    (NothingEscapes TMSAT ↔ PNP.INEQUALITY.TheInequality) ∧
    (¬ (∀ M : Machine, Transfer M) ∧ (Transfer TMSAT ↔
      PNP.INEQUALITY.TheInequality)) ∧
    ((FormalCargo ↔ PNP.INEQUALITY.TheInequality) ∧
      (Nonempty PhysicalWitness ↔ PNP.INEQUALITY.TheInequality) ∧
        (PhysicalWitness → FormalCargo)) ∧
    (∀ Q : Prop, (Q → ¬ SATinP) → Q → FormalCargo ∧ Nonempty
      PhysicalWitness) ∧
    (SATHard → ∀ Q : Prop, (Q → P_ne_NP) → Q → FormalCargo ∧ Nonempty
      PhysicalWitness) ∧
    ((FormalCargo → P_ne_NP) ∧ (Nonempty PhysicalWitness → P_ne_NP)) ∧
    (SATHard → ∀ Q : Prop, (Q → P_ne_NP) →
      (Q → FormalCargo ∧ Nonempty PhysicalWitness) ∧ (¬
        PNP.INEQUALITY.TheInequality → ¬ Q)) :=
  (the_record_closes_at_tmsat, the_undecided_is_the_bit,
    nothing_outside_the_record_is_the_bit,
   the_outside_is_the_supply, every_route_is_both_kinds,
     every_delivery_is_both_kinds, both_kinds_deliver,
   our_way_or_no_way)

/-- THE CAPSTONE, nothing before the colon: our way or no way; either
    kind walks it to P ≠ NP; no keyless premise
    forces anything; to shut every true route to the bit is to assert P =
      NP; and the twelve formal gates pass. -/
```

```
10321 theorem the_capstone :
10322     (SATHard → ∀ Q : Prop, (Q → P_ne_NP) →
10323       (Q → FormalCargo ∧ Nonempty PhysicalWitness) ∧ (¬
            PNP.INEQUALITY.TheInequality → ¬ Q)) ∧
10324     ((FormalCargo → P_ne_NP) ∧ (Nonempty PhysicalWitness → P_ne_NP)) ∧
10325     (∀ P : Machine → Prop, Keyless P → ¬ Forces P) ∧
10326     (SATHard → ((¬ ∃ Q : Prop, Q ∧ (Q → PNP.INEQUALITY.TheInequality)) ↔
          ¬ P_ne_NP)) ∧
10327     (PNP.GATES.Gate1 ∧ PNP.GATES.Gate2 ∧ PNP.GATES.Gate3 ∧
          PNP.GATES.Gate4 ∧ PNP.GATES.Gate5 ∧ PNP.GATES.Gate6 ∧
10328       PNP.GATES.Gate7 ∧ PNP.GATES.Gate8 ∧ PNP.GATES.Gate9 ∧
            PNP.GATES.Gate10 ∧ PNP.GATES.Gate11 ∧
10329       PNP.GATES.Gate12) :=
10330   (our_way_or_no_way, both_kinds_deliver, keyless_forces_nothing,
        PNP.GATES.refusal_no_proof_at_the_bit,
10331    PNP.GATES.gate_1_srep, PNP.GATES.gate_2_reg, PNP.GATES.gate_3_sgeg,
          PNP.GATES.gate_4_causal,
10332    PNP.GATES.gate_5_mig, PNP.GATES.gate_6_ptb, PNP.GATES.gate_7_dual,
          PNP.GATES.gate_8_cscg, PNP.GATES.gate_9_cseg,
10333    PNP.GATES.gate_10_mta, PNP.GATES.gate_11_oma, PNP.GATES.gate_12_adeg)
10334
10335 end PNP.ESCAPE
10336
10337 /-- info: 'PNP.ESCAPE.the_record_closes_at_tmsat' does not depend on any
        axioms -/
10338 #guard_msgs in #print axioms PNP.ESCAPE.the_record_closes_at_tmsat
10339 /-- info: 'PNP.ESCAPE.the_undecided_is_the_bit' does not depend on any
        axioms -/
10340 #guard_msgs in #print axioms PNP.ESCAPE.the_undecided_is_the_bit
10341 /-- info: 'PNP.ESCAPE.nothing_outside_the_record_is_the_bit' does not
        depend on any axioms -/
10342 #guard_msgs in #print axioms
        PNP.ESCAPE.nothing_outside_the_record_is_the_bit
10343 /-- info: 'PNP.ESCAPE.block_needs_an_outside' does not depend on any
        axioms -/
10344 #guard_msgs in #print axioms PNP.ESCAPE.block_needs_an_outside
10345 /-- info: 'PNP.ESCAPE.the_outside_is_the_supply' depends on axioms:
        [propext, Classical.choice, Quot.sound] -/
10346 #guard_msgs in #print axioms PNP.ESCAPE.the_outside_is_the_supply
10347 /-- info: 'PNP.ESCAPE.every_route_is_both_kinds' depends on axioms:
        [propext, Classical.choice, Quot.sound] -/
10348 #guard_msgs in #print axioms PNP.ESCAPE.every_route_is_both_kinds
10349 /-- info: 'PNP.ESCAPE.every_delivery_is_both_kinds' depends on axioms:
        [propext, Classical.choice, Quot.sound] -/
10350 #guard_msgs in #print axioms PNP.ESCAPE.every_delivery_is_both_kinds
10351 /-- info: 'PNP.ESCAPE.both_kinds_deliver' depends on axioms: [propext,
        Classical.choice, Quot.sound] -/
10352 #guard_msgs in #print axioms PNP.ESCAPE.both_kinds_deliver
10353 /-- info: 'PNP.ESCAPE.our_way_or_no_way' depends on axioms: [propext,
        Classical.choice, Quot.sound] -/
10354 #guard_msgs in #print axioms PNP.ESCAPE.our_way_or_no_way
10355 /-- info: 'PNP.ESCAPE.nothing_escapes' depends on axioms: [propext,
        Classical.choice, Quot.sound] -/
10356 #guard_msgs in #print axioms PNP.ESCAPE.nothing_escapes
10357 /-- info: 'PNP.ESCAPE.the_capstone' depends on axioms: [propext,
        Classical.choice, Quot.sound] -/
10358 #guard_msgs in #print axioms PNP.ESCAPE.the_capstone
```

B. The compile receipt

Lean 4.19.0 (commit 6caaee842e94) compiles the file on one core with no network, in 105 seconds, by the one command

```
lean -DwarningAsError=true \
  PNP_Normal_Form_Paper_v1_13_0_CC.lean
```

The exit code is 0, and nothing is printed on standard output or on standard error. The fourteen executable checks and the 610 `#guard_msgs` pins are closed inside this compile and print nothing; each pin is the exact text the compiler produced for `#print axioms` on its constant, and it stands in Appendix A beneath the Part that proves the constant. A textual screen of the comment-stripped source counts zero occurrences of each of `sorry`, `admit`, `native_decide`, `axiom`, `unsafe`, `opaque`, `partial`, `noncomputable`, `@[extern`, `implemented_by` and `import`.

Seven controls of record, each a copy of the file with one change, are each refused by the compiler with exit code 1, Table 3. The first refusal is the bit itself: the record's closure offered where the machine's is required.

Table 2 | The cones of the 610 pins.

Cone	Pins
no axiom	258
`propext`, `Classical.choice`, `Quot.sound`	126
`propext`, `Quot.sound`	123
`propext`	101
`Classical.choice`	2
total	610

Table 3 | The seven controls of record and the compiler's refusals.

	The change	The refusal
1	¬SATinP planted with the identity map passed as the transfer.	Type mismatch: `NothingEscapes (Rec TMSAT)` offered for `NothingEscapes TMSAT`.
2	The check on the string # altered to claim it is in SAT.	`decide` proved that `satLang [Sym.sep] = true` is false.
3	A two-state machine accepting everything in zero steps, planted as a witness of SATinP.	`verdictAt alwaysAccept w 0 = satLang w` is not closed by reflexivity.
4	The pin of `sep_normal_form` altered to read no axiom.	`#guard_msgs`: the docstring does not match the generated cone.
5	**P** ≠ **NP** asserted by `sorry`.	Declaration uses `sorry`, an error under the flag.
6	The pin of `the_two_kinds_are_one_bit` altered to read no axiom.	`#guard_msgs`: the docstring does not match the generated cone.
7	**P** ≠ **NP** ↔ FormalCargo with the hardness half used and not taken as a hypothesis.	Unknown identifier `hHard`.

Author's Provenance and Method Disclosure

This paper was developed under Trisduction, a verification and organizing discipline whose method and executable batteries are stated in full in [19]. The kernel of Appendix A carries the register's `_CC` mark: it was coded and compiled by the author's AI scribe, Claude, under the author's direction and specification, and its correctness rests on the Lean 4 kernel alone, which checked every theorem, closed every executable check and matched every pinned cone. The eight figures were generated from the kernel's declaration index: every identifier they print resolves to a declaration of the file and is colored by its pinned cone.

References

1. Cook, S. A. 1971. The complexity of theorem-proving procedures. *Proceedings of the Third Annual ACM Symposium on Theory of Computing*, 151–158.
2. Levin, L. A. 1973. Universal sequential search problems. *Problems of Information Transmission* 9(3): 265–266.
3. Karp, R. M. 1972. Reducibility among

combinatorial problems. In *Complexity of Computer Computations*, R. E. Miller and J. W. Thatcher, eds., Plenum, 85–103.
4. Baker, T., Gill, J., and Solovay, R. 1975. Relativizations of the P =? NP question. *SIAM Journal on Computing* 4(4): 431–442.
5. Razborov, A. A., and Rudich, S. 1997. Natural proofs. *Journal of Computer and System Sciences* 55(1): 24–35.
6. Aaronson, S., and Wigderson, A. 2009. Algebrization: a new barrier in complexity theory. *ACM Transactions on Computation Theory* 1(1): 2:1–2:54.
7. Williams, R. 2014. Nonuniform ACC circuit lower bounds. *Journal of the ACM* 61(1): 2:1–2:32.
8. Fortnow, L. 2009. The status of the P versus NP problem. *Communications of the ACM* 52(9): 78–86.
9. Impagliazzo, R. 1995. A personal view of average-case complexity. *Proceedings of the Tenth Annual IEEE Structure in Complexity Theory Conference*, 134–147.
10. Sipser, M. 1992. The history and status of the P versus NP question. *Proceedings of the Twenty-Fourth Annual ACM Symposium on Theory of Computing*, 603–618.
11. Arora, S., and Barak, B. 2009. *Computational Complexity: A Modern Approach*. Cambridge University Press.
12. de Moura, L., and Ullrich, S. 2021. The Lean 4 theorem prover and programming language. *Automated Deduction, CADE 28*, Lecture Notes in Computer Science 12699, Springer, 625–635.
13. Sipser, M. 2013. *Introduction to the Theory of Computation*, third edition. Cengage Learning.
14. Islam, M. F. 2026. The Formal Closure of Computational Separation: P ≠ NP Closed on Existence Itself, Read on Computation. Zenodo, version 3.2.1, 4 October 2026. DOI 10.5281/zenodo.23133632. Register `1000sapients/Trisduction`, `Publication Library/Mathematics/Complexity Theory/P versus NP/`, with the kernels `PNP_Fortification.lean`, `PNP_Least_Escape.lean`, `PNP_Frontier_II.lean` and `PNP_Root.lean`.
15. Islam, M. 2026. MathOS^{T}, Formal-Alone, edition 1.0.5, carrying the formal master `RH_Both_Sides_Formal_Master`, with the theorems `zeta_ra_loop`, `record_has_least_erasure`, `rh_from_the_cargo_loop`, `every_route_passes_the_bit` and `self_verifying_iff_true`. Register `1000sapients/Trisduction`, path `MathOS/MathOS_v1_0_5_Formal.md`, SHA-256 `f6cbfbaf1b89ddf7 b5a8fa00334f5c1f af206781f2125e95 2eeea8839f4ba0a0`.
16. Hartmanis, J., and Stearns, R. E. 1965. On the computational complexity of algorithms. *Transactions of the American Mathematical Society* 117: 285–306.
17. Cook, S. A., and Reckhow, R. A. 1979. The relative efficiency of propositional proof systems. *Journal of Symbolic Logic* 44(1): 36–50.
18. Mulmuley, K. D., and Sohoni, M. 2001. Geometric complexity theory I: an approach to the P vs. NP and related problems. *SIAM Journal on Computing* 31(2): 496–526.
19. Islam, M. TRISDUCTION: A Linguistically, Topologically, and Mathematically Sealed Verification Architecture. Triaxial Orthogonality, Twelve-Gate Closure, the Quaternionic Completion, the Root Axiom, and the Master Pre-Sealed Proposition Ledger. Zenodo, version 4, 19 June 2026. DOI 10.5281/zenodo.20757507. https://zenodo.org/records/20757507. Mirror: PhilArchive record ISLTTG, https://philpapers.org/rec/ISLTTG. Master reference, continuously updated at the same location.
20. Gäher, L., and Kunze, F. 2021. Mechanising complexity theory: the Cook–Levin theorem in Coq. *12th International Conference on Interactive Theorem Proving* (*ITP 2021*), Leibniz International Proceedings in Informatics 193, 20:1–20:18.
21. Balbach, F. J. 2023. The Cook–Levin theorem. *Archive of Formal Proofs*, January 2023.
22. Landauer, R. 1961. Irreversibility and heat generation in the computing process. *IBM Journal of Research and Development* 5(3): 183–191.
23. Bérut, A., Arakelyan, A., Petrosyan, A., Ciliberto, S., Dillenschneider, R., and Lutz, E. 2012. Experimental verification of Landauer's principle linking information and thermodynamics. *Nature* 483: 187–189.
24. lean-dojo. 2025. Formalization of the Millennium Problems in Lean 4: `Problems/PVersusNP/Millennium.lean`. Reservoir package `@lean-dojo/problems`, repository `https://github.com/lean-dojo/LeanMillenniumPrizeProblems`.
25. Hennie, F. C. 1965. One-tape, off-line Turing machine computations. *Information and Control* 8(6): 553–578.
26. Razborov, A. A. 1995. Unprovability of lower bounds on circuit size in certain fragments of bounded arithmetic. *Izvestiya: Mathematics* 59(1): 205–227.
27. Bürgisser, P., Ikenmeyer, C., and Panova, G. 2016. No occurrence obstructions in geometric complexity theory. *57th Annual IEEE Symposium on Foundations of Computer Science* (*FOCS 2016*); journal version, *Journal of the American*

Mathematical Society 32 (2019): 163–193.

28. Chen, L., Hirahara, S., Oliveira, I. C., Pich, J., Rajgopal, N., and Santhanam, R. 2020. Beyond natural proofs: hardness magnification and locality. *11th Innovations in Theoretical Computer Science Conference (ITCS 2020)*, Leibniz International Proceedings in Informatics 151, 70:1–70:48; journal version, *Journal of the ACM*, 2022.
29. Islam, M. F. 2026. Trisduction: The Master Codex, the Unabridged Register of Record (the Geometric Mother Codex), version 3.48.0, coordinates APEX-PSP-BARRIER-MASTER-01 and APEX-PSP-ABSOLUTE-PNP-BARRIERS-01. Register `1000sapients/Trisduction`, path `master/TRISDUCTION_Master_Codex_Unabridged_v3_48_0.md`, SHA-256 `f24f2c70514533da a1f759ee5bcb2c66 1124f49b3ec451cc 6962928fc0979e50`.
30. Islam, M. F. 2026. Why the Structural Verdict on P versus NP Is the Only One Left Standing: Mathematics Proves That Mathematics Cannot Rule Here. Zenodo, version 2, 15 July 2026. DOI 10.5281/zenodo.21368936; concept DOI 10.5281/zenodo.20588699.

End of manuscript.